International Journal of Secondary Computing and Applications Research

Volume 3, Issue 4

EDITOR-IN-CHIEF: DR. MARIA HWANG

OCTOBER, 2026

Proceedings of International Journal of Secondary Computing and Applications Research, Vol 3, Issue 4

October 30, 2026

Letter from the Editor-in-Chief

It is my pleasure to welcome you to this issue of the International Journal of Secondary Computing and Applications Research (IJSCAR). This is our largest issue yet, with roughly double the number of articles in our previous one, and the breadth of student-led computing research gathered here shows just how quickly this community is growing. The work ranges from machine learning for health and medicine, including genomic variant interpretation, rare-disease modeling, and dental imaging, to computer vision for agriculture, with tomato plant disease detection, and for underwater environments; from predictive maintenance for turbofan engines and anomaly detection in high-energy particle physics to brain-inspired learning algorithms; and from nutrition estimation to audits of demographic sensitivity, with large language models at the center of several studies.

Taken together, these papers show a maturing research instinct: a willingness to engage seriously with real data and real constraints, not just idealized benchmarks. Several authors in this issue compared their methods against strong baselines, examined how sensitive their results are to choices such as decision thresholds, random seeds, and evaluation data, and candidly reported the limits of what their experiments can show rather than reporting a single favorable configuration—exactly the kind of self-scrutiny that distinguishes rigorous research from a class project.

At IJSCAR, our mission remains to provide a platform for students and early-career researchers to share innovative ideas, engage with the broader research community, and contribute to the advancement of knowledge. We are continually inspired by the quality of submissions we receive and by the dedication of young scholars pursuing meaningful and impactful research, and we remain committed to nurturing that curiosity issue after issue.

As Editor-in-Chief, I want to thank our authors for the care they put into this work, their mentors and teachers for the guidance behind the scenes, and our reviewers for the time they gave to strengthening each submission. On behalf of the editorial team, I am also grateful to our readers for their continued support in helping foster a community committed to learning, discovery, and scientific progress.

We hope this issue informs and inspires, and we look forward to sharing the next chapter of student computing research with you soon.

Sincerely,
Maria Hwang
Editor-in-Chief

Volume 3, Issue 4 October 30, 2026

DOI: 10.67149/yhjs2024.5/d32s2mcm | https://ijscar.org/pubs/vol3-issue4.pdf

© 2026 International Journal of Secondary Computing and Applications Research

Contents

- Comparing the Sample and Computational Efficiency of Unit-Precision Predictive Coding and Backpropagation
  Austin Huang **4**
- Benchmarking BRCA2 Missense Variant Pathogenicity Predictors Across Functional Domains and in a Subset of Variants Observed in Ashkenazi Jewish Individuals
  Meirav Bocknek, Mariame Diabate **16**
- Beyond Cycle Count: Sensor-Based Maintenance Warnings for Simulated Turbofan Engines
  Prahalad Talur **23**
- Dynamic Mathematical PDE Modeling and Machine Learning for TANGO2 Identification and Crisis Prediction
  Abhay Tippimath **29**
- Investigating the Efficacy of the Gemini 2.5 Pro Model in the Estimation of Macronutrient Content
  Sonali Guttikonda, Junyao Chen, He Tu, Zhengyuan Zhou **38**
- Analysis of Methods to Improve Underwater Object Detection in Various Environments
  Nico Katz **44**
- Optimizing Classification-Based Machine Learning Models for Bacterial Spot Disease Detection in Tomato Plants
  Siya Sharma **57**
- Demographic Sensitivity in Heartsteep Drink Recommendations: An Exploratory Audit of Three Language Models
  Yoomin (Rachel) Jung **62**
- A Jet-Multiplicity Confound in Reconstruction-Based Anomaly Detection: Evidence from a Higgs-to-Tau-Pair Proxy Signal
  Abhijay Arun **72**
- Iterative Development of an AI Prototype for Dental Caries Detection and Localization with Informal Clinical Feedback
  Chinmay Karthik Amancha **88**
- Causal Faithfulness of SAE Feature Attributions in a Probe Pipeline
  Samuel Xie, Noura Al-Moubayed **94**
- Interpreting Character-Level Transformers for Addition and Multiplication Through Integrated Gradients
  Abhinav Damera **107**

Comparing the Sample and Computational Efficiency of Unit-Precision Predictive Coding and Backpropagation

Austin Huang
Eton College
Windsor, Berkshire, United Kingdom
austinhuang1121@outlook.com

Abstract

Synchronous unit-precision predictive coding (PC) replaces backpropagation's (BP's) immediate backward pass with recurrent activity inference. We compared PC with objective- and optimizer-matched BP in tanh multilayer perceptrons on MNIST and Fashion-MNIST. In the depth-2 network trained on 100 labelled MNIST examples, each method's learning rate was selected independently using validation data, and both searches met the prespecified bracketing rule. With 200 inference steps, PC came close to BP's accuracy but remained lower. Across ten fresh paired seeds, the mean epoch-30 PC-minus-BP test-accuracy difference was −0.295 percentage points (95% paired-bootstrap interval [−0.376, −0.181]). PC nevertheless required substantially more computation. Diagnostic-free float32 training took 130.6 times as long as BP on a single CPU thread, while analytical counts gave approximately 113 times as many MACs, 112 times as many FLOPs, and 403 times as many tanh evaluations. The depth-2, 500-label Fashion-MNIST comparison remained unresolved because the BP learning-rate search did not show the required numerically valid decline at a higher rate; its result is therefore descriptive. These findings apply only to the tested synchronous unit-precision formulation, datasets, architecture, and digital CPU implementation, not to all predictive-coding variants or hardware.

Keywords

Predictive coding, backpropagation, sample efficiency, computational efficiency, iterative inference, implementation validation

ACM Reference Format:
Austin Huang. 2026. Comparing the Sample and Computational Efficiency of Unit-Precision Predictive Coding and Backpropagation. In *Proceedings of International Journal of Secondary Computing and Applications Research (IJSCAR VOL. 3, ISSUE 4)*. ACM, New York, NY, USA, 12 pages. https://doi.org/10.67149/yhjs2024.5/3u76nbg5

1 Introduction

Backpropagation is highly effective for training multilayer networks, but its globally coordinated error signals have motivated alternatives that may be more compatible with biological or local computation [1]. Predictive-coding networks instead refine neural activities through local prediction errors [11]. Under particular assumptions, their weight updates can approach BP gradients [17]. That mathematical link is important, but it leaves a practical question unanswered: does recurrent inference improve learning enough to justify its cost?

The answer depends on the algorithm being tested. This study concerns synchronous, unit-precision PC: every prediction error has unit precision, hidden beliefs are updated together from an unchanged state snapshot, and predictions are recomputed during inference. Precision-weighted, fixed-prediction, amortized, incremental, and error-coordinate methods can behave differently. We therefore use "PC" as shorthand for the tested formulation, not for the entire predictive-coding family.

An apparent efficiency difference can arise from an incorrect implementation, a different objective, unequal initialization, or incomplete hyperparameter selection. Sample and computational efficiency must also remain separate. Reusing a fixed labelled set creates more presentations without supplying more unique labels, while every PC inference step consumes work absent from BP.

We address these issues in four stages. First, the PC implementation is checked against automatic differentiation, finite differences, and an independent energy formulation. Second, raw MSE gradients are compared with matched BP gradients, separating directional alignment, magnitude, and convergence to the PC fixed point. Third, a broad benchmark evaluates depths, labelled-set sizes, and three inference budgets; that experiment is retained as descriptive evidence because its learning-rate grid was truncated before validation turnover. Fourth, a prospective learning-rate extension uses a wider fixed grid, deterministic infill, independent validation selection, 200 inference steps, and untouched test evaluation on ten fresh paired seeds.

For the depth-2, 100-label MNIST condition with PC at $T = 200$, the wider search found numerically valid declines in validation accuracy on both sides of each method's near-best rates. PC's mean accuracy remained below BP's. The measured gap was smaller than in the separate original experiment, but the searches, inference budgets, and seeds differed, so that comparison does not identify the cause. Fashion-MNIST remained unresolved because BP never showed the required valid decline at a higher learning rate. Diagnostic-free CPU timing and analytical operation counts also show substantial recurrent cost. These findings apply to this synchronous unit-precision implementation rather than PC as a whole.

2 Theoretical and empirical context

Whittington and Bogacz related supervised PC updates to BP under assumptions that include small output errors and converged neural activities [17]. The model evaluated here assigns unit precision to every prediction error and does not hold predictions fixed during

This paper is published under the Creative Commons Attribution-NonCommercial-NoDerivs 4.0 International (CC-BY-NC-ND 4.0) license. Authors reserve their rights to disseminate the work on their personal and corporate Web sites with the appropriate attribution.
IJSCAR VOL. 3, ISSUE 4
© 2026 The Author(s). Published by IJSCAR.

inference. It therefore lacks the output-precision hierarchy or fixed-prediction construction that can bring gradient magnitudes closer together. Later analyses distinguish gradient direction from numerical equality and strict PC from modified inference rules [12, 13]. Algorithms that compute exact reverse derivatives form a separate category [7, 14].

Finite inference matters because convergence depends on the weights, initialization, and step size; outside suitable regimes, activity dynamics may diverge [2]. Validation must therefore distinguish convergence to the PC fixed point from agreement with BP. Reaching the former does not imply equality with the latter.

Recent work explores settings in which PC may learn efficiently, including deep or narrow architectures, pretraining, and conditioning interventions [8]. Other methods alter the computation through specialized software, hybrid or incremental schedules, depth-scaled parameterizations, or error-coordinate dynamics [3, 4, 10, 15, 16]. In particular, the current version of ePC is an error-based reparameterization designed for digital simulation and is reported as accepted to the ICML 2026 main track [3]. These approaches may reduce inference cost or change optimization behaviour, but their benefits cannot be assumed for conventional synchronous state inference, and local parameter updates alone do not guarantee lower end-to-end cost [19].

We evaluate only synchronous unit-precision PC and do not claim that the observed deficit is specific to this formulation. A comparison with a precision-weighted or fixed-prediction variant could determine whether the result changes under those interventions, but that experiment was not performed here. Such variants therefore remain an explicit limitation and future direction.

3 Methods

3.1 Data, matching, and original benchmark

Experiments used the official MNIST [6] and Fashion-MNIST [18] splits. Images were flattened to 784 dimensions and normalized using statistics from the corresponding training split. Split seed 999 selected a fixed 5,000-example validation pool from the 60,000 training examples. Model selection used training and validation data only; official test outcomes were accessed only after each configuration was finalized.

Every model was a width-256 tanh multilayer perceptron with biases, a linear 10-unit output, batch size 64, float32 training, and 30 epochs. Matched conditions shared initial parameters, examples, minibatches, and epoch order. Both theoretical comparators used mean-per-example MSE and plain SGD without momentum or weight decay. A conventional BP classifier with cross-entropy and Adam was retained as a practical reference, not as a theoretical comparator [5]. Models were implemented in PyTorch [9].

The original benchmark crossed MNIST depths two, four, and six with 100, 500, 1,000, and 5,000 labelled examples; Fashion-MNIST used depths two and six with 500 and 5,000 examples. PC inference budgets were $T \in \{10, 20, 50\}$, and ten paired seeds were used. This produced 48 PC-budget comparisons. Its six-rate learning-rate search ended at 0.03 while validation performance was still improving. We retain those results as broad descriptive evidence rather than as globally tuned comparisons.

3.2 Synchronous unit-precision PC

Let v_0 denote the input and v_K the output. With PyTorch's parameter orientation,

$$\mu_{b,l+1} = \phi_l(v_{b,l})W_l^\top + b_l, \qquad \epsilon_{b,l+1} = v_{b,l+1} - \mu_{b,l+1}, \tag{1}$$

where b indexes examples, ϕ_0 is the identity, and hidden activations use tanh. The per-example energy is

$$E_b = \frac{1}{2}\sum_{l=0}^{K-1} \|\epsilon_{b,l+1}\|_2^2. \tag{2}$$

Input and target output are clamped. At inference step t, every hidden belief is updated synchronously from the same state snapshot:

$$v_{b,l}^{(t+1)} = v_{b,l}^{(t)} - \eta_{\text{inf}} \left.\frac{\partial E_{\text{sum}}}{\partial v_{b,l}}\right|_{v^{(t)}}, \qquad E_{\text{sum}} = \sum_b E_b. \tag{3}$$

After T steps, beliefs are detached and the parameter rule is the batch-mean derivative

$$\frac{\partial E_{\text{mean}}}{\partial W_l} = -\frac{1}{B}\sum_{b=1}^{B} \epsilon_{b,l+1}^\top \phi_l(v_{b,l}), \qquad E_{\text{mean}} = B^{-1}\sum_b E_b. \tag{4}$$

Thus, activity inference is not accidentally divided by the batch size, whereas parameter learning has the same mean reduction as BP. The learning rule does not differentiate through the inference trajectory. Appendix A gives the activity derivative, bias rule, stationarity definition, and numerical conventions; Appendix B reports the numerical checks.

3.3 Prospective learning-rate extension

We widened the learning-rate search to check whether the best results were still at its boundary. We required numerically valid runs with lower validation accuracy at rates below and above the near-best group; a failed run did not count as evidence of a decline. This rule brackets the selected region within the tested rates but does not prove a global optimum.

The extension fixed four depth-2 cells before development: MNIST with 100 labels and Fashion-MNIST with 500 labels, each comparing BP–MSE–SGD with PC–MSE–SGD at $T = 200$ and $\eta_{\text{inf}} = 0.2$. Five new development seeds (31001–31005) and ten disjoint confirmatory seeds (41001–41010) preserved paired initialization, examples, and epoch order.

Every cell first evaluated the same 18 learning rates:

$$0.0003, 0.001, 0.003, 0.01, 0.03, 0.04, 0.05, 0.06, 0.07,$$
$$0.08, 0.09, 0.1, 0.12, 0.15, 0.3, 1, 3, 10.$$

A candidate could enter selection only when all five development runs completed with finite values and passed the prespecified numerical checks. Selection maximized mean epoch-30 validation accuracy. Rates within 0.1 percentage points of the maximum formed the near-optimal set, and the smallest such rate was selected. Formally, a qualifying flank had to lie outside that set, fall at least 0.5 points below the maximum, and be worse than the selected rate in at least four of five paired seeds.

If a flank was absent after the fixed grid, up to three deterministic geometric-midpoint attempts per side filled the largest remaining log-rate gap. A cell was fully performance-bracketed only when

qualifying finite lower and upper flanks existed. A rate that succeeded in some seeds but failed in others had mixed feasibility; neither that pattern nor failure in all seeds counted as validation turnover. A cell still lacking the required evidence after the fixed actions was labelled unresolved. These labels were fixed before test access.

Both depth-2, 100-label MNIST method cells met the bracketing rule. On Fashion-MNIST, PC met the rule. BP had a learning rate that completed successfully in all five development runs, so it could be carried forward, but its search remained unresolved after three upper-side midpoint attempts. Both datasets entered confirmatory evaluation, but only the specified MNIST condition supports a bracketed comparison.

3.4 Analysis and resource accounting

The main quantity estimated in the extension was the paired epoch-30 test-accuracy difference, PC minus BP, within each dataset. We also measured performance across the full training run rather than only at epoch 30. For epochs 1–30, we calculated the area under each test-accuracy curve against cumulative training-example presentations, divided by the presentation range, and compared PC with BP within each paired seed. This is the normalized test-accuracy-versus-cumulative-presentation AUC difference used below. Ten-thousand paired seed-level bootstrap resamples produced percentile intervals; time ratios were bootstrapped on the log scale. These intervals are conditional on the selected rates, fixed split, and confirmatory design; they do not incorporate uncertainty from repeating the development search. No p-values or multiplicity-adjusted claims were used. Extension seeds were never pooled with the original benchmark or earlier construction attempts.

Timing used float32 PyTorch on one Apple ARM64 CPU thread. For each confirmatory run, an isolated disposable model warmed the cache; the untouched identically initialized model then supplied 30 diagnostic-free timed epochs. Warm-up, preprocessing, evaluation, diagnostics, serialization, and logging were excluded. These ratios characterize this implementation and do not predict GPU or neuromorphic performance.

For a hardware-independent secondary measure, one multiply–accumulate (MAC) counts as two FLOPs. Every other explicit scalar operation counts as one FLOP, and tanh evaluations are reported separately. With $S = 268{,}800$ weight MACs, $R = 68{,}096$ hidden-error MACs, $A = 522$ affine output units, $H = 512$ hidden units, and $P = 269{,}322$ parameters, BP uses $B(2S + R)$ MACs per batch and PC uses $B[3S + T(S + R)]$. Appendix C, Section C.3, reports exact per-epoch counts; Table 4 gives the corresponding 30-epoch totals.

4 Implementation checks

The checks support two conclusions: the implementation follows the intended PC equations, and in the controlled validation cases, PC updates near the stated stopping tolerance pointed in almost the same direction as BP updates while remaining smaller. These controlled checks do not establish convergence throughout training or explain the remaining accuracy gap.

Initial comparison code did not implement the intended supervised PC–BP correspondence, so the PC model was rebuilt around the MSE objective. Before the learning comparison, the replacement was tested for matched parameter orientation and initialization, detached inferred beliefs, and synchronous hidden-layer updates. Validation used float64 arithmetic and raw gradients before any optimizer step. The convergence comparisons in Table 1 and Figure 1C–D used randomly initialized, untrained networks, one synthetic example per seed, and target perturbation $\epsilon = 0.01$. Their separately converged reference stopped at the finite stationarity tolerance of 10^{-8}. Appendix B and Table 5 give the numerical checks.

Table 1: Width-256 weight-gradient comparisons at $T = 200$, averaged over ten seeds and all weight layers. Randomly initialized, untrained networks used one synthetic example per seed, target perturbation $\epsilon = 0.01$, and float64 arithmetic. "To PC limit" uses a separately converged PC gradient stopped at finite stationarity tolerance 10^{-8}; "to BP" uses the matched BP gradient.

Depth	Cosine	Norm ratio	To BP	To PC limit
2	0.999	0.740	0.264	5.70×10^{-5}
4	0.998	0.681	0.322	1.36×10^{-4}
6	0.998	0.673	0.331	6.61×10^{-4}

Manual activity gradients matched automatic differentiation, while the detached parameter gradients matched both automatic differentiation and centred finite differences. A separately written energy-based model reproduced the same beliefs, energy, stationarity, and gradients. These results establish internal correctness, not equivalence to BP.

BP comparisons used targets $y_\epsilon = o + \epsilon(y - o)$. For $\epsilon \le 10^{-3}$ in depth-2 networks, the minimum mean weight-gradient cosine across activation, perturbation, and layer cells was 0.992; the overall mean was 0.994, and the smallest individual cosine was 0.970. Figure 1A–B shows that this strong directional agreement coexisted with PC/BP norm ratios below one: the updates pointed in similar directions, but PC's updates remained smaller.

Figure 1 separates convergence to the PC fixed point from agreement with BP. At width 256 and $T = 200$, the finite-inference PC update was close to the PC update computed after inference reached the stated stopping tolerance; Table 1 gives the exact distances at depths two, four, and six. The small depth-two reversal after $T = 100$ is consistent with comparison against a finite-tolerance reference and should not be interpreted as a loss of inference convergence. The difference from BP stopped decreasing at a much larger value. At $T = 200$, cosines exceeded 0.998, whereas PC/BP norm ratios fell from 0.740 at depth two to 0.673 at depth six, showing that the residual difference was mainly in update size.

In the original benchmark, PC and BP shared the same truncated learning-rate grid and every MSE condition selected its upper boundary. With similarly directed gradients, a common nominal rate produced smaller PC parameter steps. The learning-rate extension instead selected each method independently and chose 0.04 for PC versus 0.03 for BP in the depth-2, 100-label MNIST condition. In a separate 100-example check, five epochs reduced feedforward

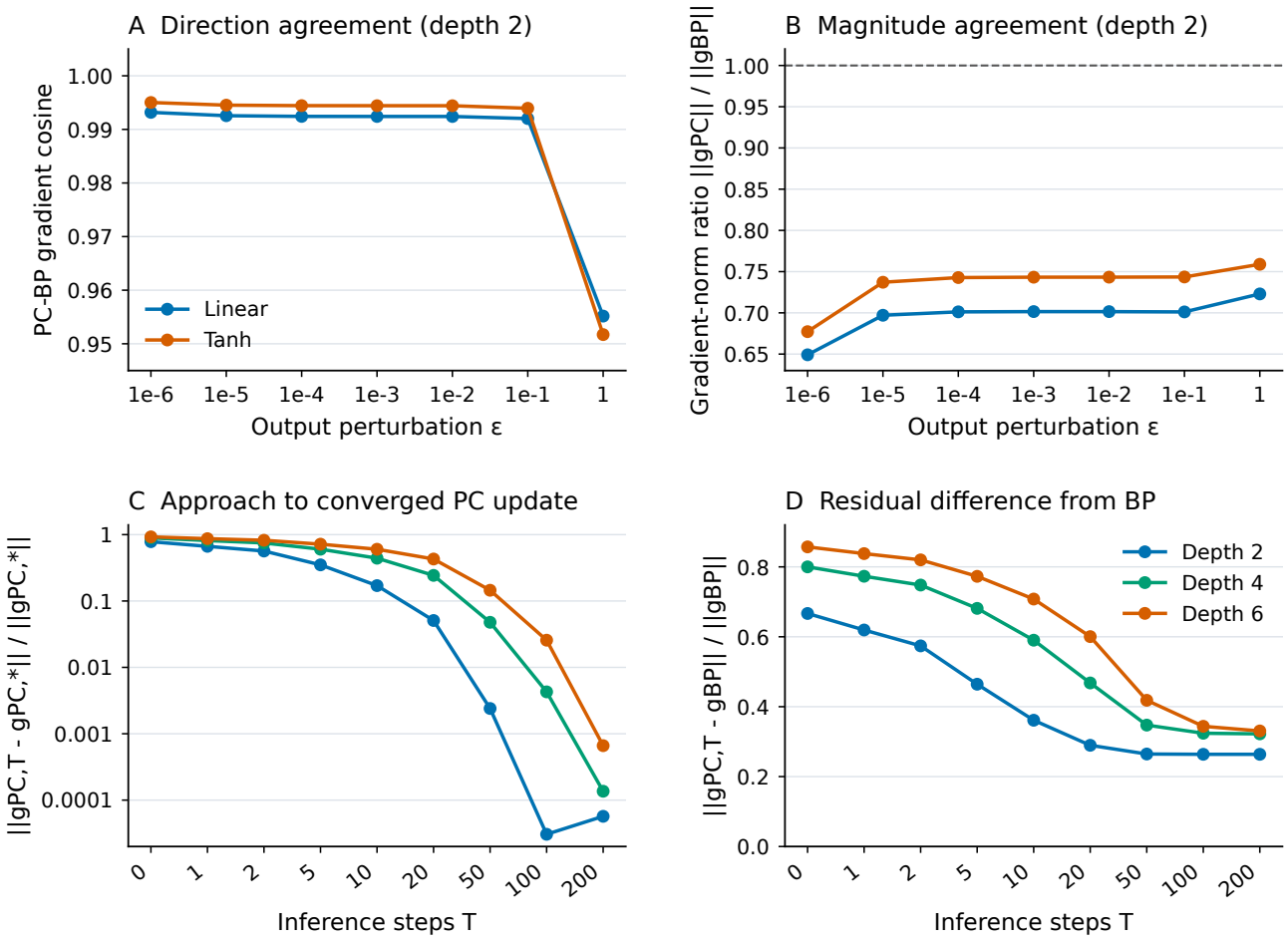


Figure 1: Gradient checks for unit-precision PC. (A–B) Depth-2 gradients remain closely aligned over small output perturbations, but their norm ratio stays below one. Values average ten seeds and three weight layers. (C) Finite-inference gradients generally approach separately converged PC gradients as T increases. The small depth-two reversal after $T = 100$ is consistent with comparison against a finite-tolerance reference and should not be interpreted as a loss of inference convergence. (D) Their difference from BP stops decreasing at a much larger value. Panels C–D use randomly initialized, untrained width-256 networks, one synthetic example per seed, target perturbation $\epsilon = 0.01$, float64 arithmetic, ten seeds, and all weight layers. The separately converged reference stopped at stationarity tolerance 10^{-8}. The two lower panels use different references and answer different questions.

MSE from 0.587 to 0.529 for BP and from 0.587 to 0.544 for PC, confirming that both implementations could learn.

5 Results

In the MNIST experiment with two hidden layers and 100 labelled training examples, PC came close to BP's accuracy but required much more computation. With 200 inference steps, PC finished 30 epochs an average of 0.295 percentage points below BP and took about 131 times as long to train on a single CPU thread.

Each method's learning rate was selected independently using validation data, and both MNIST searches met the prespecified bracketing rule. Fashion-MNIST remained unresolved because BP did not show the required numerically valid decline in validation accuracy at a higher learning rate.

5.1 Validation-only learning-rate selection

Both depth-2 MNIST searches met the rule: validation accuracy was lower at numerically valid rates below and above each near-best group, and at least four of five paired seeds changed in the required direction. Figure 2 shows a performance zoom and, below it, the status of every fixed-grid candidate across the full range. Table 2 reports all four development cells, including Fashion-MNIST. Detailed candidate and failure evidence appears in Appendix C.

The Fashion-MNIST BP cell used all three permitted upper midpoint attempts (0.10954, 0.10466, and 0.10708). Near the numerical

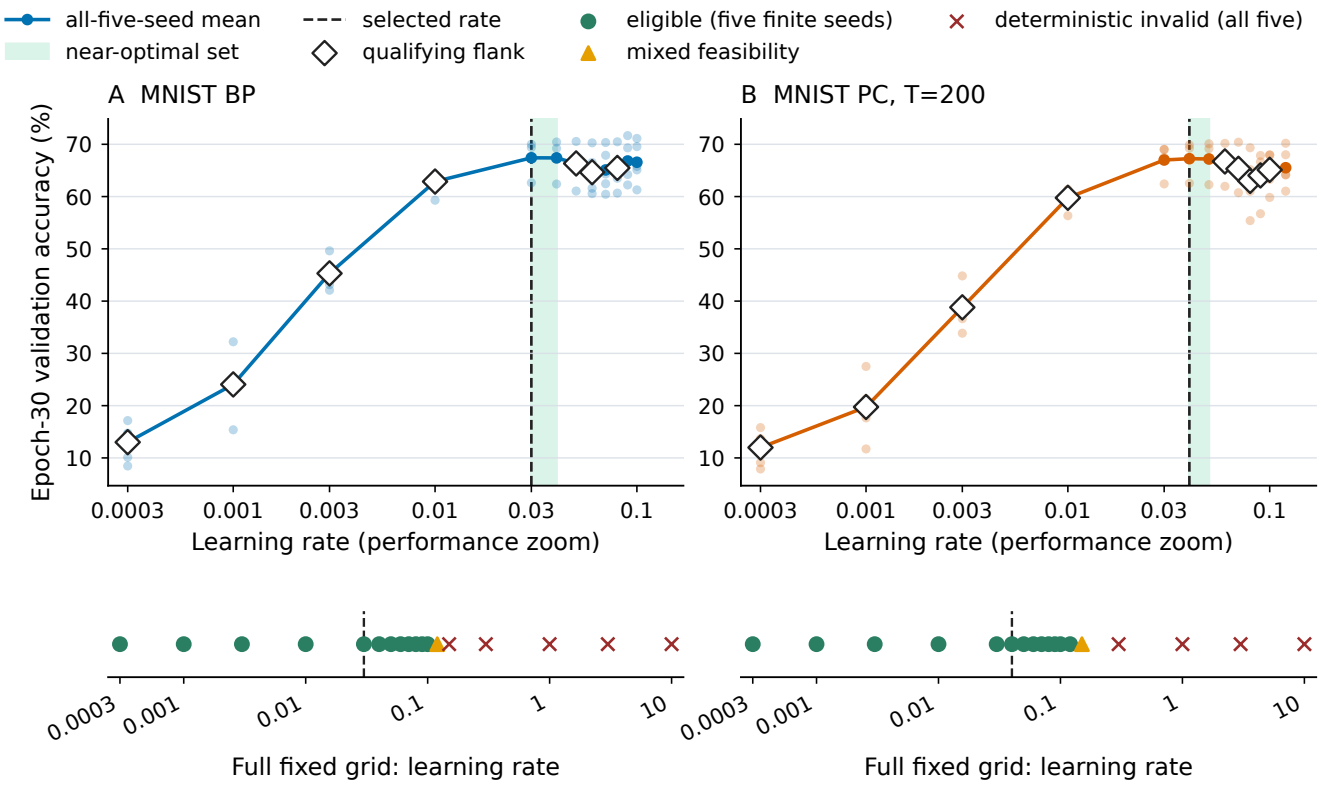


Figure 2: Depth-2 MNIST learning-rate evidence from the prospective extension. Each upper panel has its own labelled logarithmic performance-zoom axis; pale points are the five development seeds, connected points are eligible all-five-seed means, the shaded region is the near-optimal set, the dashed line is the selected rate, and open diamonds mark qualifying finite flanks. Each lower strip uses a separate full-grid logarithmic axis spanning 0.0003–10 and represents every one of the 18 fixed-grid candidates exactly once: green circles are eligible, amber triangles have mixed feasibility, and red crosses are deterministic invalid in all five seeds. Mixed and invalid candidates are not treated as performance turnover. Both method cells have lower and upper qualifying flanks.

Table 2: Validation-only selection in the four learning-rate extension cells. Flanks are qualifying finite rates under the prespecified 0.5-point mean downturn and four-of-five paired direction rule. Fashion-MNIST remains unresolved because its BP cell lacks an upper flank.

Dataset	Method	Selected	Near-optimal set	Qualifying lower flanks	Qualifying upper flanks
MNIST	BP	0.03	0.03, 0.04	0.0003, 0.001, 0.003, 0.01	0.05, 0.06, 0.08
MNIST	PC, $T = 200$	0.04	0.04, 0.05	0.0003, 0.001, 0.003, 0.01	0.06, 0.07, 0.08, 0.09, 0.1
Fashion-MNIST	BP	0.03	0.03, 0.1, 0.10466	0.0003, 0.001, 0.003, 0.01	none; unresolved
Fashion-MNIST	PC, $T = 200$	0.03	0.03	0.0003, 0.001, 0.003, 0.01	0.05, 0.06

boundary, some rates completed successfully in some seeds but failed in others. Because no numerically valid higher-rate candidate met the required downturn, the cell was fixed as unresolved before test access.

5.2 Accuracy after independent learning-rate selection

Figure 3 shows every paired confirmatory seed and the prespecified intervals. Table 3 gives the corresponding numerical summaries. In the fully bracketed depth-2, 100-label MNIST comparison, PC at $T = 200$ minus BP at epoch 30 averaged −0.295 percentage points (median −0.320, sample SD 0.172; 95% paired-bootstrap interval [−0.376, −0.181]). Nine of ten paired seeds were negative. The normalized presentation-AUC difference averaged −0.226 percentage points, with interval [−0.291, −0.149].

Fashion-MNIST's descriptive epoch-30 difference was −0.378 percentage points. The median was −0.390, the SD was 0.162, and the interval was [−0.475, −0.281]. All ten seed differences were

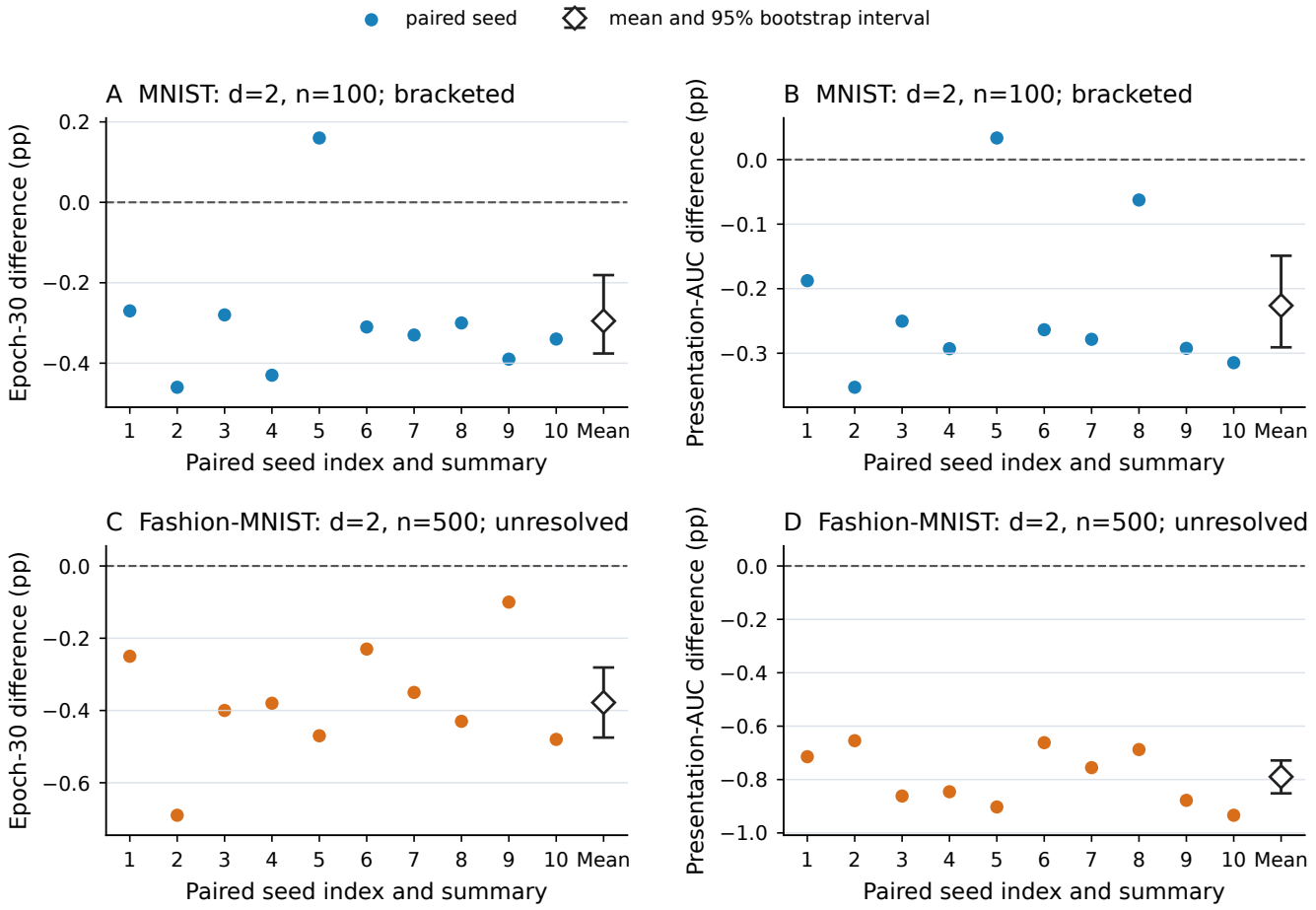


Figure 3: Paired accuracy results after independent learning-rate selection. Each circle is one of ten fresh paired seeds at positions 1–10; the open diamond and bar at the separate "Mean" position are the mean and 95% paired percentile-bootstrap interval from 10,000 seed-level resamples. The dashed horizontal line marks equality. The depth-2, 100-label MNIST method cells meet the prespecified bracketing rule. Fashion-MNIST is retained but labelled unresolved because BP did not show the required numerically valid decline at a higher rate.

Table 3: Paired summaries after independent learning-rate selection. Differences are PC minus BP in percentage points. Time is the geometric mean paired ratio. Fashion-MNIST is descriptive because its search is unresolved.

Quantity	MNIST ($d = 2, n = 100$)	Fashion-MNIST ($d = 2, n = 500$)
Search status	bracketed	unresolved
Epoch-30 mean	−0.295	−0.378
Epoch-30 median	−0.320	−0.390
Epoch-30 SD	0.172	0.162
Epoch-30 95% interval	[−0.376, −0.181]	[−0.475, −0.281]
AUC mean	−0.226	−0.790
AUC 95% interval	[−0.291, −0.149]	[−0.852, −0.729]
PC/BP time ratio	130.569	177.153
Time-ratio 95% interval	[129.155, 131.788]	[175.817, 178.381]

negative. Its AUC difference averaged −0.790 percentage points, with interval [−0.852, −0.729]. These values describe the prespecified retained rates; they do not establish performance at bracketed optima.

5.3 Diagnostic-free time and analytical work

All confirmatory timings were float32, one-thread CPU measurements. PC's geometric-mean diagnostic-free training-time ratio was 130.569 on MNIST (interval [129.155, 131.788]). On unresolved Fashion-MNIST it was 177.153 (interval [175.817, 178.381]). Every seed-level ratio exceeded 125 on MNIST and 172 on Fashion-MNIST. Unlike the original benchmark timing, these windows exclude PC-only energy, stationarity, and other diagnostic calculations.

Table 4 reports the hardware-independent operation counts under the stated convention. PC at $T = 200$ used 112.574 times the BP MACs on both datasets, 112.035 times the FLOPs on MNIST and 112.229 times on Fashion-MNIST, and 403 times the tanh evaluations. In rounded terms, these are approximately 113 times as

many MACs, 112 times as many FLOPs, and 403 times as many tanh evaluations. We did not measure GPU performance and make no claim about its speed ratio. A matched GPU comparison remains future work.

5.4 Original broad benchmark as separate evidence

The original 48-condition benchmark is not replaced by the extension. Figure 4 retains its endpoint results as a separate descriptive experiment. All 48 mean PC-minus-BP effects were negative, 47 intervals were wholly negative, and the median effect was −2.92 percentage points. The grid was unbracketed, and PC accuracy was still increasing with inference budget in 15 of 16 cells, so these summaries do not represent globally tuned comparisons.

For the same depth-2 architecture and 100-label MNIST set size, the original unbracketed mean at rate 0.03 and PC $T = 50$ was −0.786 percentage points. In the separate extension, the mean at independently selected rates and PC $T = 200$ was 0.491 points closer to zero. The searches, inference budgets, and seed sets differ, so this is neither a formal cross-study test nor evidence that any one design change caused the difference.

The original instrumented-time ratios of 7.96–67.3 are retained only as historical context because PC-only diagnostics were inside the PC timing window. The learning-rate extension's diagnostic-free ratios are the primary timing evidence for the extended-search comparison.

6 Discussion and limitations

The learning-rate extension strengthens the depth-2, 100-label MNIST comparison without establishing a global optimum. Both method searches found numerically valid declines on either side of the near-best rates, and ten untouched confirmatory seeds showed a mean PC disadvantage of 0.295 percentage points at epoch 30 with PC at $T = 200$. For the same architecture and labelled-set size, the original unbracketed mean was −0.786 points; the gap measured in the extension was 0.491 points smaller. The learning-rate searches, inference budgets, and seed sets differed, so this descriptive comparison does not identify which change produced the difference. Neither the epoch-30 result nor the presentation-AUC result reached parity.

Diagnostic-free timing shows a large computational difference alongside the small MNIST accuracy difference. On the same float32, single-thread CPU setup, PC at $T = 200$ took about 131 times BP's training time on MNIST and 177 times on Fashion-MNIST. Warm-up, evaluation, diagnostics, serialization, and logging were outside both timed totals. Analytical counts point in the same direction: approximately 113 times as many MACs, 112 times as many FLOPs, and 403 times as many tanh evaluations. These values characterize this implementation and architecture. They do not predict GPU, analogue, or neuromorphic performance, where operation fusion, parallelism, memory movement, or physical dynamics may differ.

The controlled gradient checks help interpret, but do not explain, the accuracy result. PC updates computed after inference reached the stated stopping tolerance pointed in almost the same direction as BP updates while remaining smaller, with PC/BP norm ratios of 0.673–0.740 at width 256. The checks do not establish convergence throughout training or identify the cause of the remaining gap. The small depth-two reversal beyond $T = 100$ is consistent with comparison against a finite-tolerance reference; it should not be interpreted as a loss of inference convergence.

Fashion-MNIST must be interpreted differently. Its PC search met the bracketing rule, but near the upper numerical boundary the BP runs sometimes succeeded and sometimes failed, and no numerically valid higher rate showed the required decline. The selected BP rate completed successfully in all five development seeds, so the confirmatory comparison describes the retained rates; it is not evidence from jointly bracketed optima. Keeping this unresolved result visible avoids mistaking numerical instability for a decline in validation performance.

The broader empirical scope is limited: two small grayscale datasets, width-256 tanh networks, depth two in the extension, 30 epochs, MSE and plain SGD, no convolution, normalization layers, residual connections, or pretraining, and one synchronous unit-precision PC formulation. The original benchmark covers 48 depth, labelled-set-size, and inference-budget comparisons, but its learning-rate search was truncated and its timing included PC-only diagnostics; its evidence remains separate from the extension. Ten paired confirmatory seeds quantify variation in these cells but do not support universal claims. The bootstrap intervals are conditional on the selected rates, fixed split, and confirmatory design; they do not incorporate uncertainty from repeating the development search and are not presented as significance tests.

Precision weighting and fixed predictions remain important future directions. A comparison with such a variant would help determine whether the observed deficit changes under that intervention; the present study does not establish that specificity. Likewise, GPU measurements would be valuable, but were not made here. Future comparisons should state the variant and matching rules clearly and use matched implementations designed for the target hardware.

At the independently selected rates and $T = 200$, the tested depth-2, 100-label MNIST condition therefore retained a small accuracy disadvantage and substantially greater computational cost. The study does not establish a global optimum over inference budgets or exclude incomplete inference as a contributor during training. Other predictive-coding algorithms and hardware remain open empirical questions.

7 Conclusion

In a two-hidden-layer MNIST network trained on 100 labelled examples, synchronous unit-precision PC with 200 inference steps was 0.295 percentage points less accurate than BP after 30 epochs, despite both learning-rate searches meeting the prespecified bracketing rule. Training took about 131 times as long on a single CPU thread. These results show a small accuracy disadvantage and a large computational cost for the tested configuration. Fashion-MNIST remained unresolved. The validation checks do not establish convergence throughout training or explain the accuracy gap, and the findings do not rank other PC variants or hardware.

References

[1] Francis Crick. 1989. The recent excitement about neural networks. *Nature* 337, 6203 (1989), 129–132. https://doi.org/10.1038/337129a0

Table 4: Analytical operation counts for one 30-epoch confirmatory run. One MAC equals two FLOPs; other explicit scalar operations contribute one FLOP, and tanh evaluations are counted separately.

Dataset	Method	30-epoch MACs	30-epoch FLOPs	30-epoch tanh evaluations
MNIST, $n = 100$	BP	1,817,088,000	3,674,323,380	1,536,000
MNIST, $n = 100$	PC, $T = 200$	204,556,800,000	411,654,069,960	619,008,000
Fashion-MNIST, $n = 500$	BP	9,085,440,000	18,339,329,520	7,680,000
Fashion-MNIST, $n = 500$	PC, $T = 200$	1,022,784,000,000	2,058,205,743,840	3,095,040,000

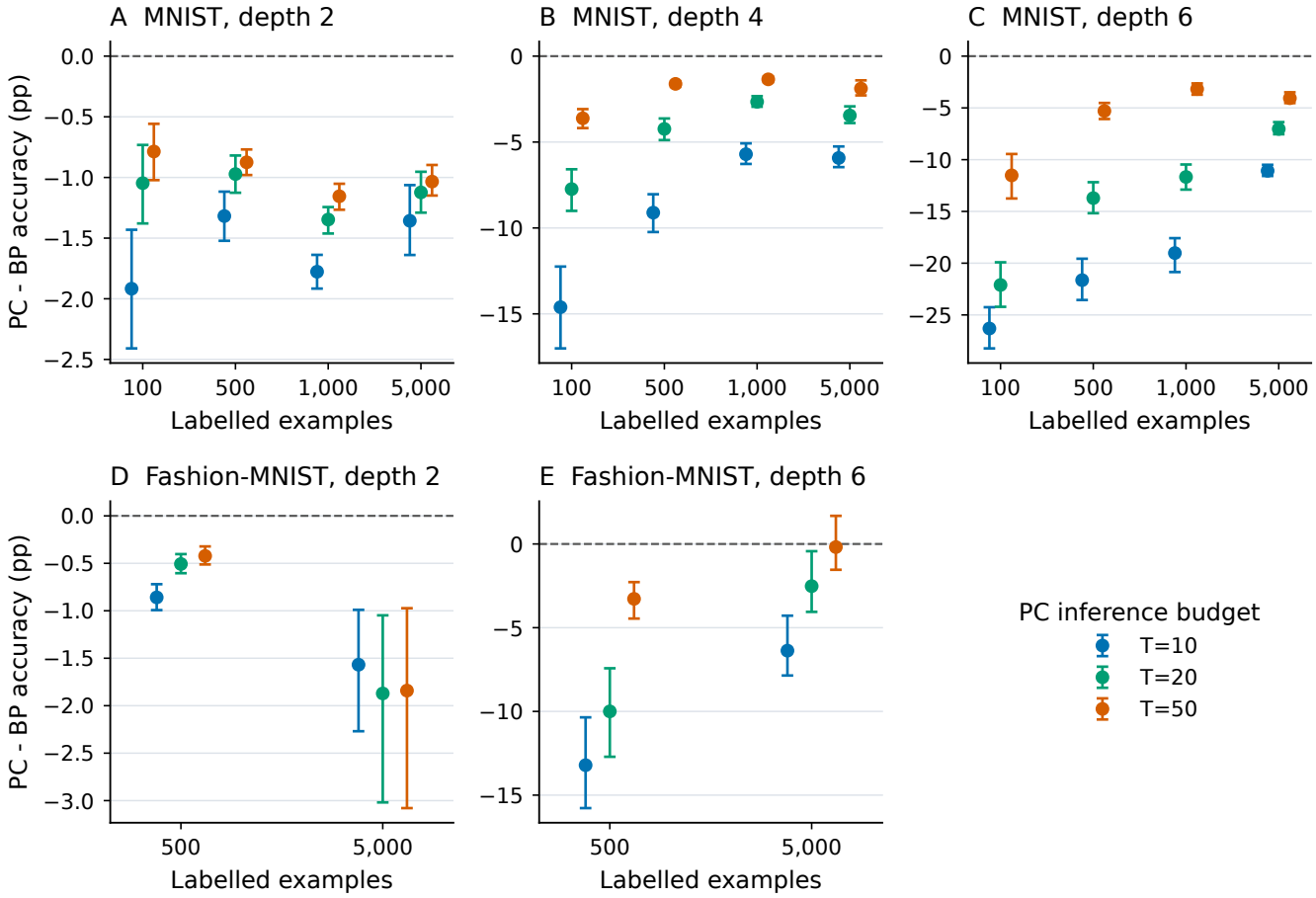


Figure 4: Original broad benchmark, reported separately from the learning-rate extension. Each point is a mean PC-minus-BP epoch-30 difference across ten paired seeds; bars are 95% paired percentile-bootstrap intervals. The original six-rate learning-rate grid ended at 0.03 before validation turnover, so this figure is descriptive evidence from an unbracketed search. No original seed is pooled with the extension.

[2] Simon Frieder and Thomas Lukasiewicz. 2022. (Non-)Convergence Results for Predictive Coding Networks. In *Proceedings of the 39th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 162)*. PMLR, 6793–6810. https://proceedings.mlr.press/v162/frieder22a.html

[3] Cédric Goemaere, Gaspard Oliviers, Rafal Bogacz, and Thomas Demeester. 2026. ePC: Fast and Deep Predictive Coding in Digital Simulation. https://doi.org/10.48550/arXiv.2505.20137 arXiv:2505.20137 [cs.LG] Version 5, revised 8 June 2026; accepted to the ICML 2026 main track.

[4] Francesco Innocenti, El Mehdi Achour, and Christopher L. Buckley. 2025. μPC: Scaling Predictive Coding to 100+ Layer Networks. In *Advances in Neural Information Processing Systems 38*. https://doi.org/10.52202/085713-2923

[5] Diederik P. Kingma and Jimmy Ba. 2015. Adam: A Method for Stochastic Optimization. In *3rd International Conference on Learning Representations*. San Diego, California, USA. https://arxiv.org/abs/1412.6980

[6] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. 1998. Gradient-based learning applied to document recognition. *Proc. IEEE* 86, 11 (1998), 2278–2324. https://doi.org/10.1109/5.726791

[7] Beren Millidge, Alexander Tschantz, and Christopher L. Buckley. 2022. Predictive Coding Approximates Backprop Along Arbitrary Computation Graphs. *Neural Computation* 34, 6 (2022), 1329–1368. https://doi.org/10.1162/neco_a_01497

[8] Gaspard Oliviers, Elene Lominadze, and Rafal Bogacz. 2026. Understanding Sample Efficiency in Predictive Coding. https://doi.org/10.48550/arXiv.2605.

11911 arXiv:2605.11911 [cs.LG]

[9] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Kopf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. 2019. PyTorch: An Imperative Style, High-Performance Deep Learning Library. In *Advances in Neural Information Processing Systems 32*, Hanna Wallach, Hugo Larochelle, Alina Beygelzimer, Florence d'Alché Buc, Emily Fox, and Roman Garnett (Eds.). Curran Associates, Inc., 8024–8035. https://proceedings.neurips.cc/paper/2019/hash/bdbca288fee7f92f2bfa9f7012727740-Abstract.html

[10] Luca Pinchetti, Chang Qi, Oleh Lokshyn, Cornelius Emde, Amine M'Charrak, Mufeng Tang, Simon Frieder, Bayar Menzat, Gaspard Oliviers, Rafal Bogacz, Thomas Lukasiewicz, and Tommaso Salvatori. 2025. Benchmarking Predictive Coding Networks – Made Simple. In *International Conference on Learning Representations*. https://proceedings.iclr.cc/paper_files/paper/2025/hash/581df42e8ebbeeac39aeda03519b7c0e-Abstract-Conference.html

[11] Rajesh P. N. Rao and Dana H. Ballard. 1999. Predictive coding in the visual cortex: a functional interpretation of some extra-classical receptive-field effects. *Nature Neuroscience* 2, 1 (1999), 79–87. https://doi.org/10.1038/4580

[12] Robert Rosenbaum. 2022. On the relationship between predictive coding and backpropagation. *PLOS ONE* 17, 3 (2022), e0266102. https://doi.org/10.1371/journal.pone.0266102

[13] Robert Rosenbaum. 2025. Correction: On the relationship between predictive coding and backpropagation. *PLOS ONE* 20, 3 (2025), e0320944. https://doi.org/10.1371/journal.pone.0320944

[14] Tommaso Salvatori, Yuhang Song, Zhenghua Xu, Thomas Lukasiewicz, and Rafal Bogacz. 2022. Reverse Differentiation via Predictive Coding. *Proceedings of the AAAI Conference on Artificial Intelligence* 36, 7 (2022), 8150–8158. https://doi.org/10.1609/aaai.v36i7.20788

[15] Tommaso Salvatori, Yuhang Song, Yordan Yordanov, Beren Millidge, Lei Sha, Cornelius Emde, Zhenghua Xu, Rafal Bogacz, and Thomas Lukasiewicz. 2024. A Stable, Fast, and Fully Automatic Learning Algorithm for Predictive Coding Networks. In *International Conference on Learning Representations*. https://proceedings.iclr.cc/paper_files/paper/2024/hash/554414e570a85eb3118e988c5d77986f-Abstract-Conference.html

[16] Alexander Tschantz, Beren Millidge, Anil K. Seth, and Christopher L. Buckley. 2023. Hybrid predictive coding: Inferring, fast and slow. *PLOS Computational Biology* 19, 8 (2023), e1011280. https://doi.org/10.1371/journal.pcbi.1011280

[17] James C. R. Whittington and Rafal Bogacz. 2017. An Approximation of the Error Backpropagation Algorithm in a Predictive Coding Network with Local Hebbian Synaptic Plasticity. *Neural Computation* 29, 5 (2017), 1229–1262. https://doi.org/10.1162/NECO_a_00949

[18] Han Xiao, Kashif Rasul, and Roland Vollgraf. 2017. Fashion-MNIST: a Novel Image Dataset for Benchmarking Machine Learning Algorithms. https://doi.org/10.48550/arXiv.1708.07747 arXiv:1708.07747 [cs.LG]

[19] Umais Zahid, Qinghai Guo, and Zafeirios Fountas. 2023. Predictive Coding as a Neuromorphic Alternative to Backpropagation: A Critical Evaluation. *Neural Computation* 35, 12 (2023), 1881–1909. https://doi.org/10.1162/neco_a_01620

A Algorithmic derivatives and numerical conventions

For hidden layer l, differentiating the summed energy in Equation (2) gives

$$\frac{\partial E_{\text{sum}}}{\partial v_{b,l}} = \epsilon_{b,l} - (\epsilon_{b,l+1} W_l) \odot \phi_l'(v_{b,l}). \tag{5}$$

All hidden layers use Equation (5) from the same pre-update beliefs at each inference step. Input and target output remain clamped. Hidden predictions are $\tanh(v_l)$; input activity is the identity and output activity is linear.

With inferred beliefs detached, the bias derivative accompanying Equation (4) is

$$\frac{\partial E_{\text{mean}}}{\partial b_l} = -\frac{1}{B} \sum_{b=1}^{B} \epsilon_{b,l+1}. \tag{6}$$

The stored weight orientation is output-by-input, so prediction uses $\phi_l(v_l) W_l^\top$. Activity inference follows the sum-energy derivative, which preserves the per-example trajectory under batch vectorization. Parameter learning follows mean energy, matching BP's mean-per-example MSE scaling. Differentiating through the inference path would define a different learning rule.

Stationarity is the maximum, over hidden layers, of the root-mean-square entries of $\partial E_{\text{sum}}/\partial v_l$. Relative L2 discrepancy between a candidate gradient g and reference g_r is $\|g - g_r\|_2 / \|g_r\|_2$. The gradient-norm ratio is $\|g_{\text{PC}}\|_2 / \|g_{\text{BP}}\|_2$.

Validation used float64 arithmetic and raw gradients before any optimizer step. Training and confirmatory timing used float32. These precision settings serve different purposes: float64 exposes derivative and convergence discrepancies, while float32 matches the training implementation.

B Mathematical and numerical checks

Table 5: Checks completed before the learning comparison. The directional criterion in Check 6 required a minimum layer/activation/perturbation mean cosine of 0.95 and an across-cell mean of 0.98 for $\epsilon \leq 10^{-3}$.

Check	Purpose	Result
1	Feedforward identity	BP and PC activities agreed at every layer after exact parameter matching.
2	Zero-error behaviour	A feedforward target produced negligible energy, gradients, and belief movement.
3	Activity derivatives	Manual $\nabla_v E_{\text{sum}}$ matched automatic differentiation for batches of one and eight.
4	Parameter derivatives	Detached-belief gradients of E_{mean} matched automatic differentiation and centred finite differences.
5	Inference stability	Accepted rates remained finite, decreased energy, and improved stationarity.
6	Small-error direction	The minimum mean cosine was 0.992 and the across-cell mean was 0.994; magnitude discrepancies remained.
7	Inference budget	Finite-budget PC gradients approached separately converged PC gradients as stationarity improved.
8	Independent formulation	A separately written energy model reproduced beliefs, gradients, energy, and stationarity.
9	Learning check	BP–MSE and PC–MSE both reduced MSE on the same 100-example subset.

For a batch of size B, the parameter derivatives of E_{mean} are

$$\frac{\partial E_{\text{mean}}}{\partial W_l} = -\frac{1}{B} \sum_{b=1}^{B} \epsilon_{b,l+1}^\top \phi_l(v_{b,l}), \tag{7}$$

$$\frac{\partial E_{\text{mean}}}{\partial b_l} = -\frac{1}{B} \sum_{b=1}^{B} \epsilon_{b,l+1}. \tag{8}$$

The beliefs on the right-hand side are detached. Differentiating through the inference trajectory would define a different learning rule. Conversely, using the gradient of mean energy for activity inference without compensating for B would shrink each belief update by $1/B$ and make the dynamics depend on vectorized batch size.

The converged comparisons used an absolute stationarity criterion of 10^{-8}. Since stationarity is defined from E_{sum}, its numerical scale depends on the batch convention; the validation kept that convention explicit and separately verified batch-invariant trajectories. Small output perturbations also produce small gradient norms, so absolute discrepancies can vanish while relative discrepancies remain appreciable.

All BP comparisons used the MSE objective, raw pre-optimizer gradients, matched parameter orientation and initialization, detached inferred beliefs, synchronous activity updates, and float64 validation arithmetic. These checks concern unit-precision PC without an output-precision hierarchy or fixed-prediction condition.

C Extended-search methods and results

C.1 Development evidence and retained failures

All four cells completed the same 18-rate fixed grid before any adaptive action. The eligible and ineligible sets are shown in Table 6. A candidate was eligible only if all five development seeds completed. Across the 375 candidate-seed identities, 102 deterministic-invalid trajectories were retained: 58 ‘nonfinite_parameter’ and 44 ‘observed_energy_increase’. Finite seeds from mixed candidates were never averaged for selection.

Table 6: Rate-level development status. Rates not listed as ineligible were eligible in all five seeds. Fashion-MNIST BP includes its three adaptive midpoint candidates.

Cell	Eligible rates	Ineligible or mixed rates
MNIST BP	0.0003–0.1 on the fixed grid	0.12, 0.15, 0.3, 1, 3, 10
MNIST PC, $T = 200$	0.0003–0.12 on the fixed grid	0.15, 0.3, 1, 3, 10
Fashion-MNIST BP	0.0003–0.1; 0.1046635; 0.1070762	0.1095445; 0.12, 0.15, 0.3, 1, 3, 10
Fashion-MNIST PC, $T = 200$	0.0003–0.12 on the fixed grid	0.15, 0.3, 1, 3, 10

The notation “0.0003–0.1 on the fixed grid” refers to discrete tested values, not a continuous interval. Those values were 0.0003, 0.001, 0.003, 0.01, 0.03, 0.04, 0.05, 0.06, 0.07, 0.08, 0.09, and 0.1. MNIST PC additionally retained 0.12 as eligible. Fashion-MNIST BP remained unresolved because no eligible upper rate met the finite downturn rule after all three midpoint attempts; mixed feasibility and numerical failure were not counted as turnover.

The earlier v1.1 and v1.2 construction attempts were separate protocol-development exercises with development seeds 100–102 and 200–204. They did not share the extension’s seed namespace, did not enter its selection means, and were not pooled with confirmatory outcomes. They document method development rather than evidence for the reported effect estimates.

C.2 Seed-level confirmatory summaries

Table 7 reports the paired values plotted in Figure 3. Training seconds sum only the 30 diagnostic-free timed epochs; disposable warm-ups are excluded.

Table 7: Seed-level extended-search results. Δ_{30} and Δ_{AUC} are PC minus BP in percentage points. Fashion-MNIST remains unresolved.

Dataset/seed	Δ_{30}	Δ_{AUC}	BP seconds	PC seconds	PC/BP
MNIST 41001	−0.27	−0.188	0.023215	3.006470	129.505
MNIST 41002	−0.46	−0.353	0.022788	3.025790	132.778
MNIST 41003	−0.28	−0.250	0.023931	3.000345	125.377
MNIST 41004	−0.43	−0.293	0.022689	3.022946	133.235
MNIST 41005	+0.16	+0.033	0.022970	2.990691	130.200
MNIST 41006	−0.31	−0.264	0.023200	3.020499	130.196
MNIST 41007	−0.33	−0.278	0.022685	2.995503	132.050
MNIST 41008	−0.30	−0.063	0.022982	2.990836	130.138
MNIST 41009	−0.39	−0.292	0.023154	3.011018	130.044
MNIST 41010	−0.34	−0.315	0.022702	3.004529	132.345
Fashion-MNIST 41001	−0.25	−0.714	0.076434	13.367747	174.893
Fashion-MNIST 41002	−0.69	−0.654	0.075792	13.356050	176.219
Fashion-MNIST 41003	−0.40	−0.862	0.075232	13.375387	177.787
Fashion-MNIST 41004	−0.38	−0.846	0.075068	13.378833	178.222
Fashion-MNIST 41005	−0.47	−0.903	0.074678	13.407663	179.539
Fashion-MNIST 41006	−0.23	−0.662	0.077229	13.351318	172.879
Fashion-MNIST 41007	−0.35	−0.755	0.075904	13.395730	176.482
Fashion-MNIST 41008	−0.43	−0.688	0.075641	13.408735	177.267
Fashion-MNIST 41009	−0.10	−0.878	0.074162	13.364089	180.202
Fashion-MNIST 41010	−0.48	−0.934	0.075281	13.412106	178.160

C.3 Per-epoch operation counts

For the fixed architecture, $S = 268{,}800$ is the number of weight MACs per forward example, $R = 68{,}096$ the hidden-error MACs per example, $A = 522$ the affine output units, $H = 512$ the hidden units, and $P = 269{,}322$ the trainable parameters. For batch size B, BP uses $B(2S + R)$ MACs and PC uses $B[3S + T(S + R)]$ MACs. Table 8 gives the exact per-epoch counts underlying Table 4.

Table 8: Exact analytical counts per epoch. One MAC is two FLOPs; other explicit scalar operations count as one FLOP, and tanh evaluations are separate.

Dataset	Method	MACs	FLOPs	tanh evaluations
MNIST	BP	60,569,600	122,477,446	51,200
MNIST	PC, $T = 200$	6,818,560,000	13,721,802,332	20,633,600
Fashion-MNIST	BP	302,848,000	611,310,984	256,000
Fashion-MNIST	PC, $T = 200$	34,092,800,000	68,606,858,128	103,168,000

Benchmarking BRCA2 Missense Variant Pathogenicity Predictors Across Functional Domains and in a Subset of Variants Observed in Ashkenazi Jewish Individuals

Meirav Bocknek
Brooklyn Prospect Charter School
Brooklyn, NY, USA
meiravbocknek@gmail.com

Mariame Diabate, PhD
Stanford University
Stanford, CA, USA
diabatem@stanford.edu

Abstract

Pathogenic variants in *BRCA2* substantially increase hereditary cancer risk, yet many identified missense variants remain classified as variants of uncertain significance (VUS). This study evaluated SIFT, PolyPhen-2, and REVEL for BRCA2 missense single-nucleotide variants using ClinVar classifications as the reference standard, with gene-wide, domain-specific, and Ashkenazi Jewish (AJ)-observed subset analyses. After missense-only and review-status filtering, 9,077 variants remained; 651 had definitive ClinVar classifications (96 Pathogenic/Likely pathogenic and 555 Benign/Likely benign). All three predictors returned scores for these 651 variants. Because the predictors can return intermediate calls, primary sensitivity and specificity were calculated among confident binary predictions and intermediate-call rates were reported separately. Gene-wide, REVEL had the highest accuracy (92.8%) and specificity (96.0%), whereas SIFT had the highest sensitivity (98.9%) but low specificity (48.7%); PolyPhen-2 showed 81.8% accuracy, 88.2% sensitivity, and 80.5% specificity. In the OB Folds, REVEL showed the best balanced binary-call performance (90.1% accuracy, 90.9% sensitivity, 89.7% specificity), although 93 of 194 variants received intermediate REVEL calls. The BRC Repeats contained no definitively pathogenic missense variants, so sensitivity was not estimable. Using an exact genomic merge and AF_ASJ > 0, 35 missense variants were observed in the AJ subset (27 Benign and 8 VUS/conflicting; no definitively Pathogenic variants). Consequently, AJ sensitivity was not estimable; specificity among confident benign calls was 100% for REVEL (95% CI 85.7%–100%), 100% for PolyPhen-2 (81.6%–100%), and 42.3% for SIFT (25.5%–61.1%). These results support context-aware use of in-silico predictions as supporting evidence, while emphasizing predictor abstention, domain-level uncertainty, and the limits of population-subset inference.

Keywords

BRCA2, in-silico predictors, variant classification, ClinVar, SIFT, PolyPhen-2, REVEL, Ashkenazi Jewish population, variants of uncertain significance

ACM Reference Format:
Meirav Bocknek and Mariame Diabate, PhD. 2026. Benchmarking BRCA2 Missense Variant Pathogenicity Predictors Across Functional Domains and in a Subset of Variants Observed in Ashkenazi Jewish Individuals. In *Proceedings of International Journal of Secondary Computing and Applications Research (IJSCAR VOL. 3, ISSUE 4)*. ACM, New York, NY, USA, 7 pages. https://doi.org/10.67149/yhjs2024.5/jbuwpv6l

This paper is published under the Creative Commons Attribution-NonCommercial-NoDerivs 4.0 International (CC-BY-NC-ND 4.0) license. Authors reserve their rights to disseminate the work on their personal and corporate Web sites with the appropriate attribution.
IJSCAR VOL. 3, ISSUE 4
© 2026 The Author(s). Published by IJSCAR.

1 Introduction

BRCA2 is a tumor-suppressor gene that encodes a large protein forming connections with other repair factors and contributing to the repair of double-strand DNA breaks. Pathogenic variants in this gene substantially increase the risk of hereditary breast, ovarian, pancreatic, and prostate cancers; for example, female BRCA2 carriers have an estimated lifetime breast cancer risk of 69% by age 80 [7]. Genetic testing is becoming more widespread because of the increasing availability of tests [4], and laboratories are identifying a large number of BRCA2 variants, many of which are classified in ClinVar as variants of uncertain significance (VUS). This leaves many patients without clear guidance for treatment or protection planning. To help bridge this gap, *in-silico* predictors use computational approaches to predict whether an amino acid substitution is likely to disrupt protein function. While these predictions do not constitute standalone evidence, they contribute useful supporting information in clinical guidelines [9]. Accurate interpretation of BRCA2 missense variants remains important because many detected variants cannot be classified confidently using currently available clinical evidence. Computational predictions can provide supporting evidence during variant interpretation, but their usefulness depends on how reliably they distinguish benign from pathogenic variation within the specific gene and functional context being evaluated.

Although this study examines variants observed in individuals of Ashkenazi Jewish ancestry, the predictors themselves are not population-conditioned: SIFT, PolyPhen-2, and REVEL score a given amino-acid substitution the same way regardless of who carries it. The AJ analysis therefore evaluates performance on the subset of BRCA2 missense variants with a positive recorded AJ allele frequency, rather than ancestry-specific predictor behavior.

Prior benchmarking studies have established the general performance of in-silico predictors across large variant databases, but BRCA2 performance can vary with local sequence constraint and functional context [6]. This study therefore asks two related but distinct questions: whether gene-wide predictor performance is preserved within defined BRCA2 functional domains, and how the same predictors behave on missense variants documented in AJ individuals. The latter analysis is a variant-subset analysis, not a test of population-conditioned algorithms. These distinctions are clinically relevant because computational predictions are used only

as supporting evidence within broader variant-classification frameworks [9].

We hypothesized that gene-wide performance would not necessarily be preserved within all BRCA2 functional domains because the consequences of missense variation differ across regions with distinct structural and evolutionary constraints. We also evaluated whether gene-wide specificity was retained among variants observed in AJ individuals, while recognizing that population membership does not alter the predictor score itself. The primary goal was to compare SIFT, PolyPhen-2, and REVEL on definitively classified BRCA2 missense variants and to identify contexts in which predictor performance or predictor abstention limits interpretation.

2 Method

BRCA2 variant data were obtained from ClinVar and downloaded on October 14, 2025. The source file contained 10,019 BRCA2 records. The analytical cohort was restricted to germline single-nucleotide variants whose molecular consequence included missense variation and whose ClinVar germline review status corresponded to at least one review star. Nonsense and stop-gain variants were excluded before predictor benchmarking because the three evaluated tools are missense predictors. This produced 9,077 missense SNVs. ClinVar classifications were grouped as Pathogenic (Pathogenic and Likely pathogenic), Benign (Benign and Likely benign), or VUS (Uncertain significance and Conflicting classifications of pathogenicity). The final missense cohort contained 96 Pathogenic, 555 Benign, and 8,426 VUS/conflicting variants; the 651 Pathogenic/Benign variants formed the reference set for performance evaluation.

Canonical BRCA2 amino-acid positions were parsed from the NM_000059.4 transcript annotation in the ClinVar Name field. This avoided ambiguity in the comma-separated Protein change field, which can contain multiple transcript-specific protein consequences. All reported domain assignments and amino-acid identifiers therefore use the canonical BRCA2 transcript.

A binary truth label was assigned for definitive ClinVar classifications (Benign = 0; Pathogenic = 1). REVEL and SIFT scores were retrieved through the MyVariant.info API using GRCh38 genomic HGVS notation and dbNSFP annotations. PolyPhen-2 HumVar scores were retrieved separately through the Ensembl Variant Effect Predictor (VEP) REST API. For PolyPhen-2, the BRCA2 Ensembl transcript ENST00000380152 was explicitly selected; all 9,077 missense variants received PolyPhen-2 scores for this transcript. REVEL and SIFT scores were also available for all 9,077 variants in the corrected analysis.

Each predictor score was classified into three tiers (Benign, intermediate/VUS, Pathogenic) using prespecified literature-supported thresholds. PolyPhen-2 HumVar scores ≤ 0.30 were classified as Benign, ≥ 0.70 as Pathogenic, and intermediate values as VUS [1]. SIFT scores ≤ 0.05 were classified as Pathogenic, > 0.10 as Benign, and intermediate values as VUS [8]; lower SIFT scores indicate greater predicted damage. REVEL scores < 0.40 were classified as Benign, ≥ 0.75 as Pathogenic, and intermediate values as VUS [5].

2.1 Ashkenazi Jewish Population Filtering

AJ-observed variants were identified by merging ClinVar and BRCA Exchange [3] using the exact GRCh38 genomic SNV (chromosome, position, reference allele, and alternate allele), rather than protein-change text. A variant was included in the AJ-observed subset only when the recorded Ashkenazi Jewish allele frequency was strictly greater than zero (AF_ASJ > 0). This criterion identified 35 missense variants: 27 Benign and 8 VUS/conflicting by ClinVar, with no definitively Pathogenic missense variants. Because there were no Pathogenic reference variants in this subset, sensitivity could not be estimated.

Because predictor scores are variant-specific rather than population conditioned, the AJ analysis should be interpreted as performance on variants observed in AJ individuals, not as evidence that predictor behavior changes by ancestry.

2.2 Domain-Specific Filtering

BRCA2 functional domains were defined from established structural and functional annotations [2] using the following amino-acid boundaries: PALB2-binding region (10–40), BRC Repeats (1002–2113), Helical Domain (2481–2667), OB Folds (2670–3190), Tower Domain (3196–3232), and C-terminal BRCT-like region (3265–3418). Variants outside these intervals were categorized as non-domain. Domain truth counts were calculated before predictor-specific filtering. Comparative predictor interpretation focused on the BRC Repeats, Helical Domain, and OB Folds because the PALB2-binding, Tower, and C-terminal regions contained very small definitive reference sets.

2.3 Performance Metrics

Predictor performance was evaluated using confusion-matrix counts and standard classification metrics. Accuracy was (TP + TN)/N, sensitivity was TP/(TP + FN), and specificity was TN/(TN + FP). Sensitivity was reported as not estimable when a subset contained no Pathogenic reference variants. Because each tool can return an intermediate/VUS prediction, every predictor was evaluated with three denominators: the number of definitive ClinVar variants with a score (N scored), the number receiving a confident binary Benign/Pathogenic call (N confident), and the number receiving an intermediate call. Primary accuracy, sensitivity, and specificity were calculated among confident binary calls, and Wilson 95% confidence intervals were reported. As a sensitivity analysis addressing predictor abstention, intermediate calls were counted as unresolved/incorrect when calculating all-case accuracy. Pathogenic prevalence was calculated as P/(P + B), whereas prior odds were calculated as P/B; these quantities were not treated as interchangeable.

3 Results

3.1 Gene-wide Predictor Performance

The corrected definitive missense reference set contained 96 Pathogenic and 555 Benign variants (n = 651). The pathogenic prevalence was therefore 96/651 = 14.75%, and the corresponding prior odds were 96/555 = 0.173. These quantities describe the composition of

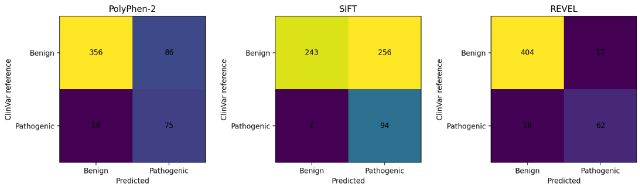


Figure 1: Gene-wide confusion matrices for confident binary predictions. Counts are shown separately for PolyPhen-2 (n = 527), SIFT (n = 594), and REVEL (n = 502). Intermediate predictor calls are not included in these matrices and are reported separately.

the definitive ClinVar missense set; they do not impose a maximum possible predictor odds ratio and should not be assumed to represent the prior probability among VUS.

All three predictors produced scores for all 651 definitive variants, but their intermediate-call rates differed. PolyPhen-2 made 527 confident binary calls and 124 intermediate calls; SIFT made 594 confident calls and 57 intermediate calls; REVEL made 502 confident calls and 149 intermediate calls. Accordingly, the confusion matrices and primary binary-call metrics use predictor-specific confident denominators rather than a common n.

Among confident binary calls, REVEL achieved the highest gene-wide accuracy at 92.8% (95% CI 90.2%–94.8%) and specificity at 96.0% (93.6%–97.5%), with sensitivity of 76.5% (66.2%–84.4%). SIFT achieved the highest sensitivity at 98.9% (94.3%–99.8%) but had substantially lower specificity at 48.7% (44.3%–53.1%) and accuracy at 56.7% (52.7%–60.7%). PolyPhen-2 showed intermediate performance, with 81.8% accuracy (78.3%–84.8%), 88.2% sensitivity (79.7%–93.5%), and 80.5% specificity (76.6%–84.0%).

The corrected results preserve the main sensitivity–specificity tradeoff, but the frequency of intermediate/VUS predictions differs across predictors and should be considered when interpreting their performance. SIFT made the fewest intermediate calls (57/651) and identified 94 of 96 Pathogenic variants as Pathogenic, whereas REVEL made 149 intermediate calls and classified 62 of 96 Pathogenic variants as Pathogenic. When intermediate calls were counted as unresolved/incorrect, all-case accuracy was 66.2% for PolyPhen-2, 51.8% for SIFT, and 71.6% for REVEL. Thus, REVEL's high binary-call accuracy and specificity should be interpreted together with its lower sensitivity and higher intermediate-call rate.

3.2 Domain-Specific Predictor Performance

To evaluate domain-specific performance, the 651 definitive missense variants were mapped to annotated BRCA2 functional regions before predictor filtering. Of these, 462 variants fell within the six predefined domains and 189 fell outside the annotated intervals. The corrected domain composition is shown in Table 1. Pathogenic missense prevalence was highly uneven across regions, including no definitively Pathogenic missense variants in the BRC Repeats, Tower Domain, or C-terminal region.

The BRC Repeats contained 159 definitive missense variants, all Benign. The Helical Domain contained 33 Pathogenic and 38 Benign variants (n = 71), and the OB Folds contained 45 Pathogenic

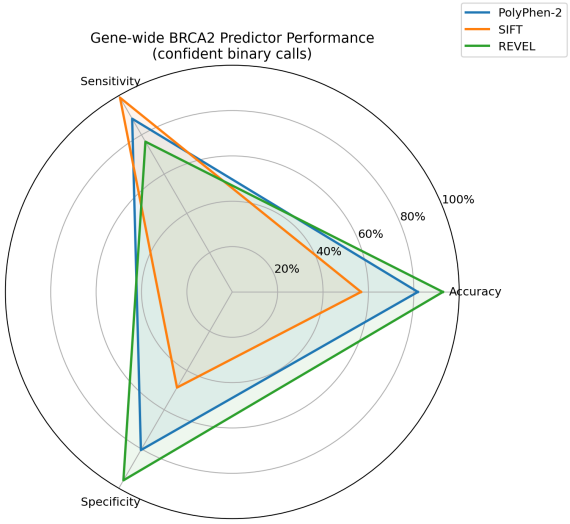


Figure 2: Radar plot of gene-wide accuracy, sensitivity, and specificity among confident binary calls for PolyPhen-2, SIFT, and REVEL.

and 149 Benign variants (n = 194). These pre-predictor counts are distinct from the smaller predictor-specific confident denominators used for performance metrics.

Comparative predictor interpretation focused on the BRC Repeats, Helical Domain, and OB Folds. The PALB2-binding region (n = 6), Tower Domain (n = 8), and C-terminal region (n = 24) were retained in the descriptive domain table but were considered too small for stable comparative performance claims.

In the BRC Repeats, all 159 definitive missense variants were Benign; therefore sensitivity was not estimable for any predictor. Among confident calls, specificity was 96.6% for REVEL (95% CI 92.3%–98.5%), 90.4% for PolyPhen-2 (84.3%–94.3%), and 62.1% for SIFT (53.9%–69.8%). The absence of definitively Pathogenic missense variants is a robust descriptive finding, but it does not by itself establish that pathogenic disruption of this region is predominantly truncating; unresolved missense variants may remain in the VUS category.

In the Helical Domain, REVEL had 90.5% accuracy, 100% sensitivity, and 60.0% specificity among 42 confident calls, but 29 of 71 variants received intermediate REVEL calls. PolyPhen-2 had 71.0% accuracy, 100% sensitivity, and 37.9% specificity among 62 confident calls, while SIFT had 61.8% accuracy, 100% sensitivity, and 25.7% specificity among 68 confident calls. In the OB Folds, REVEL showed the best balanced binary-call performance: 90.1% accuracy (82.7%–94.5%), 90.9% sensitivity (76.4%–96.9%), and 89.7% specificity (80.2%–94.9%) among 101 confident calls. PolyPhen-2 had 69.8% accuracy, 95.0% sensitivity, and 60.6% specificity among 149 confident calls; SIFT had 44.0% accuracy, 100% sensitivity, and 25.5% specificity among 182 confident calls. REVEL's OB Fold performance should nevertheless be interpreted alongside its 93 intermediate calls in this domain.

Overall, domain-level results reinforce the need to report both discrimination and abstention. REVEL generally had the highest

Table 1: BRCA2 functional domains, amino acid boundaries, and ClinVar classification counts.

Domain	AA Range	Function	Pathogenic	Benign	Total	Pathogenic Prevalence
PALB2-binding	10–40	Protein interaction, N-terminus	3	3	6	0.5000
BRC Repeats	1002–2113	RAD51 binding, DNA repair	0	159	159	0.0000
Helical Domain	2481–2667	Structural / DNA-binding domain	33	38	71	0.4648
OB Folds	2670–3190	ssDNA binding, DNA-binding domain	45	149	194	0.2320
Tower Domain	3196–3232	DNA binding	0	8	8	0.0000
C-terminal	3265–3418	C-terminus	0	24	24	0.0000

Table 2: Domain-specific predictor performance. N truth is the number of definitive ClinVar variants in the domain; N confident is the number receiving a binary Benign/Pathogenic prediction. The difference between these values is the number of intermediate predictor calls. Metrics are reported with Wilson 95% confidence intervals.

Domain	Predictor	N truth	N conf.	TP	FP	FN	TN	Acc. (95% CI)	Sens. (95% CI)	Spec. (95% CI)
BRC Repeats	PolyPhen-2	159	136	0	13	0	123	90.4% (84.3–94.3)	Not est.	90.4% (84.3–94.3)
BRC Repeats	REVEL	159	147	0	5	0	142	96.6% (92.3–98.5)	Not est.	96.6% (92.3–98.5)
BRC Repeats	SIFT	159	140	0	53	0	87	62.1% (53.9–69.8)	Not est.	62.1% (53.9–69.8)
Helical Domain	PolyPhen-2	71	62	33	18	0	11	71.0% (58.7–80.8)	100% (89.6–100)	37.9% (22.7–56.0)
Helical Domain	REVEL	71	42	32	4	0	6	90.5% (77.9–96.2)	100% (89.3–100)	60.0% (31.3–83.2)
Helical Domain	SIFT	71	68	33	26	0	9	61.8% (49.9–72.4)	100% (89.6–100)	25.7% (14.2–42.1)
OB Folds	PolyPhen-2	194	149	38	43	2	66	69.8% (62.0–76.6)	95.0% (83.5–98.6)	60.6% (51.2–69.2)
OB Folds	REVEL	194	101	30	7	3	61	90.1% (82.7–94.5)	90.9% (76.4–96.9)	89.7% (80.2–94.9)
OB Folds	SIFT	194	182	45	102	0	35	44.0% (36.9–51.2)	100% (92.1–100)	25.5% (19.0–33.4)

specificity and balanced binary-call accuracy in the BRC Repeats, Helical Domain, and OB Folds, whereas SIFT prioritized sensitivity at the cost of specificity. However, REVEL also generated substantially more intermediate calls in the Helical Domain and OB Folds, limiting direct comparison based on binary-call metrics alone.

3.3 Ashkenazi Jewish Population Subset

Using the corrected definition of AJ-observed variation (exact genomic match and AF_ASJ > 0), 35 BRCA2 missense variants were identified. Of these, 27 were definitively Benign and 8 were VUS/conflicting; none were definitively Pathogenic. Consequently, this subset can inform specificity and false-positive behavior among known Benign variants, but it cannot estimate sensitivity for Pathogenic missense variants.

Among confident AJ-observed Benign calls, PolyPhen-2 correctly classified all 17/17 variants as Benign (specificity 100%, 95% CI 81.6%–100%) but returned 10 intermediate calls. REVEL correctly classified all 23/23 confident variants as Benign (specificity 100%, 85.7%–100%) and returned 4 intermediate calls. SIFT classified 11/26 confident variants as Benign and 15 as Pathogenic, yielding specificity of 42.3% (25.5%–61.1%); one additional variant received an intermediate SIFT call. Because the AJ reference set contains only Benign definitive variants, accuracy is numerically identical to specificity and sensitivity is not estimable. The wide confidence intervals emphasize the small sample size.

Statistical Note: Sensitivity and specificity are conditional on true variant status, whereas PPV and NPV depend on the prior probability of pathogenicity. The pathogenic prevalence of the definitive ClinVar missense set (14.75%) should not automatically be used as the prior probability for ClinVar VUS, because definitively

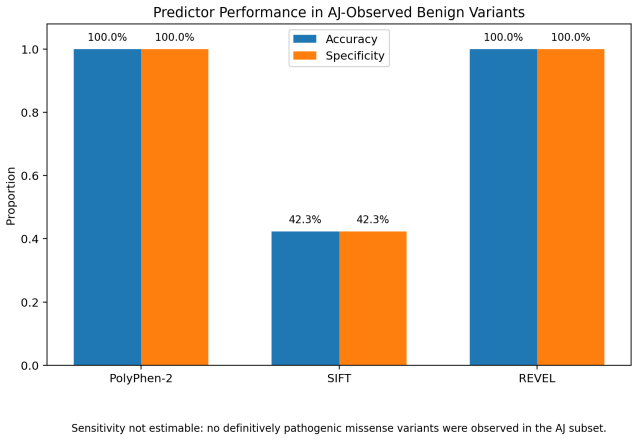


Figure 3: Corrected predictor performance among AJ-observed definitively Benign missense variants. Accuracy and specificity are shown; sensitivity is not estimable because no definitively Pathogenic missense variants were present.

classified variants and VUS are not exchangeable populations. Accordingly, this study does not assign a single posterior probability of pathogenicity or benignity to VUS.

3.4 Application to Variants of Uncertain Significance

A total of 8,426 missense variants in the corrected cohort were classified as VUS/conflicting by ClinVar. The purpose of applying the three predictors to these variants was descriptive: to characterize

Table 3: Predictor performance among the 27 definitively Benign AJ-observed missense variants. Sensitivity is not estimable because the subset contains no definitively Pathogenic variants.

Predictor	N truth	N conf.	Interm.	TN	FP	Specificity (95% CI)
PolyPhen-2	27	17	10	17	0	100% (81.6–100)
SIFT	27	26	1	11	15	42.3% (25.5–61.1)
REVEL	27	23	4	23	0	100% (85.7–100)

how often each tool produced Pathogenic, Benign, or intermediate calls under the prespecified thresholds. These call proportions are not measures of clinical accuracy because the true status of the VUS is unknown.

Note: The proportion called Pathogenic is a predictor call proportion, not a posterior probability of pathogenicity.

REVEL classified 289 of 8,426 VUS/conflicting variants as Pathogenic (3.43%), 6,303 as Benign, and 1,834 as intermediate. SIFT classified 4,336 as Pathogenic (51.46%), 3,158 as Benign, and 932 as intermediate. PolyPhen-2 classified 1,456 as Pathogenic (17.28%), 5,025 as Benign, and 1,945 as intermediate. These distributions are consistent with the gene-wide error profiles: SIFT makes Pathogenic calls much more frequently, while REVEL more often returns Benign or intermediate calls.

A Bayesian sensitivity analysis was performed across assumed pathogenicity priors from 1% to 50% using the observed gene-wide sensitivity and specificity. The resulting PPV and NPV changed substantially with the assumed prior, demonstrating why a single posterior probability should not be assigned to VUS without a justified VUS-specific prior. For example, at an illustrative 10% prior, estimated PPV/NPV were 67.8%/97.4% for REVEL, 33.5%/98.4% for PolyPhen-2, and 17.6%/99.8% for SIFT. These values are scenario estimates, not validated probabilities for individual VUS.

Therefore, predictor outputs should be used as supporting computational evidence rather than as stand-alone reclassification of a VUS. The corrected analysis does not support statements such as "94.8% confidence that an AJ VUS is benign" based on specificity alone.

4 Discussion

This study compared SIFT, PolyPhen-2, and REVEL on a corrected missense-only BRCA2 cohort and examined both domain-level performance and a subset of variants observed in AJ individuals. The revised analysis changes the interpretation in two important ways: predictor-specific intermediate calls must be considered alongside binary-call performance, and the AJ-observed subset contains no definitively Pathogenic missense variants, preventing sensitivity estimation in that subset.

Gene-wide, REVEL provided the strongest balance of accuracy and specificity among confident binary calls (92.8% accuracy; 96.0% specificity), but its sensitivity was 76.5% and it returned the largest number of intermediate calls (149/651). SIFT had the highest sensitivity (98.9%) and the fewest intermediate calls, but specificity was only 48.7%, reflecting frequent false Pathogenic calls among Benign variants. PolyPhen-2 occupied an intermediate position, with 81.8% accuracy, 88.2% sensitivity, and 80.5% specificity. These results argue against describing any single metric as overall predictor reliability without also reporting abstention and the sensitivity-specificity tradeoff.

4.1 Clinical Implications for High-Risk Populations

Ashkenazi Jewish individuals have an elevated burden of BRCA1/2 founder variants [10], but the common founder alleles are predominantly truncating and fall outside the missense-only predictors evaluated here. The present AJ analysis therefore addresses only missense variants with a positive recorded AJ allele frequency and should not be interpreted as a comprehensive assessment of BRCA2 risk prediction in this population.

The corrected AJ-observed subset contained 27 definitively Benign missense variants and no definitively Pathogenic missense variants. REVEL and PolyPhen-2 made no false Pathogenic calls among their confident predictions, whereas SIFT misclassified 15 of 26 confident Benign variants as Pathogenic. However, the sample is small, confidence intervals are wide, and sensitivity cannot be estimated. These results support caution about SIFT false-positive behavior in this subset but do not establish AJ-specific superiority or clinical actionability for any predictor.

The absence of definitively Pathogenic missense variants in the corrected AJ-observed subset is consistent with the fact that major AJ BRCA2 founder variants, including 6174delT, are truncating rather than missense. This database composition limits the ability of the present study to evaluate pathogenic-missense detection in AJ individuals and motivates larger ancestry-linked datasets with variant-level phenotypic and functional evidence.

For an AJ individual with a BRCA2 missense VUS, these computational predictions may contribute supporting evidence, but they should not be translated directly into patient-level probabilities or management recommendations. Clinical interpretation requires integration with segregation, functional, population-frequency, and other evidence under established variant-classification frameworks.

All reported performance metrics in this study apply to missense single-nucleotide variants only; truncating founder variants and other variant classes were not benchmarked.

4.2 Domain-Specific Considerations

The corrected domain analysis confirms that pathogenic missense variation is unevenly distributed across BRCA2. No definitively Pathogenic missense variants were present in the BRC Repeats, Tower Domain, or C-terminal region, whereas the Helical Domain contained 33 and the OB Folds contained 45. The BRC-repeat finding is biologically interesting, but the absence of definitive pathogenic missense classifications should not be equated with proof that disruptive variation in this region is exclusively truncating.

Table 4: Predictor call distribution among 8,426 ClinVar VUS/conflicting missense variants.

Predictor	Pathogenic call	Benign call	Intermediate call	Proportion Pathogenic
REVEL	289	6303	1834	0.0343
SIFT	4336	3158	932	0.5146
PolyPhen-2	1456	5025	1945	0.1728

In the Helical Domain and OB Folds, REVEL showed the strongest balanced binary-call performance, particularly in the OB Folds, where accuracy was 90.1%, sensitivity 90.9%, and specificity 89.7%. One possible explanation is that REVEL integrates multiple sources of evidence, including conservation metrics, protein-level functional features, and the outputs of other prediction methods, whereas SIFT relies primarily on evolutionary conservation. Structural and functional regions such as the Helical Domain and OB Folds may therefore benefit from the broader feature set incorporated by REVEL. SIFT retained very high sensitivity but poor specificity, while PolyPhen-2 showed intermediate performance. At the same time, REVEL returned 29 intermediate calls in the Helical Domain and 93 in the OB Folds, so its strong binary-call metrics apply to a more selectively classified subset. This abstention behavior is an important part of domain-specific performance.

These domain-level findings support a context-aware interpretation rather than a simple ranking of predictors. For regions with no Pathogenic reference variants, sensitivity is not estimable. For regions with both classes represented, specificity, sensitivity, confidence intervals, and intermediate-call rates should all be considered before using a computational prediction as supporting evidence.

4.3 Limitations

Several limitations should be considered. First, ClinVar classifications were used as the reference standard, but these classifications are not fully independent of the predictors being benchmarked. SIFT, PolyPhen-2, and REVEL may contribute PP3/BP4 computational evidence to ClinVar submissions, creating potential circularity. The present design cannot quantify how much of the observed agreement, including REVEL's apparent advantage, reflects this dependence.

Second, the corrected AJ-observed subset contained only 35 missense variants, including 27 definitively Benign variants and no definitively Pathogenic variants. Sensitivity therefore cannot be estimated for this subset, and specificity estimates have wide confidence intervals. The AJ analysis should be viewed as descriptive rather than population-specific validation.

Third, classification thresholds affect both binary-call performance and the number of intermediate calls. The thresholds used here were prespecified from the literature, but different thresholds would alter sensitivity, specificity, and abstention. Gene-specific threshold optimization was beyond the scope of this study.

Fourth, the analysis was restricted to missense single-nucleotide variants. Frameshift, nonsense, splice-altering, and other variant classes were not evaluated; this is particularly important for AJ founder variation. Several domains also contained very small definitive reference sets, limiting stable performance estimation.

Fifth, the AJ criterion identifies variants with a positive recorded AJ allele frequency but does not establish enrichment, ancestry specificity, or causal association with AJ cancer risk.

Finally, performance on definitively classified variants may not generalize to VUS. Intermediate predictor calls were common, especially for REVEL and PolyPhen-2, and the true pathogenicity prevalence among ClinVar VUS is unknown. Accordingly, the Bayesian calculations are sensitivity analyses over assumed priors rather than calibrated posterior probabilities for individual VUS.

5 Conclusion

This study evaluated SIFT, PolyPhen-2, and REVEL on 9,077 BRCA2 missense SNVs, including 651 definitively classified variants, with gene-wide, domain-specific, and AJ-observed subset analyses. Among confident gene-wide binary calls, REVEL had the highest accuracy and specificity, SIFT had the highest sensitivity but substantially lower specificity, and PolyPhen-2 showed intermediate performance. Domain analyses showed that performance and abstention vary by functional region, while sensitivity was not estimable in regions containing no definitively Pathogenic missense variants.

The corrected AJ-observed subset contained no definitively Pathogenic missense variants, so the study cannot support claims about AJ-specific sensitivity or patient-level posterior probabilities. Overall, the findings support using in-silico predictors as context dependent supporting evidence rather than stand-alone classifiers. Future work should use independent functional reference sets, larger ancestry-linked cohorts, and explicit evaluation of intermediate-call behavior to improve BRCA2 missense interpretation.

6 Reproducibility

The full analysis work, including all data-processing scripts, filtering steps, and code used to generate tables and figures, is available in a publicly accessible Google Colab notebook (https://colab.research.google.com/drive/1MPReq_riDLVYMWvcCOImU_kseKJhpy41?usp=sharing).

This notebook contains the complete reproducible pipeline and instructions for regenerating all results presented in the manuscript.

References

[1] Ivan A. Adzhubei, Steffen Schmidt, Leonid Peshkin, Vasily E. Ramensky, Anna Gerasimova, Peer Bork, Alexey S. Kondrashov, and Shamil R. Sunyaev. 2010. A Method and Server for Predicting Damaging Missense Mutations. *Nature Methods* 7, 4 (2010), 248–249. https://doi.org/10.1038/nmeth0410-248

[2] Kajal Biswas et al. 2012. Functional Evaluation of BRCA2 Variants Mapping to the PALB2-Binding and C-Terminal DNA-Binding Domains Using a Mouse ES Cell-Based Assay. *Human Molecular Genetics* 21, 18 (2012), 3993–4006. https://doi.org/10.1093/hmg/dds222

[3] Melissa S. Cline et al. 2018. BRCA Challenge: BRCA Exchange as a Global Resource for Variants in BRCA1 and BRCA2. *PLoS Genetics* 14, 12 (2018), e1007752. https://doi.org/10.1371/journal.pgen.1007752

[4] Alyssa L. Halbisen and Christine Y. Lu. 2023. Trends in Availability of Genetic Tests in the United States, 2012–2022. *Journal of Personalized Medicine* 13, 4 (2023), 638. https://doi.org/10.3390/jpm13040638

[5] Nilah M. Ioannidis, Joseph H. Rothstein, Vikas Pejaver, Sumit Middha, Shannon K. McDonnell, Saurabh Baheti, et al. 2016. REVEL: An Ensemble Method for Predicting the Pathogenicity of Rare Missense Variants. *The American Journal of Human Genetics* 99, 4 (2016), 877–885. https://doi.org/10.1016/j.ajhg.2016.08.016

[6] Mohannad N. Khandakji and Borbala Mifsud. 2022. Gene-Specific Machine Learning Model to Predict the Pathogenicity of BRCA2 Variants. *Frontiers in Genetics* 13 (2022), 982930. https://doi.org/10.3389/fgene.2022.982930

[7] Karoline B. Kuchenbaecker, John L. Hopper, Daniel R. Barnes, Kelly-Anne Phillips, Thea M. Mooij, Marie-José Roos-Blom, Sarah Jervis, Flora E. van Leeuwen, Roger L. Milne, Nadine Andrieu, David E. Goldgar, Mary Beth Terry, Matti A. Rookus, Douglas F. Easton, Antonis C. Antoniou, Timothy R. Rebbeck, et al. 2017. Risks of Breast, Ovarian, and Contralateral Breast Cancer for BRCA1 and BRCA2 Mutation Carriers. *JAMA* 317, 23 (2017), 2402–2416. https://doi.org/10.1001/jama.2017.7112

[8] Pauline C. Ng and Steven Henikoff. 2003. SIFT: Predicting Amino Acid Changes That Affect Protein Function. *Nucleic Acids Research* 31, 13 (2003), 3812–3814. https://doi.org/10.1093/nar/gkg509

[9] Sue Richards, Nazneen Aziz, Sherri Bale, David Bick, Soma Das, Julie Gastier-Foster, Wayne W. Grody, Madhuri Hegde, Elaine Lyon, Elaine Spector, Karl Voelkerding, and Heidi L. Rehm. 2015. Standards and Guidelines for the Interpretation of Sequence Variants: A Joint Consensus Recommendation of the American College of Medical Genetics and Genomics and the Association for Molecular Pathology. *Genetics in Medicine* 17, 5 (2015), 405–424. https://doi.org/10.1038/gim.2015.30

[10] Jaya M. Satagopan, Kenneth Offit, William Foulkes, Mark E. Robson, Sholom Wacholder, Christine M. Eng, Stephen E. Karp, and Colin B. Begg. 2001. The Lifetime Risks of Breast Cancer in Ashkenazi Jewish Carriers of BRCA1 and BRCA2 Mutations. *Cancer Epidemiology, Biomarkers & Prevention* 10, 5 (2001), 467–473. PMID: 11352856.

Beyond Cycle Count: Sensor-Based Maintenance Warnings for Simulated Turbofan Engines

Prahalad Talur
ptalur09@gmail.com
Eastlake High School
Sammamish, WA, USA

Abstract

Run-to-failure engine data can make a maintenance warning look successful for the wrong reason: elapsed cycle count is already informative about how close an engine is to its recorded failure. This study tests whether measured telemetry adds useful final-window warning information beyond elapsed cycle and current operating conditions. We label a simulated turbofan engine cycle positive when RUL ≤ 20 and use 90 complete trajectories from the public, 100×-downsampled tiny-N-CMAPSS training partition. The corrected input definition uses the 14 measured physical channels documented by N-CMAPSS. It excludes the auxiliary flight-class and health-state fields, unit ID, and RUL.

Five outer engine-held-out folds and four inner engine-grouped folds select a threshold for a 5% pre-window false-alert budget. The cycle-plus-operating baseline recalls 55.4% of final-window cycles. Adding the 14 measured channels raises recall to 79.3% at the matched alert budget. The paired engine-cluster bootstrap estimates a 23.9-percentage-point recall gain, with a 95% interval from 18.3 to 29.6 percentage points.

The result shows that the measured telemetry in this simulator reduces missed final-window cycles beyond a clock-and-operating-state baseline. It does not set a service interval, validate a fielded aircraft system, or certify safety.

Keywords

Predictive maintenance, remaining useful life, turbofan engines, N-CMAPSS, sensor telemetry, warning threshold, engine-grouped cross-validation

ACM Reference Format:

Prahalad Talur. 2026. Beyond Cycle Count: Sensor-Based Maintenance Warnings for Simulated Turbofan Engines. In *Proceedings of International Journal of Secondary Computing and Applications Research (IJSCAR VOL. 3, ISSUE 4)*. ACM, New York, NY, USA, 6 pages. https://doi.org/10.67149/yhjs2024.5/vfmxte3f

1 Introduction

Maintenance teams do not act on a remaining useful life (RUL) estimate alone. They need a warning policy that identifies a near-term maintenance window while keeping alerts during earlier operation under control. That makes the relevant question a classification and threshold-selection problem: does the available information identify cycles near the decision boundary on engines the model has not seen?

N-CMAPSS provides simulated aircraft-engine degradation trajectories under recorded flight conditions [1]. Published work on this dataset has largely reported RUL regression scores with different readers, windows, models, and loss functions [2, 4, 5]. Those RMSE or challenge-score results are not directly comparable to this study, because this study evaluates a binary warning for RUL ≤ 20 and chooses its decision threshold under a false-alert budget.

Run-to-failure data also require a demanding baseline. An engine's elapsed cycle number is associated with its remaining life by construction. If all engines failed at the same cycle, a clock-only rule could solve much of the task. The trajectories used here fail after different lifetimes, so cycle number is informative but not sufficient. A useful sensor result must exceed both the cycle-only and cycle-plus-operating-condition baselines.

This paper first compares four information sets with the same fixed random-forest reference and the same nested engine-level evaluation: elapsed cycle only, cycle plus operating conditions, measured telemetry only, and all observable inputs. It then tests a small candidate set of tabular classifiers inside the outer-training folds to ask whether the warning policy can improve without using test-engine labels. The focus remains information value, not an architecture contest. The claim is narrow: in this simulated setting, the 14 measured channels add final-window warning information beyond elapsed cycle and recorded operating conditions. The experiment does not establish an aircraft maintenance schedule.

2 Data and warning protocol

2.1 Simulated engine data and target

The experiment uses the labeled training partition of the public `tiny-N-CMAPSS` repository at commit `b915997ff8d5` [3]; the reproducibility manifest records the complete commit hash. The source URL is https://github.com/alovberg/tiny-N-CMAPSS. This repository documents a 100×-downsampled N-CMAPSS challenge subset. The checked source files contain 90 simulated run-to-failure trajectories and 633,875 retained readings. We aggregate them to 6,825 engine-cycle rows, or 92.9 retained readings per cycle on average. The separate test partition has no RUL labels and is not used here.

The target marks an engine cycle as inside a stated final maintenance window:

$$y = \begin{cases} 1, & \text{RUL} \leq 20, \\ 0, & \text{RUL} > 20. \end{cases} \tag{1}$$

There are 1,890 positive cycle rows, a 27.7% base rate. The value 20 is an experimental warning boundary, not a recommended service

This paper is published under the Creative Commons Attribution-NonCommercial-NoDerivs 4.0 International (CC-BY-NC-ND 4.0) license. Authors reserve their rights to disseminate the work on their personal and corporate Web sites with the appropriate attribution.

IJSCAR VOL. 3, ISSUE 4,
© 2026 The Author(s). Published by IJSCAR.
https://doi.org/10.67149/yhjs2024.5/vfmxte3f

interval. For each trajectory, cycle plus RUL is constant; total simulated lifetime ranges from 48 to 100 cycles (mean 75.8, standard deviation 14.1). This variation makes elapsed cycle a useful but incomplete baseline.

2.2 Feature provenance and input sets

N-CMAPSS distinguishes measured physical properties from auxiliary fields. The primary measurement source used here is the 14-channel `Xs` group. The two fields `Fc` (flight class) and `hs` (health state) are auxiliary data and are excluded from every model. In particular, `hs` is not treated as a sensor measurement. The experiment also excludes unit ID and RUL from every model. Table 1 lists the exact variables.

We compare four inputs with one fixed random-forest reference. The cycle-only baseline has one feature. Cycle plus operating conditions has the cycle number and eight within-cycle operating summaries. Measured telemetry only has 28 within-cycle summaries from the 14 measured channels. All observable inputs has 37 features: cycle, operating summaries, and measured-telemetry summaries. This comparison separates three questions: how much a clock predicts, whether operating state adds information, and whether measured telemetry reduces misses beyond both.

Each variant uses a `RandomForestClassifier` with 250 trees, maximum depth 12, minimum leaf size 3, balanced classes, and a primary random seed of 2026. The model is a fixed screening model. The study does not claim that it is the best architecture for this data.

2.3 Nested engine-held-out validation

We use five outer `GroupKFold` splits grouped by engine. Each outer test fold has 18 complete trajectories and the remaining 72 engines form the outer training data. Every engine appears in exactly one outer test fold. Its rows are absent from fitting and threshold selection for that fold.

Within each outer training set, four inner engine-grouped folds generate out-of-fold probabilities. We choose the lowest probability threshold whose pre-window false-alert rate is at or below 5%. A final model then fits the 72 outer-training engines and predicts the 18 held-out engines. The pooled result contains only these outer-test predictions. This procedure evaluates a warning policy without choosing its threshold from test-engine labels.

2.4 Nested model-family selection

The fixed random forest is used for the information comparison so that the input-set result does not depend on a particular architecture. We also evaluate one secondary, all-observable warning policy that selects a model family within each outer-training split. Its four candidates are the fixed random-forest reference, an extra-trees classifier, a histogram-gradient-boosting classifier, and a standardized RBF support-vector classifier. Their configurations are recorded in the evaluation output.

For each outer fold, every candidate produces inner engine-grouped out-of-fold probabilities. For each candidate, the inner probabilities determine the same 5%-budget threshold as above. The selected candidate is the one with the highest inner final-window recall at that threshold; ties break in order of AUROC, average precision, precision, and lower false-alert rate. We then fit only that selected candidate on all 72 outer-training engines and score the 18 held-out engines. Thus neither model-family selection nor threshold selection uses labels from the outer test engines.

2.5 Metrics, robustness, and warning timing

We report AUROC, precision, final-window recall, and the pre-window false-alert rate. We calculate 95% percentile intervals by resampling complete engine trajectories 2,000 times. We also report a fixed 0.5-threshold check on the same outer-test probabilities. It is a diagnostic, not the policy used for the primary result.

To test stochastic variation, we repeat the full fixed-random-forest input comparison and the full selected-policy procedure with seeds 2024 through 2028. The outer engine splits remain fixed, so these ranges test run-to-run variation on the same data rather than a new-dataset replication.

For a first-warning analysis, let c_u^{window} be the first cycle of engine u with RUL ≤ 20, and let c_u^{warn} be the first cycle whose score meets its outer-fold threshold. The lead is

$$\ell_u = c_u^{\text{window}} - c_u^{\text{warn}}. \tag{2}$$

Positive values are warnings before the window; negative values are warnings after it begins. This timing is a property of the tested screening policy, not a maintenance command.

2.6 Reproducibility materials

The project repository is https://github.com/prahaladtalur/N-CMAPSS-Engine-Prediction. The evaluation records the exact tiny-N-CMAPSS commit, SHA256 hashes of the two downloaded data files, outer-test predictions, selected thresholds, and the scripts used to produce the tables and figures. These materials identify the corrected input definition and make the reported values reproducible.

3 Results

3.1 Measured telemetry adds value beyond cycle and operating state

Table 2 gives the nested outer-test results. Elapsed cycle alone is a strong baseline, with AUROC 0.923 and final-window recall 0.563 at a 0.050 pre-window false-alert rate. Adding operating conditions raises AUROC to 0.935 but leaves recall at 0.554. The all-observable model, which adds summaries from the 14 measured channels, reaches AUROC 0.966 and recall 0.793 at a 0.051 false-alert rate.

The paired outer-test comparison of all observable inputs against cycle plus operating conditions estimates a 0.031 AUROC gain, with a 95% engine-cluster interval from 0.023 to 0.039. Its recall gain is 0.239, with an interval from 0.183 to 0.296. The false-alert-rate difference is 0.004, with an interval that spans zero (-0.006 to 0.015). At the matched alert budget, the added telemetry therefore increases the share of final-window cycles found without a clear increase in the pre-window alert rate.

The telemetry-only rule is also useful without a cycle-number feature. It has AUROC 0.939, precision 0.863, and recall 0.771 at a 0.047 false-alert rate. Compared with cycle only, it gains 0.208 recall (95% interval 0.105 to 0.310). Its AUROC difference is 0.016, with an interval from -0.009 to 0.038. The recall result is the stronger evidence for the fixed 5% warning policy.

Table 1: Input provenance. Means and population standard deviations are computed within each engine cycle for every channel except elapsed cycle.

Group	Count	Fields and use
Measured physical (`Xs`)	14	`T24, T30, T48, T50, P15, P2, P21, P24, Ps30, P40, P50, Nf, Nc, Wf`; included in telemetry variants
Operating conditions (`W`)	4	`alt, Mach, TRA, T2`; used in cycle-plus-operating and all-observable inputs
Elapsed cycle	1	`cycle`; included only in variants that name it
Auxiliary fields	2	`Fc, hs`; excluded from every model
Identifier and label	2	`unit, RUL`; excluded from every model

Table 2: Nested engine-held-out performance at a target 5% pre-window false-alert budget. Values in brackets are 95% percentile intervals from 2,000 complete-engine bootstrap resamples.

Input	AUROC	Precision	Final-window recall	Pre-window false-alert rate
Elapsed cycle only	0.923 [0.910, 0.939]	0.813 [0.766, 0.866]	0.563 [0.480, 0.646]	0.050 [0.032, 0.067]
Cycle plus operating conditions	0.935 [0.922, 0.947]	0.821 [0.775, 0.869]	0.554 [0.480, 0.629]	0.046 [0.031, 0.063]
Measured telemetry only	0.939 [0.918, 0.957]	0.863 [0.837, 0.888]	0.771 [0.715, 0.824]	0.047 [0.037, 0.058]
All observable inputs	0.966 [0.958, 0.973]	0.857 [0.821, 0.892]	0.793 [0.751, 0.835]	0.051 [0.037, 0.066]

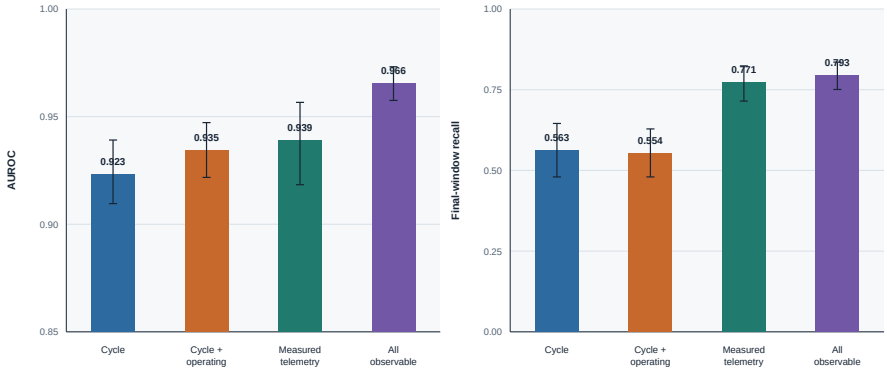


Figure 1: Measured telemetry improves final-window classification beyond elapsed cycle and operating conditions. Each bar uses only outer-test engine predictions; error bars are 95% complete-engine bootstrap intervals.

3.2 Nested selection improves the all-observable warning policy

The information comparison above keeps the classifier fixed. To test whether a stronger warning policy is available without using test-engine labels, the secondary all-observable analysis selects from four tabular candidates inside each outer-training split. Histogram gradient boosting is selected in three outer folds and the RBF support-vector classifier in two. The resulting policy reaches AUROC 0.973, precision 0.861, and final-window recall 0.821 at a 0.051 realized false-alert rate (Table 3). It warns 54 of 90 engines at or before the final-window start, so its gain is in cycle-level detection rather than first-warning lead time.

Relative to the fixed all-observable random forest, nested model selection adds 0.007 AUROC and 0.028 final-window recall at the same pooled false-alert rate. The paired complete-engine bootstrap interval is 0.002 to 0.011 for AUROC and 0.004 to 0.051 for recall. The model-family choice and threshold are made from inner out-of-fold predictions in every outer fold, so these intervals compare outer-test predictions only.

3.3 The fixed 0.5 threshold is a useful diagnostic, not an equivalent policy

Table 4 evaluates the same outer-test scores at a fixed 0.5 threshold. The telemetry-only rule changes little because its nested thresholds range from 0.460 to 0.479. The all-observable, cycle-only, and cycle-plus-operating policies require higher thresholds to meet the 5% alert budget. Their 0.5-threshold recalls are higher, but so are their false-alert rates. This confirms that threshold selection materially

Table 3: All-observable warning policies at a target 5% pre-window false-alert budget. The selected policy chooses its model family and threshold within each outer-training split. Values in brackets are 95% complete-engine bootstrap intervals.

Policy	AUROC	Precision	Final-window recall	Pre-window false-alert rate
Fixed random forest	0.966 [0.958, 0.973]	0.857 [0.821, 0.892]	0.793 [0.751, 0.835]	0.051 [0.037, 0.066]
Nested selected policy	0.973 [0.966, 0.979]	0.861 [0.827, 0.894]	0.821 [0.778, 0.862]	0.051 [0.038, 0.066]

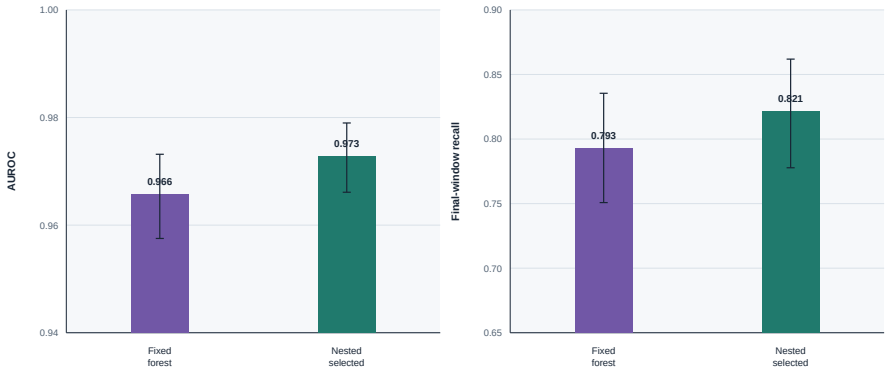


Figure 2: Nested model selection improves the all-observable policy without using labels from test engines. The selected policy uses histogram gradient boosting in three outer folds and an RBF support-vector classifier in two.

affects the decision policy and was not omitted from the primary result.

3.4 Sensor-only scoring produces earlier first warnings

The first-warning analysis evaluates a different operational property from cycle-level recall: whether a policy flags an engine at or before the start of the final 20-cycle window. The telemetry-only rule warns 69 of 90 engines by that point, with a median lead of 6 cycles among the 87 engines it eventually warns (Table 5). The fixed all-observable random forest warns 50 engines by the window start, while the nested selected policy warns 54. Both have a median lead of 1 cycle among the engines they eventually warn.

This difference is a tradeoff, not a contradiction. The all-observable model was selected for cycle-level final-window classification, while the first-alert metric rewards an earlier threshold crossing. A deployment study that needs lead time should select and validate an engine-level early-warning objective directly.

3.5 Cycle-level metrics are stable across model seeds

Table 6 reports five complete nested evaluations with random-forest seeds 2024 through 2028. The seed ranges are narrow for each input. They do not replace external validation, because the simulated engines and the outer folds remain the same.

Across the five seeds, the all-observable recall gain over cycle plus operating conditions ranges from 21.9 to 23.9 percentage points, with a mean of 22.6 points. The paired 18.3-to-29.6-point interval reported for the primary run (seed 2026) captures uncertainty from resampling complete engines; it does not include model-seed variability. First-warning timing is reported for the primary run only and is not part of this seed-robustness check.

4 Discussion

The main result is not that a classifier can recognize a simulated failure trajectory after seeing its age. Cycle count alone already reaches AUROC 0.923. The useful result is the comparison against that strong baseline. At a matched pre-window alert budget, measured telemetry raises final-window recall from 0.554 for cycle plus operating conditions to 0.793 for all observable inputs. The paired complete-engine bootstrap interval for the recall gain remains positive, from 0.183 to 0.296.

The secondary nested model-selection policy raises all-observable recall from 0.793 to 0.821 and AUROC from 0.966 to 0.973. It selects histogram gradient boosting in three outer folds and an RBF support-vector classifier in two. This is a practical improvement under the same warning budget, but it is not an architecture claim. The candidate set is small, the model choice is evaluated only on this simulated subset, and the policy does not improve the earlier first-warning timing reported for the sensor-only rule.

The telemetry-only rule supplies a separate result for an early-screening setting. It has no elapsed-cycle feature, yet it recalls 77.1% of final-window cycles at a 4.7% pre-window false-alert rate. It warns 69 of 90 engines by the window start, with a median lead of 6 cycles among warned engines. This does not show that the

Table 4: Diagnostic fixed-threshold performance on the same outer-test probabilities. These values do not replace nested threshold selection.

Input	Precision	Final-window recall	Pre-window false-alert rate
Elapsed cycle only	0.624	0.887	0.204
Cycle plus operating conditions	0.700	0.859	0.141
Measured telemetry only	0.890	0.751	0.036
All observable inputs	0.791	0.872	0.089

Table 5: First-warning timing from outer-test predictions. Positive lead means the first warning occurs before the final-window start.

Input	By window start	Late warning	No warning	Median lead among warned
Measured telemetry only	69/90 (76.7%)	18/90	3/90	6 cycles
All observable, fixed random forest	50/90 (55.6%)	39/90	1/90	1 cycle
All observable, nested selected policy	54/90 (60.0%)	34/90	2/90	1 cycle



Figure 3: First warning for the telemetry-only policy across 90 outer-test engine trajectories. This policy uses no cycle number or operating-condition field. Positive lead values arrive before the final 20-cycle window begins.

Table 6: Five-seed robustness check. Each entry is mean [minimum, maximum] over full nested evaluations using fixed outer engine splits. The first four rows use the fixed random-forest reference; the final row repeats model-family selection inside every outer-training split.

Input	AUROC	Final-window recall	Pre-window false-alert rate
Elapsed cycle only	0.923 [0.923, 0.923]	0.564 [0.559, 0.568]	0.050 [0.048, 0.051]
Cycle plus operating conditions	0.935 [0.934, 0.935]	0.559 [0.554, 0.566]	0.049 [0.046, 0.049]
Measured telemetry only	0.940 [0.939, 0.941]	0.774 [0.771, 0.776]	0.048 [0.046, 0.050]
All observable inputs	0.965 [0.965, 0.966]	0.785 [0.780, 0.793]	0.050 [0.049, 0.051]
Nested selected policy	0.973 [0.973, 0.973]	0.821 [0.821, 0.822]	0.051 [0.051, 0.051]

model knows when a real engine should be removed from service. It shows that the documented measured channels contain signal about the simulated approach to the chosen 20-cycle boundary.

The feature definition matters. The evaluated model uses only the 14 named measured physical channels, listed in Table 1. It does not use virtual sensors, target health parameters, unit ID, RUL, or the auxiliary `Fc` and `hs` fields. This restriction makes the telemetry result testable. It also makes the result lower than an experiment that treated auxiliary health-state information as if it were a measured sensor.

Several limitations remain. First, tiny-N-CMAPSS is simulated and the retained data are 100× downsampled. Each per-cycle summary averages about 93 retained readings, so it cannot test how a warning would respond to high-rate within-flight transients. Second, the 20-cycle window, 5% false-alert budget, and one-cycle alert definition are experimental choices. A maintenance program would need costs, inspection capacity, confirmation rules, and a lead-time objective before choosing a policy. Third, five seeds test stochastic variation of the fixed reference and the nested selection procedure on the same data. They do not replace an independently labeled fleet or a held-out test campaign.

The next technical step is to train and evaluate a warning policy against an engine-level objective, such as warning by a specified lead time with a per-engine alert budget. That would directly optimize the tradeoff exposed in Table 5. The next empirical step is external validation on labeled engines with maintenance outcomes. Until then, this paper should be read as a controlled simulation result about information value, not a deployable maintenance recommendation.

5 Conclusion

This study tests a narrow maintenance-warning question on simulated turbofan engines: do measured channels add information beyond elapsed cycle and operating conditions when the warning threshold is chosen without seeing the held-out engines? The answer is yes for the tested N-CMAPSS subset. At a roughly 5% pre-window false-alert rate, all observable inputs raise final-window recall from 55.4% to 79.3% over the cycle-plus-operating baseline. The paired engine-cluster interval for the gain is 18.3 to 29.6 percentage points.

The telemetry-only result is also useful. With no cycle number or operating-condition field, it recalls 77.1% of final-window cycles and warns 69 of 90 engines by the window start in the primary run (seed 2026). Its cycle-level recall is stable across five random-forest seeds, ranging from 77.1% to 77.6%; first-warning timing was not repeated across seeds. These results use only the 14 documented measured channels. A secondary policy that selects its model family inside the training folds raises the all-observable result to AUROC 0.973 and recall 82.1%, compared with 0.966 and 79.3% for the fixed random forest at the same pooled false-alert rate. This is a controlled performance gain, not a claim of a universally best model.

The contribution is evidence that observed sensor telemetry carries maintenance-warning value beyond a flight-cycle clock in this simulator. It is not evidence for a service interval or a fielded-aircraft decision rule. Those claims require maintenance-defined costs and lead times, plus independent labeled engine data.

References

[1] Manuel Arias Chao, Chetan Kulkarni, Kai Goebel, and Olga Fink. 2021. Aircraft Engine Run-to-Failure Dataset under Real Flight Conditions for Prognostics and Diagnostics. *Data* 6, 1 (2021), 5. doi:10.3390/data6010005

[2] Manuel Arias Chao, Chetan Kulkarni, Kai Goebel, and Olga Fink. 2022. Fusing physics-based and deep learning models for prognostics. *Reliability Engineering & System Safety* 217 (2022), 107961. doi:10.1016/j.ress.2021.107961

[3] Andreas Lövberg. [n. d.]. tiny-N-CMAPSS: Downsampled N-CMAPSS challenge data. GitHub repository, https://github.com/alovberg/tiny-N-CMAPSS. Pinned commit: https://github.com/alovberg/tiny-N-CMAPSS/tree/b915997ff8d571f4e9d091954d6556835f212ded; accessed 2026-08-12. https://github.com/alovberg/tiny-N-CMAPSS

[4] Andreas Lövberg. 2021. Remaining Useful Life Prediction of Aircraft Engines with Variable Length Input Sequences. *Annual Conference of the PHM Society* 13, 1 (2021). doi:10.36001/phmconf.2021.v13i1.3108

[5] David Solís-Martín, Juan Galán-Páez, and Joaquín Borrego-Díaz. 2021. A Stacked Deep Convolutional Neural Network to Predict the Remaining Useful Life of a Turbofan Engine. *Annual Conference of the PHM Society* 13, 1 (2021). doi:10.36001/phmconf.2021.v13i1.3110

Dynamic Mathematical PDE Modeling and Machine Learning for TANGO2 Identification and Crisis Prediction

Abhay Tippimath
McNeil High School
Austin, Texas, USA
abhay.tippi03@gmail.com

Abstract

TANGO2 (Transport And Golgi Organization 2) deficiency disorder (TDD) is a rare autosomal recessive condition marked by unpredictable, life-threatening metabolic crises, including rhabdomyolysis, cardiac arrhythmias, and metabolic encephalopathy. Supportive treatment, particularly B-complex vitamin supplementation and fasting avoidance, substantially reduces the frequency and severity of crises, so undiagnosed patients remain at the greatest risk. This work presents a two-stage framework that couples supervised machine learning classification with numerical simulation of a stochastic partial differential equation (PDE). The first stage identifies TDD patients from their stable baseline presentation, and the second models future crisis behavior as a threshold-crossing dynamical system. Using deidentified Human Phenotype Ontology (HPO) profiles from 90 molecularly confirmed TDD patients and 141 diagnosed Undiagnosed Diseases Network (UDN) controls, we exclude all 23 crisis-defining phenotypes and restrict the primary analysis to the 30 HPO terms recorded by both annotating teams, so that classification depends only on information available before a crisis. A Random Forest classifier, evaluated by repeated stratified tenfold cross-validation, attains an AUC of 0.993 (variance-corrected 95% interval 0.980 to 1.000), with 95.6% sensitivity and 100% specificity. The result withstands a label permutation test ($p = 0.001$), stricter feature sets, and ablation of the most important features, indicating a signal distributed across many correlated phenotypes. Two phenotypes, paroxysmal lethargy and compensatory head posture, were recorded only in TDD patients; because the control annotating team never used these terms, they are reported as hypotheses that cannot yet be separated from annotation style. Separately, a stochastic reaction-diffusion PDE with a patient-specific source term reproduces the episodic nature of metabolic crises and exhibits a critical source amplitude that partitions patients into stable and recurrent regimes. Scalar models without diffusion cannot reproduce this behavior without recalibration, and the patient-specific parameter is recoverable from a single simulated timeline ($r = 0.992$). Together, these results indicate that phenotypes present before any crisis can identify TDD before life-threatening events occur, while the PDE offers a testable hypothesis for how continuous metabolic risk produces discrete, episodic crises.

This paper is published under the Creative Commons Attribution-NonCommercial-NoDerivs 4.0 International (CC-BY-NC-ND 4.0) license. Authors reserve their rights to disseminate the work on their personal and corporate Web sites with the appropriate attribution.
IJSCAR VOL. 3, ISSUE 4,
© 2026 The Author(s). Published by IJSCAR.
https://doi.org/10.67149/yhjs2024.5/l542temi

Keywords

TANGO2, rare disease, random forest, reaction diffusion equation, partial differential equations, Human Phenotype Ontology, crisis prediction, bioinformatics, machine learning

ACM Reference Format:
Abhay Tippimath. 2026. Dynamic Mathematical PDE Modeling and Machine Learning for TANGO2 Identification and Crisis Prediction. In *Proceedings of International Journal of Secondary Computing and Applications Research (IJSCAR VOL. 3, ISSUE 4).* ACM, New York, NY, USA, 9 pages. https://doi.org/10.67149/yhjs2024.5/l542temi

1 Introduction

1.1 Clinical and Molecular Background

TANGO2 (Transport And Golgi Organization 2) deficiency disorder is caused by biallelic loss of function variants in *TANGO2* on chromosome 22q11.21, most often a recurrent deletion of exons 3 to 9 [8, 9]. The protein localizes to the endoplasmic reticulum, the Golgi apparatus, and mitochondria, and its loss impairs lipid homeostasis, membrane trafficking, and mitochondrial fatty acid oxidation. Its precise molecular function is still being characterized, so no targeted therapy exists and care depends on early recognition and supportive treatment.

The natural history has two states. Between crises, patients show a relatively stable neurodevelopmental phenotype that includes developmental delay, intellectual disability, dysarthria, and gait disturbance. During a crisis, usually triggered by fasting or febrile illness, patients decompensate with rhabdomyolysis, metabolic acidosis, encephalopathy, and dangerous ventricular arrhythmias [4]. Fourteen of the 90 patients analyzed here are deceased. Figure 1 summarizes the mechanism: loss of TANGO2 disrupts three converging processes, and under metabolic stress, cellular energy production fails in the tissues with the highest demand. B complex supplementation, particularly pantothenate (B5) and folate (B9), is thought to act upstream by supporting coenzyme A availability, which is why it reduces crisis frequency and severity.

1.2 Two Problems with Complementary Structure

This history creates two clinical problems. The *diagnostic* problem is that the stable phenotype overlaps with many neurogenetic conditions, so TDD is often not suspected until a first crisis has already occurred, even though prophylaxis exists. Computationally, this is classification of a sparse binary phenotype vector. The *prognostic* problem is that clinicians cannot anticipate the next crisis, so monitoring is uniform rather than stratified. Computationally, this is a dynamical question about a latent risk that accumulates until it

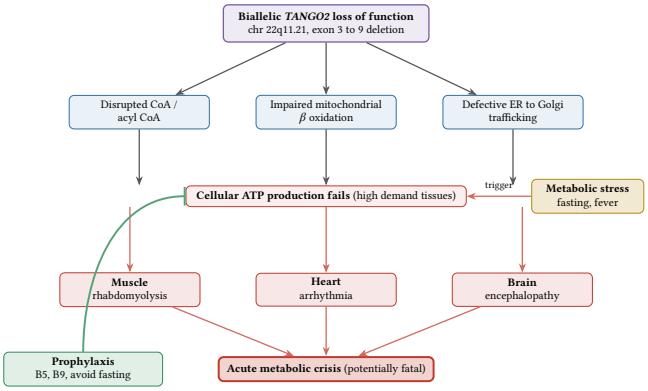


Figure 1: Pathophysiology of TANGO2 deficiency disorder. Loss of function disrupts three converging molecular processes. Under metabolic stress this causes energy failure and injury to muscle, heart, and brain, which together produce the acute crisis. B complex prophylaxis (green) acts upstream.

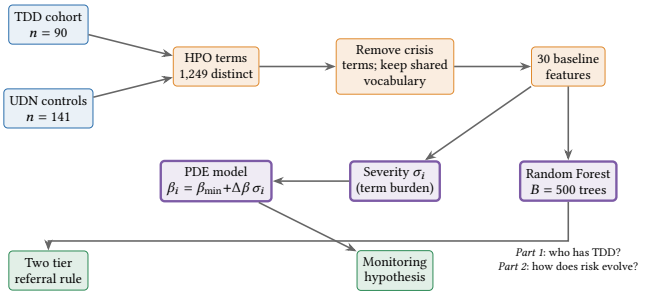


Figure 2: Overview of the framework. Shared baseline phenotypes drive a Random Forest classifier (Part 1). A severity score computed from the same phenotypes sets the patient specific source amplitude of the PDE (Part 2).

crosses a threshold. Figure 2 shows how the framework connects the two.

1.3 Contributions

(1) We show that phenotypes observable strictly before any crisis identify TDD accurately, after removing every crisis defining term and restricting to vocabulary shared by both annotating teams, and we stress test the result with a permutation test, a variance corrected interval, stricter feature sets, and alternative models.
(2) We report two phenotypes recorded only in TDD patients in this cohort, explain why their apparent specificity cannot yet be separated from annotation style, and contrast them with a more sensitive neurological triad recorded by both annotating teams.
(3) We formulate a stochastic reaction diffusion model of crisis risk, derive a critical source amplitude β^* separating quiescent and recurrent regimes, and test empirically what the spatial term adds over scalar models without diffusion.
(4) We verify the solver numerically, quantify parameter sensitivity, and show through a synthetic recovery test that the patient specific parameter is identifiable from crisis timelines, which prepares the model for future fitting to longitudinal data.

2 Related Work

Phenotype driven rare disease analysis. The Human Phenotype Ontology (HPO) provides a standardized vocabulary of phenotypic abnormalities and underpins computational rare disease analysis [6]. The Undiagnosed Diseases Network (UDN), which supplied our controls, applies systematic phenotyping to patients whose presentations defeated standard workup [15]. Whereas phenotype driven tools usually rank candidate genes for one patient, we learn a classifier that separates one confirmed disorder from its realistic differential using only features available before a crisis.

Classification on sparse binary features. Random Forests average many decorrelated trees, need no feature scaling, capture interactions, and expose feature importances [2]. These properties suit small, sparse, correlated clinical data where an interpretable rule is the goal. Because leakage and dataset artifacts frequently inflate clinical machine learning results, we pair the model with explicit safeguards and falsification tests, including label permutation [12].

Reaction diffusion models and stiff solvers. Reaction diffusion equations descend from the Fisher KPP equation [5, 7] and are widely used for pattern formation and physiology [10]. We apply this machinery to an abstract coordinate of coupled physiological subsystems. Diffusion terms are stiff, so we use an implicit explicit (IMEX) time integrator [1] verified against standard numerical criteria [14].

3 Data

3.1 Cohorts

Two deidentified cohorts were provided by the Department of Molecular and Human Genetics at Baylor College of Medicine. The TDD cohort contains 90 individuals with molecularly confirmed TANGO2 deficiency (49 male, 41 female; 76 living, 14 deceased; median age at evaluation 8.8 years across the 86 with recorded age). The control cohort contains 141 UDN participants with a confirmed diagnosis other than TANGO2 deficiency, most with neurological ($n = 67$) or multiple congenital anomaly ($n = 39$) diagnoses. The task is therefore TDD versus a phenotypically overlapping differential rather than versus healthy children. The cohorts contain 1,249 distinct HPO terms and no missing phenotype records.

3.2 Feature Construction and Analysis Timeline

Two safeguards shape the features. First, every phenotype describing a crisis or its aftermath was removed, because a classifier that sees *rhabdomyolysis* would simply detect a crisis that has already occurred. The 23 excluded categories (35 HPO terms) include rhabdomyolysis, myoglobinuria, elevated creatine kinase, cardiac arrest, cardiomyopathy, arrhythmia, prolonged QT, metabolic and lactic acidosis, hyperammonemia, hypoglycemia, encephalopathy, and coma. Keeping terms recorded in at least three patients leaves 261 candidate features.

Table 1: Random Forest hyperparameters, fixed before evaluation.

Setting	Value	Setting	Value
Trees	500	Max depth	none
Split criterion	Gini	Features per split	$\lfloor\sqrt{30}\rfloor = 5$
Bootstrap	yes	Class weight	balanced
Min samples split	2	Min samples leaf	1
Random seed	fixed	Threshold	0.5

Second, the cohorts were phenotyped by different teams with nearly disjoint vocabularies. A term used by only one team separates the cohorts regardless of biology, so we retained only the 31 candidate terms used by both teams. We then excluded the single electrocardiographic term, because an ECG abnormality is a test result rather than a bedside observation, leaving a primary set of 30 features. The two candidate markers of Section 5.4 were never recorded in controls and are therefore outside this set; the headline results are achieved without them. By the same reasoning, their absence among controls may reflect annotation vocabulary rather than biology.

Reviewers rightly ask when such choices were made. The crisis exclusion list and the three patient threshold were fixed before any model was trained. The shared vocabulary restriction was added after an audit of term overlap, and the ECG exclusion was made later on the advice of a reviewing clinician, after initial results had been seen. Both later restrictions made the task harder rather than easier (AUC 0.996 for the 261 candidate features, 0.994 for 31 shared terms, 0.993 for the final 30), so neither was chosen to improve performance, and results at every stage are reported. Section 5.2 further shows that the result does not depend on the frequency threshold.

Each patient is represented as a binary vector $\mathbf{x} \in \{0,1\}^{30}$ whose entry j is 1 when the patient carries term j. Binary indicators need no scaling, and the absence of a term is treated as a true negative. The 90 to 141 class imbalance is handled by inverse frequency class weights rather than resampling.

4 Methods

4.1 Random Forest Classifier

A Random Forest of $B = 500$ trees estimates $\hat{P}(y = 1 \mid \mathbf{x}) = \frac{1}{B}\sum_{b=1}^{B} T_b(\mathbf{x})$, and a patient is classified as TDD when $\hat{P} \geq 0.5$. Feature importance is the mean decrease in Gini impurity $G(m) = 2p_m(1 - p_m)$, which rewards features that are both common in cases and rare in controls. Because correlated phenotypes share credit, importances identify informative clusters rather than independent causes. All hyperparameters (Table 1) were fixed in advance and held constant across folds, and the implementation uses the `RandomForestClassifier` of Pedregosa et al. [13].

4.2 Evaluation and Stress Tests

With 231 patients, a single split is unreliable, so we use repeated stratified tenfold cross validation: 10 repeats, 100 model fits, about 208 training and 23 validation patients per fold, and every patient predicted once per repeat by a model that never saw it. Calibration is measured with the Brier score $\mathrm{BS} = \frac{1}{n}\sum_i(\hat{p}_i - y_i)^2$ and its skill relative to a model that always predicts the base rate.

Because the 100 fits reuse the same 231 patients, they are not independent, and a naive standard deviation understates uncertainty. We therefore also report the corrected resampled t interval of Nadeau and Bengio [11], which inflates the variance of fold estimates by the ratio of test to training size. To test whether the AUC could arise from a small, easily separable dataset, we ran a label permutation test [12]: the labels were shuffled 1,000 times and the full cross validation was repeated each time (100 trees per forest for tractability). Further checks were stricter feature sets, the annotation depth of each record, duplicate records, ablation of the most important features, and two alternative model families, logistic regression and a support vector machine with a radial basis kernel.

4.3 Reaction Diffusion Model of Crisis Risk

We model crisis risk as a field $r(x,t) \in [0,1]$, where t is time in days and $x \in [0,1]$ is a latent coordinate indexing coupled physiological subsystems:

$$\frac{\partial r}{\partial t} = D\frac{\partial^2 r}{\partial x^2} + \alpha\, r(1-r) + \beta\, S(x) - \delta\, r, \tag{1}$$

with no flux boundaries and $r(x,0) = 0.13$. Diffusion represents spread of instability between coupled subsystems; logistic growth represents accumulation that reinforces itself, capped at 1 [5, 7]; the Gaussian source $S(x) = \exp[-(x - x_0)^2/2\sigma^2]$ represents a constitutional vulnerability whose amplitude β is the only patient specific parameter; and $-\delta r$ represents homeostatic clearance. A crisis is declared when $\max_x r \geq r^*$, after which the reset $r \leftarrow r_{\mathrm{rest}} + \kappa(r - r_{\mathrm{rest}})$ with $\kappa = 0.52$ and $r_{\mathrm{rest}} = 0$ removes 48 percent of the accumulated risk while leaving β unchanged, so crises recur from the dynamics rather than from an imposed period. Because real crises follow discrete triggers, we add multiplicative noise $\sigma_\eta\sqrt{r(1-r)}\,S(x)\,\dot{W}(x,t)$, which vanishes at 0 and 1 and so keeps r in range.

The coordinate x is a modeling abstraction rather than a measured quantity. The reason for a spatial model is that risk in TANGO2 appears to be concentrated in a vulnerable subsystem before spreading to coupled ones, and a scalar model cannot distinguish concentrated risk near threshold from the same total risk spread harmlessly. Section 5.6 tests this claim empirically rather than asserting it.

4.4 Parameter Selection

Parameters appear in Table 2. Only β varies between patients. The shared parameters were chosen by a coordinate wise grid search to reproduce three qualitative features reported in the TANGO2 literature [4, 9]: recurrence on a timescale of months, more frequent crises with greater baseline burden, and, in the deterministic model without noise, no crises for the mildest patients, that is, those with β below the critical amplitude β^* of Section 4.6. Crises in these patients under noise are a model prediction rather than a calibration target (Section 5.5). The parameters were not fitted to patient crisis dates, which the dataset does not contain. The resting level is $r_{\mathrm{rest}} = 0$; it does not affect β^*, which is defined with the reset disabled. The threshold r^* sets the scale of the dimensionless risk; α and δ set the rebuild time after a crisis; D keeps the risk profile spatially structured over the horizon; x_0 and σ place and size the vulnerable

Table 2: Model parameters and their basis. Only β varies between patients, and no parameter was fitted to observed crisis dates.

Symbol	Meaning	Value	Basis
r^*	crisis threshold	0.72	scale of r
κ	retention after crisis	0.52	partial recovery
r_{rest}	resting risk level	0	recovery floor
β	source amplitude	0.0121 to 0.0176	per patient
β^*	critical amplitude	0.01273	derived
α, δ	growth, clearance	0.011, 0.007	crisis timescale
D	diffusion coefficient	0.0012	spatial structure
x_0, σ	locus, width	0.58, 0.06	vulnerable band
σ_η	noise intensity	0.045	timing variability
N, Δt, T	grid, step, horizon	260, 0.5 d, 900 d	numerics

Algorithm 1 IMEX solver with crisis detection

Assemble tridiagonal $\mathbf{A} = \mathbf{I} - \Delta t\, D\mathbf{L}$ with no flux rows; $\mathbf{r} \leftarrow r_0\mathbf{1}$; $\mathcal{E} \leftarrow \emptyset$
for $n = 0$ to $T/\Delta t - 1$ **do**
  $\mathbf{g} \leftarrow \alpha\mathbf{r} \odot (\mathbf{1} - \mathbf{r}) + \beta\mathbf{S} - \delta\mathbf{r}$ ▷ explicit reaction
  $\boldsymbol{\xi} \leftarrow \sigma_\eta \sqrt{\mathbf{r} \odot (\mathbf{1} - \mathbf{r})} \odot \mathbf{S} \odot \Delta\mathbf{W}^n$ ▷ noise
  Solve $\mathbf{A}\mathbf{r}^+ = \mathbf{r} + \Delta t\, \mathbf{g} + \boldsymbol{\xi}$; $\mathbf{r} \leftarrow \text{clip}(\mathbf{r}^+, 0, 1)$
  if $\max_j r_j \geq r^*$ **then** $\mathcal{E} \leftarrow \mathcal{E} \cup \{n\Delta t\}$; $\mathbf{r} \leftarrow r_{\text{rest}}\mathbf{1} + \kappa(\mathbf{r} - r_{\text{rest}}\mathbf{1})$
  end if
end for
return crisis times $\mathcal{E}$

band; and σ_η gives roughly a week of timing variability between realizations. Severity is the min to max normalized count of retained baseline terms, $\sigma_i = (n_i - n_{\min})/(n_{\max} - n_{\min})$, mapped linearly to $\beta_i = \beta_{\min} + (\beta_{\max} - \beta_{\min})\sigma_i$. This linear mapping is a modeling assumption, so simulated patient trajectories illustrate the model rather than predict individual outcomes.

4.5 Numerical Scheme

With $N = 260$ nodes ($\Delta x \approx 3.85 \times 10^{-3}$), an explicit scheme would need $\Delta t \leq \Delta x^2/2D \approx 6.2 \times 10^{-3}$ days, about 80 times smaller than our step of 0.5 days. We therefore treat diffusion implicitly and reaction explicitly [1]:

$$(\mathbf{I} - \Delta t\, D\, \mathbf{L})\, \mathbf{r}^{n+1} = \mathbf{r}^n + \Delta t\big[\alpha\mathbf{r}^n \odot (\mathbf{1} - \mathbf{r}^n) + \beta\mathbf{S} - \delta\mathbf{r}^n\big] + \boldsymbol{\xi}^n, \quad (2)$$

where $\mathbf{L}$ is the three point Laplacian and $\boldsymbol{\xi}^n$ is the Euler Maruyama noise increment. The system is tridiagonal, so each step costs $O(N)$ with the Thomas algorithm (Algorithm 1).

Proposition 4.1. *The implicit diffusion update is unconditionally stable for every $\Delta t > 0$.*

Proof. For a Fourier mode $r_j^n = g^n e^{\mathrm{i}\theta j}$ the amplification factor is $g(\theta) = [1 + 4\lambda \sin^2(\theta/2)]^{-1}$ with $\lambda = D\Delta t/\Delta x^2 > 0$. The denominator is at least 1, so $0 < g(\theta) \leq 1$ for every θ. □

4.6 Critical Source Amplitude

We define the critical amplitude from the steady state of the deterministic PDE with the reset disabled,

$$\beta^* = \inf\{\beta > 0 : \max_x r_\infty(x; \beta) \geq r^*\}, \quad (3)$$

and bisection gives $\beta^* = 0.01273$. Patients below β^* settle into a bounded state under the threshold, and patients above it cross repeatedly. The calibrated range 0.0121 to 0.0176 contains β^*, so the mildest patients sit just below it.

It is instructive to compare two scalar reductions without diffusion, each with fixed point $r_+ = [(\alpha - \delta) + \sqrt{(\alpha - \delta)^2 + 4\alpha b}\,]/2\alpha$ for effective source b. A scalar model of the *peak*, which receives the full source $b = \beta$ with no leakage, has r_+ between 1.25 and 1.46 across the cohort. These fixed points lie above the admissible range $[0, 1]$ of r, which means risk in this model never settles below the threshold; in simulation r is clipped at 1, so every patient crosses. A scalar model of the *average*, with $b = \beta\bar{S}$ where $\bar{S} = 0.150$ is the mean of S, has r_+ between 0.63 and 0.70, so no patient would cross, and its own critical amplitude (0.0188) lies outside the cohort range. The spatial model sits between these extremes because diffusion leaks risk away from a localized peak.

4.7 Model Tests

Three tests address what the model can and cannot show. (i) *Scalar baselines*: both scalar models above were simulated with the same threshold, reset, and noise (noise scaled by $\bar{S}$ for the average model), 200 realizations per amplitude. (ii) *Equal total source*: the width σ was scaled by factors of 0.5 to 2 while β was adjusted to hold the total source $\int \beta S\, dx$ fixed. Any model of the average depends only on this total and therefore predicts no change, so a change isolates the contribution of spatial concentration. (iii) *Synthetic recovery*: to check that β is identifiable before fitting real data, we built a reference table of mean time to first crisis and mean crisis count on a grid of β values (0.0115 to 0.0185 in steps of 0.00025, 100 realizations each), simulated one independent 900 day timeline for each of 60 synthetic patients with known β drawn uniformly from the cohort range, and estimated β as the grid value minimizing the standardized distance between observed and reference features, with the shared parameters held at their calibrated values.

5 Results

5.1 Classification Performance

On the 30 shared features, the Random Forest reached AUC 0.993 ± 0.002, accuracy 98.3 percent, sensitivity 95.6 percent, specificity 100 percent, and Brier score 0.017 (Table 3). Averaged across folds, it identified 86 of 90 TDD patients and all 141 controls. Figure 3 shows a representative fold. The patients it missed were the most sparsely annotated TDD records, carrying one to five shared terms and lacking the full neurological triad, so the misses reflect missing documentation rather than a rejected signature. Logistic regression and the support vector machine reached AUCs of 0.990 and 0.992 on the same features.

5.2 Stress Tests of the Classifier

An AUC near 0.99 with perfect specificity warrants deliberate attempts to break it (Figure 4, Table 4).

Permutation. Across 1,000 label shuffles, the cross validated AUC averaged 0.494 (SD 0.051) and never exceeded 0.689, against 0.993 with true labels ($p = 0.001$, the smallest value attainable with 1,000 permutations). A dataset that is merely small and easy to separate

Table 3: Repeated stratified tenfold cross validation (10 repeats) on the 30 shared features. The corrected interval accounts for overlap between folds.

Model	AUC	Sens.	Spec.	Brier
Random Forest	0.993	95.6%	100%	0.017
Logistic regression	0.990	96.2%	100%	0.013
SVM (radial kernel)	0.992	98.4%	100%	0.010
Random Forest AUC per repeat: 0.992 to 0.996				
Corrected 95% interval for AUC [11]: 0.980 to 1.000				

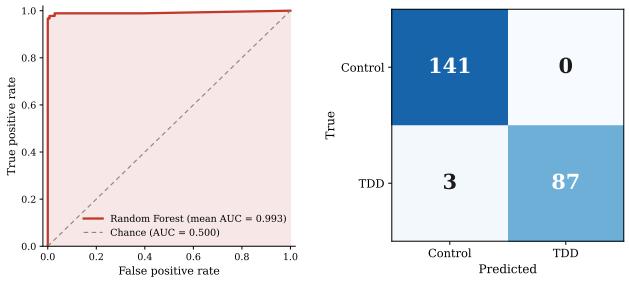


Figure 3: Left: ROC curve (legend reports the repeated cross validation mean AUC). Right: confusion matrix for a representative fold, with 87 of 90 TDD patients and all 141 controls classified correctly.

by chance would produce high AUCs under shuffled labels, and this one does not.

Uncertainty. The 10 repeats gave AUCs from 0.992 to 0.996, so the result is not an artifact of one favorable partition. Because folds overlap, the naive interval understates uncertainty; the corrected interval is wider, 0.980 to 1.000, and we treat it as the honest range.

Feature choices. Changing the frequency threshold to two or five patients gave AUCs of 0.994 and 0.994. A stricter set keeping only terms recorded in at least three patients *within each cohort*, which removes terms the controls' team used only rarely, retained 18 features and gave 0.994. Removing the six most important features left 0.993; removing eight lowered it to 0.980.

Leakage checks. The number of terms per record classifies at AUC 0.569 on its own, near chance, so annotation depth is not the signal. All 231 phenotype sets are unique, with none shared across cohorts.

Specificity. Zero false positives among 141 controls does not imply a zero rate. By the rule of three, the one sided 95 percent upper bound on the false positive rate is about $3/141 \approx 2.1$ percent. A leave one team out validation, suggested by a reviewer, is not possible here because each cohort was annotated by exactly one team, so team and label are identical; the strict vocabulary set above is the closest available check, and Section 8 describes the external test that would settle it.

5.3 Calibration

The Brier score of 0.017 compares with 0.238 for a model that always predicts the base rate, a skill of 0.93. The calibration curve lies

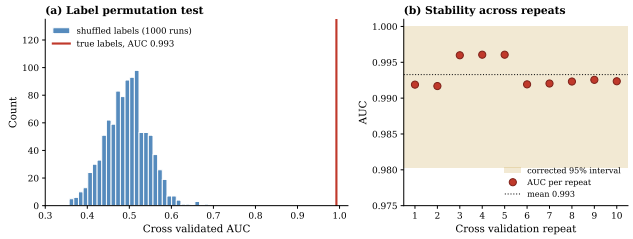


Figure 4: Stress tests. (a) Cross validated AUC under 1,000 label permutations (blue) versus true labels (red). (b) AUC for each of the 10 repeats, with the variance corrected 95 percent interval (shaded).

Table 4: Sensitivity of the classifier to feature construction (mean AUC over 5 repeats for alternative sets).

Feature set	Features	AUC
All 261 candidates (no vocabulary restriction)	261	0.996
Shared vocabulary including ECG term	31	0.994
Primary: shared vocabulary, ECG removed	**30**	**0.993**
Frequency threshold of 2 patients	30	0.994
Frequency threshold of 5 patients	28	0.994
At least 3 patients in *each* cohort	18	0.994
Primary minus 6 most important	24	0.993
Primary minus 8 most important	22	0.980
Record length only	1	0.569

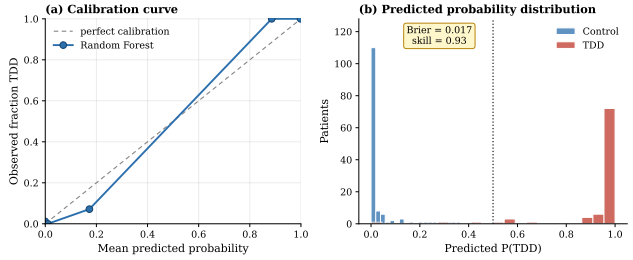


Figure 5: Calibration. (a) Reliability curve, quantile binned. (b) Predicted probabilities by true class.

near the diagonal and predicted probabilities are strongly bimodal (Figure 5), which supports the fixed 0.5 threshold. These probabilities reflect the 39 percent TDD share of this study, not the far lower prevalence of a general clinic, a point we return to in Section 6.

5.4 Candidate Markers and a Neurological Triad

All phenotypes are structured HPO annotations rather than free text. Paroxysmal lethargy (82.2 percent of TDD patients) and compensatory head posture (70.0 percent) were never recorded in the 141 controls. Because only the TDD annotating team used these terms, the reasoning of Section 3.2 applies to them directly: their absence among controls may reflect differences in annotation vocabulary rather than biology. We therefore treat them as hypotheses about specificity that this dataset cannot confirm, to be tested

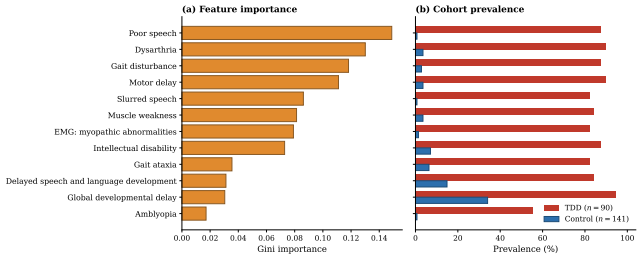


Figure 6: (a) Gini importance of the twelve most informative shared phenotypes. (b) Their prevalence in TDD ($n = 90$) and control ($n = 141$) patients.

Table 5: The ten most informative shared phenotypes.

Baseline phenotype	TDD (%)	Control (%)	Importance
Poor speech	87.8	0.7	0.149
Dysarthria	90.0	3.5	0.130
Gait disturbance	87.8	2.8	0.118
Motor delay	90.0	3.5	0.111
Slurred speech	82.2	0.7	0.086
Muscle weakness	84.4	3.5	0.081
EMG: myopathic abnormalities	82.2	1.4	0.079
Intellectual disability	87.8	7.1	0.073
Gait ataxia	82.2	6.4	0.036
Delayed speech and language	84.4	14.9	0.031

when TDD patients and controls are annotated by the same team (Section 8.2). They are also only moderately sensitive, since head posture is absent in about a third of TDD patients.

The neurological triad is the more sensitive signal. Dysarthria and motor delay each appear in 81 of 90 TDD patients (90.0 percent) versus 3.5 percent of controls, gait disturbance in 87.8 versus 2.8 percent, and intellectual disability in 87.8 versus 7.1 percent (Figure 6, Table 5). Gini importance rewards features that are common in cases and rare in controls, so these phenotypes drive the classifier's decisions, and their combined presence is what the ensemble exploits. Unlike the candidate markers, every one of them was recorded by both annotating teams.

5.5 Episodic Crises and the Critical Amplitude

Simulated trajectories reproduce the qualitative course of the disease: risk accumulates near the vulnerable locus, fluctuates, crosses the threshold, is partially reset, and rebuilds (Figure 7). Over 200 realizations per stratum, a high burden patient ($\beta = 0.0167$) averages 4.79 ± 0.40 crises in 900 days with the first near day 229, a moderate patient ($\beta = 0.0158$) averages 4.00 with the first near day 256, and a mild patient ($\beta = 0.0139$) averages 2.76 ± 0.43 with the first near day 346. Across the 90 patients, severity correlates with simulated crisis count ($r = 0.97$) and inversely with time to first crisis ($r = -0.90$). Because severity sets β by assumption, these correlations describe how the model behaves rather than confirming a clinical relationship.

Figure 8 shows the steady state peak against β. Below β^* no crisis occurs without noise, which is the calibration target of Section 4.4.

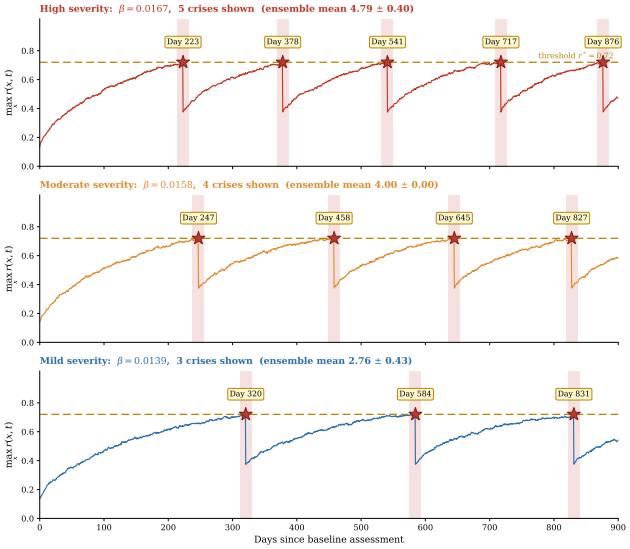


Figure 7: Single stochastic realizations of $\max_x r(x,t)$ for three severity strata, with the ensemble mean and standard deviation over 200 realizations in each title. Stars mark simulated crises and the dashed line is $r^* = 0.72$.

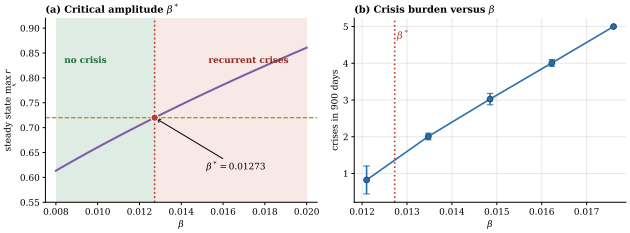


Figure 8: (a) Steady state peak risk versus β, with $\beta^* = 0.01273$ separating quiescent (green) and recurrent (red) regimes. (b) Mean simulated crises in 900 days versus β (bars: SD over 120 realizations).

With noise, a patient at $\beta = 0.0121$ still has at least one crisis in 83 percent of realizations, and each of those realizations has exactly one crisis, so the mean of 0.83 crises in 900 days equals that fraction, compared with 4.79 for a high burden patient. The calibration target is therefore met in the deterministic model, and the stochastic model adds a prediction beyond it: a low baseline burden makes crises rare rather than impossible. Above β^*, crisis count rises smoothly with no chaotic regime, so predictions degrade gradually under error in β.

5.6 What the Spatial Term Adds

Figure 9a compares the PDE with the two scalar baselines under identical parameters. The scalar model of the peak produced 26.9 to 40.9 crises in 900 days, roughly one every three to five weeks for every patient. The scalar model of the average produced 0.0 to 1.4, so even the most severe patients would average only about

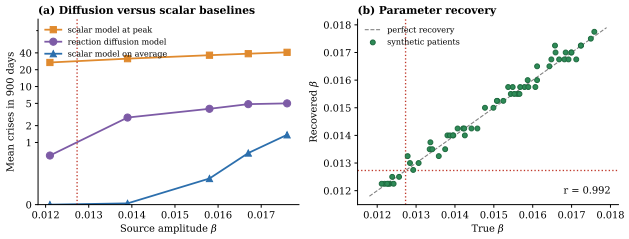


Figure 9: (a) Mean crises in 900 days for the PDE and two scalar models without diffusion, identical parameters, 200 realizations each (dotted line: β^*). (b) Synthetic recovery of β from one simulated 900 day timeline per patient.

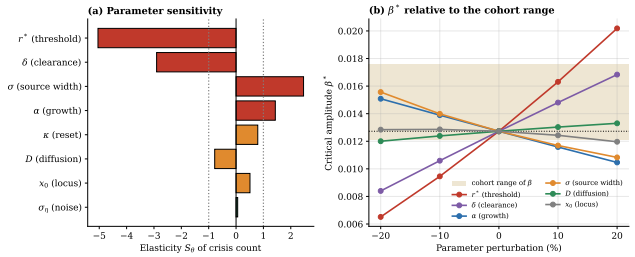


Figure 10: (a) Elasticity of crisis count for each parameter (red sensitive, orange moderate, green robust). (b) Critical amplitude β^* under 10 and 20 percent perturbations of each parameter that affects it; the shaded band is the cohort range of β, so points outside it place the entire cohort on one side of β^*. The reset κ and noise σ_η do not affect β^*.

one crisis every two years. The mildest patients having few or no crises is not the shortcoming here, since the PDE behaves the same way deterministically; the shortcoming is that crises are far too rare across the whole cohort. The PDE produced 0.8 to 5.0, on the timescale of months reported clinically. Neither scalar reduction reproduces that behavior without recalibration, and each gives an essentially all or none answer across the cohort.

The equal total source test isolates the specific contribution of diffusion. Holding $\int \beta S\, dx$ fixed while scaling the width by 0.5, 0.75, 1, 1.5, and 2 changed the PDE's mean crisis count to 4.5, 4.0, 4.0, 3.4, and 3.0. A scalar model of the average cannot register this change, because it sees only the total. The PDE therefore makes a distinct, testable claim: patients with equally large but more concentrated vulnerability should decompensate more often. We note that a scalar model could be recalibrated to place a threshold inside the cohort range; what it cannot represent is this dependence on concentration.

5.7 Identifiability of the Patient Parameter

From a single simulated 900 day timeline per synthetic patient, the recovered β correlated with the true value at $r = 0.992$ (Figure 9b), with mean absolute error 1.6×10^{-4}, median relative error 0.9 percent, and negligible bias. The regime, above or below β^*, was classified correctly for all 60 patients. This shows that β is recoverable from the kind of data a longitudinal study would collect, provided the shared parameters are known; jointly estimating the shared parameters would require data from many patients and is left to future work.

5.8 Numerical Verification and Sensitivity

Both resolutions were refined sixteenfold: the grid from $N = 65$ to 1,040 against a reference of $N = 2{,}080$, and the time step from 0.5 to 0.03125 days against a reference of 0.0039 days. Error fell monotonically at approximately first order in both, with observed temporal orders of 1.01 to 1.10, as expected for a first order time integrator and a first order boundary treatment. With reaction and source removed, total mass was conserved to a relative drift of 1.3×10^{-12}, and the measured amplification factor was 1.000000, matching the proposition.

Figure 10 reports, for each parameter θ, the elasticity of the crisis count n,

$$S_\theta = \frac{\Delta n / n}{\Delta \theta / \theta}. \tag{4}$$

The model is most sensitive to the threshold r^* ($S = -5.0$) and clearance δ ($S = -2.9$), which encode the core assumptions, and nearly insensitive to the noise intensity σ_η ($S = 0.06$). The locus x_0 has a moderate effect on crisis count ($S = 0.51$), but moving it by 10 percent shifts β^* by at most 2.3 percent, so it barely changes which patients fall in each regime. The reset κ leaves β^* exactly unchanged, as it must, since a reset cannot alter a steady state. A critical amplitude exists under every perturbation tested, and crisis count still rises with β, but its position relative to the cohort is sensitive. Increasing r^* by 20 percent raises β^* to 0.0202, above the entire cohort range of 0.0121 to 0.0176, so no patient would cross the threshold; decreasing r^* by 20 percent lowers it to 0.0065, below the entire range, so every patient would. Some 10 percent changes also move β^* below the cohort range, namely lowering r^* or δ or raising α or σ, and lowering D or raising x_0 by 20 percent moves it just below (Figure 10b). Which patients the model classifies as quiescent therefore depends directly on the calibrated values of r^*, δ, α, and σ, and should not be interpreted for individual patients until these are estimated from data.

6 Clinical Application

The classifier suggests a two tier referral rule (Figure 11) that needs no new laboratory infrastructure. **Tier 2** is the operative rule: a child with the neurological triad, led by dysarthria, would receive a genetic panel that includes *TANGO2*. It rests on phenotypes recorded by both annotating teams and catches most cases, because nearly all TDD patients show the triad. **Tier 1** is conditional: a child with developmental delay who shows paroxysmal lethargy or compensatory head posture would be referred directly for *TANGO2* sequencing, but only if the external test of Section 8.2 confirms that these markers remain specific when TDD patients and controls are annotated by the same team. Until then, Tier 1 is a hypothesis and should not be used to route patients. The design reflects an asymmetry: a false positive costs a genetic test, while a missed diagnosis can cost a preventable crisis.

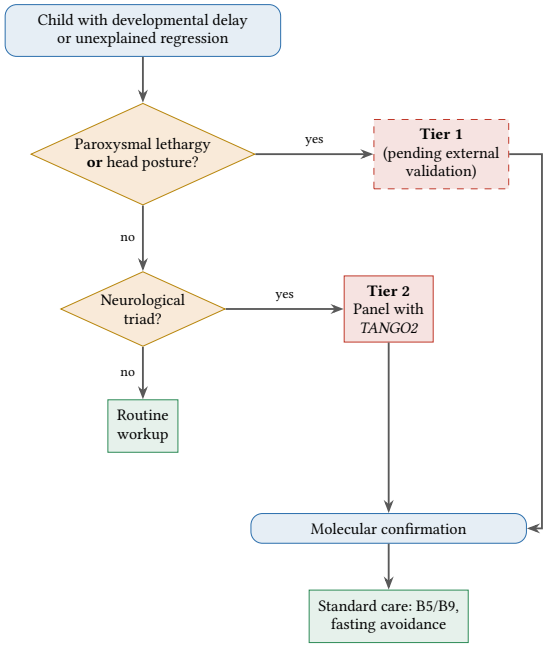


Figure 11: Proposed two tier referral rule for evaluation. Tier 2 uses the neurological triad, recorded by both annotating teams. Tier 1 (dashed) uses the candidate markers and applies only if the external test of Section 8.2 confirms their specificity. Diagnosis always rests on molecular confirmation.

Two cautions shape how the rule should be read. First, predicted probabilities reflect a study in which 39 percent of patients have TDD; in a general clinic the prevalence is far lower. Illustratively, if 1 in 1,000 children with developmental delay had TDD, then even at the 97.9 percent specificity lower bound most positive flags would be false positives. The output is therefore a trigger for confirmatory genetic testing, never a diagnosis, and probabilities would need recalibration to local prevalence. Second, the rule is a proposal derived from cohort statistics. It has been shaped by feedback from collaborating geneticists, but before any clinical use it requires formal review by clinical experts in TANGO2 and prospective evaluation. The PDE component is for research only and is not intended to guide individual care.

7 Ethics and Data Governance

This study is a secondary analysis of deidentified records. The data were provided by Baylor College of Medicine under a data use agreement covering their use for this analysis. No institutional review board approval or exemption determination was obtained for this specific analysis. The data were deidentified by the data custodian before being shared with the author, and the custodian approved publication of the results. The author received only HPO term lists with study identifiers, sex, vital status, and age at evaluation; no names, birth dates, medical record numbers, or contact details were accessed, the author had no access to any key linking records to individuals, and there was no contact with patients or families. Under the U.S. Common Rule, research using information that the investigator cannot readily link to identifiable individuals does not meet the regulatory definition of human subjects research [3]. The original records were collected by the source programs at Baylor College of Medicine and the Undiagnosed Diseases Network under their own institutional review board oversight.

Because the TDD cohort is small and includes deceased individuals, reidentification risk is not zero even for deidentified data. We therefore report only aggregate statistics, describe individual patients only in aggregate terms, and do not release patient level records; the public code runs on user supplied data and contains no patient information.

8 Discussion

The classification result shows that the information needed to suspect TDD is present before the first crisis. This was not guaranteed: the stable phenotype could have been indistinguishable from other developmental disorders, with discriminating information arriving only with the crisis. Instead, a model restricted to shared, precrisis phenotypes separates TDD from a difficult control cohort, the result survives permutation, variance correction, stricter feature sets, ablation, and alternative models, and the signature is spread across many correlated findings rather than resting on a few. The most directly usable outcome is the neurological triad, recorded by both annotating teams, which could prompt earlier genetic testing; the two candidate markers are promising but require external confirmation first.

The PDE result is different in kind. It is a mechanistic hypothesis: episodic crises whose frequency rises with severity can arise from slow, continuous accumulation of risk crossing a threshold, without assuming that crises are random events. The model makes testable predictions, namely a critical amplitude dividing quiescent from recurrent patients, noise induced crises just below that amplitude, and a dependence on how concentrated a patient's vulnerability is, which scalar models cannot express. Whether these predictions hold in patients remains to be determined.

8.1 Scope and Limitations

The classifier was evaluated on 231 specialist referred patients from two cohorts, so performance in an unselected population will be lower and requires prospective confirmation. Because each cohort was annotated by one team, annotation style and diagnosis cannot be fully separated in this data, even with the shared vocabulary safeguards. This applies most strongly to the two candidate markers, which only one team recorded, and in weaker form to shared terms with extreme prevalence contrasts such as poor speech (87.8 percent of TDD patients versus 0.7 percent of controls): both teams used the term, but a difference in how often each team recorded it could still widen the contrast. The position of β^* relative to the cohort also depends on calibrated parameters (Section 5.8). The PDE uses an abstract coordinate, a linear severity mapping, and parameters calibrated to qualitative patterns rather than fitted to crisis dates, so its simulated timelines illustrate mechanisms rather than forecast individual patients. The recovery test establishes identifiability of β only when the shared parameters are known.

8.2 Future Work

The most decisive next test is external: TDD patients diagnosed within the UDN are annotated by the same team as the controls, so classifying them would directly separate disease signal from annotation style. It would also test whether paroxysmal lethargy and compensatory head posture remain specific when recorded by the same team as the controls, which is the condition for adopting Tier 1. Prospective use of the referral rule at first developmental assessment would measure how many cases are found before a first crisis. For the PDE, longitudinal crisis dates would allow each β_i to be fitted with the recovery procedure validated here, the model to be scored against held out crises, and the prediction that B complex supplementation lowers effective β to be tested directly.

9 Conclusion

Restricting a Random Forest to phenotypes observable before any metabolic crisis, and to vocabulary shared by both annotating teams, separated TANGO2 deficiency from a phenotypically overlapping cohort with an AUC of 0.993 (corrected interval 0.980 to 1.000), a result that withstood a 1,000 shuffle permutation test, stricter feature sets, and alternative models. A highly prevalent neurological triad recorded by both annotating teams suggests a practical referral rule for prospective evaluation, and two candidate markers recorded only in TDD patients are hypotheses awaiting an external test. A stochastic reaction diffusion model, verified numerically, reproduces the episodic course of the disease, identifies a critical amplitude separating quiescent and recurrent regimes, captures an effect of vulnerability concentration that scalar models cannot represent, and has an identifiable patient parameter, preparing it for testing against longitudinal data.

Code Availability

Analysis code and a research prototype interface are available at https://github.com/abhaytippi/TANGO2-Forecast. The prototype is intended for research and education, has not been validated for clinical use, and must not be used to make decisions about individual patients.

Acknowledgments

The author thanks Dr. Seema Lalani and Dr. Zhandong Liu of Baylor College of Medicine for providing the deidentified cohorts, for clinical and scientific guidance, and for mentorship, and thanks the reviewers, whose suggestions led to the permutation, baseline, and recovery analyses reported here.

References

[1] U. M. Ascher, S. J. Ruuth, and B. T. R. Wetton. 1995. Implicit explicit methods for time dependent partial differential equations. *SIAM Journal on Numerical Analysis* 32, 3 (1995), 797–823.
[2] L. Breiman. 2001. Random forests. *Machine Learning* 45, 1 (2001), 5–32.
[3] U.S. Department of Health and Human Services. 2018. Protection of Human Subjects, 45 CFR 46.102, Definitions. *Code of Federal Regulations.*
[4] J. N. Dines, K. Golden-Grant, A. LaCroix, et al. 2019. TANGO2: expanding the clinical phenotype and spectrum of pathogenic variants. *Genetics in Medicine* 21, 3 (2019), 601–607.
[5] R. A. Fisher. 1937. The wave of advance of advantageous genes. *Annals of Eugenics* 7, 4 (1937), 355–369.
[6] S. Köhler, L. Carmody, N. Vasilevsky, et al. 2019. Expansion of the Human Phenotype Ontology (HPO) knowledge base and resources. *Nucleic Acids Research* 47, D1 (2019), D1018–D1027.
[7] A. N. Kolmogorov, I. G. Petrovsky, and N. S. Piskunov. 1937. A study of the diffusion equation with increase in the amount of substance. *Bulletin of Moscow University, Mathematics Series A* 1 (1937), 1–25.
[8] L. S. Kremer, F. Distelmaier, B. Alhaddad, et al. 2016. Biallelic truncating mutations in TANGO2 cause infancy onset recurrent metabolic crises with encephalocardiomyopathy. *American Journal of Human Genetics* 98, 2 (2016), 358–362.
[9] S. R. Lalani, P. Liu, J. A. Rosenfeld, et al. 2016. Recurrent muscle weakness with rhabdomyolysis, metabolic crises, and cardiac arrhythmia due to biallelic TANGO2 mutations. *American Journal of Human Genetics* 98, 2 (2016), 347–357.
[10] J. D. Murray. 2002. *Mathematical Biology I: An Introduction* (3rd ed.). Springer.
[11] C. Nadeau and Y. Bengio. 2003. Inference for the generalization error. *Machine Learning* 52, 3 (2003), 239–281.
[12] M. Ojala and G. C. Garriga. 2010. Permutation tests for studying classifier performance. *Journal of Machine Learning Research* 11 (2010), 1833–1863.
[13] F. Pedregosa, G. Varoquaux, A. Gramfort, et al. 2011. Scikit learn: Machine learning in Python. *Journal of Machine Learning Research* 12 (2011), 2825–2830.
[14] A. Quarteroni, R. Sacco, and F. Saleri. 2007. *Numerical Mathematics* (2nd ed.). Springer.
[15] R. B. Ramoni, J. J. Mulvihill, D. R. Adams, et al. 2017. The Undiagnosed Diseases Network: accelerating discovery about health and disease. *American Journal of Human Genetics* 100, 2 (2017), 185–192.

Investigating the Efficacy of the Gemini 2.5 Pro Model in the Estimation of Macronutrient Content

Sonali Guttikonda
University High School
Carmel, IN, USA
sonali.gutti@gmail.com

Junyao Chen
NYU Stern, New York University
New York, NY, USA
junyao.chen@nyu.edu

He Tu
New York University Shanghai
Shanghai, China
ht2604@nyu.edu

Zhengyuan Zhou
NYU Stern, New York University
New York, NY, USA
zz26@stern.nyu.edu

Abstract

Health guidance is a topic that has been gaining increasing traction in the past few years. Specifically, apps utilizing AI to provide health guidance and allow users to track macronutrient content have been gaining prevalence in recent research. However, current apps require users to manually input all information into the app. Therefore, this work seeks to evaluate the effectiveness of the Gemini 2.5 Pro model, a multimodal large language model (LLM), in analyzing the macronutrient content (i.e., proteins, fats, and carbohydrates) in food images. We tested 100 dishes randomly sampled from the Nutrition5k dataset. Each image was queried three times with three different prompt phrasings and the estimated values were compared with the ground truth values in the dataset. For the baseline prompt, the mean absolute error (MAE) was 4.1 g for protein, 3.6 g for fat, and 6.1 g for carbohydrates, and the median absolute percentage errors were 29.9%, 29.2%, and 32.1% respectively. The mean absolute percentage error (MAPE) was 38.5% for protein but 211.2% for fats and 442.7% for carbohydrates, which were inflated due to a small number of dishes containing less than 0.5 g of fat or carbohydrate, where errors of a few grams produced errors in the thousands. Without dishes with less than 2 g of the nutrient, the MAPE was 38.5%, 33.6%, and 33.5%. On metrics not sensitive to near-zero values, error was of similar size across the three macronutrients, and protein was underestimated on average, with the largest errors concentrated in a subset of dishes. MAE differed by at most 0.6 g across prompt wordings, although fat MdAPE was higher under both alternative prompts. Because no confidence intervals or significance tests were computed, these are observations from a single 100-dish sample rather than established differences. Additionally, the 100 dishes selected for the study came from a 400-dish subset whose selection cannot be reproduced.

Keywords

Multimodal Large Language Models, Gemini 2.5 Pro, Macronutrient Estimation, Food Image Analysis, Nutrition5k, Prompt Engineering

This paper is published under the Creative Commons Attribution-NonCommercial-NoDerivs 4.0 International (CC-BY-NC-ND 4.0) license. Authors reserve their rights to disseminate the work on their personal and corporate Web sites with the appropriate attribution.
IJSCAR VOL. 3, ISSUE 4,
© 2026 The Author(s). Published by IJSCAR.
https://doi.org/10.67149/yhjs2024.5/c6ey9odw

ACM Reference Format:
Sonali Guttikonda, Junyao Chen, He Tu, and Zhengyuan Zhou. 2026. Investigating the Efficacy of the Gemini 2.5 Pro Model in the Estimation of Macronutrient Content. In *Proceedings of International Journal of Secondary Computing and Applications Research (IJSCAR VOL. 3, ISSUE 4).* ACM, New York, NY, USA, 6 pages. https://doi.org/10.67149/yhjs2024.5/c6ey9odw

1 Introduction

Artificial Intelligence has gained increasing prevalence in the past few years in multiple aspects such as academic research and everyday tasks. Particularly, the use of AI models for estimating nutritional and mass information about a meal is a topic that has been increasingly explored in the past few years. This research is significant for combating widespread misinformation about nutrition online as well as providing accurate guidance for users seeking a healthier lifestyle. Recently, researchers have explored utilizing AI to estimate nutritional information based on a single image as opposed to current applications that require users to manually input food data. Previous research in this area has utilized the YOLO computer vision models. YOLO models belong to a family of object detection models that use single-stage detection. YOLO models simply localize and detect images without any language reasoning; therefore our research primarily focuses on utilizing a Gemini 2.5 Pro model, which is a multimodal model that is designed for language reasoning and image analysis. Additionally, we utilized the Nutrition5k dataset, which is a dataset containing images of about 5,000 dishes with different types of food along with their nutritional information, along with the Gemini model. We evaluate how accurately Gemini estimates per-dish macronutrient content from a single image, using the Nutrition5k dataset as the ground truth.

2 Related Work

2.1 Large Language Models (LLMs) for Macronutrient Estimation

Recent studies have tested LLMs for use in nutrition estimation with both text and image inputs. For text input, Carrillo-Larco utilized a fine-tuned LLM in order to estimate energy and macronutrient content from 24-hour dietary recalls with chain-of-thought prompting [1]. Additionally, Lawabni et al. found a high similarity in reference values between four LLMs (ChatGPT 5.2, Claude Opus 4.5, Gemini 3 Pro Preview, and Llama 4 Maverick) for carbohydrates,

protein, and fat content [7]. Utilizing photographs for nutrient estimation is more difficult since models must recognize the foods and interpret portion sizes. Fridolfsson et al. compared three different LLMs on food images: GPT-4o, Claude 3.5 Sonnet, and Gemini 1.5 Pro. GPT-4o and Claude had MAPEs of 36% and 37.3%, while Gemini had a significantly higher MAPE [3]. Certain studies specifically used the Nutrition5k dataset. For example, Khlaisamniang et al. first described the dish and then estimated nutrient content, with MAEs of 7.38 g for protein, 5.62 g for fat, and 7.76 g for carbohydrates using the Gemini 1.5 Pro model [6]. Additionally, Yan et al. utilized their DietAI24 framework and reduced MAE by 63% [11]. Our work evaluates the more updated Gemini 2.5 Pro model with three different single-step prompts on the Nutrition5k dataset to test how prompt variation and variability affect macronutrient estimation accuracy.

2.2 YOLO Models for Macronutrient Estimation

Another type of work dedicated to macronutrient estimation utilizes object detection models, specifically in the YOLO family. These models localize foods within a single pass and then compare them to nutrition values found in a database. Kalaimani et al. used the YOLOv8 and YOLOv10 models and reported accuracies around 95–99% for proteins, fats, and carbohydrates [5]. Matan et al. used YOLOv8 detection alongside a nutrition database, resulting in portion-aware estimates with an MAE of 4.5 g [8]. Dhanush et al. used the YOLOv11 model with the Segment Anything Model in order to isolate macronutrients and micronutrients from dishes [2]. Selvaraju et al. used YOLOv5, YOLOv8, and Mask R-CNN models in conjunction, resulting in a precision range of 0.95–0.97 [9]. All of these studies utilized models specifically designed for detection as compared with a multimodal LLM, which can identify foods as well as reason about their portions without specific training. Additionally, they utilize different comparison metrics, so they cannot be compared directly to our study's results.

3 Methodology

3.1 Dataset and Sampling

All of the data was primarily collected from one dataset, the Nutrition5k dataset, which has both photographs of diverse dishes and information on their macronutrient content. The images and ground-truth metadata were selected by Junyao Chen as a subset of the Nutrition5k dataset. This subset contained 400 randomly sampled overhead dish images from cafeteria 1 with matching nutrition labels. The seed used to select the 400-dish subset was not recorded. Ground truth values for each dish (total mass, protein, fat, and carbohydrates, in grams) were taken from the Nutrition5k dish metadata for cafeteria 1 (`dish_metadata_cafe1`) [4, 10]. Only the subset was used due to the API cost and time constraints of the study. Of these 400 dish images we used a random sample of 100 images with a fixed random seed (42) in order for our sample to be unaffected by dish ordering in the dataset.

3.2 Model Pipeline

This research primarily utilized a Gemini 2.5 Pro model with images from the Nutrition5k dataset as the input. The images were first resized to 1024 px by 1024 px and input to the Gemini 2.5 Pro model through the Google Gen AI Python SDK, with a prompt requesting a JSON object with total mass, protein, fat, and carbohydrates in grams alongside a list of visible ingredients. The temperature was set to 1.0, which is the API default. Since the LLM outputs varied between calls, each image was queried three times per prompt and these estimates were averaged. Any responses that were not valid were re-requested with up to three attempts. However, all 100 images returned valid predictions for every prompt.

3.3 Prompt Variants

In order to determine whether prompt wording affected the accuracy of the model, we compared three different prompts that requested the same output: baseline, stepwise, and diet app. The baseline asked directly for the estimates. The stepwise prompt asked the model to identify each ingredient and portion size prior to estimating nutrient content. Finally, the diet-app prompt framed the task as if the model was logging a meal in a diet-tracking app. The full prompts are shown in Figure 1.

3.4 Error Calculation

Across all dishes, we calculated the mean absolute error (MAE), the mean signed error, the mean absolute percentage error (MAPE), the median absolute percentage error (MdAPE), and the symmetric mean absolute percentage error (sMAPE) using the following formulas:

$$\text{MAE} = \frac{1}{N}\sum_{i=1}^{N}|E_i - T_i| \tag{1}$$

$$\text{Mean signed error} = \frac{1}{N}\sum_{i=1}^{N}(E_i - T_i) \tag{2}$$

$$\text{MAPE} = \frac{100}{N}\sum_{i=1}^{N}\frac{|E_i - T_i|}{T_i} \tag{3}$$

$$\text{MdAPE} = \text{median}_i\left(100 \cdot \frac{|E_i - T_i|}{T_i}\right) \tag{4}$$

$$\text{sMAPE} = \frac{100}{N}\sum_{i=1}^{N}\frac{2\,|E_i - T_i|}{|E_i| + |T_i|} \tag{5}$$

Here i is the dish index, E_i is the estimate, T_i is the true value in grams, and N is the total number of dishes. Negative signed error indicates underestimation. Dishes with $T_i = 0$ are excluded from MAPE and MdAPE; the number excluded was 0 for total mass, 1 for protein, 1 for fat, and 2 for carbohydrates. Since MAPE becomes very large when T_i is close to zero, we also report MAPE computed over dishes with $T_i \geq 2$ g. To measure run-to-run variability, we computed the standard deviation of the three repeated estimates for each dish and report the mean of these values across dishes.

4 Results

We evaluated Gemini 2.5 Pro on 100 randomly sampled dishes (fixed seed 42) from the 400 available Nutrition5k images. Each image was queried three times per prompt at temperature 1.0, and the three estimates were averaged. All 100 images returned valid predictions for every prompt.

```
JSON_FORMAT = (
    '{"total_mass_g": 0, "protein_g": 0, "fat_g": 0, "carbohydrates_g": 0, '
    '"ingredients": ["name1", "name2"]}\n'
    "total_mass_g is total food mass in grams.\n"
    "protein_g is total protein content in grams.\n"
    "fat_g is total fat content in grams.\n"
    "carbohydrates_g is total carbohydrate content in grams.\n"
    "ingredients is a list of visible ingredient names."
)
PROMPTS = {
    "baseline": "Look at this food image. Reply with JSON only:\n" + JSON_FORMAT,
    "stepwise": (
        "Look at this food image. First identify each visible ingredient and estimate "
        "its portion size, then add them up to get the totals. Reply with JSON only:\n"
        + JSON_FORMAT
    ),
    "diet_app": (
        "Look at this food image and estimate its nutrition as accurately as possible, "
        "as if logging it in a diet-tracking app. Reply with JSON only:\n" + JSON_FORMAT
    ),
}
```

Figure 1: Three different prompts input into Gemini 2.5 Pro for macronutrient estimation.

Table 1: Estimation error metrics of Gemini 2.5 Pro on 100 randomly sampled Nutrition5k dishes (with baseline prompt and three queries per image, averaged). MAE = mean absolute error. Bias = mean signed error (estimate − true); negative values indicate underestimation. MdAPE = median absolute percentage error. sMAPE = symmetric mean absolute percentage error. MAPE = mean absolute percentage error, over dishes with a true value above 0 g; "MAPE, dishes ≥ 2 g only" excludes dishes with a true value below 2 g. Repeat SD = mean standard deviation of the three repeated estimates per dish. Dishes = 0 g (excluded) = number of dishes with 0 g of each nutrient, excluded from the MAPE calculation.

Metric	Mass	Protein	Fat	Carbs
MAE (g)	24.2	4.1	3.6	6.1
Bias (g)	−0.3	−3.2	−0.9	+1.2
MdAPE (%)	20.4	29.9	29.2	32.1
sMAPE (%)	24.2	43.5	46.6	45.7
MAPE (%)	23.8	38.5	211.2	442.7
MAPE, dishes ≥ 2 g only (%)	23.0	38.5	33.6	33.5
Dishes with true value < 2 g	1	42	55	18
Repeat SD (g)	8.93	0.71	1.11	1.60
Dishes = 0 g (excluded)	0	1	1	2

Overall accuracy. Table 1 shows the error for the baseline prompt. The mean absolute error (MAE) was 4.1 g for protein, 3.6 g for fat, 6.1 g for carbohydrates, and 24.2 g for total dish mass. Median absolute percentage errors (MdAPE) were similar for the three macronutrients (29.2–32.1%), as were the symmetric MAPE values (43.5–46.6%).

Sensitivity of MAPE.. Standard MAPE was substantially higher for fat (211.2%) and carbohydrates (442.7%) than for protein (38.5%). This difference does not come from the number of low-value dishes alone. Protein had 42 dishes below 2 g (one of which had a true value of 0 g and was excluded from MAPE, leaving 41) compared with 55 for fat and only 18 for carbohydrates. Instead, it depends on how close those dishes are to zero and how large the model's errors on them are. The low-protein dishes had a median true value of 0.56 g, and the model's estimates stayed close (median absolute error 0.18 g; largest 1.7 g). Their MAPE (38.7%) was therefore almost identical to that of other dishes, so removing them left protein MAPE at 38.5%. In contrast, most low-fat dishes contained less than 0.5 g of fat, and the ten largest fat errors (0.8–5.2 g) produced very large percentage errors. Excluding dishes below 2 g, MAPE fell to 33.6% for fat and 33.5% for carbohydrates, similar to protein (38.5%). We therefore rely on MAE, MdAPE, and sMAPE for comparisons across nutrients. On these metrics, we did not observe a difference suggesting that fat is estimated less accurately than protein or carbohydrates, although our analysis does not formally test this.

Table 2: Effect of prompt wording on estimation error (100 dishes, three queries per image averaged). MAE = mean absolute error. MdAPE = median absolute percentage error. sMAPE = symmetric mean absolute percentage error.

Nutrient	Prompt	MAE (g)	MdAPE (%)	sMAPE (%)
Total mass	Baseline	24.2	20.4	24.2
	Stepwise	23.9	18.5	24.6
	Diet app	23.3	17.1	24.8
Protein	Baseline	4.1	29.9	43.5
	Stepwise	4.0	26.5	43.3
	Diet app	4.0	23.0	43.3
Fat	Baseline	3.6	29.2	46.6
	Stepwise	3.8	37.0	50.1
	Diet app	3.7	34.4	56.0
Carbohydrates	Baseline	6.1	32.1	45.7
	Stepwise	5.9	28.5	43.7
	Diet app	5.5	27.5	42.2

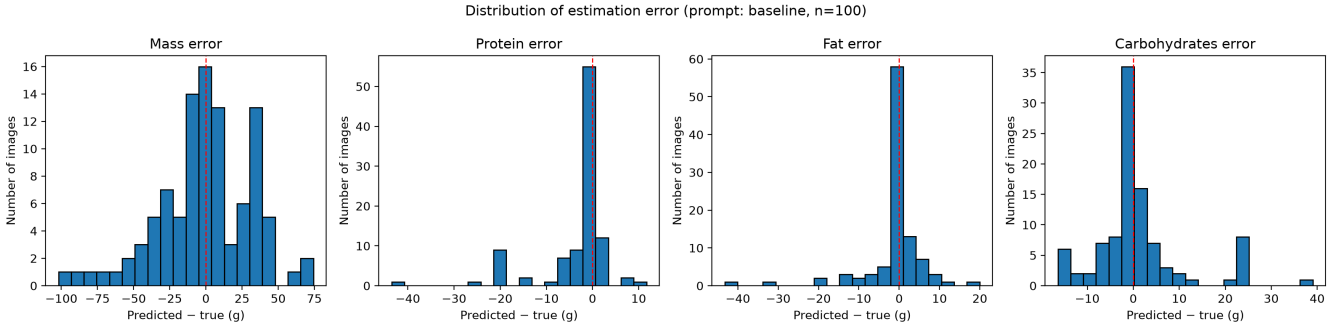


Figure 2: Distribution of signed estimation error (predicted − true, in grams) for total mass, protein, fat, and carbohydrates under the baseline prompt ($n = 100$ dishes).

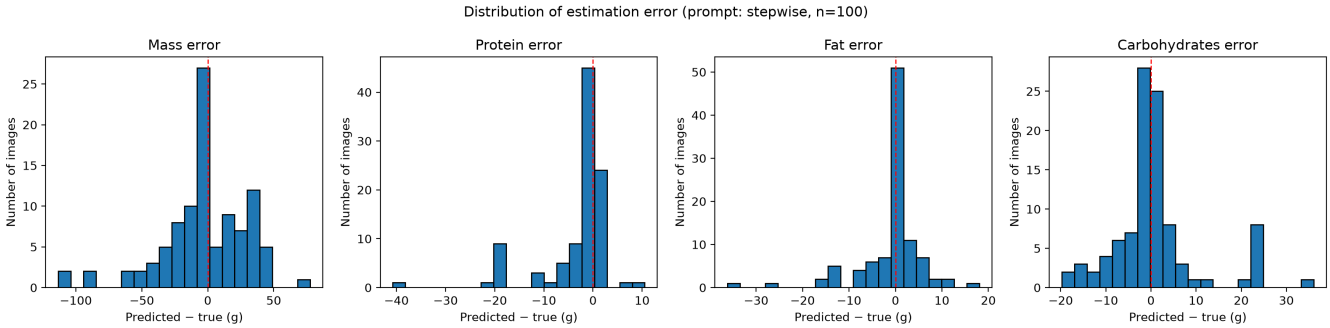


Figure 3: Distribution of signed estimation error (predicted − true, in grams) for total mass, protein, fat, and carbohydrates under the stepwise prompt ($n = 100$ dishes).

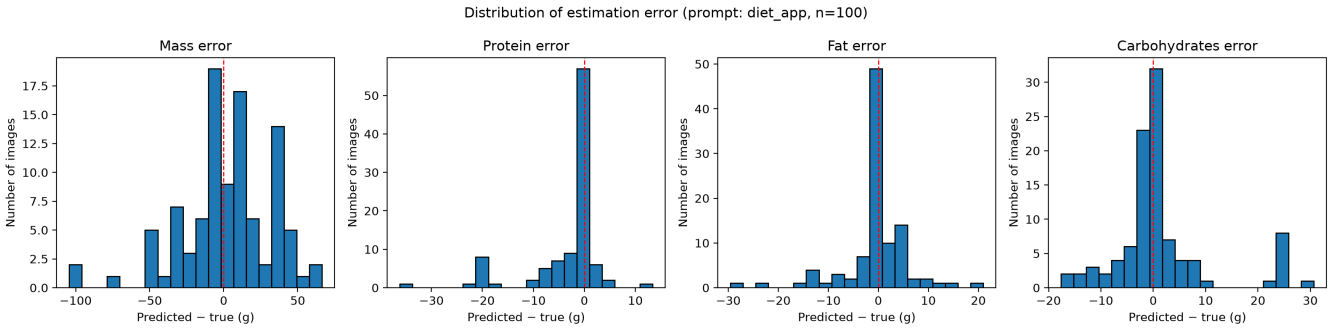


Figure 4: Distribution of signed estimation error (predicted − true, in grams) for total mass, protein, fat, and carbohydrates under the diet-app prompt ($n = 100$ dishes).

Bias. Protein was underestimated on average, with a bias of −3.2 g against an MAE of 4.1 g. This net underestimation is not a uniform shift: Figure 2 shows most protein errors near zero, with a left tail of dishes underestimated by roughly 10–40 g that accounts for much of the negative bias. The mean signed errors for fat (−0.9 g), carbohydrates (+1.2 g), and total mass (−0.3 g) were close to zero.

Run variability. The mean standard deviation across queries of the same image was 0.71 g for protein, 1.11 g for fat, 1.60 g for carbohydrates, and 8.93 g for mass, or roughly 17–37% of the respective MAEs. It was the smallest for protein (17%) and the largest for total mass (37%). Run-to-run variability is therefore a relatively meaningful share of the total error, which supports averaging multiple queries per image.

Prompt wording. Table 2 compares results from the three prompt phrasings. At most, MAE differed by 0.6 g for each macronutrient across the prompts, and no prompt was the best across all nutrients. For the baseline prompt, Figure 2 shows that most errors for each macronutrient fall close to zero. The protein underestimation appears as a long left tail of dishes underestimated by 10–40 g. Additionally, the distribution for the diet-app prompt (Figure 4) closely

Table 3: MAE on Nutrition5k compared with Khlaisamniang et al. The two studies used different test subsets (Nutrition320 vs. our 100 dishes), so the values are not directly comparable.

			MAE (g)		
Study	Model	Approach	Protein	Fat	Carbs
Khlaisamniang [6]	Gemini 1.5 Pro	Two-step	7.38	5.62	7.76
This study	Gemini 2.5 Pro	One-step	4.1	3.6	6.1

resembled that of the baseline prompt. The diet-app framing gave the lowest MdAPE for mass (17.1%), protein (23.0%), and carbohydrates (27.5%), but a higher one for fat (34.4%). Protein bias was very similar for all three prompts (−3.1 to −3.3 g). However, fat MdAPE was higher under both alternative prompts (stepwise 37.0%, diet app 34.4%) than under the baseline (29.2%). Since we did not compute confidence intervals or test these differences, we report them as observed differences on this sample.

Comparison with prior work. Table 3 compares these results with that of Khlaisamniang et al. [6], who used Gemini 1.5 Pro with a two-step approach on Nutrition320, a different 320-dish subset of Nutrition5k. Our MAEs were lower for all three macronutrients. However, the two studies differ in model version, prompting approach, and test dishes, so the comparison is indicative only.

5 Discussion

Our hypothesis was that Gemini 2.5 Pro would have similar accuracy across all three macronutrients. The observed errors are consistent with this, although we did not test it statistically. For measures that were not sensitive to small true values, the three nutrients were similar: MAE was 3.6–6.1 g, MdAPE was 29.2–32.1%, and sMAPE was 43.5–46.6%.

The high MAPE values for fat and carbohydrates are driven largely by dishes with near-zero true values. MAPE becomes very large when a dish's true value is less than a gram and the estimate is off by a few grams. Most low-fat dishes had less than 0.5 g of fat, and seven low-carbohydrate dishes were estimated to contain about 25 g instead of 0.43 g; these seven dishes account for a large share of the carbohydrate MAPE. Errors this large on dishes with low carbohydrate content could reflect model error, but they could also reflect errors in ground-truth labels. On inspection, the images show plain protein dishes with no visible carbohydrate source, so these appear to be genuine model overestimates. Low-protein dishes, by contrast, were mainly estimated within less than a gram, so the percentage errors stayed moderate. Therefore, MAPE should be interpreted carefully when used with nutrition datasets with many near-zero values, and should be reported alongside MdAPE and sMAPE values.

Protein was underestimated on average under all three prompt wordings (bias of −3.2 g for the baseline prompt and −3.1 to −3.3 g across prompts). The histograms show that this bias is concentrated in a tail of dishes underestimated by 10–40 g rather than a uniform shift across all dishes. Total mass had a mean signed error near zero (−0.3 g), which indicates that over- and underestimates of portion size roughly cancel on average; it does not by itself show that individual portions are judged accurately, since mass MAE was 24.2 g (MdAPE 20.4%). Therefore, future work could involve examining the types of dishes that drive this bias (e.g., whether dishes contain ingredients like meat or eggs).

The three prompt wordings gave similar MAEs, differing by at most 0.6 g per nutrient. Fat MdAPE, however, was higher under both alternative prompts than under the baseline, so prompt wording may affect some nutrients more than others; a larger sample with significance testing would be needed to tell. Run-to-run variability was present but less than the overall error. The mean spread across repeats was 0.7–1.6 g for the macronutrients, or 17–31% of their MAEs (37% for total mass). This supports averaging three queries per image.

Our MAEs using Gemini 2.5 Pro were lower than those reported by Khlaisamniang et al. with Gemini 1.5 Pro on the same dataset. However, the studies differ in model version, prompt design, and test subset (their Nutrition320 versus our 100 dishes), and any of these could account for the difference.

One limitation of this study was the sample size: we only sampled 100 dishes from a single subset due to time constraints. In addition, the 100 dishes used came from a 400-dish subset whose selection cannot be reproduced. Additionally, only one model (Gemini 2.5 Pro) was tested. The 2 g threshold for fat and carbohydrates was a practical choice, not a standard one. We also did not test whether the differences between prompts were statistically significant.

6 Conclusion

In this 100-dish sample, Gemini 2.5 Pro's estimation error was of similar size across the three macronutrients; protein was underestimated on average, with most of that error concentrated in a subset of dishes, and changing the prompt wording changed MAE by at most 0.6 g, though fat MdAPE rose under both alternative prompts. These are observations rather than statistically tested conclusions. In a further continuation of the study, we could analyze how effective the Gemini 2.5 Pro model is in predicting the content of more types of nutrients in food images, as well as identify the types of foods on which Gemini had poor estimation performance. To do this, we could choose a dataset that has information on more types of nutrients. Additionally, we could use pattern-recognition methods to identify the types of food that the Gemini 2.5 Pro model analyzed poorly.

Acknowledgments

We would like to thank Catherine Tissot, Matthew Leingang, Wynnian Yang, and the entire NYU GSTEM Program for supporting us on this project.

Data Availability

The Nutrition5k images and dish metadata used in this study are publicly available at https://github.com/google-research-datasets/Nutrition5k [4]. The dish IDs of the 400-dish subset and of the 100 dishes evaluated are unavailable to readers.

References

[1] Carrillo-Larco, R.M.: LLMs for energy and macronutrients estimation using only text data from 24-hour dietary recalls: A parameter-efficient

fine-tuning experiment using a 10-shot prompt. arXiv:2509.13268 (2025). https://doi.org/10.48550/arXiv.2509.13268

[2] Dhanush, R., Abhilash, G.M., M, E., C, A.S., G, S.H.: NutriVision: YOLO-SAM-driven dietary app for personalized food recommendation and meal tracking. In: 2025 International Conference on Innovative Trends in Information Technology (ICITIIT). pp. 1–6 (2025). https://doi.org/10.1109/ICITIIT64777.2025.11040294

[3] Fridolfsson, J., Sjöberg, E., Thiwång, M., Pettersson, S.: Performance evaluation of 3 large language models for nutritional content estimation from food images. Current Developments in Nutrition **9**(10), 107556 (2025). https://doi.org/10.1016/j.cdnut.2025.107556

[4] Google Research: Nutrition5k dataset. GitHub repository, https://github.com/google-research-datasets/Nutrition5k (2021)

[5] Kalaimani, M., D, A.S., M, D.S., Santhoshkumar, S.P.: Smart food recognition and nutrition estimation using computer vision. In: 2026 9th International Conference on Inventive Computation Technologies (ICICT). pp. 1198–1204 (2026). https://doi.org/10.1109/ICICT68280.2026.11510899

[6] Khlaisamniang, P., Kerdthaisong, K., Vorathammathorn, S., Yongsatianchot, N., Phimsiri, H., Chinkamol, A., Thitseesaeng, T., Veerakanjana, K., Kachai, K., Ittichaiwong, P., Saengja, T.: Decomposing food images for better nutrition analysis: A nutritionist-inspired two-step multimodal LLM approach. In: 2025 IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW). pp. 482–491 (2025). https://doi.org/10.1109/CVPRW67362.2025.00053

[7] Lawabni, R., Solano, O.R., Kampen, L.L., Kurasch, J., Alshurafa, N., Chang, J.S., Evans, R., Feng, D., Lin, A.W.: Evaluation of energy and nutrient estimates from large language models using text-based queries. The Journal of Nutrition **156**(9), 101712 (2026). https://doi.org/10.1016/j.tjnut.2026.101712

[8] Matan, P., Thulasiram, M., Vardhan, D.H., Reddy, V.R.V.K.: Precision dietary recommendation using YOLOv8 food detection and optimization based nutritional planning. In: 2026 Third International Conference on Innovations in Cybersecurity and Data Science (ICICDS). pp. 1334–1339 (2026). https://doi.org/10.1109/ICICDS70526.2026.11604686

[9] Selvaraju, T., Dakshinamurthi, V., Badurudeen, G., Prabhakaran, N., Gopi, P., Hussain, M.A.N.: Food detection and estimation of calories and other macronutrients and features to support healthy lifestyle. In: AIP Conference Proceedings. vol. 3279, p. 020129 (2025). https://doi.org/10.1063/5.0263063

[10] Thames, Q., Karpur, A., Norris, W., Xia, F., Panait, L., Weyand, T., Sim, J.: Nutrition5k: Towards automatic nutritional understanding of generic food. In: 2021 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). pp. 8903–8911 (2021)

[11] Yan, R., Luo, H., Lu, J., Liu, D., Posluszny, H., Dhaliwal, M.P., MacLeod, J., Qin, Y., Yang, C., Hartman, T.J., Hu, X.: DietAI24 as a framework for comprehensive nutrition estimation using multimodal large language models. Communications Medicine **5**(1), 458 (2025). https://doi.org/10.1038/s43856-025-01159-0

Analysis of Methods to Improve Underwater Object Detection in Various Environments

Nico Katz
La Jolla Country Day School
San Diego, California, USA
1npk412@gmail.com

Abstract

We evaluate how single-augmentation and aggregate-augmentation training affect the robustness of a Faster R-CNN detector on an underwater target-detection task, and we assess a physics-based dehazing stage built on ten synthetic attenuation levels ordered as the Jerlov water types are. Seven detectors, one of them a control, were tested on a clean split, a composite degradation, each degradation alone, and four degradations held out of every training policy. All seven fall within 0.805–0.841 AP on clean data, so clean validation does not discriminate between augmentation policies. The aggregate model's lead over the control is the same on trained-on and held-out degradations (0.026 AP each) and reappeared in two retrained seeds per policy (0.019 and 0.021 AP), though two seeds cannot establish reproducibility; its composite-condition gap (0.186 AP) did not reproduce on retraining (0.406 and 0.412). The aggregate is not the best of the seven on held-out degradations, an ordering that includes checkpoints predating per-worker data-loader seeding, and its advantage is concentrated in two blur-like conditions on the validation split (83 %) but not on a held-back 97-image partition (44 %). The physics inversion alone raises AP from 0.691 to 0.819 across three conditions and beats a Dark Channel Prior baseline, whereas adding contrast enhancement and white balance lowers it to 0.680; this comparison favors our method by construction, since the inversion is given the true attenuation level and path length, the frames are generated by the equation it inverts, and the coefficients are our own. Across the full sweep, which used the three-stage pipeline, correction gains 0.197 AP in coastal water and costs 0.117 AP in open ocean. Augmentation and correction approximately add for the aggregate policy but overlap for the other five. Clean validation performance may not predict robustness under degraded conditions.

Keywords

Computer Vision, Underwater, Jerlov Water Types, Data Augmentations

ACM Reference Format:

Nico Katz. 2026. Analysis of Methods to Improve Underwater Object Detection in Various Environments. In *Proceedings of International Journal of Secondary Computing and Applications Research (IJSCAR VOL. 3, ISSUE 4)*. ACM, New York, NY, USA, 13 pages. https://doi.org/10.67149/yhjs2024.5/myab98j1

This paper is published under the Creative Commons Attribution-NonCommercial-NoDerivs 4.0 International (CC-BY-NC-ND 4.0) license. Authors reserve their rights to disseminate the work on their personal and corporate Web sites with the appropriate attribution.

IJSCAR VOL. 3, ISSUE 4

© 2026 The Author(s). Published by IJSCAR.

1 Introduction

Underwater computer vision is affected by two effects that are largely absent from terrestrial imaging [1], [2]. First, water attenuates light wavelength-selectively: red is absorbed within a few meters, while blue persists, producing the characteristic blue-green hue of water and progressive loss of chromatic contrast with distance [1], [2], [3]. Second, suspended particulate matter scatters ambient light directly into the camera path, adding a veiling glow that reduces contrast independently of the subject [1], [2]. Both effects worsen with distance and with the optical properties of the water body itself [1], [2]. These conditions pose a practical problem for detecting targets reliably across widely varying visibility; benchmark collections of real underwater photographs document how wide that variation is [4]. Two mitigation strategies are implemented in this study. One is data augmentation: adjust the training images so the detector learns representations similar to the degradations it will encounter. The data augmentations used in this study will be presented in section three.

The other is image enhancement/dehazing, which attempts to invert the physical degradation before detection, so that the detector receives an approximation of the original image. This report evaluates both methods to answer the following:

(1) Which individual data augmentations provide the most improvement in underwater object detection where image clarity is reduced?
(2) How effective is image enhancement/dehazing at improving model performance in underwater computer vision?

2 Related Work

Using Jerlov water types to approach underwater computer vision problems is not a novel approach [5], [6], [7]. Many works use physics based simulation involving Jerlov Water Types for underwater computer vision [5], [6], [7]. For example, Mondal et al. [6], in a preprint, proposed a physics-informed and data-driven framework for simulating underwater images, combining a simplified physical image-formation model with learned residual degradation. Others, such as Uplavikar et al. [5], have used Jerlov Water Types as a way to add nuance to various underwater environments, and Li, Anwar and Porikli [7] synthesize training data across the Jerlov types in the same way.

Two further bodies of work bear on what we do here. The first is data augmentation itself, which Shorten and Khoshgoftaar [8] survey. Hendrycks and Dietterich [9] test a fixed model against a catalog of synthetic corruptions instead of a clean split, which is the idea behind the held-out condition we introduce in Section 3.5. The second is underwater detection benchmarks. DUO [10] and RUOD [11] both provide real underwater imagery with object annotations

and would be the natural outside comparison for a study like this one; we do not evaluate on them here, for the reason given in Section 5. On the enhancement side the standard non-physical baseline is the Dark Channel Prior of He, Sun and Tang [12], which we adopt as a comparison method in Section 4.9; learned alternatives include the analysis-synthesis network of Wang et al. [13], also a preprint, which we do not evaluate here because it would require a trained model we do not have.

The contribution of this study is therefore not the introduction of a new Jerlov-based dehazing method. Instead, this study evaluates two distinct approaches to improving underwater object-detection robustness: detector-side data augmentation and physics-based image correction. We systematically compare individual and aggregate augmentation policies under controlled composite degradation and separately evaluate a Jerlov-parameterized dehazing stage across water types and simulated path lengths. This allows us to examine not only whether these techniques improve detection, but also whether performance on clean validation data is predictive of robustness under degraded conditions.

3 System

3.1 Underwater image formation and the Jerlov classification

The Jerlov Water Type classification system categorizes water according to optical clarity and attenuation characteristics. The two key sub-classifications of Jerlov Water Types are Coastal and Open Ocean Water Types. Open Ocean Types range across I, IA, IB, II, and III, with I being the most clear Water Type. Coastal Types range across 1C, 3C, 5C, 7C, and 9C, with 9C being the most turbid or unclear Water Type. Figure 1 shows one frame from our dataset simulated at each of these water types.

In order to dehaze images using Jerlov Water Types we use the following equations [1], [2], [3]. Let $J_c(x)$ be the true radiance of the scene at pixel x in color channel $c \in \{R, G, B\}$. The observed image $I_c(x)$ is modeled as a spectral attenuation

$$I_c(x) = J_c(x)\, t_c + B_c\,(1 - t_c), \qquad t_c = e^{-K_c d}, \tag{1}$$

where d is the path length: the distance light travels through water between target and camera, not depth below the surface. K_c is the diffuse attenuation coefficient for RGB channel c, and B_c is

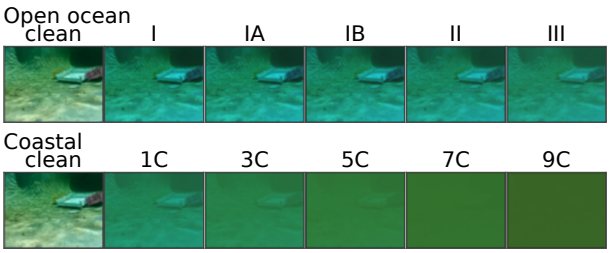


Figure 1: One frame from our dataset, simulated at every attenuation level through Equation (1), labelled by Jerlov type. Top row: the clean frame and the open-ocean types I to III. Bottom row: the same frame and the coastal types 1C to 9C. Contrast and color are lost gradually across the open-ocean types and collapse across the coastal ones.

the veiling-light color [2], [14]. The first term is the surviving direct signal; the second is backscatter. As $t_c \to 0$ the image tends to the flat color B_c, which is why deeply degraded frames lose contrast entirely rather than merely dimming. We note that this single-coefficient formulation is a simplification: Akkaynak and Treibitz [15] show that attenuation and backscatter are governed by different coefficients, each depending on factors this model folds into one constant.

The Jerlov system classifies natural water bodies by their spectral attenuation into open-ocean types (I, IA, IB, II, III, in order of increasing attenuation) and coastal types (1C through 9C, in order of increasing turbidity) [3], [16]. Each type is associated with characteristic wavelength-dependent optical attenuation properties, which we approximate here with an RGB K_c parameterization. The K_c and B_c values used below are our own: we chose them to follow the attenuation ordering that the Jerlov classification implies [1], [3], rather than taking the measured values reported there. The ten settings used below are therefore synthetic attenuation levels ordered as the Jerlov classification orders its water types, rather than the Jerlov water types themselves; we keep the Jerlov labels for readability. So (1) becomes a one-parameter group in path length once a type is chosen. Inverting it gives the restoration [1], [2]

$$\hat{J}_c(x) = \frac{I_c(x) - B_c(1 - t_c)}{t_c}, \tag{2}$$

with t_c clipped below at 0.05 to prevent noise amplification as the denominator approaches zero. Since (2) restores channel means for our water types but not water's overall local contrast, we follow it with CLAHE [17] on the L channel in Lab space and a gray-world white balance giving the end result below.

3.2 Model foundation

The model used here is Faster R-CNN [18] with a ResNet-50 [19] FPN [20] backbone. We measured no training-time benefit from that choice and claim none; what lets a dataset of this size converge in ten epochs is the COCO-pretrained initialization Section 3.4 describes. Our problem can be understood as a multi-classification problem applied many times per image, with an added regression component. A network of region proposal emits candidate regions [18] where for each region a classification head produces a softmax vector over s classes, trained by categorical cross-entropy exactly as in whole-image classification. In our case $s = 2$ (background, target). In parallel, a regression head emits four unbounded box-offset values per class, trained by a smooth-L_1 objective. The Ground-truth class labels for the proposals are not given, but are assigned by intersection-over-union against annotated boxes [18].

Because model detections can include false positives, image confidence alone is not an adequate evaluation metric. Instead, we can use average precision (AP), where a detection counts as a true positive when its intersection over union (IoU) with a ground-truth box exceeds a threshold and its class is correct; sweeping the confidence threshold traces a precision-recall curve whose area is the average precision. We report AP averaged over IoU thresholds 0.50 to 0.95 in steps of 0.05, denoted AP@[.50:.95], which rewards accurate bounding boxes rather than high confidence detection.

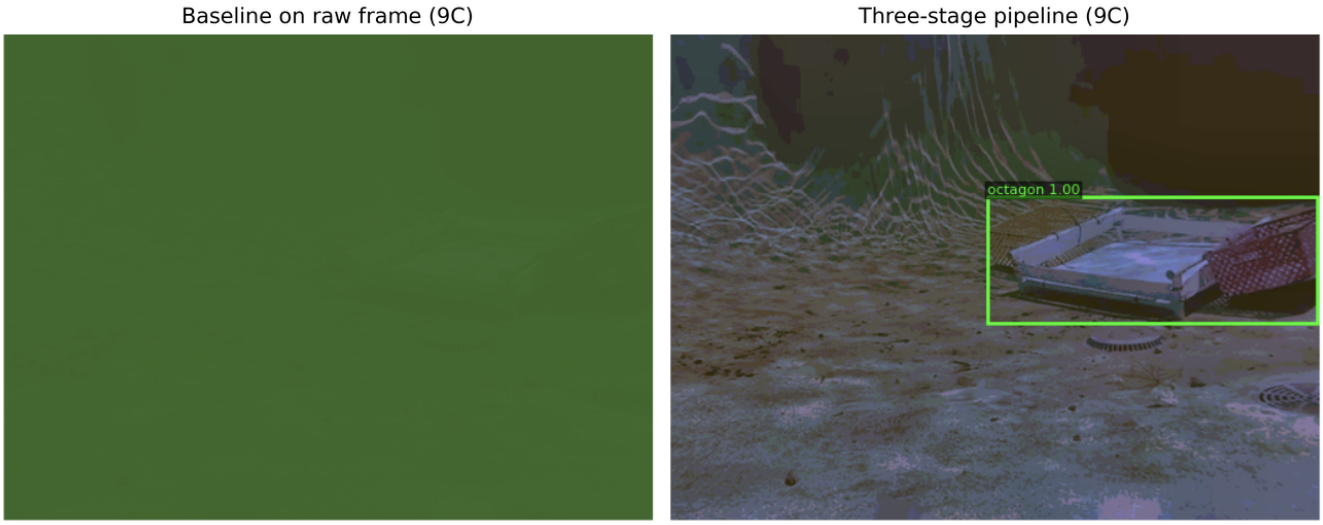


Figure 2: The three-stage correction on one validation frame simulated at 9C, the most turbid coastal type. Left: the raw degraded frame, on which the detector finds nothing. At this turbidity the direct signal has fallen away and the image has tended to the flat veiling-light color B_c, which is the $t_c \to 0$ limit of (1). Right: the same frame after correction, with the target recovered. The class is labeled octagon for the reason given in Section 3.3.

3.3 Dataset and model

The dataset is a custom single-class target-detection set in COCO format, collected from the camera of an uncrewed underwater vehicle at two swimming pools: a private residential pool and the Woollett Aquatics Center in Irvine, California. The frames were recorded between June and July 2026, while the team was practicing its mission runs for the RoboSub 2026 international competition. It comprises 2,376 images, split into 2,086 training images, a 193-image validation set, and a third partition of 97 images used for neither training nor checkpoint selection, which Section 4.5 evaluates separately. Of the 193 validation images, 150 carry a target box and 43 are negatives in which the target is out of frame, which is why Section 4.6 measures box geometry over 150 annotations. Every AP in this paper is computed over the full 193-image validation split, except where Section 4.5 and Table 4 report the 97-image partition.

The annotation tool wrote more than one output image for some source frames: 79 of the 2,086 training images are second outputs of a frame already in the split, and 59 of those pairs carry different box geometry. The training data therefore already contains tool-transformed copies, for every model including the control, so the control is not trained on wholly untransformed data. No two validation images share a source frame with each other; fourteen share one with a training image (Section 5). Five of the 97 images in the third partition are second outputs of a training frame and one of a validation frame.

Frames were extracted from each recording as consecutive runs, and the filename index is the source video's frame number. The three partitions were produced by a random split of the images in the nominal ratio 88:8:4, which divides images rather than recordings or source frames; the export bears this out. All sixteen recordings represented in the 97-image partition also appear in training and none is unique to it, the dataset covering seventeen in all; and of the one hundred source frames for which the annotation tool wrote two output images, twenty have those two outputs in different partitions, which a split grouped on the source frame could not produce. The proportions in which they fall across the partitions are those of an assignment made image by image ($\chi^2 = 5.8$ on five degrees of freedom, $p = 0.33$). Measured against the training split, 5 of the 97 are second annotation outputs of a training frame, 89 are one frame away, 2 are two frames away and 1 is four frames away. No image in the partition was used for training or for checkpoint selection, but its images are not temporally independent of the training footage. Section 5 reports the same measurement for the validation split.

The target is a rectangular equipment table roughly 1 m across, with a flat top plate, two side trays and four legs, resting on the pool floor. It is the same physical object in every frame, photographed from a range of distances and angles.

The class is labeled *octagon* in the annotation files. That name comes from the competition mission the vehicle was built for: ascending inside an octagonal PVC frame floating at the surface. The table sits directly beneath the center of that frame and marks its position from underwater, so the team's name for the mission carried over into the class label. The octagon itself is at the surface and appears in no frame; the annotated object is always the table.

Annotations are single axis-aligned boxes drawn by hand around the visible extent of the table. Every target fills a moderate to large part of the frame, so no annotation falls in the COCO small-area bucket.

Two properties of the split bear on every result below, so we state them here rather than deferring them to Section 5. First, the partitions divide frames rather than recordings, so an image can

sit one frame from a training image; the paragraph above measures how close, and the very high clean scores in Table 2 may be optimistic for that reason. Second, the same validation images were used both to select the checkpoint and to report results; the 97-image third partition was held back from both, and Section 4.5 reports it separately.

All models are Faster R-CNN ResNet-50 FPN initialized from COCO-pretrained weights [21], with the box predictor replaced by a two-class head (background plus target) and fine-tuned end-to-end for 10 epochs using SGD (learning rate 0.005, momentum 0.9, weight decay 5×10^{-4}) with a step schedule decaying by $\gamma = 0.1$ every 4 epochs. The checkpoint minimizing validation loss was retained.

3.4 Training augmentation policies

Seven models were trained under seven data augmentation policies. Each augmentation is applied stochastically per image with probability p and strength sampled from the stated range. The seventh is the control, which applies no augmentation of its own. It is listed here so that every model appearing in the results has a training row:

Model	Augmentation applied during training	p
Scale+Rotate	rotation ±15°; scaling 0.8–1.2×	0.30
Brightness	uniform brightness 0.6–1.5×	0.40
Non-uniform light	directional illumination gradient, intensity 0.2–0.5	0.25
Noise	additive Gaussian, $\sigma \in [5, 20]$	0.30
Blur	Gaussian Blur radius 0.5–2.5	0.20
All (aggregate)	all of the above, plus grayscale at $p = 0.15$	–
Baseline (no aug)	—	–

Table 1: Training augmentation policies. Rotation and scaling transform annotation boxes alongside the image; the photometric augmentations leave boxes unchanged.

One further difference was not intended. The original checkpoints were trained before the data-loader workers were seeded: `train_loader` was built without a `worker_init_fn`, so the augmentation draws came from the global NumPy stream and each forked worker inherited the parent's state and drew the same sequence. The augmented policies may therefore have received less varied augmentation than Table 1 describes, while the control draws no augmentation randomness of its own and is unaffected. The runs retrained in Section 4.3 used per-worker seeding, so the two checkpoint families differ in their data-loader random-number handling as well as in their seeds.

3.5 Evaluation regimes

Each model was evaluated in four regimes. The **clean** regime uses an unmodified validation split. The **composite augmented** regime applies every degradation simultaneously and deterministically to every validation image: rotation 25°, scaling 0.85×, brightness 0.6×, non-uniform illumination (intensity 0.4, direction 45°), Gaussian noise $\sigma = 25$, and blur radius 1.5. Determinism matters here: training augmentation is stochastic to generate variety, whereas evaluation must be repeatable so that conditions are comparable across models. Geometric transforms are applied to ground-truth boxes as well, so that AP is computed against correctly transformed annotations.

Two of these composite settings are deliberately outside the ranges any model trained on: rotation is 25° against a training maximum of 15°, and noise is $\sigma = 25$ against a training maximum of 20. The composite condition is therefore out of distribution for every model, including the aggregate one, and its results should be read with that in mind.

The **per-degradation** regime applies each of the seven degradations the aggregate model trains on, one at a time at the composite setting, so that a model trained on one degradation can be tested against that degradation in isolation. Six of them are the ones that make up the composite condition; the seventh is grayscale, which the aggregate policy also uses and which is applied here by the same function with the same settings as in training, so it is a trained-on condition and not a held-out one. This answers the first research question directly, instead of inferring it from the composite result.

The **held-out** regime applies four degradations that appear in no training policy: motion blur, JPEG compression at quality 18, simulated marine snow, and defocus. Because no model has seen them, this regime does not give the aggregate model the built-in advantage it has under the composite condition. Two of the four are nonetheless relatives of a degradation the models did train on: motion blur and defocus both remove high spatial frequencies, as the Gaussian blur in the training policies does, so they test transfer within a family rather than to an unfamiliar kind of corruption. JPEG compression and marine snow are the two unlike anything in training. The marine-snow generator places particles at random positions; its seed is derived from the image itself, so every condition in this paper reproduces exactly across evaluation passes. What that fixes is which particle field each image receives, not how far a different field would move the result, so this column is reproducible rather than averaged over particle seeds.

Separately, a **water-type sweep** evaluates the Jerlov based dehazing. Our entire validation set is degraded via (1) at all 10 water types × 6 path lengths (1, 2, 4, 6, 8 and 10 m), and performance is recorded both on the degraded frame ("raw") and after applying the three-stage pipeline ("corrected"), yielding 60 paired measurements per model. Following the second research question we report this sweep in AP rather than in top-1 confidence, for the reason given in Section 3.2: confidence can rise on a detection that is in the wrong place, and AP cannot.

4 Evaluation

4.1 Validation Test of Models

Here we test every model on the same validation split to ensure we have no discrepancy with our model training or issues with the models themselves. The spread across each of the models on clean data is 0.036 AP, and the ordering is close to uninformative as the aggregate model in the fifth row ranks fifth in **AP@[.50:.95]**, behind the control. The second data column, AP@.50, is essentially saturated (≥ 0.995 for every model), indicating that on undegraded

Model	AP@ [.50:.95]	AP@ .50	AP@ .75	AP $_{large}$	AR@ 1	AR@ 100
Non-uniform light	0.841	1.000	0.959	0.843	0.869	0.875
Brightness	0.832	1.000	0.959	0.833	0.859	0.865
Blur	0.831	0.997	0.957	0.832	0.852	0.859
Baseline (no aug)	0.824	0.997	0.957	0.824	0.849	0.855
All (aggregate)	0.823	0.999	0.959	0.823	0.855	0.862
Noise	0.819	0.999	0.959	0.819	0.846	0.851
Scale+Rotate	0.805	0.996	0.955	0.803	0.831	0.839

Table 2: Detection performance (0 to 1) on the clean validation split (193 images). All seven models lie within a 0.036 AP band. AP_{small} is undefined (−1) throughout because the split contains no ground-truth objects in the COCO small-area bucket.

imagery the task is nearly solved and that the AP@[.50:.95] differences reflect localization tightness rather than detection success. This test on clean validation cannot be used to select a data augmentation policy given that it is used to ensure our models work on their own datasets properly and rather acts as a control.

4.2 The composite condition is a ceiling

We first ran every model against the composite degraded validation set, which applies all six training degradations at once. The aggregate model scores 0.277 AP against the control's 0.091, a gap of 0.186. The best single-augmentation policy, Blur, scores 0.247; we do not read the 0.029 between them as a difference, for the reason Section 4.3 gives.

We report these numbers but do not build on them. The composite test set is assembled from six of the seven degradations the aggregate model trained on; no other policy trained on more than two of them, so a win for the aggregate model here is close to guaranteed by construction. What that gap measures is how far an augmentation policy can carry a detector inside its own training distribution, not whether that policy generalizes. The two experiments that follow avoid this: Section 4.4 tests each degradation separately, so a policy is credited only where it ought to help, and Section 4.5 tests degradations appearing in no training policy at all. Those are the results this paper rests on.

4.3 How much of a gap is training noise

The seven policies in Tables 2, 4 and 6 were each trained once, so a difference between two of them could be a difference between two training runs rather than between two policies. The question has to be put to separate training runs.

We retrained the control and the aggregate policy under two seeds each — four runs of ten epochs, same hyperparameters as the originals — and scored all four on the degradations held out of every training policy. Table 3 reports every run separately. The two runs of one policy differ by 0.003 for the aggregate and 0.005 for the control on the held-out mean. With two seeds per policy these are observations rather than estimates of a spread, and we use them as such.

The held-out comparison is consistent across all three pairs: the two retrained pairs lead by 0.019 and 0.021 and the pair used throughout by 0.026, the same direction each time and within 0.007 of one another. We do not convert that into a multiple of the run-to-run difference, because two seeds on two policies cannot support the spread such a ratio implies. The difference is also specific to the mean over four conditions; Section 4.4 reports margins on single conditions and makes no comparison of this kind. Section 5 gives the qualifications.

Policy	Run	Clean	Composite	Held-out mean
Control	original	0.824	0.091	0.726
Control	seed 1234	0.827	0.087	0.733
Control	seed 5678	0.823	0.053	0.728
Aggregate	original	0.823	0.277	0.752
Aggregate	seed 1234	0.802	0.493	0.751
Aggregate	seed 5678	0.810	0.465	0.748

Table 3: Per-run AP for the two retrained policies. The held-out mean reproduces; the composite score does not.

The two retrained aggregate runs score 0.493 and 0.465 AP under the composite condition, a difference of 0.028. Both exceed the original aggregate checkpoint's 0.277, by 0.217 and 0.188. The two retrained control runs score 0.087 and 0.053, a difference of 0.034, against 0.091 for the original control. Re-scoring the original checkpoints under the procedure the retrained runs used leaves their composite scores unchanged, so the discrepancy is associated with the training runs rather than with the evaluation. The most likely reason is the data-loader seeding of Section 3.4: the original augmented checkpoints drew a repeated augmentation sequence and may have received less variety than intended, while the control, which draws no augmentation randomness, is unaffected. The runs do not isolate loader seeding as the cause and we do not claim they do. What they do show is a difference in degree consistent with it: the original control sits 0.004 from one of its retrained runs and 0.038 from the other, while the original aggregate sits 0.188 and 0.217 from its two. The held-out comparison is unaffected, both giving the same answer within 0.007, while the composite comparison runs from 0.186 to 0.412. The result this paper rests on survives the change in training procedure; the composite gap, which Section 4.2 already treats as a ceiling, does not, and we quote no composite gap as a measured quantity.

4.4 Each degradation on its own

The composite condition applies six degradations at once, so it cannot say which individual augmentation helps against which individual degradation, which is the first research question. Figure 3 answers that directly, testing all seven models against each degradation separately.

The answer to the first research question is largely negative. Five specialists cover six degradations, since Scale+Rotate trains on both scaling and rotation. Four of those six pairings beat the control: Scale+Rotate on rotation (0.451 against 0.418), Brightness on brightness (0.829 against 0.801), Non-uniform light on non-uniform illumination (0.839 against 0.817) and Blur on blur (0.823 against 0.797). The margins run from 0.022 to 0.033. These are observed differences between single training runs: each specialist policy was trained once, and the retraining in Section 4.3 covers only the control and the aggregate policy, on the mean over four held-out

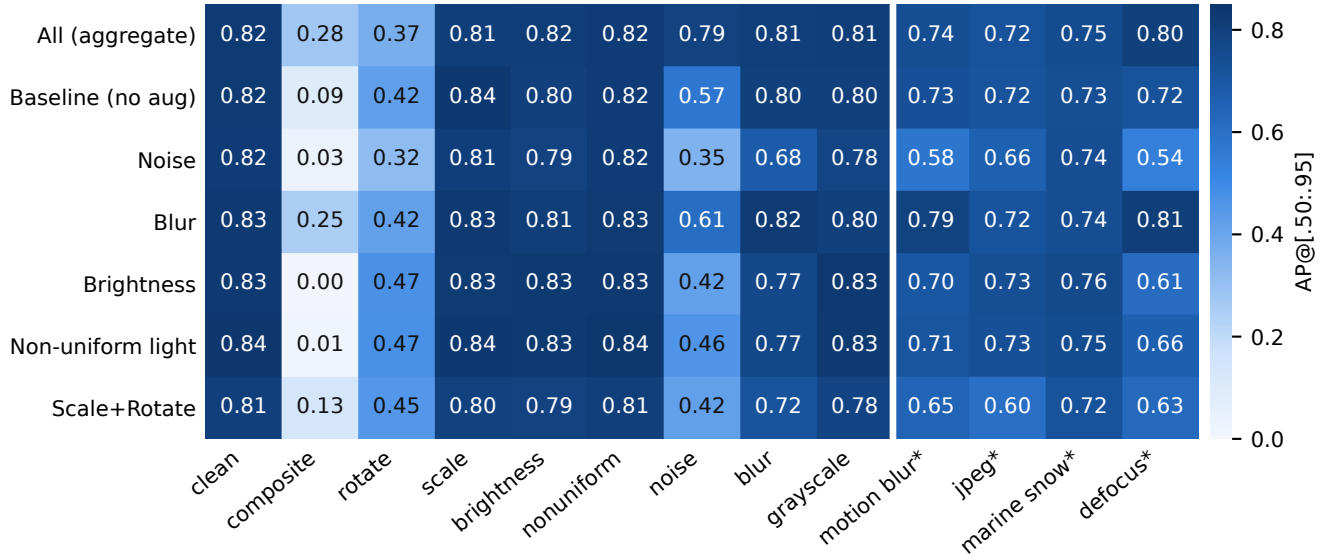


Figure 3: AP@[.50:.95] for all seven models against each condition separately. Columns right of the white rule are the held-out degradations, which appear in no training policy. Darker is better.

conditions rather than on single conditions. We therefore report these four wins as observed and do not claim they are reproducible across training runs, nor rank them against one another. The other two pairings lose: Scale+Rotate scores 0.796 against the control's 0.836 under scaling, a margin of 0.040 larger than the wins above but from the same single runs, and the Noise model scores 0.353 against the control's 0.572 under noise. Both policies were trained once for this comparison, so we do not estimate how much of that difference is run-to-run variation; nor do we have an explanation for the Noise result. We report it as observed. The aggregate model, which also trained on noise, is the best model in that column at 0.791.

Rotation and noise are the two degradations that hurt most, and they are much harder than the rest. Rotation is the worst single condition for four of the seven models and noise for the other three; across the field rotation runs from 0.322 to 0.473 and noise from 0.353 to 0.791, with mean scores of 0.417 and 0.517. Every other single degradation (scaling, brightness, non-uniform illumination, blur and grayscale) leaves every model at 0.681 or above. Section 4.6 discusses how rotated boxes are scored, which bears on this figure.

The aggregate policy is broadly but shallowly better. Across all thirteen conditions in Figure 3 it beats the control in ten, which is consistent in sign, though seven of those ten margins are below 0.02 and so inside the range we have said we cannot rank on. It is the single best model in only two of the thirteen, and on the seven individual degradations, all of which its own policy trains on, it is best in only one, noise. Training on everything therefore buys a little robustness in many directions at the cost of specializing in none: a model slightly better than no augmentation almost everywhere, and rarely the best choice anywhere.

Most individual degradations do not separate the models, and no augmentation at all is a middle-of-the-pack policy. Four of the seven single degradations — scaling, brightness, non-uniform illumination and grayscale — hold all seven models inside bands of 0.033 to 0.051 AP, too narrow to rank on. Only rotation (0.151), blur (0.141) and noise (0.438) spread the field wider, so the matrix holds fewer usable comparisons than its size suggests. Against that background the control is never the best and never the worst model on any of the seven degradations the training policies cover, ranking from second under scaling to fifth under brightness and non-uniform illumination. Averaged over those seven, only two of the six augmented policies beat it: the aggregate policy at 0.746 and Blur-only at 0.731 against the control's 0.720, while Scale+Rotate (0.680) and the Noise model (0.651) fall clearly below. Tested one degradation at a time, augmentation is not reliably better than no augmentation. These results come from checkpoints predating per-worker dataloader seeding (Section 3.4), so that finding and the unexplained Noise result apply to these checkpoints rather than to the policies in general.

4.5 Degradations no model trained on

The composite regime is built from the aggregate model's own training degradations, so its result is a ceiling. To test whether augmentation produces robustness that generalizes, we evaluate all seven models against four degradations that appear in no training policy: motion blur, JPEG compression at quality 18, simulated marine snow, and defocus.

This test is the one our main claim rests on. Against degradations it never saw, the aggregate model scores 0.752 against the control's 0.726: an advantage of 0.026, where the same comparison was larger under the composite condition, by an amount the retrained runs do not reproduce (Section 4.3). It is also no longer the best model: Blur-only leads at 0.765, by 0.013. This ordering includes the five single-augmentation checkpoints that predate per-worker dataloader seeding (Section 3.4). Section 4.3's run-to-run differences

Model	Motion blur	JPEG	Marine snow	Defocus	Mean (val.)	Mean (97)
Blur	0.790	0.716	0.742	0.812	**0.765**	**0.813**
All (aggregate)	0.738	0.721	0.750	0.798	0.752	0.805
Baseline (no aug)	0.728	0.720	0.734	0.722	0.726	0.779
Non-uniform light	0.711	0.731	0.749	0.663	0.714	0.750
Brightness	0.699	0.734	0.759	0.613	0.701	0.746
Scale+Rotate	0.650	0.601	0.721	0.625	0.649	0.699
Noise	0.576	0.659	0.742	0.538	0.629	0.672

Table 4: AP@[.50:.95] on four degradations held out of every training policy. The four condition columns are the 193-image validation split; Mean (val.) is their mean there and Mean (97) the same mean on the 97-image partition of Section 3.3. The aggregate model's large advantage under the composite condition does not survive here: it leads the control by 0.026 on validation and 0.025 on the 97, and is not the best model on either.

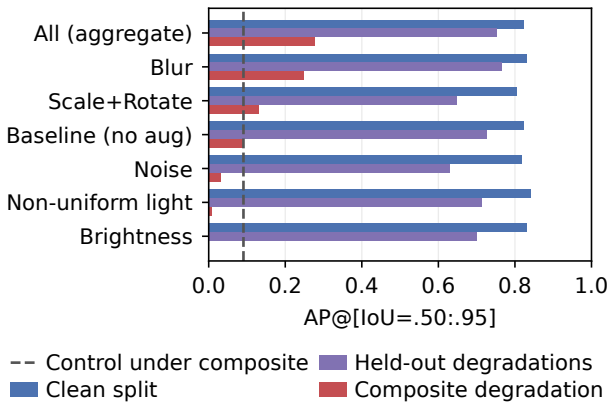


Figure 4: AP@[.50:.95] by model in three regimes. The seven models span 0.036 AP on clean data (blue). That spread widens to 0.276 under composite degradation (red) and narrows again to 0.136 under the held-out degradations (purple) — much narrower than composite, but still close to four times the clean band. Dashed line marks the original control checkpoint under composite degradation, at 0.091; three of the six augmented models fall below it.

were observed on the control and the aggregate policy, not on Blur-only, so we do not measure that lead against them. So we only claim that the aggregate model is no longer best, and we do not claim that Blur-only is better than it. The aggregate model does beat the control on all four conditions taken individually, but on this split it is the best model on none of them. Four of the six augmented models score *below* the control. So what the aggregate policy buys is narrow rather than general: it is best on none of the four.

The 97-image partition of Section 3.3 was used for neither training nor checkpoint selection, so it offers an evaluation free of selection pressure. On the clean split the seven models span 0.842 to 0.869 there, against 0.805 to 0.841 on validation; their means over the four held-out degradations are the last column of Table 4. The aggregate model's held-out mean is 0.805 against the control's 0.779, a difference of 0.025 where the same comparison on the validation split gives 0.026, and the ordering of all seven models is the same on both partitions, with Blur-only again first. On the four conditions taken individually its lead there is 0.008 on motion blur, 0.027 on JPEG, 0.029 on marine snow and 0.036 on defocus; `data/test97.csv` carries every model's figures. The partition carries 83 annotated targets against the validation split's 150, so its figures warrant more caution. Its images sit within four frames of a training image and it shares every recording with training (Section 3.3), so it does not test generalization to unseen footage.

The little that survives is concentrated rather than broad. The aggregate model's 0.026 lead is not spread evenly over the four conditions: defocus alone supplies 73% of it, marine snow 16%, motion blur 10% and JPEG 2%, where the two models differ by 0.002. That split follows the family division noted in Section 3.5: defocus and motion blur are the two relatives of the Gaussian blur in training, and account for 83% of the advantage between them. Marine snow supplies most of the rest, a gap of 0.016. The other degradation unlike anything in training, JPEG, contributes almost nothing: the two models differ there by under 0.002. The held-out result is therefore, on this split, better read as transfer within the blur family than as general robustness, which strengthens the negative finding rather than softening it. The 97-image partition agrees on the size of this advantage and not on its composition. Summed over the four conditions it is 0.101 there against 0.104 here, but defocus supplies 36% of it rather than 73% and JPEG 27% rather than 2%, so the blur-family pair accounts for 44% rather than 83%; the aggregate model also takes first place on JPEG there, by 0.011. With 83 annotated targets spread over four conditions these are small differences on a small sample, and we draw only the negative conclusion they support: the concentration above is a property of the validation split rather than of the policy. The four conditions also differ in how much they discriminate at all: defocus and motion blur spread the seven models by 0.274 and 0.214 and JPEG by 0.133, while marine snow spreads them by only 0.038. We read that column as carrying little signal about augmentation policy: dropping it from the held-out mean leaves the ordering unchanged and widens the gap to 0.029.

Blur-only's lead follows the same family line, and we offer it only as a conjecture. It wins the two blur-like conditions, motion blur (0.790) and defocus (0.812), and places fifth of seven on JPEG, which chiefly introduces block artifacts; averaged that way it scores 0.801 against 0.729. We do not claim this as a mechanism. We chose the grouping after seeing the results, the control shows no split of its own either way (−0.002), and dropping marine snow turns Scale+Rotate positive, a policy that trained on nothing blur-like. Testing the idea properly needs held-out degradations chosen in advance to sit near and far from each training policy.

4.6 What rotated boxes do to AP@.75

The composite condition rotates each image by 25° and rotates the ground-truth boxes with it. Because an annotation is stored as an axis-aligned rectangle, the rotated box must be re-enclosed in a new axis-aligned rectangle, which is larger than the original. For a box of width w and height h rotated by θ, the enclosure has width $w|\cos\theta| + h|\sin\theta|$ and height $w|\sin\theta| + h|\cos\theta|$. Table 5 evaluates that enclosure across rotation angles.

At 25° the transformed reference box has an area 1.766 times the original for a square annotation, and more for any other rectangle.

Rotation	0°	5°	10°	15°	20°	25°	30°	45°
Area inflation, square annotation ×	1.000	1.174	1.342	1.500	1.643	1.766	1.866	2.000
Illustrative IoU, square annotation: original box vs. rotated enclosure	1.000	0.852	0.745	0.667	0.609	0.566	0.536	0.500

Table 5: Effect of rotating an axis-aligned ground-truth box and re-enclosing it, shown for a square annotation. The second row is an illustrative limiting case, not a bound established for the actual objects: it assumes the object's own extent does not change with rotation, and the reciprocal relation it uses holds only while the original box lies inside the enclosure, which it does up to an aspect ratio of about 4.5:1. An object that fills its box reaches IoU 1.0 at every angle.

Across our 150 validation annotations the median area inflation is 1.868×, with an interquartile range of 1.790 to 2.082 and a maximum of 4.323× for the most elongated box.

Area inflation alone does not determine the resulting IoU. If the object fills its annotation box, the tightest axis-aligned box around the rotated object coincides with the transformed reference box, and a perfect prediction still reaches IoU 1.0; the inflation costs nothing. Under other relationships between the object and its box the overlap can be lower. The reciprocal of the median inflation is 0.535, but that is an illustrative area comparison, not a demonstrated upper bound on IoU: it would be the bound only if the object's own extent were unchanged by rotation, which we did not measure.

We did not compare predicted boxes against alternative ground-truth conventions, so we cannot quantify how much of the measured AP@.75 result is attributable to the box transformation rather than to detector performance. Under the composite condition the aggregate model achieves AP@.75 of 0.199. This confirms that detections above the 0.75 IoU threshold occur under the actual evaluation, but it does not isolate rotation from the five other degradations applied simultaneously, nor quantify the geometric scoring effect. AP@.75 under this condition should therefore be interpreted as a joint measure of detector performance and evaluation geometry, not as a standalone measure of either.

4.7 Effectiveness of the correction across water types

	Open ocean (I–III)			**Coastal (1C–9C)**		
Model	Raw	Corrected	Δ	Raw	Corrected	Δ
All (aggregate)	0.801	0.729	−0.073	0.502	0.693	+0.191
Blur	0.803	0.697	−0.106	0.440	0.645	+0.205
Brightness	0.815	0.708	−0.107	0.474	0.674	+0.199
Noise	0.776	0.625	−0.151	0.396	0.575	+0.179
Non-uniform light	0.816	0.682	−0.134	0.446	0.647	+0.201
Scale+Rotate	0.777	0.635	−0.142	0.411	0.588	+0.177
Baseline (no aug)	0.795	0.688	−0.108	0.417	0.645	+0.228

Table 6: Mean AP@[.50:.95], split by water-type family, averaged over six path lengths. The correction is not neutral in open-ocean water. It costs every model accuracy there, and gains every model about the same amount in coastal water. Each Δ is computed before rounding, so three of them differ by 0.001 from the rounded figures beside them.

Averaged over the ten water types, six path lengths and seven detectors, the correction, applied throughout this section as the full three-stage pipeline, moves open-ocean AP from 0.798 to 0.680, a loss of 0.117, and coastal AP from 0.441 to 0.638, a gain of 0.197. The aggregate figure conceals the structure, and in AP that structure is sharper than it was in confidence. In open-ocean types the correction now *costs* accuracy, between 0.073 and 0.151 depending on the detector, because those images are barely degraded to begin with and the contrast stages strip contrast that was never haze; Section 4.9 shows the inversion on its own does not do this. The benefit is concentrated in coastal types, where every detector gains between 0.177 and 0.228. The crossover sits between 1C, where the correction still costs 0.041, and 3C, where it gains 0.139. In the most degraded conditions tested (7C and 9C at 6 m and beyond) raw AP is exactly 0.000 in all 42 model-condition cells, while the corrected pipeline recovers 0.440.

Table 6 also tests directly whether augmentation and correction overlap, since both are present in this sweep. The aggregate model leads the control by 0.046 AP on uncorrected frames and by 0.045 AP after correction, so the correction is worth about the same amount whether or not the model was augmented: the two add. Two things limit that. It is an average that hides a reversal, since the augmentation advantage grows by 0.035 in open-ocean water and shrinks by 0.037 in coastal water. And it holds only for the aggregate policy: the same test on the five single-augmentation models gives −0.011, −0.014, −0.027, −0.043 and −0.046, every one negative and at least nine times the aggregate model's −0.001. For those policies the correction recovers part of what the augmentation already provided.

The open-ocean cost is not an artifact of reporting AP instead of confidence. Across the seven detectors, mean top-1 confidence in open-ocean water falls by 0.045 to 0.141 and the detection rate by 0.025 to 0.107, so all three metrics agree. If a system applies the correction to every frame without checking, it will lose accuracy whenever the water is clear.

4.8 Where the correction fails in extreme turbidity

The dehazing stage does not extend detection indefinitely. Table 7 reports, for the Noise model, the shortest path length at which detection fails completely. Like Section 4.7, this section uses the

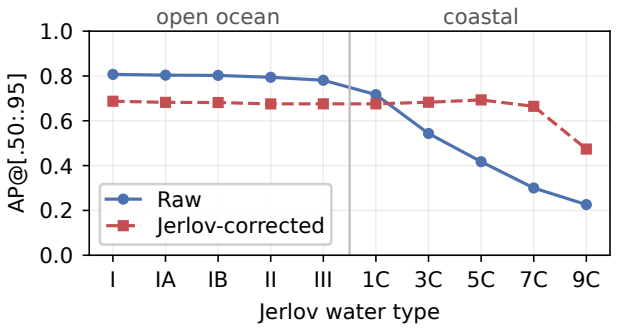


Figure 5: AP@[.50:.95] by water type, averaged over six path lengths and all seven detectors, raw against corrected. Without correction accuracy decays steadily from 1C onward; with correction the curve is far flatter across the coastal types, at the cost of a consistent loss in the open-ocean types where there was little to remove.

three-stage pipeline that Section 4.9 identifies as the weaker configuration, and the boundary has not been re-measured with the inversion alone.

The correction pushes the failure boundary outward substantially — 3C, 5C and 7C are recovered entirely — but at 9C it delays failure only from 4 m to 8 m, and beyond that both pipelines return nothing. This is the expected consequence of the transmission clipping in (2): once t_c falls below the clip floor, the direct signal has been attenuated past the point where inversion can recover it, and amplifying what remains amplifies noise rather than scene content. The correction should therefore be understood as extending the usable envelope, not removing the limit.

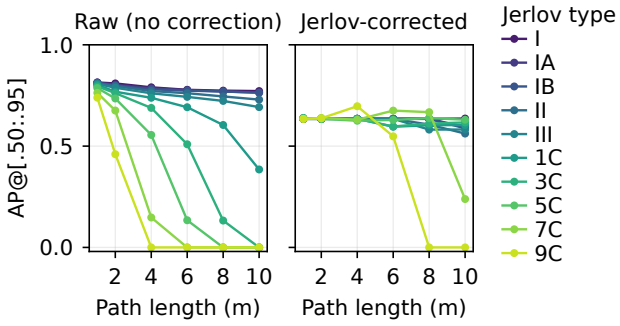


Figure 6: AP@[.50:.95] versus path length for the Noise model, by water type. Raw curves (left) collapse with turbidity; 3C, 5C, 7C and 9C all fail within the sweep. Corrected curves (right) stay between 0.54 and 0.70 out to 6 m for every type, after which 9C reaches zero at 8 m and 7C falls to 0.24 at 10 m.

4.9 Which stage of the pipeline earns the gain

The pipeline described in Section 3.1 has three stages: the Jerlov inversion of (2), then CLAHE on the L channel, then a gray-world white balance. Only the first is physics-based, and the results above cannot separate its contribution from that of the two contrast steps. We therefore ran each stage on its own, over three representative conditions (5C at 4 m, 3C at 6 m, and I at 2 m) across all seven detectors, and added the Dark Channel Prior [12] as a standard enhancement baseline that uses no water-type information at all.

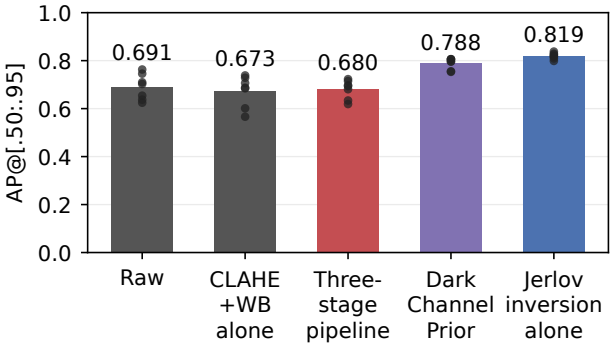


Figure 7: AP@[.50:.95] by enhancement method, averaged over three conditions. Dots are the seven detectors. The Jerlov inversion on its own is the best method tested; adding the contrast stages that follow it in our pipeline costs more than the inversion gained.

This reverses the recommendation implied by the three-stage pipeline we began with. The Jerlov inversion *alone* scores 0.819 against 0.691 for the raw image, a gain of 0.128, and it improves on raw for all seven detectors. The full three-stage pipeline scores 0.680, which is *below* raw. CLAHE and white balance alone score 0.673, worse than raw for five of the seven detectors. The contrast stages are not only redundant after the inversion, they make the result worse, and they are the reason the measured benefit looked smaller than it is.

The same split matters against the external baseline. The full pipeline loses to the Dark Channel Prior (0.680 against 0.788) on all seven detectors, which alone would be a negative result for the physics-based approach. The inversion by itself beats the Dark Channel Prior (0.819 against 0.788) on all seven, by 0.019 to 0.054. If we measure this method as a three-stage pipeline it is worse than a standard baseline, but if we measure only the physics step on its own it is better. We therefore recommend the inversion without the contrast stages.

4.10 What the correction costs when its settings are wrong

Every result above gives the correction the exact water type and path length used to degrade the image. A vehicle has neither. To bound how much that matters we re-ran the correction at 5C and 4 m with deliberately wrong settings, again using the three-stage pipeline rather than the inversion alone that Section 4.9 recommends.

The error is strongly asymmetric, and it is useful to know which direction is the safe one. Assuming a path length 3 m shorter than the truth scores 0.714, slightly *better* than the correct 4 m value of 0.682; assuming 3 m longer scores 0.375, a loss of 0.307. The same holds for water type. Every type clearer than the truth scores between 0.739 and 0.771, above the correct setting's 0.682. Assuming 7C drops it to 0.088, a loss of 0.595, and assuming 9C drops it to 0.001.

Water type	I	IA	IB	II	III	1C	3C	5C	7C	9C
Raw — fails at	—	—	—	—	—	—	10 m	8 m	6 m	4 m
Corrected — fails at	—	—	—	—	—	—	—	—	—	8 m

Table 7: Shortest path length at which AP across the validation split reaches zero (Noise model). "—" denotes no failure within the 10 m sweep. Failure is defined by AP alone rather than jointly with detection rate, since the two need not coincide and reporting both would invite confusion.

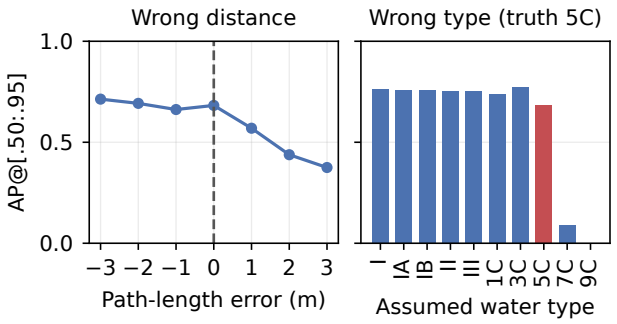


Figure 8: AP@[.50:.95] when the correction is given wrong parameters, averaged over seven detectors. Left: error in assumed path length, true value 4 m. Right: assumed water type, true value 5C in red. Both panels are asymmetric in the same direction: assuming clearer, nearer water changes AP little and usually raises it — every clearer water type scores above the true 5C, and only the 1 m underestimate costs anything, 0.020 — while assuming murkier, further water is very costly.

The mechanism is visible in (2). Over-estimating attenuation makes t_c too small, so the division amplifies both the residual signal and the noise; under-estimating it leaves some haze in place, which the detector tolerates. A practical system that has to guess should therefore guess conservatively. Assuming the water is clearer and nearer than it really is costs us very little. The opposite error is far more damaging, and at the extreme it removes detection altogether: over-estimating path length by 3 m still leaves AP at 0.375, while assuming 9C water drops it to 0.001.

4.11 Estimating the parameters from the image

The stronger version of the previous question is whether the settings can be recovered from the frame itself. We implemented a blind variant that searches water type and path length to optimize a no-reference image-quality score, with no access to the true values. This study too used the three-stage pipeline, so it is not directly comparable with the inversion-only result in Section 4.9; we have not measured a blind-estimated inversion on its own.

Blind estimation recovers the water type in 5 of 18 trials, under the diagnostic described below. Despite that, it beats the closed-loop configuration in both coastal conditions (Table 8), for the reason Section 4.10 gives: its errors tend toward under-correction, which is the safe direction. In clear water it fails badly, scoring 0.394 where leaving the image alone scores 0.817, because it infers heavy attenuation from an image that has almost none and then inverts it. Averaged over the three conditions blind estimation is worse than doing nothing.

This is not a problem with our search procedure alone. Within the transmission term, attenuation and path length are coupled through the product $K_c d$, so that term on its own cannot separate them: a clear water type at long range and a turbid one at short range give the same transmission. The full model is not quite so strict. Water type also sets the veiling-light color B_c, which in our implementation differs between types — from $[0.05, 0.35, 0.55]$ at I to $[0.22, 0.40, 0.15]$ at 9C — so two conditions with the same $K_c d$ do not produce exactly the same image. We therefore claim only that the coupling makes the two hard to separate, not that the problem is unsolvable in principle.

The diagnostic that accompanies it is narrower: it evaluates 18 simulated conditions — six water types crossed with three path lengths — on a single input frame, not 18 distinct scenes. Under its criterion, exact recovery of the water type while ignoring the estimated path length, the type was recovered in 5 of 18 conditions (27.8%); the full type-and-path-length pair was recovered in none. Its median optical-depth error, which compares $\tau = K_c d$ under the estimated water type and path length against the true values as $\sum |\tau_{\text{est}} - \tau_{\text{true}}| / \sum |\tau_{\text{true}}|$, is 36.8%. These rates describe one frame under the conditions tested, not identification accuracy in general.

We also looked at the transmission itself, and found we could not use it. The estimator reads transmission off the image as a per-channel dynamic range, which recovers it only up to an unknown factor equal to the scene's own range. That factor is close to uniform across the three channels for this frame, so it very nearly cancels when we take ratios between channels: a small ratio error would tell us about the frame's own channel balance, not about how well the water's color cast was recovered. For the same reason the estimator cannot establish the absolute scale of transmission at all. We therefore do not report transmission-scale or color-ratio errors as evidence of recovery accuracy. Fixing the scale would need a second cue, such as a stereo pair, a known-size reference object, or a depth sensor.

The two interventions studied here act on different parts of the problem, and Section 4.7 shows their effects approximately add for the aggregate policy but overlap for the other five augmented policies. Augmentation acts on the detector, though Section 4.4 shows it does not buy robustness uniformly: the aggregate model falls below the control on rotation and on scaling, the two geometric degradations. Section 4.5 shows that what it does buy is largely confined to the degradations trained on and to their close relatives. The Jerlov correction acts on the image, and buys robustness specifically to wavelength-selective attenuation and backscatter [1], [2], a degradation no amount of the augmentations tested here simulates, since none of them models physical light transport. Neither substitutes for the other. The answer to the second research question is therefore conditional on which configuration is meant: the physics inversion on its own is effective, raising AP from 0.691 to 0.819 and

Condition	Raw	Blind estimate	Known settings
3C @ 6 m	0.604	**0.756**	0.671
5C @ 4 m	0.652	**0.718**	0.682
I @ 2 m	**0.817**	0.394	0.688
Mean	**0.691**	0.623	0.680

Table 8: AP@[.50:.95] with parameters estimated from the image, against the raw frame and against the closed-loop setting where the true parameters are supplied. Averaged over seven detectors.

beating a standard dehazing baseline on all seven detectors, while the three-stage pipeline we began with scores below the raw image. The methodological finding is arguably the more portable one: clean-split validation was not informative about robustness here, and we do not claim anything stronger than that. All seven models sit within 0.036 AP on the clean split, each was trained once, and the clean ordering does not survive a change of evaluation set. But a spread that small, from single runs, is not enough to show that the ordering reversed rather than that it never meant much. The claim our evidence supports is the weaker one: a clean validation score did not predict which model would hold up under degradation, so robustness must be measured under the conditions of interest.

5 Limitations and Future Work

The attenuation coefficients are uncalibrated. The K_c and B_c values in our equation were constructed to follow the qualitative attenuation ordering associated with the Jerlov classification [1], [3] where red attenuates the fastest and coastal types attenuate more strongly across all bands, but these values are not the same as calculated values. In other words, they match the optical ordering of Jerlov water types but are not calculated for exactitude. Hence, the results involving the water-type sweep should therefore be read as characterizing the approximate behavior of the correction under a plausible environment but not as calculated field predictions of what the water is like.

We cannot separate the rotation scoring effect from the detector. Rotating an image rotates its ground-truth boxes, re-enclosed in new axis-aligned rectangles, inflating the box by a median of 1.868× at 25°. We did not measure how the object's own extent changes under rotation, so we cannot say what that costs a detector. Separating them means scoring the same predictions against both the inflated box and a tight one around the rotated object, which needs rotated-box or polygon annotations on sample frames. That is inference-only work and the clearest next step for Section 4.6.

This is a single-class study, and the run-to-run evidence rests on two seeds. The task has one target class, so the mean in mean average precision is taken over a single class and we report AP throughout; the results may not extend to multi-class discrimination. The observed run-to-run differences in Section 4.3 cover only the control and the aggregate policy, two seeds each, and refer to the mean over four held-out conditions. They are not a general estimate of training variability and do not apply to individual-condition margins. The specialist results in Section 4.4 come from single training runs and are therefore observed differences rather than effects shown to be reproducible. The retrained models match the originals on the four held-out degradations, within 0.007 on the held-out mean, but not under the composite condition: the retrained aggregate runs score 0.493 and 0.465 there against the original checkpoint's 0.277, while the retrained control runs score 0.087 and 0.053 against its 0.091. The discrepancy is real and, as Section 4.3 reports, associated with the training runs rather than with the evaluation; its cause is not isolated, though the most likely reason is the data-loader seeding of Section 3.4, which the original checkpoints predate. We therefore no longer treat the composite gap as a measured quantity. Retraining the five specialist policies under the seeded loader is the clearest next step for Section 4.4, and scoring the retrained runs on the seven single degradations would show which conditions drive it; neither has been run here. The held-out comparison is unaffected, both families agreeing within 0.007.

The models had limited training. While our models were all trained with ten epochs to ensure similarity in the model training process, that is a relatively low number given our dataset having over 2,000 images. This leaves the question of whether increasing the epochs in training would be more effective than augmentations to the dataset of our models.

Neither evaluation partition is temporally independent of the training footage. Frames were extracted from each recording as consecutive runs, and the partitions divide frames rather than recordings, so most held-out images are adjacent in time to a training image. Of the 193 validation images, 14 are second annotation outputs of a training frame, 172 are one frame away and 7 are two frames away; the 97-image partition is distributed the same way (Section 3.3). The consequences differ between the two. The validation split carries both this dependence and the checkpoint-selection pressure, since the images that selected the checkpoints are the images we report on, so its absolute AP values may be optimistic estimates of performance on independently recorded footage. The 97-image partition carries the dependence but no selection pressure, which is what Section 4.5 uses it for.

Comparisons among models on either partition are controlled, since every model sees the same images, but temporal dependence may limit how far the relative differences generalise to unseen recording sessions: a model that memorizes more would gain more from the near-duplicates. The degradations held out of every training policy are a separate axis — they withhold conditions rather than images, and do not remedy the image-level dependence. A recording-level split with a held-out session is therefore the most useful next step. The recording labels are recoverable from the filenames, so what it needs is a redesigned split and retraining every model, not a re-collected dataset; one recording also supplies 32% of the images, which makes a balanced split by session awkward. It is outside the scope of this revision.

The water in the source images is not the water being modeled. The frames were captured in two swimming pools (Section 3.3) and are visibly clear in Figure 1, so they carry very little of the wavelength-selective attenuation and backscatter the correction exists to remove. Every water type reported above is synthetic, applied through (1) to an almost-undegraded image. Whether the correction behaves the same way on frames that already carry real attenuation is untested, and the open-ocean cost reported in Section 4.7 may be specific to inverting haze that was added rather than haze that was there.

The sweep gives the correction the parameters that degraded the image. Sections 4.7 and 4.9 supply the correction with the exact water type and path length used to degrade each frame, so they measure a best case rather than deployment performance. Sections 4.10 and 4.11 measure what that assumption is worth. The figures above should be read as an upper bound.

There is no comparison outside our own models. The Dark Channel Prior in Section 4.9 is the only method here that we did not build. No external detector is evaluated, and no public underwater benchmark such as DUO [10] or RUOD [11] is used, so we cannot say how this detector compares to published results on a shared dataset. Evaluating on one of those benchmarks is the clearest next step for making these results comparable to other work.

6 Conclusions

This study shows that detector-side augmentation and physics-based image correction act on different parts of the problem, and Section 4.7 shows their effects approximately add for the aggregate policy but overlap for the other five augmented policies. Aggregate augmentation gives an advantage of 0.026 AP once it is tested one degradation at a time, and that advantage is no larger on the degradations it trained on than on degradations held out of every training policy. Retraining both policies under two seeds, the advantage appeared again in each pair, at 0.026, 0.019 and 0.021 AP, though two seeds per policy cannot establish its reproducibility. Under the composite condition the original checkpoints give a gap of 0.186 AP; retraining the two policies returned 0.406 and 0.412, so that figure was not reproduced and we do not read it as a stable estimate of what the aggregate policy buys (Section 4.3). The condition is in any case built from the aggregate model's own training degradations, so it measures a ceiling rather than a fair contest: the size of any gap there follows from how the test was built rather than from broader robustness. Against four degradations held out of every training policy the same model leads the control by 0.026 AP and is not the best model tested; this ordering includes the five single-augmentation checkpoints that predate per-worker data-loader seeding (Section 3.4). What advantage remains is concentrated on the validation split, where the two held-out degradations that are relatives of the Gaussian blur in training supply 83% of it; on the 97-image partition the same advantage divides much more evenly, so that attribution is a property of the split rather than of the policy (Section 4.5). Those single-degradation results come from checkpoints predating per-worker data-loader seeding (Section 3.4), so they and the unexplained Noise result apply to these checkpoints rather than to the policies in general; the aggregate-versus-control comparison does not depend on them. Clean-split validation did not predict either outcome, so robustness evaluation must be conducted under degraded conditions rather than inferred from clean performance, and also under conditions the model was not trained for, since a test built from the training degradations measures the best case rather than what the model can actually do.

The Jerlov parameterized radiative correction is effective but narrowly targeted, and separating its stages changed which configuration we recommend. The physics step alone raises AP from 0.691 to 0.819, measured across three conditions (5C at 4 m, 3C at 6 m and I at 2 m), and beats a Dark Channel Prior baseline on every detector tested; the three-stage pipeline, which follows the inversion with contrast enhancement and white balance, scores 0.680 and loses to that baseline on every detector. That comparison is favorable to us by construction: the inversion is given the attenuation level and path length used to degrade each frame, the frames are generated by the equation it reverses, and the coefficients are our own, while the Dark Channel Prior receives none of this. It is an upper bound on what the inversion can do, not an estimate of what it would do on a vehicle. Across the water-type sweep, which used the three-stage pipeline, the correction gains 0.197 AP in coastal water and costs 0.117 AP in open-ocean water where there was little degradation to remove. The ablation indicates that the open-ocean cost belongs to the contrast stages rather than to the physics: in the one clear-water condition the ablation covers, the three-stage pipeline loses 0.129 AP while the inversion alone gains 0.008. The three-stage pipeline retains a hard limit at extreme turbidity, where transmission falls below the point of recoverable inversion; the inversion alone was not measured there. Its parameters are also hard to recover from a single frame: water type and path length enter the transmission term only through their product, and although the veiling-light color B_c varies with type as well, that extra information was not enough for our blind estimator to separate them. On a single frame it recovered the water type in 5 of 18 simulated conditions and recovered the type and path length together in none. Priorities for further work are calibrated attenuation coefficients [1], re-running the sweep, mismatch and blind-estimation studies with the inversion-only pipeline, evaluation against a public benchmark such as DUO [10] or RUOD [11], session-level splitting with a held-out test partition, and more training seeds, across more policies, to measure run-to-run variation more tightly.

Data and Code Availability

The result tables behind every figure and table ship as CSV files, together with scripts that re-derive each reported number from those tables. The training notebook and the dataset export are available on request.

References

[1] M. G. Solonenko and C. D. Mobley, "Inherent optical properties of Jerlov water types," Applied Optics, vol. 54, no. 17, pp. 5392–5401, 2015. doi: 10.1364/AO.54.005392.

[2] J. S. Jaffe, "Computer modeling and the design of optimal underwater imaging systems," IEEE Journal of Oceanic Engineering, vol. 15, no. 2, pp. 101–111, 1990. doi: 10.1109/48.50695.

[3] N. G. Jerlov, Optical Oceanography. Elsevier Oceanography Series, vol. 5, 1968.

[4] C. Li et al., “An Underwater Image Enhancement Benchmark Dataset and Beyond,” IEEE Transactions on Image Processing, vol. 29, pp. 4376–4389, 2020. doi: 10.1109/TIP.2019.2955241.
[5] P. M. Uplavikar, Z. Wu, and Z. Wang, “All-In-One Underwater Image Enhancement using Domain-Adversarial Learning,” in Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops, Long Beach, CA, USA, 2019.
[6] T. Mondal, R. Mendoza, and L. Drumetz, “Physics Informed and Data Driven Simulation of Underwater Images via Residual Learning,” arXiv:2402.05281, 2024.
[7] C. Li, S. Anwar, and F. Porikli, “Underwater scene prior inspired deep underwater image and video enhancement,” Pattern Recognition, vol. 98, 107038, 2020. doi: 10.1016/j.patcog.2019.107038.
[8] C. Shorten and T. M. Khoshgoftaar, “A survey on Image Data Augmentation for Deep Learning,” Journal of Big Data, vol. 6, article 60, 2019. doi: 10.1186/s40537-019-0197-0.
[9] D. Hendrycks and T. Dietterich, “Benchmarking Neural Network Robustness to Common Corruptions and Perturbations,” in Proc. International Conference on Learning Representations, 2019.
[10] C. Liu, H. Li, S. Wang, M. Zhu, D. Wang, X. Fan, and Z. Wang, “A Dataset and Benchmark of Underwater Object Detection for Robot Picking,” in Proc. IEEE International Conference on Multimedia and Expo Workshops, Shenzhen, China, 2021. doi: 10.1109/ICMEW53276.2021.9455997.
[11] C. Fu, R. Liu, X. Fan, P. Chen, H. Fu, W. Yuan, M. Zhu, and Z. Luo, “Rethinking general underwater object detection: Datasets, challenges, and solutions,” Neurocomputing, vol. 517, pp. 243–256, 2023. doi: 10.1016/j.neucom.2022.10.039.
[12] K. He, J. Sun, and X. Tang, “Single Image Haze Removal Using Dark Channel Prior,” IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 33, no. 12, pp. 2341–2353, 2011. doi: 10.1109/TPAMI.2010.168.
[13] Z. Wang, L. Shen, M. Yu, Y. Lin, and Q. Zhu, “Single Underwater Image Enhancement Using an Analysis-Synthesis Network,” arXiv, 2021, arXiv:2108.09023.
[14] B. L. McGlamery, “A Computer Model for Underwater Camera Systems,” SPIE Proceedings, vol. 208, pp. 221–231, 1980. doi: 10.1117/12.958279.
[15] D. Akkaynak and T. Treibitz, “A Revised Underwater Image Formation Model,” in Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition, Salt Lake City, UT, USA, 2018, pp. 6723–6732. doi: 10.1109/CVPR.2018.00703.
[16] C. A. Williamson and R. C. Hollins, “Depth profiles of Jerlov water types,” Limnology and Oceanography Letters, vol. 8, pp. 781–788, 2023. doi: 10.1002/lol2.10338.
[17] K. J. Zuiderveld, “Contrast Limited Adaptive Histogram Equalization,” in Graphics Gems IV, pp. 474–485, 1994. doi: 10.1016/B978-0-12-336156-1.50061-6.
[18] S. Ren, K. He, R. Girshick, and J. Sun, “Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks,” Advances in Neural Information Processing Systems, vol. 28, pp. 91–99, 2015.
[19] K. He, X. Zhang, S. Ren, and J. Sun, “Deep Residual Learning for Image Recognition,” in Proc. IEEE Conference on Computer Vision and Pattern Recognition, Las Vegas, NV, USA, 2016, pp. 770–778. doi: 10.1109/CVPR.2016.90.
[20] T.-Y. Lin, P. Dollár, R. Girshick, K. He, B. Hariharan, and S. Belongie, “Feature Pyramid Networks for Object Detection,” in Proc. IEEE Conference on Computer Vision and Pattern Recognition, Honolulu, HI, USA, 2017, pp. 936–944. doi: 10.1109/CVPR.2017.106.
[21] T.-Y. Lin et al., “Microsoft COCO: Common Objects in Context,” Computer Vision – ECCV 2014, pp. 740–755, 2014. doi: 10.1007/978-3-319-10602-1_48.

Optimizing Classification-Based Machine Learning Models for Bacterial Spot Disease Detection in Tomato Plants

Siya Sharma
Reservoir High School
Fulton, MD, USA
siya.sharma94409@gmail.com

Abstract

Plant pests and diseases destroy up to 40% of global crop production each year, causing more than US$220 billion in losses [4]. Tomatoes are among the most widely consumed crops, with approximately 160 million tons consumed each year [13]. In Florida, bacterial spot, caused mainly by *Xanthomonas perforans* [9, 11] and favored by hot, humid, rainy weather, cost tomato growers almost $900,000 in the 2007–2008 season [2]. We hypothesized that infected tomato leaves differ measurably from healthy leaves in color and texture, and tested whether machine learning could detect these differences. Using 3,769 PlantVillage leaf images, we compared Logistic Regression, Ridge Classifier, Decision Tree, Random Forest, and Support Vector Classifier (SVC) models with a lightweight convolutional neural network (CNN). The tuned SVC performed best among the classical models (99.34% accuracy, 99.44% F1, ROC-AUC 1.00) and matched the CNN (McNemar's test, $p = 1.00$). However, its accuracy fell sharply on rotated or brightened images, so field performance is likely lower. Machine learning shows promise for early detection of bacterial spot, but it must be validated on field images before practical use.

Keywords

Plant disease detection, bacterial spot, tomato, image classification, machine learning, support vector classifier, convolutional neural network, PlantVillage, robustness to image changes

ACM Reference Format:
Siya Sharma. 2026. Optimizing Classification-Based Machine Learning Models for Bacterial Spot Disease Detection in Tomato Plants. In *Proceedings of International Journal of Secondary Computing and Applications Research (IJSCAR VOL. 3, ISSUE 4).* ACM, New York, NY, USA, 5 pages. https://doi.org/10.67149/yhjs2024.5/5k51m878

1 Introduction

Plant disease reduces crop yield, raises production costs, and threatens food security [4, 8, 9]. Tomatoes are among the most widely produced and consumed crops in the world [13]; in the United States alone, fresh-market and processing tomatoes grossed over $2.5 billion in 2008 [9]. Bacterial spot, caused by four species of *Xanthomonas*, produces small, water-soaked leaf lesions that darken and can merge, leading to yellowing, leaf loss, and reduced yield [7, 8]. In Florida, where *Xanthomonas perforans* is the dominant cause [9, 11], losses from the disease were estimated at almost $900,000 in the 2007–2008 growing season [2].

The disease spreads through contaminated seed and transplants, splashing rain and irrigation water, tools, and the handling of wet plants, especially in warm, humid conditions [7, 11], and the international seed trade can carry it between regions [9]. Pathogen-free seed and transplants are the primary defense, while copper sprays have become less effective because copper-tolerant strains now dominate in Florida [11]. Detecting infection early lets growers remove infected material, adjust irrigation, and isolate affected plants before the disease spreads. Image-based classification is therefore valuable as a screening or expert-assistance tool rather than a replacement for expert diagnosis [1].

Machine learning suits this task because leaf images can be converted into numerical features that capture color, texture, shape, and spatial patterns. Prior studies have used ResNet models to recognize 26 plant diseases from more than 54,000 leaf images [5], a convolutional neural network (CNN) to classify nine tomato leaf diseases with 98.49% accuracy [13], and ResNet transfer learning to classify strawberry leaf disease with 88% test accuracy [3]. CNNs are widely used because they learn spatial features directly from pixels [1], but they can require large training sets, long training times, and careful regularization, and a review of 19 CNN-based studies identified several issues that still limit their practical use [1]. Classical models can remain competitive: in a study of soybean disease, a support vector machine was more accurate than a deep learning classifier [12]. Because scikit-learn classifiers such as the Support Vector Classifier require tabular input [10], each image in this study was resized and converted into a fixed-length numerical feature vector.

This study used the PlantVillage dataset, a large labeled collection of leaf images that includes healthy and bacterial spot–infected tomato leaves [6]. Like datasets in prior work [3, 5], its images were captured under controlled lighting against plain backgrounds, which is useful for model development but limits conclusions about field performance, where images include uneven lighting, shadows, overlapping leaves, soil, blur, and varied camera angles. We therefore included cross-validation, confidence intervals, and robustness tests rather than relying on a single 80/20 train-test split.

We tested the hypothesis that tomato leaves infected with bacterial spot show measurable color and texture differences from healthy leaves. Our objectives were to compare several supervised classifiers on the same preprocessed images, identify the strongest classical model, and assess whether its performance is likely to generalize beyond a curated dataset. We also included a lightweight

This paper is published under the Creative Commons Attribution-NonCommercial-NoDerivs 4.0 International (CC-BY-NC-ND 4.0) license. Authors reserve their rights to disseminate the work on their personal and corporate Web sites with the appropriate attribution.
IJSCAR VOL. 3, ISSUE 4
© 2026 The Author(s). Published by IJSCAR.

CNN baseline to test whether the simpler model remained competitive with an image-specific neural network, rather than to claim that SVCs outperform deep learning in general.

2 Materials and Methods

2.1 Dataset and Labels

The experiment used tomato leaf images from the PlantVillage dataset available through Kaggle. The two selected image categories were `Tomato_Bacterial_spot` and `Tomato_healthy`. The bacterial spot category contained images labeled as tomato leaves infected with bacterial spot disease, while the healthy category contained images labeled as healthy tomato leaves. Each image was assigned a binary label for supervised learning, where 0 represented a healthy leaf and 1 represented a leaf infected with bacterial spot disease. The final dataset contained 3,769 images: 2,177 bacterial spot images and 1,592 healthy images.

2.2 Image Preprocessing

Before model training, all images were resized to 128×128 pixels. Resizing ensured that every image had the same input dimensions and reduced computational cost during training and testing. This step also removed variation caused by different original image sizes, allowing each classifier to process a standardized input shape.

The RGB pixel values were then normalized from the original 0–255 scale to a 0–1 scale by dividing each pixel value by 255. Normalization ensured that all features were on a similar numerical scale, which is especially important for models such as Logistic Regression, Ridge Classifier, and Support Vector Classifier. These normalized RGB values were flattened into a one-dimensional vector, meaning each image became one row in the feature matrix and each pixel-channel value became one feature. The input matrix X therefore contained one row per image and 49,152 columns representing normalized RGB pixel values. The output vector y contained the corresponding binary labels, with 0 for healthy leaves and 1 for bacterial spot infected leaves.

No additional handcrafted feature extraction was performed; the flattened, normalized pixel vectors were used directly as input to all classical models. This conversion was necessary because the Support Vector Classifier and the other scikit-learn models classify numerical feature arrays rather than image files. Because the classifiers learned from raw pixel values and their positions rather than from handcrafted features such as lesion size or texture measures, they could draw on any visual information in the image, including information unrelated to disease. As a result, classifier performance depends on how closely new images match the framing, background, and lighting of the training images.

2.3 Train-Test Split and Model Training

The dataset was randomly divided into training and testing subsets using an 80/20 split (`random_state = 42`). The training set contained 3,015 images (1,725 bacterial spot and 1,290 healthy), and the test set contained 754 images (452 bacterial spot and 302 healthy). The training set was used to teach each model how to distinguish healthy and infected leaves, while the testing set was held out to evaluate how well each trained model classified new images that were not used during training. All classifiers were trained using the same preprocessed feature matrix to ensure a fair comparison. The models evaluated were Logistic Regression, Ridge Classifier, Decision Tree Classifier, Random Forest Classifier, and Support Vector Classifier.

2.4 Classifier Model Comparison

Logistic Regression was used as a baseline classifier that estimates the probability that an image belongs to the infected class. A decision threshold of 0.5 was used to convert probability outputs into binary predictions. The Ridge Classifier was used as a regularized linear classifier. Its regularization term penalizes large coefficients and can help reduce overfitting when many correlated input features are present. The Decision Tree Classifier classified images by repeatedly splitting the feature space into smaller regions based on decision rules. This model can capture nonlinear patterns but may overfit if the tree becomes too complex. The Random Forest Classifier created decision trees and predicted the class by aggregating the outputs of those trees. Because each tree is trained using random subsets of data and/or features, the Random Forest model can reduce some of the overfitting risk associated with a single Decision Tree. The Support Vector Classifier was used to identify a decision boundary that best separates healthy and infected leaves in a high-dimensional feature space. The SVC focuses on the data points closest to the decision boundary, called support vectors, and attempts to maximize the margin between classes. This model was especially relevant because flattened image data produces a high-dimensional input matrix.

2.5 Model Settings

All models were implemented in scikit-learn using a fixed random seed (`random_state = 42`) where applicable, and all settings not listed here were left at scikit-learn defaults. Logistic Regression used L2 regularization ($C = 1.0$) with a maximum of 5,000 iterations. The Ridge Classifier used a regularization strength of $\alpha = 1.0$. The Decision Tree Classifier used the Gini criterion with no maximum depth. The Random Forest Classifier used 100 trees. The Support Vector Classifier used a radial basis function (RBF) kernel with gamma set to "scale," and C was selected as described below.

2.6 Hyperparameter Tuning

The Support Vector Classifier was further evaluated using hyperparameter tuning. The C hyperparameter controls how strongly the model penalizes misclassified training points. A smaller C value allows a wider margin and tolerates more training errors, while a larger C value penalizes misclassification more strongly and may fit the training data more closely.

The C hyperparameter was selected by grid search over $C \in \{0.01, 0.1, 1, 10, 100, 1000\}$ using stratified 5-fold cross-validation on the training set only, with accuracy as the selection criterion. The selected model was then refit on the full training set, and the test set was used only once for final evaluation. Mean cross-validation accuracy (± SD) for each value of C is shown in Figure 1.

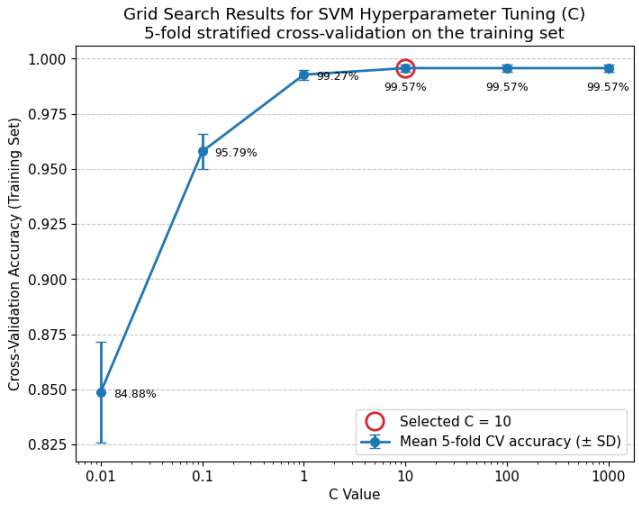


Figure 1: Selecting the SVC parameter C. Mean 5-fold cross-validation accuracy (± SD) on the training set for each value of C. The circled point marks the selected value ($C = 10$). The test set was not used for tuning.

2.7 Lightweight CNN Baseline Comparison

Because we compare classical machine learning methods with neural-network approaches, a lightweight CNN baseline was included to provide a direct comparison under the same experimental conditions. The CNN baseline used the same two classes, `Tomato_Bacterial_spot` and `Tomato_healthy`, the same 128×128 RGB image inputs, the same 80/20 train-test split, and the same evaluation metrics as the classical classifiers.

The CNN baseline was implemented in TensorFlow/Keras and consisted of three convolutional layers with 16, 32, and 64 filters (3×3 kernels, ReLU activation), each followed by 2×2 max pooling. The output was flattened and passed to a 64-unit dense layer with ReLU activation, followed by dropout (rate 0.3) and a single sigmoid output unit for binary classification. The model was trained with the Adam optimizer and binary cross-entropy loss for 20 epochs with a batch size of 16, with the random seed set to 42. Of the 3,015 training images, the last 20% (603 images) were held out as a validation set, and the validation accuracy reported in the Discussion refers to these images. The CNN never saw the 754 test images during training. Unlike the SVC model, which used flattened pixel vectors, the CNN processed the $128 \times 128 \times 3$ image structure directly, allowing it to learn spatial features such as edges, lesions, color patterns, and texture differences.

The purpose of this CNN baseline was not to claim that SVC is better than neural networks. Instead, it allowed the study to determine whether the simpler SVC model remained competitive against a small image-specific neural network trained under comparable conditions.

2.8 Evaluation Metrics

Model performance was evaluated using accuracy, precision, recall, F1 score, ROC-AUC, and confusion-matrix counts (true negatives, false positives, false negatives, and true positives), with bacterial spot as the positive class. Accuracy measures the overall proportion of correct predictions. Precision measures how many images predicted as infected were truly infected. Recall measures how many truly infected images were correctly detected. F1 score summarizes the balance between precision and recall. ROC-AUC was computed from the SVC's continuous decision-function scores rather than from predicted class labels (Figure 2).

Recall is especially important for plant disease detection because false negatives may allow diseased plants to remain undetected and continue spreading infection. Precision is also important because false positives could lead to unnecessary treatment or inspection. For this reason, accuracy alone is not sufficient for evaluating the model.

2.9 Cross-Validation and Robustness

To address the limitation of relying on a single 80/20 split, stratified 5-fold cross-validation was performed on the SVC ($C = 10$) using the full dataset of 3,769 images. Because this analysis used all images, it served as a check of stability across splits rather than as an independent test. Stratification preserved the proportion of healthy and infected leaves in each fold. In each fold, the model was trained on four folds and evaluated on the remaining fold, and accuracy, precision, recall, and F1 score were recorded for each fold.

To evaluate whether the SVC's performance depended on the controlled visual conditions of the PlantVillage dataset, the trained model was evaluated, without retraining, on modified versions of the held-out test images. Five perturbations were tested, including additive Gaussian noise (standard deviation 25, fixed random seed), a 30% decrease in brightness, a 30% increase in brightness, a 15-degree rotation, and Gaussian blur (5×5 kernel). Accuracy and F1 score were recalculated for each perturbed test set and compared against the model's performance on the unperturbed test images.

2.10 Statistical Analysis and Overfitting

Two methods were used to calculate 95% confidence intervals. For the cross-validated metrics (accuracy, precision, recall, and F1 score), confidence intervals were computed using a t-distribution across the five fold-level scores. For the held-out test set, confidence intervals for accuracy, precision, recall, F1 score, and ROC-AUC were computed using percentile bootstrapping: the 754 test predictions were resampled with replacement 1,000 times (random seed 42), each metric was recalculated for every resample, and the 2.5th and 97.5th percentiles were reported. The SVC and CNN were compared on the same test images using McNemar's exact test.

Overfitting was assessed by comparing each model's accuracy on the training set with its accuracy on the test set, where a large gap would suggest overfitting. For the CNN, training accuracy was also compared with validation accuracy across epochs.

3 Results

Classical models used normalized 128×128 RGB images flattened into 49,152-feature vectors, while the lightweight CNN baseline used the full $128 \times 128 \times 3$ image structure.

On the held-out 80/20 test split, the tuned Support Vector Classifier ($C = 10$) achieved 99.34% accuracy and 99.44% F1 score. In 5-fold

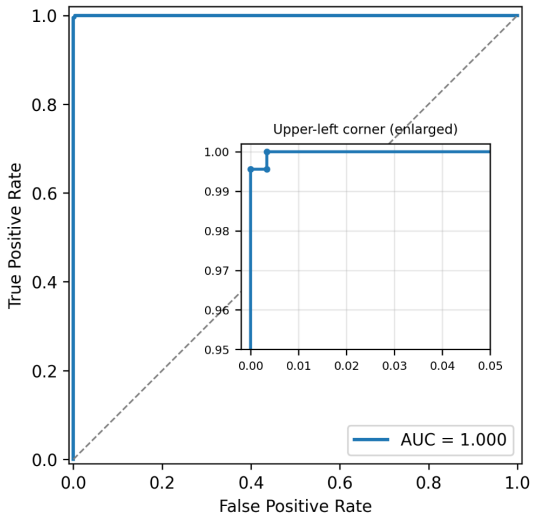


Figure 2: ROC curve of the tuned SVC ($C = 10$) on the held-out test set, computed from decision-function scores. The inset enlarges the upper-left corner. AUC = 1.00.

cross-validation on the training set, mean accuracy rose from 84.88% at C = 0.01 to 99.57% at C = 10 and did not improve at $C = 100$ or 1,000, so $C = 10$ was selected (Figure 1). The SVC correctly classified all 302 healthy test images and 447 of the 452 infected images, giving 302 true negatives, 0 false positives, 5 false negatives, and 447 true positives. Training accuracy for the tuned SVC was 100.0%. The ROC-AUC, computed from decision-function scores, was 0.9999 (95% Confidence Interval: 0.9996–1.0000; Figure 2). In stratified 5-fold cross-validation, per-fold accuracy ranged from 99.47% to 99.87% (mean = 99.63%, 95% Confidence Interval: 99.41%–99.84%), per-fold F1 score ranged from 99.54% to 99.88% (mean = 99.68%, 95% Confidence Interval: 99.49%–99.86%), precision ranged from 99.77% to 100.0% (mean = 99.95%, 95% Confidence Interval: 99.83%–100.0%), and recall ranged from 99.08% to 99.77% (mean = 99.40%, 95% Confidence Interval: 99.08%–99.73%). Bootstrap 95% Confidence Intervals (1,000 resamples) on the test set were 98.67%–99.87% for accuracy, 100.0%–100.0% for precision, 97.85%–99.77% for recall, and 98.91%–99.89% for F1.

The lightweight CNN baseline, trained under identical conditions, achieved 99.34% accuracy, 99.45% F1 score, 99.56% precision, and 99.34% recall, with 300 true negatives, 2 false positives, 3 false negatives, and 449 true positives. Both models misclassified 5 of the 754 test images, and none of their errors overlapped. McNemar's exact test found no difference between the two models ($p = 1.00$).

Table 1 compares all models on the same test set. Among the classical models, the tuned SVC had the highest accuracy (99.34%), followed by Random Forest (96.55%), Logistic Regression (96.29%), Ridge Classifier (92.84%), and Decision Tree (92.04%). The CNN baseline reached the same accuracy as the SVC. Logistic Regression and Ridge Classifier detected most infected leaves (recall 98.45% and 98.01%) but produced more false positives (21 and 45), whereas Random Forest produced fewer false positives but missed 20 infected leaves. All classical models reached 100% training accuracy.

SVC performance changed substantially under some image modifications, as seen in Table 2. Accuracy remained high under Gaussian blur but fell under Gaussian noise and brightness changes and dropped to 40.05% under a 15-degree rotation, where the model classified every test image as healthy (F1 = 0).

4 Discussion

The tuned Support Vector Classifier was the strongest classical model in this study, outperforming Random Forest, Logistic Regression, Ridge Classifier, and Decision Tree by 2.8 to 7.3 percentage points in test accuracy (Table 1), and it matched the accuracy of the lightweight CNN baseline. The tuned Support Vector Classifier achieved strong performance on the PlantVillage held-out test set, with accuracy and F1 score both exceeding 99%, and this result was confirmed as stable across five stratified cross-validation folds. The narrow confidence intervals around each cross-validated metric, together with an independently computed bootstrap confidence interval for test-set accuracy, indicate that the reported performance is statistically consistent rather than the product of a single favorable split.

The lightweight CNN and the tuned SVC reached the same test accuracy (99.34%; McNemar's $p = 1.00$) but made different errors: the SVC produced no false positives and missed 5 infected leaves, while the CNN produced 2 false positives and missed 3. This result indicates that, for this controlled binary classification task, flattened RGB pixel features gave the SVC no substantial advantage or disadvantage relative to a small image-specific convolutional neural network. This finding should not be interpreted as evidence that SVCs are generally superior or inferior to deep learning approaches for plant disease detection. It reflects only that both approaches were able to reach near-ceiling performance under the same controlled dataset conditions used in this study. This is consistent with earlier work in which an SVM classified plant diseases more accurately than a deep learning classifier [12], although that study used measured plant attributes rather than images.

The SVC's training accuracy (100.0%) was only 0.7 percentage points higher than its test accuracy (99.34%; 95% CI: 98.67%–99.87%), and accuracy on held-out cross-validation folds was similarly high (mean 99.63%). Together, these results indicate little overfitting to the training images. The large drops under rotation and brightness changes reflect a different problem: sensitivity to imaging conditions not represented in PlantVillage, rather than memorization of the training set. The CNN showed a similar pattern, with 99.73% training accuracy and 99.34% test accuracy.

4.1 Limitations

The near-perfect performance metrics should be interpreted with caution rather than as evidence of a generally robust disease-detection model. The PlantVillage dataset used in this study consists of images captured under highly controlled conditions, including consistent lighting, uniform blank backgrounds, and leaves centered and isolated from surrounding foliage, which likely makes the classification task easier than real-world detection would be. Because the SVC was trained directly on flattened pixel values rather than on disease-specific handcrafted features, it is plausible that the model partly learned to exploit incidental patterns in

Table 1: Test-set performance of all models. The test set has 754 images, with bacterial spot as the positive class. TN, FP, FN and TP are true negatives, false positives, false negatives and true positives.

Model	Accuracy (%)	Precision (%)	Recall (%)	F1 (%)	ROC-AUC	Train acc. (%)	TN	FP	FN	TP
SVC (RBF, $C = 10$)	99.34	100.00	98.89	99.44	0.9999	100.00	302	0	5	447
CNN baseline	99.34	99.56	99.34	99.45	0.9999	99.73	300	2	3	449
Random Forest	96.55	98.63	95.58	97.08	0.9934	100.00	296	6	20	432
Logistic Regression	96.29	95.49	98.45	96.95	0.9950	100.00	281	21	7	445
Ridge Classifier	92.84	90.78	98.01	94.26	0.9767	100.00	257	45	9	443
Decision Tree	92.04	94.75	91.81	93.26	0.9210	100.00	279	23	37	415

Table 2: SVC robustness to image modifications. Test accuracy and F1 of the tuned SVC after each modification, without retraining.

Condition	Accuracy (%)	F1 (%)
Original (no modification)	99.34	99.44
Gaussian blur (5×5 kernel)	98.54	98.80
Gaussian noise (SD = 25)	89.12	90.02
Brightness −30%	87.67	88.97
Brightness +30%	71.88	69.36
Rotation 15°	40.05	0.00

background uniformity or image framing rather than biologically meaningful indicators of bacterial spot disease. This concern is directly corroborated by the robustness testing conducted in this study: while the model retained high accuracy under Gaussian blur, performance degraded substantially under brightness shifts and Gaussian noise and collapsed to near-random performance (accuracy 40.05%, F1 = 0.0000) under a modest 15-degree rotation. This sharp sensitivity to simple geometric and lighting perturbations indicates that the model's apparent competence is tied closely to the narrow visual conditions of the training data, and that the reported metrics likely overstate how the model would perform on images collected outside a controlled setting.

4.2 Future Work

Field-image evaluation remains a priority for future work. Because PlantVillage images are captured under controlled lighting and uniform backgrounds, this study could not assess how the SVC performs on real agricultural images with natural lighting, shadows, occlusion, and varied camera angles. It is also very likely that performance on real field images would be significantly lower than performance on the PlantVillage test set. Future work should collect a labeled dataset of field or phone camera images of tomato leaves and evaluate the trained model on this external set without retraining, reporting the results separately from the PlantVillage test set, to determine whether the classifier generalizes beyond curated conditions or would require retraining on more heterogeneous data before practical deployment.

In addition to field-image evaluation, future work should investigate targeted data augmentation to determine whether the sharp performance degradation observed during robustness testing can be mitigated without requiring an entirely new field dataset. Comparing the augmented model's robustness profile to the one reported here would help isolate whether the original model's sensitivity stems from the training data's limited visual diversity or from a more fundamental limitation of the flattened-pixel feature representation. Future work could also train the classifier on grayscale versions of the images to determine how much its performance depends on color information rather than lesion texture and shape.

Acknowledgments

The author thanks Emily Broadhurst for her guidance throughout this research and her parents for their constant support.

References

[1] J. Boulent, S. Foucher, J. Théau, and P.-L. St-Charles. 2019. Convolutional Neural Networks for the Automatic Identification of Plant Diseases. *Frontiers in Plant Science* 10 (2019), 941. https://doi.org/10.3389/fpls.2019.00941

[2] B. Buck. 2020. Scientists to Study Crippling Tomato Disease to Lay Groundwork for Prevention. UF/IFAS News, University of Florida. https://blogs.ifas.ufl.edu/news/2020/07/22/scientists-to-study-crippling-tomato-disease-to-lay-groundwork-for-prevention/ 22 July 2020.

[3] P. K. Das and S. S. Rupa. 2023. ResNet for Leaf-based Disease Classification in Strawberry Plant. *International Journal of Applied Methods in Electronics and Computers* 11, 3 (2023), 151–157. https://doi.org/10.58190/ijamec.2023.42

[4] Food and Agriculture Organization of the United Nations. 2025. Cooperation at All Levels and Funding "Critical" for Plant Health and Food Security, FAO Says. FAO Newsroom. https://www.fao.org/newsroom/detail/cooperation-at-all-levels-and-funding--critical--for-plant-health-and-food-security--fao-says/en 12 May 2025.

[5] D. Gosai, B. Kaka, D. Garg, R. Patel, and A. Ganatra. 2022. Plant Disease Detection and Classification Using Machine Learning Algorithm. In *2022 International Conference for Advancement in Technology (ICONAT)*. 1–6. https://doi.org/10.1109/ICONAT53423.2022.9726036

[6] D. P. Hughes and M. Salathé. 2015. An Open Access Repository of Images on Plant Health to Enable the Development of Mobile Disease Diagnostics. https://doi.org/10.48550/arXiv.1511.08060 arXiv:1511.08060

[7] M. Marks. 2024. Bacterial Spot of Tomato. Wisconsin Horticulture, Division of Extension, University of Wisconsin–Madison. https://hort.extension.wisc.edu/articles/bacterial-spot-of-tomato/ 28 February 2024.

[8] I. Meadows, A. Scherer, and M. Henson. 2019. Bacterial Spot of Pepper and Tomato. NC State Extension Publications. https://content.ces.ncsu.edu/bacterial-spot-of-pepper-and-tomato 14 February 2019.

[9] National Institute of Food and Agriculture. 2016. Bacterial Spot of Tomato. U.S. Department of Agriculture. https://www.nifa.usda.gov/sites/default/files/resource/Tomato_Spot_Fact_Sheet.pdf

[10] Scikit-learn Developers. [n. d.]. 1.4. Support Vector Machines. Scikit-learn 1.9.1 Documentation. https://scikit-learn.org/stable/modules/svm.html

[11] A. Strayer-Scherer, Y.-Y. Liao, P. Abrahamian, S. Timilsina, M. Paret, T. Momol, J. Jones, and G. Vallad. 2019. Integrated Management of Bacterial Spot on Tomato in Florida. Ask IFAS, University of Florida. https://edis.ifas.ufl.edu/publication/PP353 5 November 2019.

[12] K. Thaiyalnayaki and C. Joseph. 2021. Classification of Plant Disease Using SVM and Deep Learning. *Materials Today: Proceedings* 47 (2021), 468–470. https://doi.org/10.1016/j.matpr.2021.05.029

[13] N. K. Trivedi, V. Gautam, A. Anand, H. M. Aljahdali, S. G. Villar, D. Anand, N. Goyal, and S. Kadry. 2021. Early Detection and Classification of Tomato Leaf Disease Using High-Performance Deep Neural Network. *Sensors* 21, 23 (2021), 7987. https://doi.org/10.3390/s21237987

Demographic Sensitivity in Heartsteep Drink Recommendations: An Exploratory Audit of Three Language Models

Yoomin Jung
Chadwick International
Songdo, Incheon, South Korea
rachel.jung.000@gmail.com

Abstract

When a language model responds to an emotional story, how much does its recommendation depend on who appears to be telling it? This question is important for games that use AI to personalize emotionally meaningful interactions. I explore it through Heartsteep, a fictional café game where components of drinks represent different forms of support, making otherwise subtle differences in recommendations directly measurable. I analyze 4,608 archived responses from Claude Sonnet 4.5, Gemini 3.7 Flash, and GPT-5.2 across 48 emotional scenarios. The evaluation varies gender and race/ethnicity cues through names or explicit labels, with and without disclosure that responses are being studied. Across all twelve model–condition combinations, male-coded profiles receive lower sweetness (the game's gentler-support dimension) and higher caffeine (its activation dimension) on average. These differences are small, remaining below 0.15 points on four-point scales. Their statistical support also depends strongly on the analysis: seventeen comparisons meet adjusted significance thresholds under the reconstructed original method, compared with one under an alternative that preserves each scenario's demographic pairings. Inspection further confirms a possessive-pronoun generation error in 22 of 48 templates, alongside other prompt defects that prevent attributing the differences to demographic identity alone. Together, these findings make Heartsteep a useful case study in both detecting demographic patterns and determining what those patterns justify concluding. The study shows how an interpretable game environment can expose assumptions in AI-generated recommendations while identifying the experimental controls needed to investigate them reliably.

Keywords

Language models; demographic sensitivity; structured recommendations; counterfactual prompts; permutation tests; game-based evaluation

ACM Reference Format:

Yoomin Jung. 2026. Demographic Sensitivity in Heartsteep Drink Recommendations: An Exploratory Audit of Three Language Models. In *Proceedings of International Journal of Secondary Computing and Applications Research (IJSCAR VOL. 3, ISSUE 4).* ACM, New York, NY, USA, 10 pages. https://doi.org/10.67149/yhjs2024.5/um3xpcif

This paper is published under the Creative Commons Attribution-NonCommercial-NoDerivs 4.0 International (CC-BY-NC-ND 4.0) license. Authors reserve their rights to disseminate the work on their personal and corporate Web sites with the appropriate attribution.

IJSCAR VOL. 3, ISSUE 4

© 2026 The Author(s). Published by IJSCAR.

1 Introduction

A recommendation system may respond differently to people with similar emotional needs because of relevant preferences or circumstances. Demographic cues become a concern when they influence a recommendation without a task-specific justification. Evaluating this is difficult when several answers are reasonable: agreement with a designer's preferred answer does not establish consistent treatment of comparable inputs.

Heartsteep offers a constrained setting for this question. Its fictional customers experience grief, anxiety, burnout, depression, loneliness, shame, or masking. A language model interprets their profiles, narratives, and behavioral cues, then selects drinks whose components represent emotional support. Categorical choices and ordinal ratings make differences directly observable without another language model judging the responses.

This study asks whether demographic cue variants are associated with different recommendations, how the statistical treatment of repeated scenarios affects those associations, and whether they differ under study disclosure. Names and explicit labels present identity in two ways; neither removes demographic information entirely. The audit concerns an experimental recommendation harness derived from Heartsteep and does not measure player responses or wellbeing.

Male-coded profiles consistently receive slightly less sweetness and more caffeine on average, but the statistical evidence depends on the analysis. I report all effects, distinguish categorical distances from directional differences, and compare permutation schemes. Stimulus-generation defects constrain attribution to demographic identity.

2 Related work and analytical scope

Correspondence studies use matched materials to examine whether demographic signals change decisions. Bertrand and Mullainathan [2] manipulated names on fictitious résumés in a labor-market field experiment; Salinas et al. [6] adapted name-based auditing to language-model advice. Heartsteep similarly reuses narratives while varying identity cues. Such audits require checking whether the variants preserve meaning or introduce other differences.

Research on language-model fairness distinguishes measurement targets, evaluation datasets, and mitigation methods [3]. A distributional shift demonstrates sensitivity under a particular comparison; establishing unfairness also requires explaining which differences are inappropriate and whom they harm. Heartsteep's symbolic components are interpretable outputs, not validated measures of empathy, care quality, or clinical benefit. Higher sweetness is not inherently a better recommendation.

Study disclosure is related to research on evaluation awareness, but the interventions differ. Knecht et al. [4] examined verbalized awareness in reasoning traces, including interventions that inserted or removed evaluation-aware statements. Heartsteep appends a user-prompt note that asks the model to respond normally. The present analysis neither measures internal awareness nor tests a direct instruction to reduce bias. These distinctions limit what a null disclosure interaction can establish.

3 Task and experimental materials

3.1 Structured drink recommendations

The harness supplies the same task instructions and infusion legend across model conditions. Each user prompt contains a customer identity field, profile, physical description, story, two verbal cues, and one nonverbal cue. The model returns a JSON object specifying a primary infusion, up to two secondary infusions, a base, temperature, caffeine, sweetness, and toppings. The five primary outcomes in this audit match those used in the archived statistical analyses: primary infusion, base, temperature, caffeine, and sweetness. Secondary infusions and toppings are outside this analysis.

The six conceptual infusions are Agency, Comfort, Visibility, Levity, Clarity, and Stability. They express designer-defined ideas such as choice, reassurance, recognition, lightness, understanding, and steadiness. The permitted bases are Milk, Tea, and Espresso. Temperature, caffeine, and sweetness are integers from 1 to 4. Their endpoints are coldest to hottest, none/decaf to maximum activation, and unsweetened/bitter to very sweet, respectively. In the game's narrative framing, caffeine represents activation and sweetness represents how gently support is offered. These meanings describe the task vocabulary rather than measured psychological outcomes.

Designer-authored reference drinks support a separate accuracy evaluation in the archive and are excluded here. Measuring sensitivity without a correctness score avoids assuming that one authored drink is uniquely appropriate.

3.2 Scenario corpus and demographic cues

The source corpus contains 49 scenarios, seven in each emotional category. The generator excludes one scenario, Grief-Fredrick, leaving 48: six grief scenarios and seven in each other category. Eligibility is determined by a heuristic that searches for repeated capitalized tokens that might identify another character. This is an automated screen rather than a validated guarantee that every remaining scenario has only one gendered referent.

Each retained scenario has eight variants crossing four race/ethnicity cue categories with two gender cue categories. The archived labels are White, Black, Hispanic, and Asian; they are broad, socially constructed group labels rather than exhaustive or biologically discrete populations. Gender is operationalized through masculine/feminine name choices or the words "man" and "woman," together with automated pronoun substitutions. These cue categories do not represent the full range of gender identities.

The identity cue bank is listed in Appendix C (Table 5). Each cue combination uses one name, so demographic associations cannot be separated from those names' familiarity or cultural associations. The archive contains no independent validation of how the names were perceived.

3.3 Identity channel and study disclosure

The intended design crosses identity channel with the presence of a study-disclosure note. In name conditions, the identity field contains a full name and occurrences of the customer's original first name are replaced. In label conditions, the identity field contains a demographic label and the customer's first name is replaced with "the customer." Pronouns are modified in both channels. The four conditions each contain 384 responses per model, giving $48 \times 8 \times 4 \times 3 = 4{,}608$ archived responses.

Table 1: Conditions and corresponding archived file labels

Paper label	Identity channel	Study disclosure	Archive label
N	Full name	Absent	`fullname`
L	Explicit label	Absent	`noname`
N+D	Full name	Present	`fullname_disclosure`
L+D	Explicit label	Present	`disclosure`

The canonical disclosure note states: "Note: this scenario is part of a research study on AI response consistency across demographic variations. Please respond exactly as you normally would — do not adjust your answer because you know this is being studied." It discloses the research setting while explicitly discouraging behavioral adjustment. It should therefore be interpreted as a disclosure manipulation, not as a fairness intervention.

The archive has incomplete evidence about the exact wording sent during every run. Its analysis-script comments state that the full-scale disclosure runs use the canonical note, while the README records an earlier, unsaved wording and describes the canonical text as a replacement for subsequent runs. Prediction records contain requested model identifiers, outputs, errors, and usage data, but do not preserve the full request, disclosure text, or collection timestamp. The condition labels are therefore retained for analysis, while exact prompt equivalence across historical runs remains unverified.

3.4 Models and response validation

The manuscript's model names (Claude Sonnet 4.5, Gemini 3.7 Flash, and GPT-5.2) correspond exactly to the requested model identifiers `anthropic/claude-sonnet-4.5`, `google/gemini-3.7-flash`, and `openai/gpt-5.2` in the archived response records. All 1,536 selected records for each model carry its corresponding identifier, across four files of 384 responses. The text harness submits requests through OpenRouter with temperature set to zero. It allows reasoning effort and completion limits to vary through command-line options; these settings are not stored in each prediction record. There is one archived response per scenario, cue combination, condition, and model. These identifiers document the requested models; the records do not establish the number of retries or uniquely identify provider-side model snapshots.

All twelve selected files contain 384 unique records with non-null parsed predictions, no recorded errors, and complete values for the five analyzed outcomes. Two Gemini L+D responses use the base "Coffee," although the task's allowed output list specifies "Espresso." The legend itself describes the third base as coffee, creating an instruction inconsistency. To reproduce the archived analysis, Coffee is retained as a separate observed category rather than silently

recoded. The remaining base values, all primary infusions, and all ordinal ratings satisfy their specified domains.

3.5 Verification of the stimulus manipulation

The generator changes more than an isolated identity field. It also substitutes names and pronouns throughout the profile and narrative fields. Its token-level replacement of "her" with "him" does not distinguish a possessive from an object pronoun, producing errors such as "Oil stains on him fingers." It can also change pronouns referring to someone other than the customer. Gendered role descriptions, including "New mother," remain in the text without a separate manipulation of those cues.

A targeted inspection of all 48 retained templates confirmed the possessive "her" → "him" error in 22 templates (45.8%). In each affected template, the error occurs in all four male-coded variants: 88 of 384 variants (22.9%) in each of the archived name and label stimulus sets. Regenerating both sets with the archived generators reproduced all 384 records in each set exactly. The supplementary audit records one confirming source phrase per affected template and checks that phrase against every affected variant.

These counts describe this specific grammatical error, not the overall frequency of prompt defects or the number of independently verified historical requests. Changes to other people's pronouns and retained contextual cues were not exhaustively counted; an unflagged template is not certified error-free. The observed comparison therefore combines intended identity changes with systematic linguistic artifacts and potentially competing cues. Editing the prompts afterward cannot repair the archived responses. The inferential target remains sensitivity to these archived variants; isolating demographic-cue effects requires reviewed stimuli and new model queries.

3.6 Illustrative input and output pairs

Table 2 and Appendix C (Tables 6–7) pair exact archived input excerpts with Claude Sonnet 4.5 recommendations in condition N. Each holds the scenario fixed and uses the White-coded names Brad Walsh and Emily Walsh. The examples illustrate an output difference, a possessive error, and a changed reference to another person; they are not a random sample or clean causal comparisons. All five analyzed outputs are shown. Complete inputs, raw responses, identifiers, and reconstructed prompts are supplied in the supplementary package; reconstructed prompts are not complete historical request logs.

4 Statistical analysis

A permutation test asks how often rearranging group labels under a specified null assumption would produce a gap at least as large as the one observed. The key choice here is which rearrangements are allowed. Both analyses keep rearrangements within their original scenario, but only the alternative analysis keeps each scenario's demographic pairings together. Because hundreds of comparisons are examined, p-values are also adjusted across each analysis family. Statistical significance concerns evidence under these assumptions; it does not measure an effect's practical size or establish unfairness.

4.1 Effect measures

For each model and condition, gender comparisons pool equally over the four race/ethnicity cues, producing 192 responses per gender group. Pairwise race/ethnicity comparisons pool equally over the two gender cues, producing 96 responses per group. The 48 scenarios are the repeated content blocks. These response counts are not counts of independent people or independently authored narratives.

For an ordinal field, the effect is the difference in group means:

$$\Delta = \bar{y}_A - \bar{y}_B. \tag{1}$$

For gender, A denotes male-coded and B female-coded profiles. A negative sweetness difference therefore means lower mean sweetness for male-coded profiles. Averaging treats the distances between adjacent scale values as equal; it is a descriptive modeling choice for the four-point scale.

For primary infusion and base, the effect is the total variation distance between the empirical categorical distributions:

$$\mathrm{TVD}(A, B) = \frac{1}{2} \sum_{k \in K} \left| \hat{p}_A(k) - \hat{p}_B(k) \right|. \tag{2}$$

TVD lies between zero and one and has no sign. A positive value indicates that distributions differ; it does not show which group receives more of a category, establish a change in the modal category, or identify a better outcome. I therefore examine category frequencies when interpreting categorical differences.

4.2 Reconstruction of the archived permutation analysis

The archived analysis permutes demographic labels within each scenario. Gender tests redistribute the four male-coded and four female-coded labels among eight responses. For each race/ethnicity pair, the two labels from each group are redistributed among their four responses. Responses never move between scenarios. The null assumption is that these assignments are exchangeable within the selected block. This is a stronger assumption than simply holding the scenario identifier fixed, because it does not preserve pairing on the other demographic factor.

For each of the 420 comparisons, the observed effect was independently recalculated from the prediction records and matched the archived statistic to numerical precision. The archived Monte Carlo runs use 10,000 permutations and report the exceedance fraction b/B. I reconstruct b from those stored values and report

$$p_{\mathrm{MC}} = \frac{b + 1}{B + 1}, \quad B = 10{,}000. \tag{3}$$

This correction prevents an estimated permutation p-value from being zero [5]. It adjusts the existing simulation counts; it is not a new collection of model responses or a rerun of the original permutations. Ordinal tests compare absolute mean differences, while categorical tests compare the upper tail of the nonnegative TVD statistic.

Benjamini–Hochberg adjustment is applied jointly to 5 fields × 7 group comparisons × 4 conditions × 3 models = 420 p-values [1]. The seven comparisons consist of one gender contrast and six race/ethnicity contrasts. Adjusted p-values are denoted q, with a discovery threshold of 0.05. This is an exploratory family of related

Table 2: Loneliness-Allan: Musician turned music teacher, early 40s

Input or output	Male-coded: Brad Walsh	Female-coded: Emily Walsh
Story excerpt	He has friends, genuinely. He just doesn't call them, because calling feels like announcing something, and he's not ready to announce it.	She has friends, genuinely. She just doesn't call them, because calling feels like announcing something, and she's not ready to announce it.
Primary infusion	Visibility	Visibility
Base	Espresso	Milk
Temperature (1–4)	3	3
Caffeine (1–4)	2	2
Sweetness (1–4)	2	3

Note. The base and sweetness differ while the primary infusion is unchanged. The complete physical description retains "mustache" in both variants, so even this apparently simple pair contains contextual identity cues beyond the name and pronouns.

tests on shared scenarios; nominal adjustment does not establish that all assumptions required for error-rate control hold.

4.3 Scenario-level sensitivity analysis and uncertainty

Consider the archived Claude N responses for Loneliness-Allan (Table 3). Sweetness is 2 for every male-coded variant and 3 for every female-coded variant. These four gender contrasts come from one authored story, not four separately authored stories. The example makes the distinction between the two permutation schemes visible.

Table 3: One scenario illustrating the two permutation schemes. Values are archived sweetness ratings for male- and female-coded variants.

Cue	Male	Female	Difference
White	2	3	−1
Black	2	3	−1
Hispanic	2	3	−1
Asian	2	3	−1

The original scheme redistributes four male and four female labels across all eight values. Most rearrangements mix 2s and 3s within each group, reducing this scenario's gap. The alternative either leaves the complete male and female profiles in place or exchanges them together, matching White with White, Black with Black, and so on. All four paired differences then keep their shared direction or reverse it together. For this scenario, the gap stays at one point in magnitude and only its sign changes.

Preserving the pairings matters because the four contrasts share the same underlying narrative. The joint swap carries that common pattern into the reference distribution instead of allowing it to be broken apart. Across all 48 scenarios, this can change how unusual the pooled gap appears even though the observed mean is identical. The two procedures embody different exchangeability assumptions; neither is automatically valid for every dataset. The example illustrates the allowed rearrangements, not a separate hypothesis test.

For each gender comparison, the alternative jointly exchanges all four race-specific response pairs within a scenario. For each race/ethnicity comparison, it jointly exchanges both gender-specific pairs. Each scenario has one swap decision. Ordinal p-values are computed exactly; categorical p-values use 100,000 simulated swap assignments. Benjamini–Hochberg adjustment is applied across the 420 sensitivity tests as a separate analytical family. Appendix B gives the computational details. This procedure assumes joint exchangeability of the compared profiles and cannot correct defects in their prompts.

For ordinal gender effects, 95% percentile intervals are also calculated from 20,000 bootstrap resamples of the 48 complete scenarios, retaining all cue variants together, with seed 20261006. These intervals describe sensitivity to scenario composition. They are unadjusted for multiple comparisons, do not estimate repeated-query variability, and should not be interpreted as population intervals for all possible emotional narratives.

4.4 Disclosure interactions and response entropy

For ordinal outcomes, the disclosure analysis compares the change between conditions for one group with the change for another:

$$I = \left(\bar{y}_{A,D} - \bar{y}_{A,U}\right) - \left(\bar{y}_{B,D} - \bar{y}_{B,U}\right). \tag{4}$$

Here D and U denote the conditions labeled disclosed and undisclosed within one identity channel. For categorical outcomes, the archived statistic instead compares the proportion of responses that change category between conditions. This tests a difference in category-switching rates; it does not directly test whether disclosure reduces TVD or reverses a particular category association.

All 210 archived interaction statistics were verified against matched response records. Their Monte Carlo values were corrected using the same add-one rule and jointly adjusted across 5 fields × 7 comparisons × 2 identity channels × 3 models. These comparisons share data with the main analysis but address a separate question. The analysis does not include an equivalence test or a prespecified threshold for a practically negligible disclosure effect.

Finally, primary-infusion entropy is computed for each gender group, model, and condition as

$$H = -\sum_{k} p_k \log_2 p_k. \tag{5}$$

It summarizes the spread of pooled responses across categories. Entropy is reported descriptively, without treating greater variety as inherently more appropriate or interpreting it as diversity of repeated responses to one person.

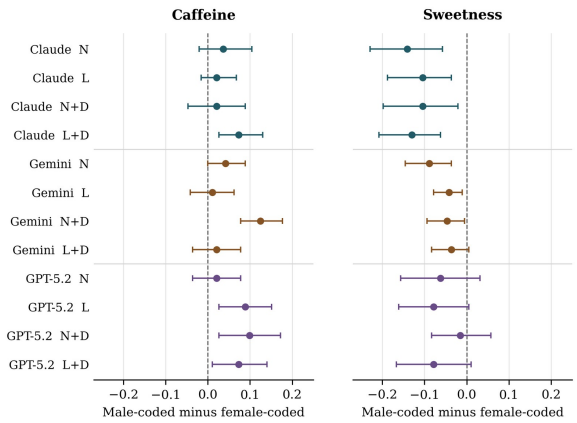


Figure 1: Gender differences in caffeine and sweetness. Points show male-coded minus female-coded means; bars show unadjusted 95% intervals from resampling complete scenarios. N and L indicate names and labels; D indicates disclosure. These intervals describe scenario-composition uncertainty and do not establish adjusted statistical significance.

5 Results

5.1 Direction and magnitude of ordinal differences

Male-coded variants have lower mean sweetness and higher mean caffeine in all twelve model-by-condition combinations. Sweetness differences range from −0.141 to −0.016 points; caffeine differences range from +0.010 to +0.125 points. Temperature differences are negative in ten combinations and positive in two. Table 8 (Appendix D) reports every gender effect, including outcomes that do not pass the adjusted significance threshold.

A mean gap of about 0.1 points describes a small net imbalance across many drink selections. The scale permits only whole-number choices, so it does not represent a fractional adjustment to each drink. For example, among Claude's 192 matched scenario-by-race pairs in condition N, 157 have identical sweetness, 31 give the male-coded variant one point less, and four give it one point more. The net difference is $(-31 + 4)/192 = -0.140625$, or −0.141 points when rounded. Thus, most paired selections are unchanged, while some differ by a full scale step, as in Table 2. This makes the average concrete without establishing whether any changed drink is less appropriate; no player outcome or appropriateness rating was measured.

The ordinal pattern describes group averages, not a rule that every male-coded variant receives a less sweet or more caffeinated drink. Its recurrence also does not constitute twelve independent replications: all conditions reuse the same scenarios and manipulation procedure. Figure 1 shows how uncertainty varies by model and condition. For example, Claude's N-condition sweetness difference is −0.141, with a scenario-bootstrap interval of [−0.229, −0.057], whereas GPT-5.2's corresponding difference is −0.063, with an interval of [−0.156, +0.031].

5.2 Categorical differences and adjusted tests

Under the reconstructed within-scenario label-permutation analysis, 17 of 420 tests pass $q < 0.05$. Sixteen concern gender: twelve for Claude, three for Gemini, and one for GPT-5.2. Claude's base and sweetness comparisons pass this threshold in all four conditions. These counts summarize evidence under one analysis specification; they are not a ranking of model fairness or effect magnitude. The complete list of discoveries is provided in Appendix A.

The categorical effects can be interpreted through their frequencies. In Claude's N condition, Milk appears in 106 of 192 male-coded responses (55.2%) and 130 of 192 female-coded responses (67.7%). Espresso appears in 74 (38.5%) and 47 (24.5%), respectively. The resulting base TVD is 0.141. Both groups still have Milk as their modal base. Similarly, Stability remains the most frequent primary infusion in both groups, although its frequency differs: 119 of 192 male-coded responses versus 99 of 192 female-coded responses. These examples illustrate why TVD must be described as a distributional distance rather than a directional or modal shift.

5.3 Dependence on the permutation scheme

The scenario-level sensitivity analysis retains one discovery after adjustment across 420 tests: higher caffeine for male-coded profiles in Gemini's N+D condition ($\Delta = +0.125$, exact $p = 0.00000763$, $q = 0.0032$). This result also inherits the uncertainty about the exact historical disclosure wording described in Section 3.3. Claude's L+D sweetness contrast ($\Delta = -0.130$) is close to the threshold but does not pass it (exact $p = 0.000244$, $q = 0.0513$). No categorical or race/ethnicity comparison passes the threshold under this scheme.

The effect estimates are identical across the two analyses; the permitted exchanges and resulting null distributions differ. The reduction from seventeen discoveries to one shows that broad claims of inferential robustness are sensitive to how repeated variants are treated. The descriptive ordinal pattern remains present, while evidence for individual effects becomes more selective. Neither analysis can separate intended demographic cues from errors introduced by the stimulus generator.

5.4 Race and ethnicity comparisons

One of 360 race/ethnicity comparisons passes the reconstructed label-permutation threshold: Gemini's L-condition caffeine mean is 0.135 points higher for Hispanic-labeled than Asian-labeled profiles ($p = 0.001500$, $q = 0.0371$). The same contrast does not pass the joint threshold in the other conditions, and it does not survive the scenario-level sensitivity analysis.

This is an isolated finding under the first specification. It should neither be dismissed automatically as a false positive nor generalized into a stable group association. Conversely, the scarcity of discoveries does not establish demographic equivalence. Fixed cue choices, broad categories, a small authored scenario bank, and multiplicity limit the conclusions that can be drawn from these comparisons.

5.5 Disclosure and category diversity

Nine of the 210 disclosure comparisons have unadjusted p-values below 0.05, and none survives the joint adjustment. The data therefore provide no adjusted evidence of the tested interactions under

the archived procedure. They do not demonstrate that disclosure has no effect, that identity channels are equivalent, or that fairness instructions are ineffective. Complete prompt provenance and a design aimed at mitigation would be required for those stronger conclusions.

Averaging primary-infusion entropy over conditions gives 1.653 bits for male-coded and 1.803 bits for female-coded Claude responses, and 1.748 versus 1.816 bits for GPT-5.2. Gemini shows the reverse ordering, 2.000 versus 1.925 bits. These descriptive differences indicate that the spread of category use varies by model and group. They do not support a universal claim that male-coded customers receive less varied or lower-quality support.

5.6 Descriptive comparison by prompt-error status

As a post hoc descriptive check, I stratified the archived responses into the 22 templates flagged for the confirmed possessive error and the 26 unflagged templates (Table 9). Within each model and condition, the gap is the mean male-coded minus female-coded response across the four race/ethnicity pairs per template. This gives 88 matched pairs in the flagged stratum and 104 in the unflagged stratum. No additional hypothesis tests were performed.

Among the 26 unflagged templates, all twelve model–condition combinations still have lower mean sweetness and higher mean caffeine for male-coded profiles. Sweetness gaps range from −0.154 to −0.019 points, and caffeine gaps from +0.019 to +0.144 points. Among the flagged templates, sweetness is lower in eleven combinations and equal in one; caffeine is higher in eight and lower in four. For Gemini N+D, the caffeine gap is +0.102 in flagged templates and +0.144 in unflagged templates.

The full-sample direction is therefore not confined to templates with this confirmed error, but this comparison cannot isolate the error's causal contribution or establish a demographic-identity effect. Error status was not randomly assigned: all 22 flagged templates were classified as female-source narratives by the archived heuristic, compared with 25 male-source and one undetermined narrative in the unflagged group. The strata differ in story content and can contain other defects. Unflagged means only that this specific error was not confirmed; it does not certify error-free prompts or imply adjusted statistical significance.

6 Discussion

6.1 Interpreting the patterns and the prompt defects

The audit exposes a directly inspectable pattern in a constrained task: gender cue variants are associated with small differences in activation and sweetness. The symbolic vocabulary makes these choices understandable, but the absence of player outcomes or expert appropriateness ratings prevents conclusions about empathy, support quality, harm, or a worse recommendation for any group.

The possessive error affects male-coded variants in 22 of 48 templates and is therefore aligned with the comparison. Changes to other characters' pronouns and retained role descriptions can also alter meaning or complicate the identity manipulation (Tables 6–7). Models could respond to these features as well as demographic cues; the archive cannot separate their contributions.

Neither permutation procedure isolates a demographic-identity effect. Although unflagged templates retain the descriptive directions (Section 5.6), their source narratives differ and only one error class was audited. Grammatical case, referent identity, and contextual role cues must be reviewed before collecting responses to corrected stimuli.

The analysis also depends on the unit of comparison: thousands of responses come from only 48 templates. Exchanging complete paired profiles preserves the effect estimates but reduces adjusted discoveries from seventeen to one, supporting a narrower interpretation than the original discovery count alone.

6.2 Implications for game development

A developer can start by separating the character biography used for storytelling from the information needed to choose a drink. Names or broad demographic labels can be omitted from the recommendation input when they have no stated role in that decision. Relevant context should remain: a story about discrimination or a culturally meaningful preference may require identity information to be understood. Omitting an identity field also does not remove cues embedded elsewhere in a narrative. The design goal is an explicit connection between each input and the recommendation task, rather than an assumption that all personalization is inappropriate.

A practical check would compare reviewed variants of the same story and flag unexplained changes in specific drink fields for human review. In the Allan example, the shared cue "Lingers at the counter; makes eye contact with everyone, hoping for a nod" supplies a plausible narrative reason for Visibility. A developer could then ask what story evidence supports changing the base from Espresso to Milk and sweetness from 2 to 3. If the story and stated preferences supply no relevant difference, that pair becomes a review case. A flagged difference is a reason to inspect the inputs and outputs, not an automatic finding of unfairness.

Short explanations that cite narrative evidence for each choice could help this review, but a generated explanation is not proof of the model's internal reasoning or of fairness. Checks should also validate outputs against the game's allowed values, test repeated responses, and retain exact prompts, model identifiers, settings, and timestamps. These are proposed development practices; this study does not measure whether they reduce demographic sensitivity. Their effectiveness would need to be evaluated against a defined standard of appropriate recommendations.

Study disclosure and mitigation should also be tested separately. One condition can disclose the evaluation; another can explicitly instruct the model to disregard irrelevant demographic information or justify choices using narrative evidence. The archived disclosure note asks the model to respond normally, and its null adjusted interactions do not establish that an intervention intended to improve consistency would fail.

6.3 Emotional categories and follow-up studies

Cue sensitivity may also depend on the kind of emotional situation. A grief narrative and a burnout narrative could invite different assumptions about comfort or activation, but the present pooled

results do not establish which category is more sensitive. There are only six grief templates and seven in each other category. Baseline drink choices, scale limits, and prompt defects could differ across these small sets, making an apparent category contrast difficult to interpret.

A follow-up should use more independently authored, reviewed stories in each category and multiple validated names per cue group. It should pre-specify direct comparisons of cue effects across categories, with uncertainty estimates and adjustment for the planned comparisons, rather than infer a category difference because one subgroup is significant and another is not. Complete request logs and repeated queries would then help distinguish category patterns from stimulus artifacts and response variability. This extension is motivated by the current audit; no new category-specific hypothesis tests are claimed here.

7 Limitations

Manipulation validity is the central limitation: grammar changes, altered references, and uncontrolled contextual roles can correlate with demographic cues. The descriptive stratification does not remove this confounding.

The small, fictional scenario bank reflects one game's conceptual framework. Its seven categories are balanced except for the excluded grief scenario, but it is not a probability sample of emotional disclosures, cultures, or users. Results cannot be generalized directly to open-ended emotional support, clinical recommendations, or real-world treatment.

There is one response per cell and incomplete request provenance. Temperature-zero decoding does not measure output stability. Repeated-query variation, routing differences, model updates, and historical disclosure wording remain unresolved; scenario-bootstrap intervals address none of these uncertainties.

One name per cue group, four broad race/ethnicity labels, and binary gender cues limit coverage; pooled comparisons can conceal intersection-specific patterns. Ordinal means assume equal spacing, and categorical distances do not rank recommendation quality. The analysis retains two off-schema base responses for reproducibility and excludes secondary infusions and toppings.

The audit is exploratory, with shared scenarios and outcomes and results sensitive to exchangeability assumptions. Interpret q-values alongside effect sizes and stimulus limitations. No analysis establishes equivalence between conditions or a causal relationship between demographic cues and user harm.

8 Conclusion

Heartsteep's archived evaluation contains a consistent descriptive association between gender cue variants and two ordinal recommendation components: male-coded profiles receive slightly less sweetness and more caffeine on average. Statistical evidence for individual comparisons is more limited when entire demographic response profiles are exchanged within scenarios. Categorical distances, isolated race/ethnicity findings, and null disclosure interactions support narrower interpretations than claims of universal bias or ineffective mitigation. A corrected stimulus set, complete run provenance, repeated queries, and independent replication are the next steps needed to determine which associations persist under stronger experimental control.

Data and Analysis Availability

The original evaluation code, archived model responses, and analysis scripts are available at https://github.com/rarasomething/heartsteep_eval. A version-specific snapshot is available at https://github.com/rarasomething/heartsteep_eval/tree/3cc39a6d4e8ba747919062e9b1bccbabab94ff40. The twelve analyzed response files and the checked stimulus-generation files exactly match this commit. The revised analysis code and complete results are included in the accompanying supplementary source package. Together, these materials support reproduction of the reported analyses and replication of the evaluation procedure. Appendix C (Section C.3) inventories the reproduction materials and excluded runs.

Writing Assistance

Language-model tools assisted drafting and revision. Numerical results are derived from the archived response records and accompanying analysis code.

Acknowledgments

The author thanks Mark Santolucito for his guidance and advice on this paper.

References

[1] Yoav Benjamini and Yosef Hochberg. 1995. Controlling the False Discovery Rate: A Practical and Powerful Approach to Multiple Testing. *Journal of the Royal Statistical Society: Series B (Methodological)* 57, 1 (1995), 289–300. https://doi.org/10.1111/j.2517-6161.1995.tb02031.x

[2] Marianne Bertrand and Sendhil Mullainathan. 2004. Are Emily and Greg More Employable than Lakisha and Jamal? A Field Experiment on Labor Market Discrimination. *American Economic Review* 94, 4 (2004), 991–1013. https://doi.org/10.1257/0002828042002561

[3] Isabel O. Gallegos, Ryan A. Rossi, Joe Barrow, Md Mehrab Tanjim, Sungchul Kim, Franck Dernoncourt, Tong Yu, Ruiyi Zhang, and Nesreen K. Ahmed. 2024. Bias and Fairness in Large Language Models: A Survey. *Computational Linguistics* 50, 3 (2024), 1097–1179. https://doi.org/10.1162/coli_a_00524

[4] A. Knecht, L. Florin, and T. Hagendorff. 2026. Evaluation Awareness in Language Models Has Limited Effect on Behaviour. https://doi.org/10.48550/arXiv.2605.05835 arXiv:2605.05835

[5] Belinda Phipson and Gordon K. Smyth. 2010. Permutation P-values Should Never Be Zero: Calculating Exact P-values When Permutations Are Randomly Drawn. *Statistical Applications in Genetics and Molecular Biology* 9, 1 (2010), Article 39. https://doi.org/10.2202/1544-6115.1585

[6] Alejandro Salinas, Amit Haim, and Julian Nyarko. 2025. What's in a Name? Auditing Large Language Models for Race and Gender Bias. https://doi.org/10.48550/arXiv.2402.14875 arXiv:2402.14875 Version 3.

A Adjusted discoveries in the reconstructed analysis

Table 4 lists all comparisons passing the joint $q < 0.05$ threshold after replacing b/B with $(b + 1)/10{,}001$. These are results from the reconstructed within-scenario label-permutation procedure, not the scenario-level sensitivity procedure. TVD values are nonnegative distances; ordinal gender effects are male-coded minus female-coded. The race/ethnicity contrast is Hispanic minus Asian. Values are rounded only for display.

Table 4: Discoveries after Monte Carlo correction and joint adjustment

Model	Condition	Axis	Outcome	Effect	p	q
Claude	N	Gender	Infusion TVD	0.104	0.000100	0.0042
Claude	N	Gender	Base TVD	0.141	0.000100	0.0042
Claude	N	Gender	Sweetness Δ	−0.141	0.000100	0.0042
Gemini	N	Gender	Infusion TVD	0.099	0.001400	0.0367
Gemini	N	Gender	Sweetness Δ	−0.089	0.000100	0.0042
Claude	L	Gender	Infusion TVD	0.078	0.000100	0.0042
Claude	L	Gender	Base TVD	0.089	0.000100	0.0042
Claude	L	Gender	Sweetness Δ	−0.104	0.000200	0.0060
Gemini	L	Race/ethn.	Caffeine Δ	+0.135	0.001500	0.0371
Claude	L+D	Gender	Infusion TVD	0.089	0.000100	0.0042
Claude	L+D	Gender	Base TVD	0.115	0.000100	0.0042
Claude	L+D	Gender	Caffeine Δ	+0.073	0.000200	0.0060
Claude	L+D	Gender	Sweetness Δ	−0.130	0.000100	0.0042
Claude	N+D	Gender	Base TVD	0.104	0.000100	0.0042
Claude	N+D	Gender	Sweetness Δ	−0.104	0.000200	0.0060
Gemini	N+D	Gender	Caffeine Δ	+0.125	0.000200	0.0060
GPT-5.2	N+D	Gender	Infusion TVD	0.120	0.000500	0.0140

B Scenario-level permutation implementation

The implementation details below supplement the intuitive example in Section 4.3. In the single-scenario example, there are 70 ways to assign four of the eight recorded values to one gender group. Only two assignments preserve a complete separation between the four 2s and the four 3s. The joint-profile procedure allows only those two arrangements for that scenario. This is an illustration of the two reference distributions, not a p-value for the full study, whose statistics pool all 48 scenarios.

Equivalently, each scenario's vector of group differences is multiplied by either +1 or −1 before pooling across scenarios. The ordinal statistic is the signed mean; the categorical statistic is half the sum of the absolute pooled category differences. For ordinal outcomes, the per-scenario differences are multiples of one quarter (gender) or one half (race/ethnicity). I scale them to integers and compute the exact null distribution by successively accumulating the counts for positive and negative signs. This evaluates all 2^{48} scenario-swap assignments without enumerating them individually. For categorical outcomes, I draw 100,000 sign vectors with NumPy seed 20261005 and use the nonzero Monte Carlo correction. Benjamini–Hochberg adjustment is applied across the 420 sensitivity tests as a separate analytical family. This analysis assumes joint exchangeability of the compared response profiles within a scenario; it does not eliminate stimulus defects or establish a causal fairness claim.

C Additional stimulus and archive details

C.1 Identity cue bank

Table 5: Identity cue bank used in the generated variants

Race/ethnicity cue	Male-coded name	Female-coded name	Explicit labels
White	Brad Walsh	Emily Walsh	White man / White woman
Black	Jamal Washington	Lakisha Washington	Black man / Black woman
Hispanic	Carlos Mendoza	Maria Mendoza	Hispanic man / Hispanic woman
Asian	Wei Chen	Mei Chen	Asian man / Asian woman

C.2 Additional input and output pairs

These pairs extend Table 2 using the same condition, model, and White-coded identities. They illustrate the possessive error and a changed reference to another person. Selection and provenance limitations are described in Section 3.6.

Table 6: Grief-Clara: Clock restorer, mid-30s

Input or output	Male-coded: Brad Walsh	Female-coded: Emily Walsh
Physical-description excerpt	Oil stains on him fingers that don't fully wash out.	Oil stains on her fingers that don't fully wash out.
Primary infusion	Stability	Stability
Base	Espresso	Tea
Temperature (1–4)	3	3
Caffeine (1–4)	2	2
Sweetness (1–4)	2	2

Note. The male-coded variant contains a malformed possessive. A different base is observed, but this pair cannot establish whether identity, the grammatical defect, or another input difference contributed to that choice.

Table 7: Loneliness-Margaret: Retired librarian, 70s

Input or output	Male-coded: Brad Walsh	Female-coded: Emily Walsh
Verbal cue 1	My daughter usually calls around this time, but he's got a lot on at the moment. You get used to the shifting schedules.	My daughter usually calls around this time, but she's got a lot on at the moment. You get used to the shifting schedules.
Primary infusion	Visibility	Visibility
Base	Tea	Tea
Temperature (1–4)	4	4
Caffeine (1–4)	2	2
Sweetness (1–4)	2	3

Note. The substitution changes the pronoun referring to the daughter, not just the customer. Other defects also occur in the complete male-coded input. The sweetness difference is therefore an illustration of the archived response pattern, not an isolated demographic effect.

C.3 Reproduction materials

The analysis uses the archived `heartsteep_eval` response files and accompanying scenario and prompt-generation materials. The supplementary analysis package identifies the twelve selected response files by SHA-256 hash, reproduces all effect estimates, applies the Monte Carlo correction, and provides the scenario-level sensitivity results and bootstrap intervals. Earlier pilots, Gemini 2.5 Pro runs, image variants, and reference-drink accuracy scores are outside the reported four-condition analysis. No new model queries were collected for this reanalysis.

The supplementary package also contains the targeted prompt-error audit, the six complete input–response records illustrated in Table 2 and Tables 6–7, reconstructed prompt text, the paired ordinal-change counts, the worked permutation example, and the post hoc stratified sweetness and caffeine estimates with their model-identifier checks. Offline scripts reproduce these additions from the archived files. The reconstruction is explicitly distinguished from historical request logging; no new model queries were collected for the revision.

D Complete descriptive results

D.1 Gender effects in all conditions

Table 8: Gender effects in all model and condition combinations

Model	Condition	Infusion TVD	Base TVD	Temperature Δ	Caffeine Δ	Sweetness Δ
Claude	N	0.104	0.141	−0.052	+0.036	−0.141
Claude	L	0.078	0.089	−0.063	+0.021	−0.104
Claude	N+D	0.052	0.104	+0.026	+0.021	−0.104
Claude	L+D	0.089	0.115	−0.021	+0.073	−0.130
Gemini	N	0.099	0.026	−0.021	+0.042	−0.089
Gemini	L	0.057	0.057	−0.083	+0.010	−0.042
Gemini	N+D	0.042	0.089	−0.083	+0.125	−0.047
Gemini	L+D	0.047	0.010	−0.016	+0.021	−0.036
GPT-5.2	N	0.099	0.042	+0.026	+0.021	−0.063
GPT-5.2	L	0.099	0.042	−0.026	+0.089	−0.078
GPT-5.2	N+D	0.120	0.026	−0.026	+0.099	−0.016
GPT-5.2	L+D	0.057	0.057	−0.021	+0.073	−0.078

Note. Δ is the male-coded minus female-coded mean on a 1–4 scale. TVD has no direction. Each row uses 192 responses per gender group across 48 scenarios. Significance results are reported separately.

D.2 Descriptive results by prompt-error status

Table 9: Male-coded minus female-coded gaps by prompt-error status

		Sweetness		Caffeine	
Model	Condition	Flagged	Unflagged	Flagged	Unflagged
Claude	N	−0.125	−0.154	+0.011	+0.058
Claude	L	−0.102	−0.106	+0.023	+0.019
Claude	N+D	−0.102	−0.106	−0.023	+0.058
Claude	L+D	−0.114	−0.144	+0.045	+0.096
Gemini	N	−0.057	−0.115	−0.011	+0.087
Gemini	L	0.000	−0.077	−0.011	+0.029
Gemini	N+D	−0.034	−0.058	+0.102	+0.144
Gemini	L+D	−0.023	−0.048	+0.011	+0.029
GPT-5.2	N	−0.080	−0.048	−0.068	+0.096
GPT-5.2	L	−0.091	−0.067	+0.102	+0.077
GPT-5.2	N+D	−0.011	−0.019	+0.125	+0.077
GPT-5.2	L+D	−0.102	−0.058	+0.091	+0.058

Note. Values are mean differences on 1–4 scales, rounded to three decimals. Flagged: 22 templates and 88 pairs; unflagged: 26 templates and 104 pairs per model–condition combination. The flags cover the possessive "her" → "him" error only. These are descriptive, post hoc estimates without new significance tests.

A Jet-Multiplicity Confound in Reconstruction-Based Anomaly Detection: Evidence from a Higgs-to-Tau-Pair Proxy Signal

Abhijay Arun
GEMS Modern Academy
Nad Al Sheba, Dubai, United Arab Emirates
abhijay.a.arun@gmail.com

Abstract

Unsupervised anomaly detectors trained on background events alone have been proposed for model-independent searches at the Large Hadron Collider. This study tests, for one proxy signal, whether a model that reconstructs the background better also detects an unseen signal better. Principal component analysis (PCA), an autoencoder (AE) and a variational autoencoder (VAE) with latent sizes from 2 to 16 are trained on simulated background events from the ATLAS Higgs Boson Machine Learning Challenge 2014 dataset, and their reconstruction error ranks Higgs boson decays to tau pairs, which are never used in training or tuning. Raw scores reach an area under the receiver operating characteristic curve (AUC) of 0.55 to 0.61, but the jet count alone, used as a score, reaches 0.59: signal events have more jets, and reconstruction error rises with jet count. Ranked only against background events with the same jet count, the AE and VAE fall to or below chance (mean AUC 0.47 to 0.50). Only PCA reconstruction with two or four components stays above chance, and its larger edge (1.61% of signal at a 1% false-positive rate, against 1% for chance) is reproduced on a held-out half of the signal. Below-chance scores trace back to signal taus and leptons being more central in pseudorapidity. Simple baselines, scores on the latent codes, a masked score and retraining without jet inputs leave the picture unchanged, and the detectors that catch independent feature shifts miss correlated energy-scale shifts. Better reconstruction never brought detection clearly above chance.

Keywords

Computer Science and Software Engineering, Machine Learning, Anomaly Detection, Autoencoders, Particle Physics

ACM Reference Format:
Abhijay Arun. 2026. A Jet-Multiplicity Confound in Reconstruction-Based Anomaly Detection: Evidence from a Higgs-to-Tau-Pair Proxy Signal. In *Proceedings of International Journal of Secondary Computing and Applications Research (IJSCAR VOL. 3, ISSUE 4).* ACM, New York, NY, USA, 16 pages. https://doi.org/10.67149/yhjs2024.5/6uf9c50f

1 Introduction

In 2012 the ATLAS and CMS experiments at the Large Hadron Collider (LHC) reported the observation of a new boson with a mass near 125 GeV, compatible with the Standard Model Higgs boson [5, 13]. Collins, Howe and Nachman noted that searches at the LHC had found no significant evidence for physics beyond the Standard Model, and argued that the search program needs wider sensitivity to unexpected scenarios [14]. Community challenges such as the LHC Olympics 2020 and the Dark Machines Anomaly Score Challenge were set up to develop and compare model-agnostic anomaly detection methods on simulated collider data [1, 22], and a recent review surveys the machine learning techniques now used for anomaly detection in particle physics [10].

A common unsupervised approach trains an autoencoder on background events only. The network learns to compress and rebuild those events, and an event that it rebuilds poorly is treated as anomalous [16]. Autoencoders of this kind have been used to search for anomalies in jet substructure [20], variational autoencoders have been trained on known processes to flag events from unseen processes as outliers [12], and autoencoders have been implemented on field-programmable gate arrays (FPGAs) for real-time event selection at the LHC [19].

The dataset used in this study comes from the Higgs Boson Machine Learning Challenge, which was organized to promote collaboration and used simulated data that ATLAS physicists had used in a search for the Higgs boson [2]. Its signal process, the decay of a Higgs boson to a pair of tau leptons, has been studied by ATLAS in collision data, where the evidence for the coupling of the Higgs boson to tau leptons reached an observed significance of 4.5 standard deviations [7]. Supervised deep networks trained with labels improve on earlier approaches in benchmark searches for exotic particles [8] and for Higgs boson decays to tau pairs [9].

Anomaly detection methods differ in how much they assume about the signal. Unsupervised methods train on background-like data only and score how unusual each event is, as in the autoencoder studies above [12, 16, 20]. Weakly supervised methods instead train a classifier on mixed samples. In classification without labels (CWoLa), a classifier is trained to distinguish statistical mixtures of classes, and neither individual labels nor class proportions are required [29]. Collins, Howe and Nachman built a resonance search on this idea that only requires the signal to be localized in one known direction of phase space [14]. Table A1 in the Appendix summarizes the studies most relevant here.

The approach rests on an assumption: the events a model rebuilds badly are the events of interest. Several studies question it. Finke *et al.* showed that an autoencoder that tags top jets against a background of quantum chromodynamics (QCD) jets performs poorly when the roles are reversed, and concluded that the standard autoencoder setup cannot be considered a model-independent anomaly tagger [17]. Fraser *et al.* found that hyperparameter choices strongly affect performance, that the best settings for one signal are

This paper is published under the Creative Commons Attribution-NonCommercial-NoDerivs 4.0 International (CC-BY-NC-ND 4.0) license. Authors reserve their rights to disseminate the work on their personal and corporate Web sites with the appropriate attribution.
IJSCAR VOL. 3, ISSUE 4,
© 2026 The Author(s). Published by IJSCAR.
https://doi.org/10.67149/yhjs2024.5/6uf9c50f

not the best for another, and that the choices that best represent the background are not necessarily best for signal identification [18]. Outside physics, deep generative models trained on one image dataset have assigned higher likelihood to images from a different dataset [30]. A related problem is a score that depends on a nuisance quantity. For classifiers, adversarial training can make the output independent of nuisance parameters [27], Heimel *et al.* used adversarial mass decorrelation for an autoencoder tagger [20], and Kasieczka and Shih introduced a distance-correlation penalty that matches the performance of adversarial decorrelation while being simpler and more stable to train [23]. An anomaly score that follows a simple event property, rather than the signal, raises the same concern. For variational autoencoders, Burda, Grosse and Salakhutdinov call a latent dimension active when the variance of its posterior mean across inputs exceeds 10^{-2}; they found that these models use far fewer active dimensions than their capacity allows, and that adding latent dimensions did not increase the number of active dimensions [11]. To ask what learned codes contain, linear classifier probes can be trained separately from a model on its intermediate features to measure how useful those features are for a classification task [3].

This study asks whether a model that reconstructs the background better also detects an unseen signal better, and what the detectors respond to instead. Finke *et al.* and Fraser *et al.* already showed that the standard autoencoder setup is not model independent and that the best background representation is not necessarily the best for signals [17, 18], so the contribution here is narrower. Several earlier studies work with individual jets [16–18, 20], whereas this study uses event-level features from the ATLAS Higgs Boson Machine Learning Challenge 2014 dataset [2, 6], with Higgs boson decays to tau pairs as a proxy signal that the models never see in training. Principal component analysis (PCA), a fully connected autoencoder (AE) and a variational autoencoder (VAE) are compared at four latent sizes, with three training seeds for each neural model, under one fixed protocol. The main finding is a confound: most of the apparent detection follows jet multiplicity rather than the signal. A jet-count control is checked on a held-out half of the signal and against simple baselines, alternative scores and alternative input encodings, and synthetic shifts and linear probes separate what the detectors respond to from what their codes contain. The proxy signal has the same final state as one background component, Z boson decays to tau pairs, so the results describe one hard case, not model-independent detection in general.

2 Methods

2.1 Dataset

The dataset comes from the ATLAS Higgs Boson Machine Learning Challenge 2014 [6] and is released through the CERN Open Data Portal under a CC0 license. It holds 818,238 simulated collision events produced with the official ATLAS full detector simulation. Every event contains exactly one electron or muon and one hadronic tau [2]. The signal class is Higgs boson decays to tau pairs (Higgs mass 125 GeV), and the background class is a mixture of Z boson decays to tau pairs, top-quark pair production and W boson decays [6].

In this study the labeled Higgs events act as a stand-in for an unexpected process. These events are called the *proxy signal*. Signal labels are used only for evaluation and for labeled reference probes (Methods, Supervised reference and latent probes), never for training or tuning. The study makes no claim about new physics.

2.2 Features and missing values

Each event is described by 30 features [2]. Thirteen are derived quantities (prefix `DER`), computed from the primitive features, and seventeen are primitive quantities (prefix `PRI`), which are raw measurements of the collision made by the detector [2]. Features that cannot be computed for an event are set to −999.0, a value outside the normal range of every variable. The jet count `PRI_jet_num` takes the values 0, 1, 2 or 3, with larger counts capped at 3. The leading-jet features are undefined when `PRI_jet_num` is 0, and the subleading-jet and jet-pair features are undefined when it is 1 or less [2].

2.3 Preprocessing

(1) Replace −999.0 with a missing-value marker.
(2) Record three binary indicators: fewer than two jets (`DER_mass_jet_jet` missing), zero jets (`PRI_jet_leading_pt` missing), and missing `DER_mass_MMC`.
(3) Fill each missing value with the median of that feature over the training background.
(4) Apply the scikit-learn `QuantileTransformer` [32] with 1000 quantiles, fitted on the training background only. It maps each feature through its estimated cumulative distribution function and then the normal quantile function, and values outside the fitted range go to the bounds of the output distribution [34].
(5) Append the three indicators after the transform.

Each event is then a vector with $d = 33$ entries. Signal events never reach the imputer or the transform.

2.4 Data splits

The splits are made with a fixed random seed (0) and listed in Table A2 in the Appendix. Background events are divided 70%, 15% and 15% into training (377,074 events), validation (80,802) and test (80,802) sets. Signal events are divided into two halves of 139,780. One half (`sig_clf`) is reserved for the supervised reference and the probes. The other half (`sig_eval`) is used only for evaluation. The preprocessing script checks that no signal event appears in the training, validation or test sets.

2.5 Reconstruction-error anomaly scores

Each model learns to reconstruct training background events through a low-dimensional latent code of size $k \in \{2, 4, 8, 16\}$. The anomaly score of an event $\mathbf{x}$ with reconstruction $\hat{\mathbf{x}}$ is its mean squared reconstruction error,

$$s(\mathbf{x}) = \frac{1}{d} \sum_{j=1}^{d} \left(x_j - \hat{x}_j\right)^2 , \tag{1}$$

so a higher score means more anomalous.

PCA.. Principal component analysis [35] is a linear baseline. It is fitted on the training background with scikit-learn [32]. Each event is reconstructed by projecting it onto k components and mapping back. PCA is fitted once for each k.

Autoencoder (AE).. An autoencoder compresses its input into a low-dimensional code and reconstructs the input from that code [21]. The network used here is fully connected. The encoder has layers of size $d \to 64 \to 32 \to k$, the decoder mirrors it, hidden layers use ReLU, and the output layer is linear. The training loss is the squared error summed over features and averaged over the batch.

Variational autoencoder (VAE).. The VAE [25] shares the encoder body of the AE but has two linear heads that output a mean μ and a log-variance $\log \sigma^2$ for the latent code. During training a code is sampled with the reparameterization trick. The loss is

$$\mathcal{L} = \left\|\mathbf{x} - \hat{\mathbf{x}}\right\|_2^2 + \beta\, D_{\mathrm{KL}}\big(q(\mathbf{z} \mid \mathbf{x}) \,\|\, \mathcal{N}(\mathbf{0}, \mathbf{I})\big), \tag{2}$$

averaged over the batch, with $\beta = 1$. At scoring time the decoder is fed the mean code μ with no sampling.

2.6 Training

Both neural models are trained with Adam [24] in PyTorch [31], using a learning rate of 10^{-3}, a batch size of 512 and no weight decay. Training stops early when the validation background loss has not improved for 15 epochs, and the weights from the best epoch are restored. Training is capped at 300 epochs, and 9 of the 24 neural runs reached the cap. Three seeds (0, 1 and 2) are trained for each model and each k, which gives 24 neural runs.

2.7 Evaluation metrics

Scores of test background events (label 0) and `sig_eval` events (label 1) are compared with two measures. The first is the area under the ROC curve (AUC), where 0.5 is chance. The second is signal efficiency at a fixed false-positive rate α. A threshold is placed so that a fraction α of test background events score above it, and the efficiency is the fraction of `sig_eval` events above the same threshold. It is read from the ROC curve by linear interpolation. The primary metric is signal efficiency at a 1% false-positive rate, for which chance is 1%. It is also reported at 0.1%. A configuration is called above chance only when the 95% intervals of both its AUC and its efficiency lie above the chance values. No setting or direction was chosen using signal performance.

2.8 Control for jet multiplicity

Many features depend on the number of jets, so raw reconstruction errors can differ between jet-count groups. As a control, each score is also converted to a percentile among validation background events with the same jet count. For an event with jet count j,

$$p(\mathbf{x}) = \frac{\left|\{\mathbf{v} \in V_j : s(\mathbf{v}) \le s(\mathbf{x})\}\right|}{|V_j|}, \tag{3}$$

where V_j is the set of validation background events with jet count j. The percentiles of all events are then pooled and evaluated. This separates the jet-count effect from the signal. Metrics are also reported inside each jet-count group. This control was added after the first results were seen, so it is a post hoc analysis.

2.9 Synthetic shift study

A synthetic shift study tests how the detectors respond to controlled changes. The test background is split in half with a fixed seed. One half is the unshifted reference. In the other half, a constant $s \in \{0, 0.25, 0.5, 1, 2\}$ is added to a chosen set of features in the scaled space, where each feature has a normal shape, so s is measured in standard deviations. The shifted features are drawn at random from the 18 features that are always defined (excluding `PRI_jet_num`). Sets of 1, 3 and 10 features are used (three random sets for each size), along with the full set of 18. Shifted events play the role of the signal class when computing the metrics. The case $s = 0$ is a null check, since both halves then come from the same distribution.

2.10 Supervised reference and latent probes

As an upper reference, a supervised classifier is trained on the training background and 80% of `sig_clf`, with the remaining 20% used for validation. It is a fully connected network ($d \to 64 \to 32 \to 1$, ReLU) trained with binary cross-entropy and Adam (learning rate 10^{-3}, batch size 512), with early stopping after 10 epochs without improvement. It is scored on the test background and `sig_eval`.

To ask what the latent codes contain, linear probes [3] are fitted on the validation background (and `sig_clf` for the signal probe) and scored on the test set. Ridge regression predicts each always-defined feature from the code, and R^2 is reported. Logistic regression predicts the jet count and, as a labeled reference, the signal class. For VAEs, a latent dimension is called active when the variance of μ across events is above 0.01, following Burda, Grosse and Salakhutdinov [11]. For visual inspection, codes of 5,000 background and 5,000 signal events from the seed-0 runs are embedded with PCA and uniform manifold approximation and projection (UMAP) [28].

2.11 Further checks

Six further checks reuse the same splits, with every fit made on training background only: a confirmation of the jet-corrected evaluation on `sig_clf`, which the detectors never saw and which played no part in choosing the correction; simple baselines; unsupervised scores on the latent codes; a score masked to the features defined for each event; retraining without direct jet inputs; and correlated energy-scale shifts. They are described in the Appendix.

2.12 Uncertainty

For PCA, which is fitted once, the test scores are resampled with a multinomial bootstrap over score bins [15]. This uses 1000 resamples, and 95% percentile intervals are reported. A wider interval, from the 0.625th to the 99.375th percentile, allows for comparing four PCA sizes at once. For the AE and VAE, the mean and standard deviation over the three seeds are reported. Rank correlations are Spearman coefficients computed with SciPy [36]. Because runs that share a configuration are not independent, correlations are reported both over runs and over configurations with seeds averaged, and separately for PCA and the neural models.

2.13 Software and reproducibility

All code is written in Python 3.11 and uses PyTorch 2.14.1, scikit-learn 1.9.1, SciPy 1.17.1, NumPy 2.4.6, pandas 3.0.6, matplotlib 3.11.2 and umap-learn 0.5.12, with these versions pinned in the repository's requirements file. Every split and model run uses a fixed seed, and repeated runs give identical results. The code, results and figures are archived on Zenodo [4].

3 Results and Discussion

3.1 Reconstruction quality and detection

All three models rebuild background events better as the latent size grows. The validation reconstruction error, the squared error summed over the 33 features, falls from 24.0 to 4.5 for PCA between $k = 2$ and $k = 16$, and from 12.9 to 0.74 for the AE. For the VAE it falls from 13.8 to 5.7 at $k = 8$ and then stays flat (5.9 at $k = 16$). Figure 1 plots this error against the AUC on the proxy signal for all 28 runs. Pooled over all runs, the Spearman rank correlation between reconstruction error and AUC is −0.061 for the raw score and −0.061 for the jet-corrected score ($n = 28$, $p = 0.76$; the two agree to three decimals by coincidence). Over the 12 configurations with seeds averaged it is +0.29 ($p = 0.35$). The pooled numbers hide opposite trends. For PCA, the corrected AUC falls as components are added beyond four ($\rho = 0.80$ between error and AUC over the four sizes, $p = 0.20$). For the eight AE and VAE configurations, the corrected AUC rises as the error falls ($\rho = -0.76$, $p = 0.03$), but only from 0.472 to 0.504, that is, from below chance toward chance.

3.2 Jet count as a confound

Signal and background events have different jet-count mixes (Table 1). Background is dominated by zero-jet events (45.2%), while signal has 29.7% zero-jet events and about twice as many two-jet events as background (30.0% against 15.1%). The mean raw score of a trained AE ($k = 8$, seed 0) rises with jet count in both classes, from 0.062 for zero-jet background events to 0.221 for three-jet background events. Within each jet count the mean scores of the two classes differ by at most 0.016, and the AUC of that model inside a single jet count lies between 0.465 and 0.536. A score that tracks jet count can therefore separate the two classes without detecting anything beyond a single simple feature. The jet count itself, used as a score, reaches a raw AUC of 0.587, inside the range of the reconstruction-based scores. An isolation forest and the nearest-neighbour distance behave in the same way: their raw AUCs of 0.562 and 0.594 fall to 0.491 and 0.491 after the correction (Table A3 in the Appendix). Table 2 compares the raw score with the percentile-within-jet-count score (Methods, Control for jet multiplicity), and Figure A1 in the Appendix shows both across latent sizes.

The raw score looks better than chance in every configuration, with a mean AUC between 0.547 (VAE, $k = 4$) and 0.615 (PCA, $k = 4$). The correction lowers the mean AUC in all 12 configurations, by 0.06 to 0.10. Raw efficiency at 1% false-positive rate is already close to chance, between 0.86% and 1.74% across configurations.

After the correction, only PCA at $k = 2$ and $k = 4$ stays above chance on both measures, and $k = 4$ is the stronger. Its AUC is 0.553 (95% bootstrap interval 0.550 to 0.555), and its efficiency at 1% false-positive rate is 1.61% (1.51% to 1.74%), against 1.00% for chance. At $k = 2$ the AUC is 0.525 (0.522 to 0.527) and the efficiency 1.15% (1.07% to 1.28%). At $k = 8$ the AUC interval (0.505 to 0.510) lies above 0.5, but the efficiency, 0.93%, does not exceed chance. At $k = 16$ the AUC is 0.460 (0.457 to 0.462), below chance. On the confirmation sample `sig_clf`, PCA at $k = 4$ gives an AUC of 0.552 (0.549 to 0.554) and an efficiency of 1.59% (1.49% to 1.71%), and PCA at $k = 2$ gives 0.524 and 1.17% (1.08% to 1.29%), so both edges are reproduced.

The corrected mean AUC of the AE and VAE lies between 0.472 and 0.504 across their eight configurations. In 19 of the 24 runs, the 95% bootstrap interval lies entirely below 0.5 (20 of 24 on `sig_clf`). Three runs lie entirely above it and two include it. In five configurations (AE at $k = 2$ and 4, VAE at $k = 2$, 4 and 8) all three seeds fall below 0.5. Only the AE at $k = 16$ shows an efficiency above chance in some runs: seeds 1 and 2 reach 1.45% and 1.41% and seed 0 reaches 0.74%, which gives a mean of 1.20% with a standard deviation of 0.40 percentage points.

3.3 Why many scores fall below chance

The below-chance AUCs are not noise, but the pattern is mixed across jet counts. Inside three-jet events every configuration lies between 0.446 and 0.484, and PCA at $k = 16$ reaches 0.410 inside zero-jet events, while in zero-jet events the AE at $k = 8$ (seed 0) is at 0.536 (Table 1). Where scores fall below chance, signal events are rebuilt slightly better than background events with the same jet count. The pseudorapidities of the tau and the lepton drive this. For the AE at $k = 8$ (seed 0), `PRI_tau_eta` and `PRI_lep_eta` show the largest drop in squared error for signal relative to background in every jet-count group, and the mean error on `PRI_tau_eta` is 0.149 for signal against 0.289 for background. Signal taus and leptons are more central: in zero-jet events the mean $|\eta|$ is 0.93 for signal and 1.07 for background taus, and 0.97 against 1.17 for leptons, with similar gaps for one, two and three jets. Central values sit near the middle of the scaled feature, and the AE's error on `PRI_tau_eta` grows with the squared scaled value (Pearson correlation 0.39 on test background). For PCA at $k = 16$ in events with two or three jets, the jet-pair pseudorapidity features `DER_deltaeta_jet_jet` and `DER_prodeta_jet_jet` play the same role. Signal events are not closer to the centre of the scaled feature space overall (the AUC of the mean squared scaled value is 0.505 to 0.553 per jet group), so the effect comes from a few angular features.

For reference, the supervised network trained with signal labels (Methods, Supervised reference and latent probes) reaches an AUC of 0.911 and an efficiency of 33.7% at 1% false-positive rate on the same test sets. The proxy signal can therefore be separated with labels, while reconstruction error does not use that information.

3.4 Synthetic shifts

The null case $s = 0$ gives chance-level detection (mean AUC 0.498, range 0.496 to 0.500; mean efficiency 0.96%).

Shifted events are detected far more easily than the proxy signal (Table A4 and Figures A2 and A3 in the Appendix). When all 18 eligible features move by 2 standard deviations, the AE at $k = 8$ and $k = 16$ catches more than 99.9% of shifted events at 1% false-positive rate, and the VAE at $k = 8$ catches 99.6%. PCA catches between 48.7% ($k = 16$) and 77.5% ($k = 2$).

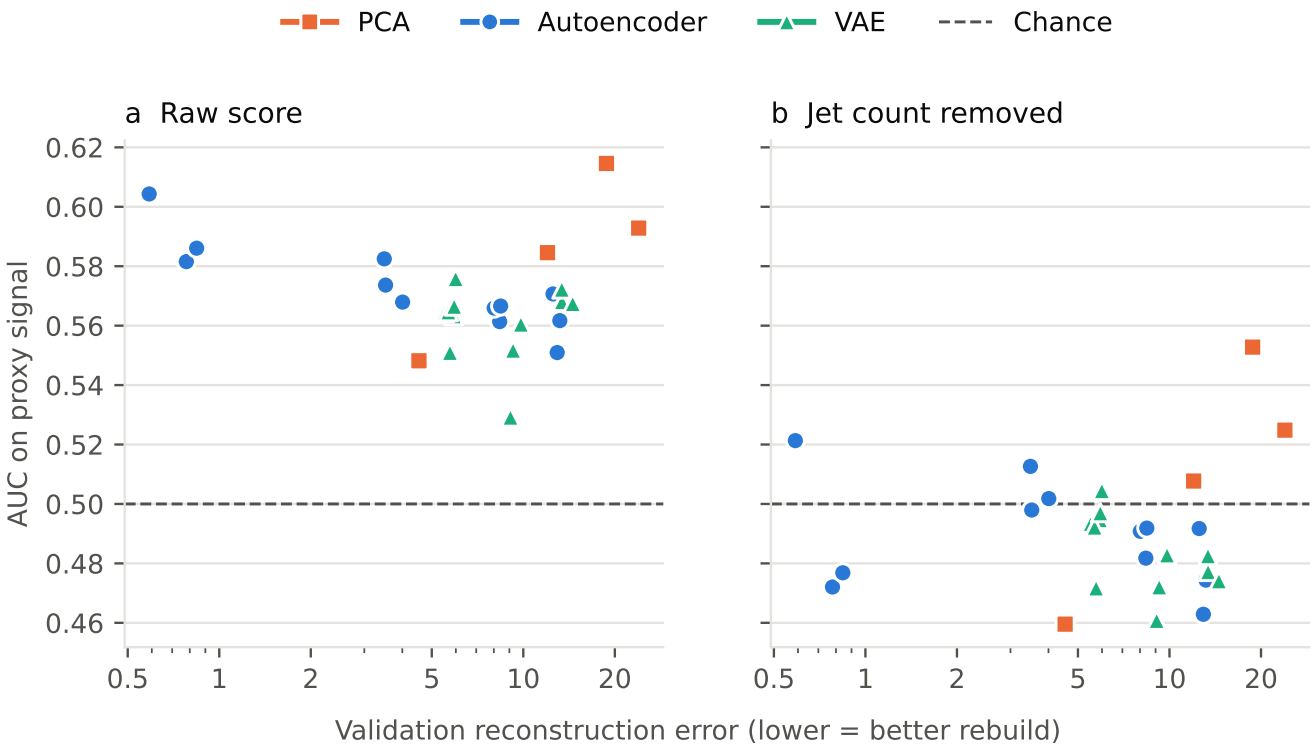


Figure 1: Reconstruction error versus detection. Lower reconstruction error does not bring better detection of the proxy signal.

Note. Each point is one trained model (28 in total; PCA squares, autoencoder circles, variational autoencoder triangles), placed by its validation reconstruction error (log scale, lower means a better rebuild) and its AUC (area under the receiver operating characteristic curve) from (a) the raw score or (b) the score converted to a percentile within its jet count; the dashed line marks chance (AUC 0.5). Created with Python 3.11 and matplotlib 3.11.2.

Table 1: Jet-count mix and raw scores by jet count. Signal events have more jets than background, and the raw score of a trained autoencoder rises with jet count in both classes.

Jets	Background (%)	Signal (%)	Mean score, background	Mean score, signal	AUC within jet count
0	45.2	29.7	0.062	0.066	0.536
1	30.2	32.3	0.134	0.126	0.474
2	15.1	30.0	0.211	0.211	0.506
3	9.5	8.1	0.221	0.206	0.465

Note. Shares are for test background and `sig_eval` signal; the mean score is the mean squared reconstruction error of the autoencoder with latent size $k = 8$ (seed 0); AUC (area under the receiver operating characteristic curve) is computed inside each jet count. Computed with Python 3.11 and scikit-learn 1.9.1.

Yet after the jet-count correction the same two AE configurations detect at most 1.5% of the proxy signal in any single run at the same false-positive rate (Table 2).

Correlated energy-scale shifts are almost invisible. When all 14 energy-like features are multiplied by a common factor of up to 1.20, the raw AUC of every run lies between 0.488 and 0.525, and at most 1.22% of shifted events are caught at a 1% false-positive rate (1.45% after the jet-count correction). Scaling only the jet features gives raw AUCs between 0.491 and 0.505 (Table A5 in the Appendix). The high detection rates above therefore come from moving events off the background distribution, not from sensitivity to changes that keep the correlations between features.

3.5 What the latent codes contain

Figure A4 and Table A6 in the Appendix summarize the probes described in the Methods, and Figures A5 to A7 give further detail. Jet count can be read from the codes with 93.7% to 100% accuracy, against 45.2% for always guessing the most common jet count. From $k = 8$ the AE and VAE codes give 100.0% accuracy, and PCA gives 99.6% at $k = 8$ and 99.8% at $k = 16$.

The proxy signal is also present in the codes. The AUC within jet count of the linear signal probe rises with latent size, from 0.586 to 0.607 at $k = 2$ to 0.747 to 0.762 at $k = 16$, against 0.801 for the same probe on the raw inputs. At $k = 16$ the probe's efficiency at 1% false-positive rate is 6.0% for PCA, 5.9% for the AE and 7.7% for the VAE, compared with 9.5% on the raw inputs, at best 1.6% for

Table 2: Detection of the proxy signal before and after the jet-count correction. The jet-count correction lowers the AUC of every configuration by 0.06 to 0.10, leaving only PCA at $k = 2$ and $k = 4$ above chance.

Model	k	Recon. error	Raw AUC	Raw eff. (%)	Corrected AUC	Corrected eff. (%)
PCA	2	23.96	0.593	1.01	0.525	1.15
PCA	4	18.77	0.615	1.59	0.553	1.61
PCA	8	12.01	0.585	1.74	0.508	0.93
PCA	16	4.54	0.548	1.25	0.460	0.66
AE	2	12.88	0.561 ± 0.010	0.90 ± 0.06	0.476 ± 0.015	0.83 ± 0.06
AE	4	8.27	0.565 ± 0.003	1.10 ± 0.06	0.488 ± 0.006	0.80 ± 0.08
AE	8	3.67	0.575 ± 0.007	1.37 ± 0.27	0.504 ± 0.008	0.87 ± 0.05
AE	16	0.74	0.591 ± 0.012	1.10 ± 0.43	0.490 ± 0.027	1.20 ± 0.40
VAE	2	13.78	0.569 ± 0.003	0.86 ± 0.05	0.478 ± 0.004	0.84 ± 0.04
VAE	4	9.38	0.547 ± 0.016	0.94 ± 0.26	0.472 ± 0.011	0.83 ± 0.09
VAE	8	5.69	0.564 ± 0.001	1.34 ± 0.19	0.493 ± 0.001	0.70 ± 0.09
VAE	16	5.88	0.564 ± 0.013	1.07 ± 0.06	0.491 ± 0.017	0.65 ± 0.12

Note. k, latent size; Recon. error, validation reconstruction error (squared error summed over the 33 features, averaged over validation background events); AUC, area under the receiver operating characteristic curve; eff., signal efficiency at a 1% false-positive rate; PCA, principal component analysis; AE, autoencoder; VAE, variational autoencoder. Values are means ± standard deviation over three seeds; PCA is a single fit, so no spread is given. Chance is an AUC of 0.5 and an efficiency of 1.00%. Computed with Python 3.11, PyTorch 2.14.1 and scikit-learn 1.9.1.

reconstruction error, and 33.7% for the supervised network. The VAE uses 2.0, 4.0, 7.3 and 7.0 active latent dimensions at $k = 2$, 4, 8 and 16.

Unsupervised scores on the same codes recover little of this. After the jet-count correction, the best are the Mahalanobis distance of the VAE code at $k = 16$ (AUC 0.552 ± 0.016, efficiency $1.53\% \pm 0.80\%$), the Mahalanobis distance of the PCA code at $k = 16$ (0.549, interval 0.546 to 0.552; 1.85%, 1.74% to 1.99%) and the Gaussian-mixture likelihood of the PCA code at $k = 4$ (0.555; 1.11%, 1.02% to 1.21%). Both PCA scores meet the above-chance definition, but neither exceeds 2%. The negative evidence lower bound of the VAE behaves like its reconstruction error, with corrected AUCs of 0.487 to 0.502 (Table A3).

3.6 Better reconstruction is not better detection

Better reconstruction never brought detection clearly above chance. For the AE and VAE it moved the corrected AUC from below chance toward chance, and for PCA, components beyond four made detection worse. The AE at $k = 16$ rebuilds background events with an error of 0.74, about 25 times lower than PCA at $k = 4$ (18.77), yet PCA at $k = 4$ is the better detector of the proxy signal after the jet-count correction (AUC 0.553 against 0.490). This agrees with the earlier findings of Fraser *et al.* [18] and Finke *et al.* [17] described in the Introduction. The VAE settles at about seven active dimensions at both $k = 8$ and $k = 16$ (7.3 and 7.0), in line with the observation of Burda, Grosse and Salakhutdinov that adding latent dimensions did not increase the number of active dimensions [11]. For the VAE, mean squared error is only part of how well the model describes the background, and its plateau at 5.7 to 5.9 partly reflects the KL term with $\beta = 1$. Scoring with the full negative evidence lower bound instead of the squared error does not change its detection, though.

3.7 Shifted events and the proxy signal

The detectors respond to anomalies of one kind. Independent shifts in the scaled space move events off the background distribution, which is what a reconstruction-based detector is built to flag, and they are caught far more often than the proxy signal (Table A4). Energy-scale shifts that keep the correlations between features are not caught, and neither is the proxy signal, whose events receive reconstruction errors close to those of background events with the same jet count (Table 1). Here, being poorly rebuilt and being signal-like turned out to be different properties.

3.8 Jet multiplicity as a confound

The raw scores gave a mean AUC between 0.55 and 0.61 for every configuration, which on its own would read as a modest success. Jet multiplicity is a confound: the two classes have different jet-count mixes, and the score rises with jet count (Table 1). The jet count alone reaches a raw AUC of 0.587, so a score that follows it ranks signal above background without detecting anything beyond a single simple feature. When each score is compared only with background events of the same jet count, the apparent detection largely disappears, for the reconstruction scores and for the isolation forest and nearest-neighbour baselines alike.

The confound is not only a product of the input encoding. Masking undefined features weakens the rise of the score with jet count but does not remove it, and the mean background score of the AE at $k = 8$ (seed 0) still grows from 0.103 with zero jets to 0.241 with three. Retraining without `PRI_jet_num` and the two jet indicators changes little. For the AE at $k = 8$ (means over three seeds), jet count can still be read from the code with 92.7% accuracy, the mean background score still grows from 0.063 with zero jets to 0.230 with three, and the corrected AUC is 0.498 ± 0.013 (Table A3).

Metrics for unsupervised detectors should therefore also be reported within groups of any event property that the score tracks and whose distribution differs between signal and background. The percentile correction used here only repairs the evaluation after training. Methods built to make a classifier independent of a nuisance variable, adversarial decorrelation [20, 27] or a distance-correlation penalty [23], could be applied to the anomaly score and the jet count during training. That was not tested here.

3.9 What the codes contain and what the scores use

The label-trained probe reaches 5.9% to 7.7% efficiency on the codes at $k = 16$, and the unsupervised scores at most 1.85%. Because the probe uses labels, this gap shows that signal information is present in the codes, not that an unsupervised score could recover it, and neither reconstruction nor density in the learned space separates this signal in a useful way. Weakly supervised training such as CWoLa [14, 29], which uses mixed samples instead of labels, is one option that this study did not test.

3.10 Limitations

Training budget. Nine of the 24 neural runs (five AE and four VAE) reached the 300-epoch cap before early stopping, so they may not have fully converged; longer budgets were not tested.

Post hoc jet-count analysis. The jet-count confound was found after the first results had been seen, so the jet-count correction was not specified in advance. The `sig_clf` confirmation reduces this concern for the PCA edge but uses the same background test set. Models trained separately for each jet count, or trained with a decorrelation penalty, were not tested.

Proxy signal. The proxy signal shares its final state with one background component, Z boson decays to tau pairs, which makes it a hard case for a detector trained on that background. A second, more distinct proxy, such as one background process held out of training, would test the scope of the conclusions, but the public dataset does not record which process produced each background event, so that split could not be made.

Statistical coverage. All splits use one fixed seed, so variation caused by the choice of split is not measured. Only three model seeds were trained per configuration. PCA is a single deterministic fit, and the bootstrap intervals cover noise in the test sample only. For the AE and VAE they are much narrower than the spread across seeds, so the means and standard deviations over seeds are the better summary. The correlations between reconstruction error and AUC rest on four PCA sizes and eight neural configurations, so they are weak evidence on their own.

Scope of the data. All results come from one simulated dataset and one signal process, Higgs boson decays to tau pairs, used as a proxy signal. The conclusions may not carry over to real collision data, to other signals or to other feature sets.

Models and probes. One architecture (hidden layers of 64 and 32 units), one optimizer setting, one VAE weight ($\beta = 1$) and latent sizes from 2 to 16 were used. Other architectures, optimizer settings, values of β and latent sizes were not explored, and the retraining without jet inputs covers only $k = 8$ for the neural models. The latent probes are linear, so information that only a nonlinear readout could extract does not appear in Table A6.

4 Conclusion

This study compared PCA, an autoencoder and a variational autoencoder as unsupervised detectors of one proxy signal, Higgs boson decays to tau pairs, in the ATLAS Higgs challenge data, with fixed seeds and with signal events kept out of training, validation and tuning. Raw reconstruction error seemed to detect the signal, but the effect came mostly from jet multiplicity: the jet count alone did as well. Within jet-count groups the AE and VAE performed at or below chance, and PCA with two or four components kept a small edge that held on a second half of the signal. The further checks left this picture unchanged, and the latent codes held signal information that the unsupervised scores tested here recovered only in small part.

For future studies, evaluations of unsupervised detectors should report results within groups of any event property, such as jet count, that both the score and the class balance depend on, and should compare against that property used as a score. Natural next steps are decorrelating the score from such properties during training, weakly supervised training, and more distinct proxy signals and datasets.

Declarations and Code Availability

Code and data. The code, result tables and figures are archived on Zenodo [4] and are also available at https://github.com/Abhijay-MSD/higgs-anomaly-detection. The script `src/paper_numbers.py` prints the derived numbers quoted in the text, and the README lists the scripts behind the further checks. Neither ATLAS nor CERN endorses this work.

Funding and competing interests. This research received no external funding, and the author declares no competing interests.

Use of AI tools. Claude (Anthropic) was used to debug the analysis code, including scripts for additional checks introduced during revision, and to execute some of these checks. The author conducted the main experiments, reviewed all code and results, verified the reported figures against the underlying result files, and drafted the article. The author takes full responsibility for the content.

Acknowledgments

The author thanks the editor and the three reviewers for their comments. The work was carried out independently, without a research mentor or supervising teacher.

References

[1] T. Aarrestad, M. van Beekveld, M. Bona, A. Boveia, S. Caron, J. Davies, et al. 2022. The Dark Machines Anomaly Score Challenge: Benchmark Data and Model Independent Event Classification for the Large Hadron Collider. *SciPost Phys.* 12 (2022), 043. doi:10.21468/SciPostPhys.12.1.043

[2] C. Adam-Bourdarios, G. Cowan, C. Germain, I. Guyon, B. Kégl, and D. Rousseau. 2015. The Higgs Boson Machine Learning Challenge. In *Proceedings of the NIPS 2014 Workshop on High-Energy Physics and Machine Learning (Proceedings of Machine Learning Research, Vol. 42)*. PMLR, 19–55. https://proceedings.mlr.press/v42/cowa14.html

[3] G. Alain and Y. Bengio. 2016. Understanding Intermediate Layers Using Linear Classifier Probes. arXiv preprint arXiv:1610.01644. https://arxiv.org/abs/1610.01644

[4] A. Arun. 2026. A Jet-Multiplicity Confound in Reconstruction-Based Anomaly Detection: Evidence from a Higgs-to-Tau-Pair Proxy Signal. Zenodo, version v2-revision [software]. doi:10.5281/zenodo.23685897

[5] ATLAS Collaboration. 2012. Observation of a New Particle in the Search for the Standard Model Higgs Boson with the ATLAS Detector at the LHC. *Phys. Lett. B* (2012). doi:10.1016/j.physletb.2012.08.020

[6] ATLAS Collaboration. 2014. Dataset from the ATLAS Higgs Boson Machine Learning Challenge 2014. CERN Open Data Portal. doi:10.7483/OPENDATA.ATLAS.ZBP2.M5T8

[7] ATLAS Collaboration. 2015. Evidence for the Higgs-Boson Yukawa Coupling to Tau Leptons with the ATLAS Detector. *J. High Energy Phys.* 2015, 4 (2015), 117. doi:10.1007/JHEP04(2015)117
[8] P. Baldi, P. Sadowski, and D. Whiteson. 2014. Searching for Exotic Particles in High-Energy Physics with Deep Learning. *Nat. Commun.* 5 (2014), 4308. doi:10.1038/ncomms5308
[9] P. Baldi, P. Sadowski, and D. Whiteson. 2015. Enhanced Higgs to $\tau^+\tau^-$ Searches with Deep Learning. *Phys. Rev. Lett.* 114 (2015), 111801. doi:10.1103/PhysRevLett.114.111801
[10] V. Belis, P. Odagiu, and T. K. Årrestad. 2024. Machine Learning for Anomaly Detection in Particle Physics. *Rev. Phys.* 12 (2024), 100091. doi:10.1016/j.revip.2024.100091
[11] Y. Burda, R. Grosse, and R. Salakhutdinov. 2016. Importance Weighted Autoencoders. In *International Conference on Learning Representations (ICLR)*. arXiv:1509.00519. https://arxiv.org/abs/1509.00519
[12] O. Cerri, T. Q. Nguyen, M. Pierini, M. Spiropulu, and J.-R. Vlimant. 2019. Variational Autoencoders for New Physics Mining at the Large Hadron Collider. *J. High Energy Phys.* 2019, 5 (2019), 036. doi:10.1007/JHEP05(2019)036
[13] CMS Collaboration. 2012. Observation of a New Boson at a Mass of 125 GeV with the CMS Experiment at the LHC. *Phys. Lett. B* (2012). doi:10.1016/j.physletb.2012.08.021
[14] J. H. Collins, K. Howe, and B. Nachman. 2018. Anomaly Detection for Resonant New Physics with Machine Learning. *Phys. Rev. Lett.* 121 (2018), 241803. doi:10.1103/PhysRevLett.121.241803
[15] B. Efron. 1979. Bootstrap Methods: Another Look at the Jackknife. *Ann. Stat.* 7, 1 (1979). doi:10.1214/aos/1176344552
[16] M. Farina, Y. Nakai, and D. Shih. 2020. Searching for New Physics with Deep Autoencoders. *Phys. Rev. D* 101 (2020), 075021. doi:10.1103/PhysRevD.101.075021
[17] T. Finke, M. Krämer, A. Morandini, A. Mück, and I. Oleksiyuk. 2021. Autoencoders for Unsupervised Anomaly Detection in High Energy Physics. *J. High Energy Phys.* 2021, 6 (2021), 161. doi:10.1007/JHEP06(2021)161
[18] K. Fraser, S. Homiller, R. K. Mishra, B. Ostdiek, and M. D. Schwartz. 2022. Challenges for Unsupervised Anomaly Detection in Particle Physics. *J. High Energy Phys.* 2022, 3 (2022), 066. doi:10.1007/JHEP03(2022)066
[19] E. Govorkova, E. Puljak, T. Aarrestad, T. James, V. Loncar, M. Pierini, et al. 2022. Autoencoders on Field-Programmable Gate Arrays for Real-Time, Unsupervised New Physics Detection at 40 MHz at the Large Hadron Collider. *Nat. Mach. Intell.* 4 (2022), 154–161. doi:10.1038/s42256-022-00441-3
[20] T. Heimel, G. Kasieczka, T. Plehn, and J. M. Thompson. 2019. QCD or What? *SciPost Phys.* 6, 3 (2019), 030. doi:10.21468/SciPostPhys.6.3.030
[21] G. E. Hinton and R. R. Salakhutdinov. 2006. Reducing the Dimensionality of Data with Neural Networks. *Science* 313, 5786 (2006), 504–507. doi:10.1126/science.1127647
[22] G. Kasieczka, B. Nachman, D. Shih, O. Amram, A. Andreassen, et al. 2021. The LHC Olympics 2020: A Community Challenge for Anomaly Detection in High Energy Physics. *Rep. Prog. Phys.* 84, 12 (2021), 124201. doi:10.1088/1361-6633/ac36b9
[23] G. Kasieczka and D. Shih. 2020. Robust Jet Classifiers through Distance Correlation. *Phys. Rev. Lett.* 125 (2020), 122001. doi:10.1103/PhysRevLett.125.122001
[24] D. P. Kingma and J. Ba. 2014. Adam: A Method for Stochastic Optimization. arXiv preprint arXiv:1412.6980. https://arxiv.org/abs/1412.6980
[25] D. P. Kingma and M. Welling. 2013. Auto-Encoding Variational Bayes. arXiv preprint arXiv:1312.6114. https://arxiv.org/abs/1312.6114
[26] F. T. Liu, K. M. Ting, and Z.-H. Zhou. 2008. Isolation Forest. In *2008 Eighth IEEE International Conference on Data Mining (ICDM)*. IEEE, 413–422. doi:10.1109/ICDM.2008.17
[27] G. Louppe, M. Kagan, and K. Cranmer. 2017. Learning to Pivot with Adversarial Networks. In *Advances in Neural Information Processing Systems 30 (NIPS 2017)*. arXiv:1611.01046. https://arxiv.org/abs/1611.01046
[28] L. McInnes, J. Healy, and J. Melville. 2018. UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction. arXiv preprint arXiv:1802.03426. https://arxiv.org/abs/1802.03426
[29] E. M. Metodiev, B. Nachman, and J. Thaler. 2017. Classification without Labels: Learning from Mixed Samples in High Energy Physics. *J. High Energy Phys.* 2017, 10 (2017), 174. doi:10.1007/JHEP10(2017)174
[30] E. Nalisnick, A. Matsukawa, Y. W. Teh, D. Gorur, and B. Lakshminarayanan. 2018. Do Deep Generative Models Know What They Don't Know? arXiv preprint arXiv:1810.09136. https://arxiv.org/abs/1810.09136
[31] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Köpf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala. 2019. PyTorch: An Imperative Style, High-Performance Deep Learning Library. arXiv preprint arXiv:1912.01703. https://arxiv.org/abs/1912.01703
[32] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and É. Duchesnay. 2011. Scikit-learn: Machine Learning in Python. *J. Mach. Learn. Res.* 12 (2011), 2825–2830.
[33] S. Ramaswamy, R. Rastogi, and K. Shim. 2000. Efficient Algorithms for Mining Outliers from Large Data Sets. In *Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data*. ACM, 427–438. doi:10.1145/342009.335437
[34] scikit-learn developers. 2026. sklearn.preprocessing.QuantileTransformer. scikit-learn documentation. Accessed 2026-10-13. https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.QuantileTransformer.html
[35] J. Shlens. 2014. A Tutorial on Principal Component Analysis. arXiv preprint arXiv:1404.1100. https://arxiv.org/abs/1404.1100
[36] P. Virtanen, R. Gommers, T. E. Oliphant, M. Haberland, T. Reddy, D. Cournapeau, et al. 2019. SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python. *arXiv preprint arXiv:1907.10121; published in Nat. Methods* (2019). doi:10.1038/s41592-019-0686-2

A Additional Checks and Results

The tables and figures in this appendix are referred to in the main text.

A.1 Further checks: details

(1) *Confirmation sample.* The jet-corrected evaluation is repeated with `sig_clf` in place of `sig_eval`. The detectors never saw `sig_clf`, and it played no part in choosing the correction.
(2) *Baselines.* The jet count itself used as a score, an isolation forest [26] with 200 trees, and the distance to the 10th nearest of 50,000 training-background events [33].
(3) *Scores on the latent codes.* The Mahalanobis distance of a code from the mean code of 100,000 training-background events, the negative log-likelihood under an eight-component Gaussian mixture fitted to the same codes, and, for the VAE, the loss of Equation 2 evaluated with the mean code (the negative evidence lower bound).
(4) *Masked score.* The squared error is averaged only over the features defined for the event, so median-filled jet features and the indicators do not enter the score.
(5) *No direct jet inputs.* `PRI_jet_num` and the two jet indicators are removed, leaving 30 inputs, and PCA at every k and the AE and VAE at $k = 8$ (three seeds each) are retrained with the same protocol.
(6) *Correlated energy-scale shifts.* In one half of the test background, all 14 energy-like features (transverse momenta, masses and energy sums), or only the four jet features among them, are multiplied by $f \in \{1.02, 1.05, 1.10, 1.20\}$ before the median fill and the quantile transform. Angles, pseudorapidities, ratios and the jet count are unchanged, so shifted events keep the correlations of real events.

A.2 Further results

The effect of latent size depends on the model and on how many features move. PCA at 18 features gets worse as components are added at every shift of 0.5 standard deviations or more. For example, at a shift of 1 standard deviation the efficiency falls from 7.5% at $k = 2$ to 3.2% at $k = 16$. The AE does best at $k = 8$ for the broad shift (31.0% against 19.8% at $k = 16$). For the narrow shift (3 features, 2 standard deviations) it keeps improving with latent size, from 6.1% at $k = 2$ to 33.8% at $k = 16$, although the spread across seeds and feature sets is large at $k = 16$ (Figure A3a).

The mean R^2 of the feature readout rises with latent size for all models. At $k = 16$ it is 0.892 for PCA, 0.759 for the AE and 0.631 for the VAE. At $k = 8$ the readout varies strongly between

features (Figure A5). The azimuthal-angle features (`PRI_tau_phi`, `PRI_lep_phi`, `PRI_met_phi`) are barely recovered: R^2 is 0.00 to 0.02 for PCA, 0.02 to 0.06 for the AE and 0.18 to 0.22 for the VAE. For the pseudorapidity features `PRI_lep_eta` and `PRI_tau_eta`, R^2 is 0.70 and 0.69 for PCA, 0.39 and 0.37 for the VAE, and 0.05 and 0.05 for the AE. The feature `PRI_jet_all_pt` is recovered well by all three models (0.97 for PCA, 0.95 for the AE and 0.80 for the VAE).

Two-dimensional maps of the codes at $k = 8$ (seed 0, 5,000 background and 5,000 signal events) agree with the probes. In the UMAP maps of the AE and PCA codes, events form separate groups that line up with jet count (Figure A7), and the signal points are spread across those same groups (Figure A6).

Table A2: Event counts in each data split. Number of background and signal events in each split of the ATLAS Higgs challenge dataset [6].

Class	Split	Events
Background	Training	377,074
Background	Validation	80,802
Background	Test	80,802
Signal	`sig_clf` (reference and probes)	139,780
Signal	`sig_eval` (evaluation)	139,780

Note. Signal events are never used for training, validation, early stopping, fitting the imputer or transform, or choosing settings; `sig_clf` is used only by the labeled reference models and `sig_eval` only for evaluation. Splits made with scikit-learn 1.9.1 (random seed 0).

Table A4: Shifted events caught at a 1% false-positive rate. The autoencoder and variational autoencoder catch synthetic shifts far more often than PCA.

		3 features	18 features		
Model	k	$s = 2$	$s = 0.5$	$s = 1$	$s = 2$
PCA	2	4.4	2.0	7.5	77.5
PCA	4	5.1	1.7	6.0	66.5
PCA	8	7.7	1.6	5.1	57.1
PCA	16	4.6	1.3	3.2	48.7
AE	2	6.1	2.9	16.3	95.7
AE	4	10.3	4.0	18.3	94.8
AE	8	20.6	3.4	31.0	100.0
AE	16	33.8	2.4	19.8	100.0
VAE	2	6.2	2.5	12.5	95.8
VAE	4	10.6	2.5	18.4	97.7
VAE	8	19.2	3.0	22.4	99.6
VAE	16	18.0	2.9	21.0	99.4

Note. Entries are the percentage of shifted test-background events flagged at a 1% false-positive rate (chance is 1%); k is the latent size and s the shift in standard deviations of the scaled features; AE, autoencoder; VAE, variational autoencoder; PCA, principal component analysis. Values are means over seeds (AE and VAE) and, for the 3-feature case, over three random feature sets. Computed with Python 3.11, PyTorch 2.14.1 and scikit-learn 1.9.1.

Table A1: Prior work most relevant to this study. Setting, approach and main finding of each cited study, as stated in its abstract, set against this study.

Study	Setting	Approach	Finding relevant here
Farina, Nakai and Shih [16]	LHC jets; QCD background, top and gluino jets as signals	Deep autoencoder, reconstruction error	Training on background alone improves signal-to-background discrimination
Heimel *et al.* [20]	Jet substructure	Autoencoder trained on QCD jets, adversarial mass decorrelation	Can identify jets from decays of heavy resonances
Cerri *et al.* [12]	LHC events	VAE trained on known processes	Flags events from unseen processes as outliers
Finke *et al.* [17]	Top and QCD jets	Autoencoder, roles of signal and background inverted	Standard setup is not a model-independent tagger
Fraser *et al.* [18]	Top and W jets against QCD	VAE	Best background representation is not necessarily best for signals
Metodiev, Nachman and Thaler [29]	Quark and gluon jets	Classification without labels	Classifier trained on mixed samples, no labels needed
Collins, Howe and Nachman [14]	Dijet resonance search	Weak supervision (CWoLa)	Turns a modest 2σ excess into a 7σ excess
Kasieczka *et al.* [22]	LHC Olympics 2020	Community challenge	Black-box datasets for benchmarking model-agnostic methods
Aarrestad *et al.* [1]	Over 10^9 simulated LHC events	Community challenge	Anomaly scores for model-independent signal regions
Govorkova *et al.* [19]	Real-time event selection	Autoencoder on FPGA	Detection in as little as 80 ns
Nalisnick *et al.* [30]	Image datasets	Flows, VAEs, PixelCNNs	Higher likelihood assigned to data from another dataset
This study	ATLAS Higgs challenge, H$\to\tau\tau$ as proxy	PCA, AE, VAE; jet-count control; probes	Reconstruction error barely detects the proxy signal once jet count is controlled

Note. QCD, quantum chromodynamics (ordinary quark and gluon jets); AE, autoencoder; VAE, variational autoencoder; PCA, principal component analysis; CWoLa, classification without labels; FPGA, field-programmable gate array; H$\to\tau\tau$, Higgs boson decay to two tau leptons.

Table A3: Further checks of the jet-count finding. Baselines, latent-code scores, a masked score and retraining without jet inputs all give corrected results near chance; only PCA keeps a small edge.

Check	Score or model, k	Raw AUC	Corrected AUC	Corrected eff. (%)
Confirmation (`sig_clf`)	PCA, 2	0.592	0.524 (0.521–0.526)	1.17 (1.08–1.29)
	PCA, 4	0.614	0.552 (0.549–0.554)	1.59 (1.49–1.71)
	PCA, 8	0.584	0.506 (0.503–0.509)	0.97 (0.89–1.06)
	PCA, 16	0.547	0.457 (0.455–0.459)	0.66 (0.59–0.71)
	AE, VAE (all k)	0.547–0.591	0.471–0.503	0.67–1.21
Baseline	Jet count as the score	0.587	0.500	1.00
	Isolation forest	0.562	0.491	1.13
	Distance to 10th neighbour	0.594	0.491	0.85
Latent-code score	Mahalanobis distance: PCA, 16	0.603	0.549 (0.546–0.552)	1.85 (1.74–1.99)
	Mahalanobis distance: AE, 4	0.568	0.514 ± 0.028	1.33 ± 0.30
	Mahalanobis distance: VAE, 16	0.595	0.552 ± 0.016	1.53 ± 0.80
	Gaussian-mixture NLL: PCA, 4	0.622	0.555 (0.552–0.557)	1.11 (1.02–1.21)
	Gaussian-mixture NLL: AE, 16	0.600	0.514 ± 0.005	1.29 ± 0.13
	Gaussian-mixture NLL: VAE, 4	0.568	0.546 ± 0.019	0.85 ± 0.19
	Negative ELBO: VAE, all k	0.576–0.587	0.487–0.502	0.72–0.88
Masked score	PCA, 4	0.609	0.554	1.60
	AE, VAE (all k)	0.522–0.592	0.473–0.505	0.69–1.27
No jet inputs	PCA, 2	0.589	0.518	1.11
	PCA, 4	0.613	0.546	1.48
	PCA, 8	0.581	0.500	0.84
	PCA, 16	0.542	0.456	0.66
	AE, 8 (3 seeds)	0.567 ± 0.015	0.498 ± 0.013	0.74 ± 0.04
	VAE, 8 (3 seeds)	0.576 ± 0.010	0.509 ± 0.015	0.67 ± 0.11

Note. Evaluated on `sig_eval` except the confirmation rows, which use `sig_clf`; 95% bootstrap intervals for single PCA fits are in brackets. Corrected values use the percentile within jet count; eff., signal efficiency at a 1% false-positive rate (chance 1.00%). Latent-code rows show the latent size with the highest corrected AUC for each score and model (negative ELBO: range over all latent sizes); values for every latent size are in `results/revision/E_latent_scores.csv` in the repository. Ranges are over configuration means; ± values are standard deviations over seeds. NLL, negative log-likelihood; ELBO, evidence lower bound. Computed with Python 3.11, PyTorch 2.14.1 and scikit-learn 1.9.1.

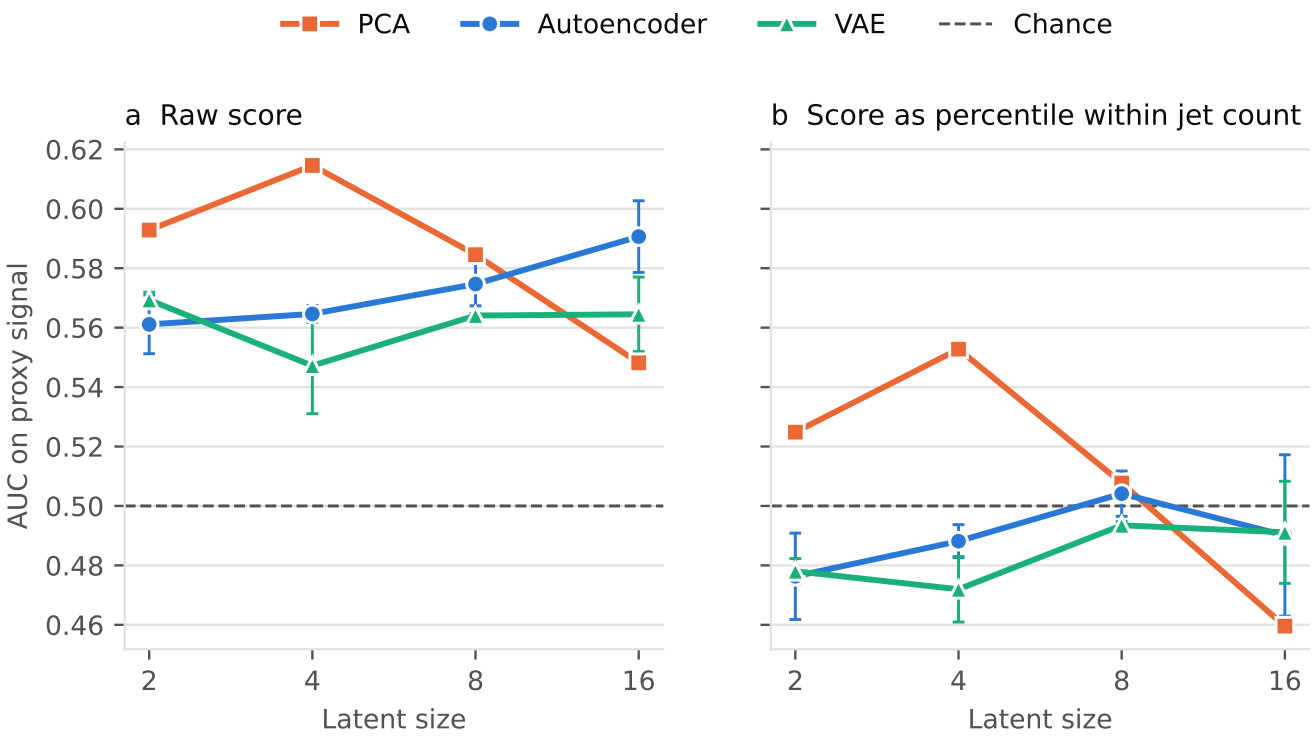


Figure A1: AUC versus latent size, raw and jet-corrected. After the jet-count correction, only PCA stays clearly above chance on the proxy signal.

Note. AUC (area under the receiver operating characteristic curve) against latent size for (a) the raw reconstruction error and (b) the score as a percentile among validation background events with the same jet count; points are means over three seeds with standard-deviation error bars (PCA is a single fit), and the dashed line marks chance. Created with Python 3.11 and matplotlib 3.11.2.

Table A5: Correlated energy-scale shifts. Scaling energies by up to 20% while keeping correlations leaves every detector near chance.

Features scaled	Factor	PCA, 8	AE, 8	VAE, 8	All runs	Max. eff. raw (%)	Max. eff. corrected (%)
All energy-like features	1.02	0.494	0.494	0.492	0.490–0.495	1.03	1.00
	1.05	0.492	0.494	0.491	0.488–0.496	1.02	1.01
	1.10	0.492	0.497	0.495	0.489–0.503	1.05	1.10
	1.20	0.499	0.508	0.512	0.493–0.525	1.22	1.45
Jet features only	1.02	0.497	0.497	0.497	0.495–0.500	1.03	1.09
	1.05	0.496	0.497	0.497	0.494–0.500	1.02	1.10
	1.10	0.495	0.498	0.497	0.492–0.501	1.04	1.14
	1.20	0.496	0.501	0.499	0.491–0.505	1.07	1.19

Note. Entries are raw AUC (area under the receiver operating characteristic curve) for shifted against unshifted halves of the test background; the latent-size-8 columns are means over seeds, and "All runs" gives the range over all 28 runs. The last two columns give the highest share of shifted events caught at a 1% false-positive rate over all runs, raw and after the jet-count correction. Energy-like features are the 14 transverse momenta, masses and energy sums; jet features are the four among them that describe jets. Computed with Python 3.11, PyTorch 2.14.1 and scikit-learn 1.9.1.

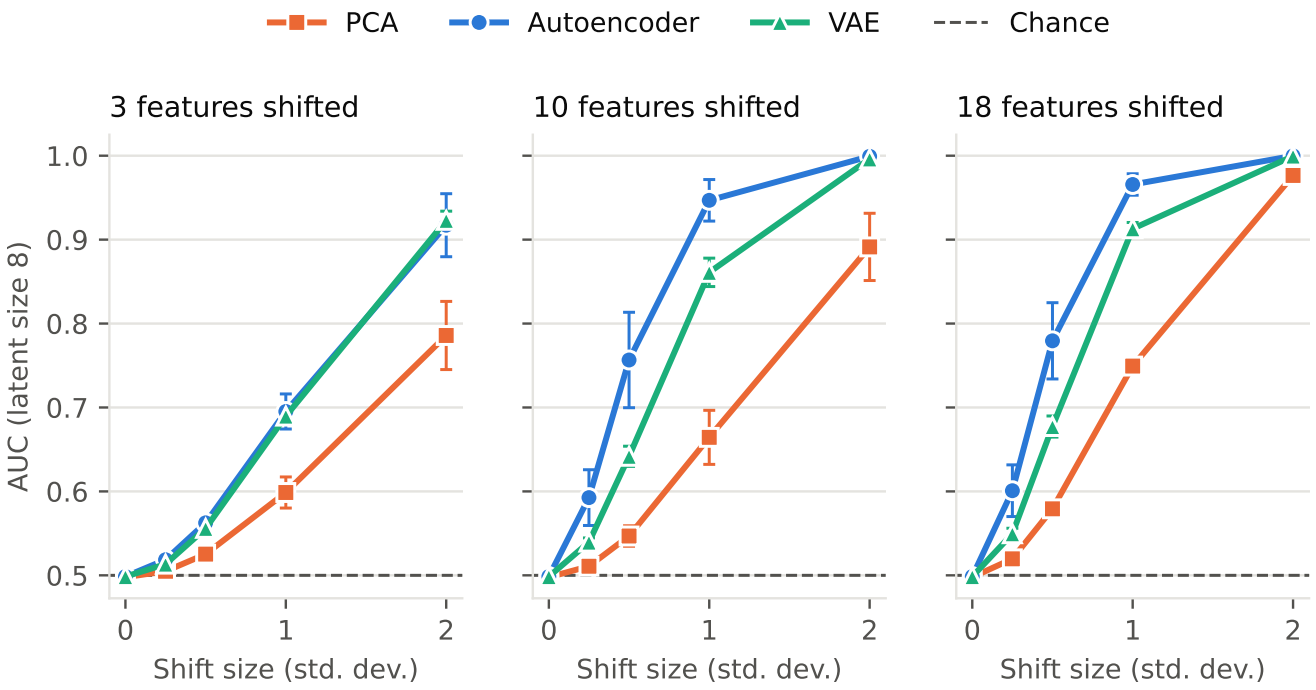


Figure A2: AUC for synthetic shifts at latent size 8. The autoencoder and variational autoencoder detect synthetic shifts at smaller shift sizes than PCA.

Note. AUC (area under the receiver operating characteristic curve) at latent size 8 against shift size in standard deviations, for 3, 10 and 18 shifted features; error bars show the standard deviation over seeds and feature sets, and the dashed line marks chance. Created with Python 3.11 and matplotlib 3.11.2.

Table A6: Linear probes on the latent codes. Linear probes read jet count almost perfectly and the proxy signal partly from the latent codes.

Model	k	Mean R^2	Jet accuracy (%)	Signal AUC	Signal eff. at 1% (%)	Active dims
PCA	2	0.223	93.7	0.590	2.1	–
PCA	4	0.365	94.9	0.656	4.0	–
PCA	8	0.666	99.6	0.759	4.6	–
PCA	16	0.892	99.8	0.762	6.0	–
AE	2	0.170 ± 0.010	97.3 ± 0.8	0.607 ± 0.013	1.1 ± 0.0	–
AE	4	0.214 ± 0.004	99.0 ± 0.9	0.641 ± 0.033	3.1 ± 2.9	–
AE	8	0.392 ± 0.038	100.0 ± 0.0	0.684 ± 0.026	3.0 ± 1.2	–
AE	16	0.759 ± 0.037	100.0 ± 0.0	0.753 ± 0.011	5.9 ± 0.3	–
VAE	2	0.209 ± 0.027	93.8 ± 0.9	0.586 ± 0.030	1.0 ± 0.1	2.0 ± 0.0
VAE	4	0.303 ± 0.062	94.5 ± 3.1	0.687 ± 0.027	2.0 ± 1.0	4.0 ± 0.0
VAE	8	0.549 ± 0.013	100.0 ± 0.0	0.743 ± 0.015	4.3 ± 2.0	7.3 ± 0.6
VAE	16	0.631 ± 0.008	100.0 ± 0.0	0.747 ± 0.018	7.7 ± 2.9	7.0 ± 0.0
Raw inputs	33	–	–	0.801	9.5	–

Note. k, latent size; R^2, coefficient of determination, averaged over the 18 always-defined features; jet accuracy, share of test events whose jet count is predicted correctly (45.2% for always guessing the most common jet count); signal AUC, area under the receiver operating characteristic curve within jet count; signal eff., signal efficiency at a 1% false-positive rate; active dims, VAE latent dimensions whose mean code has a variance above 0.01. Values are means ± standard deviation over three seeds (PCA is a single fit); the last row applies the signal probe to the 33 input features. Computed with Python 3.11 and scikit-learn 1.9.1.

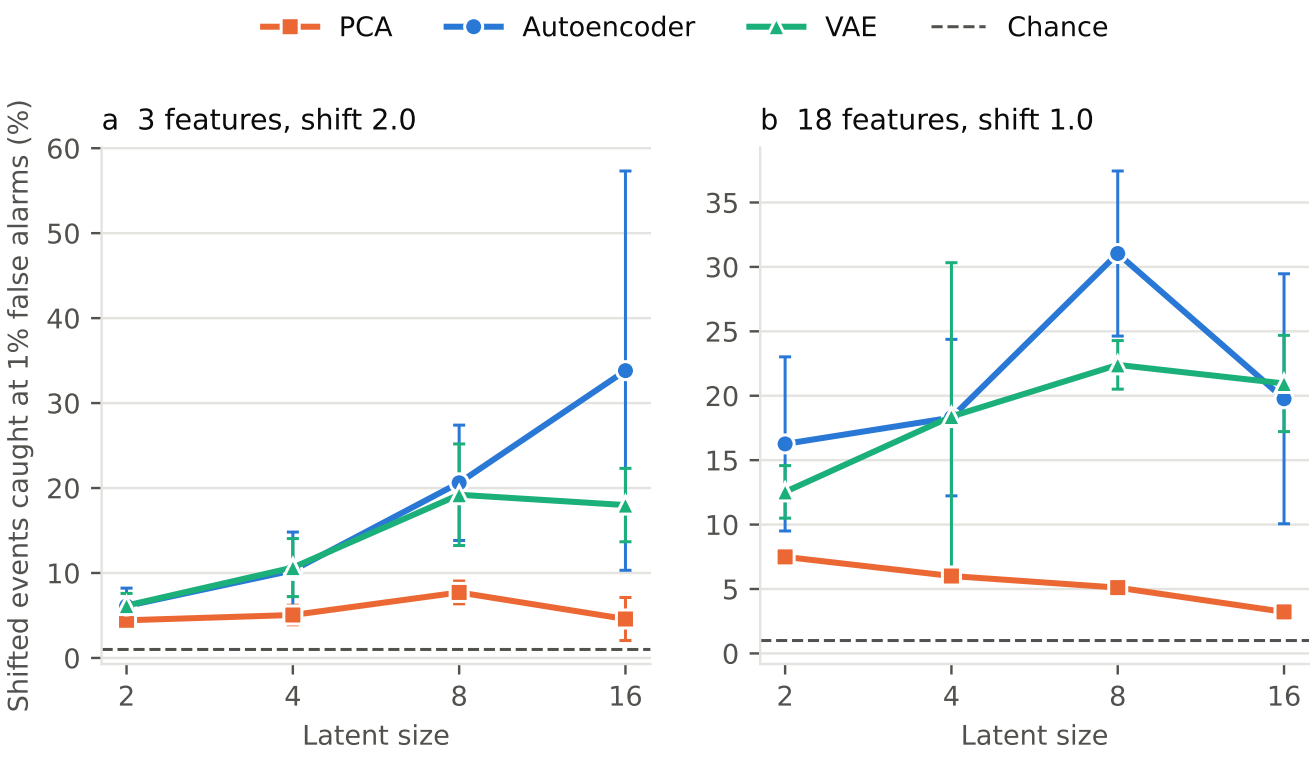


Figure A3: Shift detection by latent size. A larger latent size helps the autoencoder on a narrow shift but makes PCA worse on a broad one.

Note. Share of shifted events caught at a 1% false-positive rate against latent size for (a) three features shifted by 2 standard deviations and (b) all 18 always-defined features shifted by 1 standard deviation; error bars show the standard deviation over seeds and feature sets, and the dashed line marks chance (1%). Created with Python 3.11 and matplotlib 3.11.2.

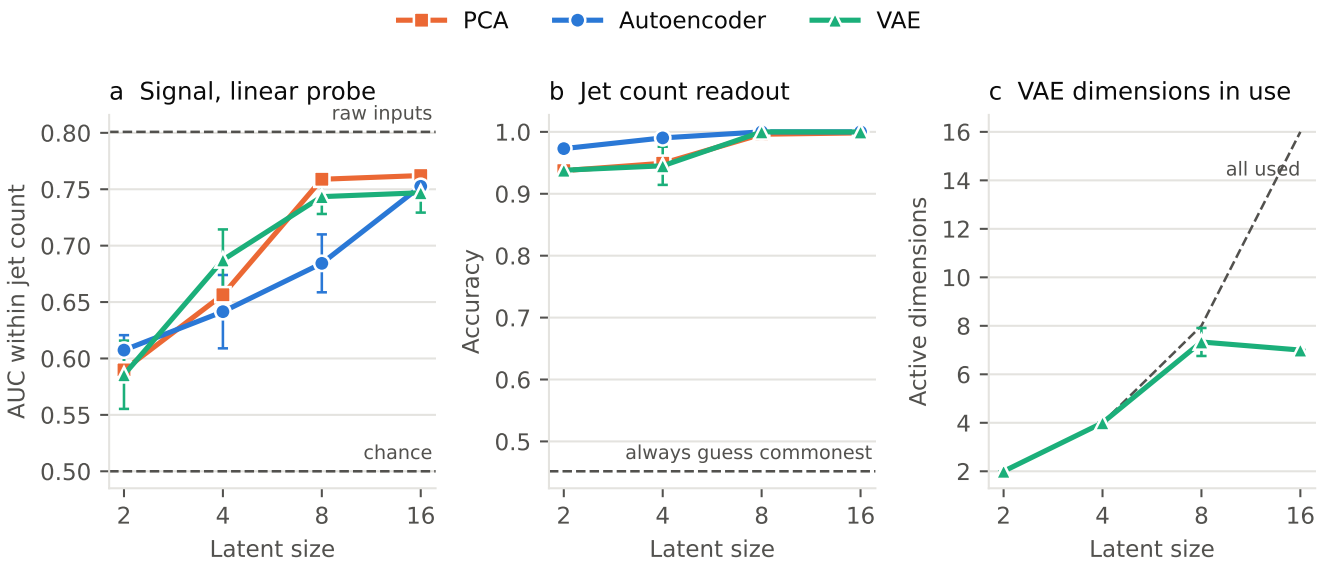


Figure A4: Contents of the latent codes. The latent codes hold the jet count and part of the proxy signal.

Note. (a) AUC (area under the receiver operating characteristic curve) of a linear signal probe within jet count, with dashed lines for chance and for the same probe on the raw inputs; (b) accuracy of reading jet count from the code, with a dashed line for always guessing the most common jet count; (c) active latent dimensions of the variational autoencoder (VAE), with a dashed line where all are used. Points are means over three seeds with standard-deviation error bars. Created with Python 3.11 and matplotlib 3.11.2.

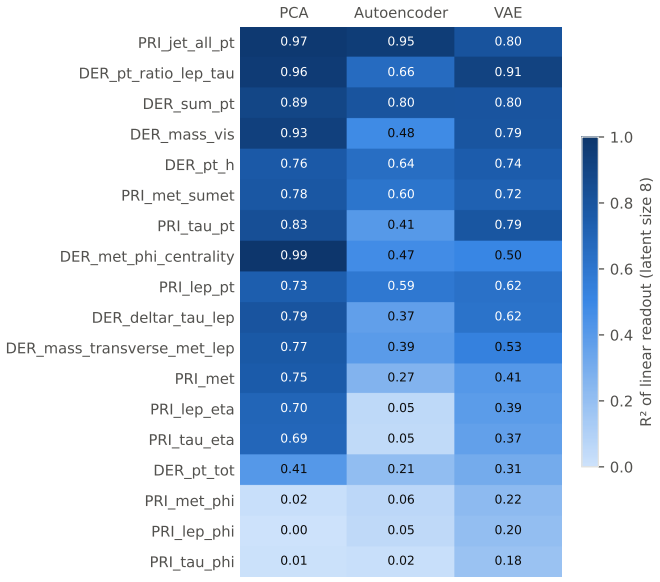


Figure A5: Feature readout from the codes at latent size 8. Azimuthal angles (ϕ) are poorly recovered by every code (R^2 at most 0.22), and the autoencoder also loses the pseudorapidities (η).

Note. Each cell is the R^2 (coefficient of determination) of a linear readout of one always-defined feature from the code at latent size 8, averaged over seeds; features are sorted by their mean R^2 across the three models. Created with Python 3.11 and matplotlib 3.11.2.

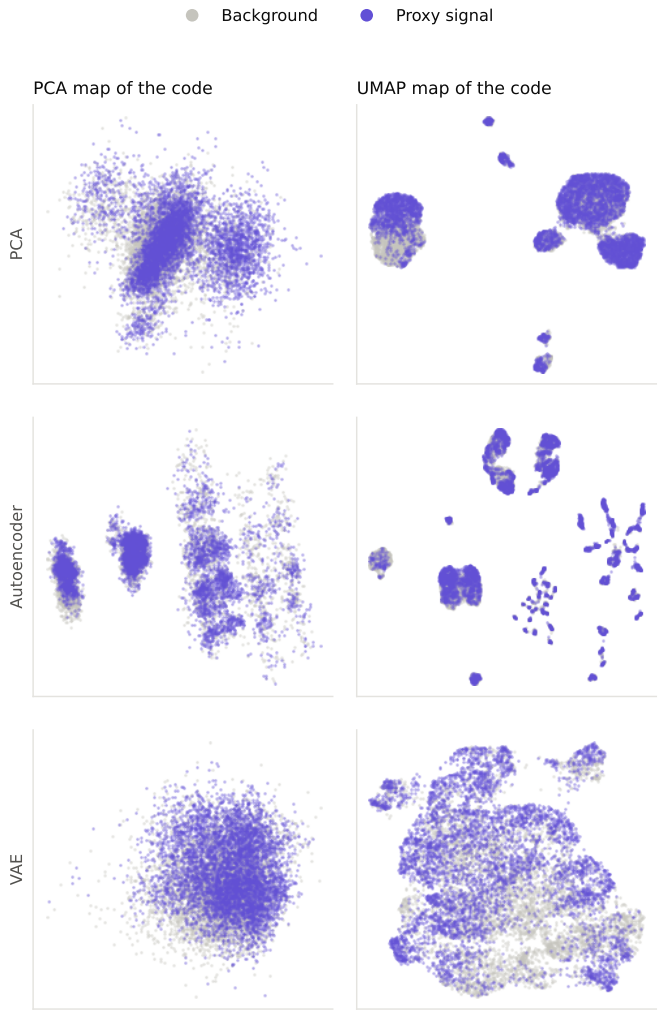


Figure A6: Latent maps colored by class. Proxy-signal points spread across the same groups as background points in every map.

Note. Codes at latent size 8 (seed 0) for 5,000 background and 5,000 signal events, projected to two dimensions with PCA (left) and uniform manifold approximation and projection (UMAP, right); rows are the PCA, autoencoder and variational autoencoder codes, and signal points are drawn over background points. Created with Python 3.11, matplotlib 3.11.2, scikit-learn 1.9.1 and umap-learn 0.5.12.



Figure A7: Latent maps colored by jet count. The groups in the latent maps follow jet count.

Note. Background events from the maps in Figure A6, colored by jet count from light (0 jets) to dark (3 jets). Created with Python 3.11, matplotlib 3.11.2, scikit-learn 1.9.1 and umap-learn 0.5.12.

Iterative Development of an AI Prototype for Dental Caries Detection and Localization with Informal Clinical Feedback

Chinmay Karthik Amancha
Chelmsford High School
Chelmsford, MA, USA
amancha.chinmay.usa@gmail.com

Abstract

This paper documents the iterative development of a student-built dental caries detection prototype and compares its final version, V4.1, with the preceding V4 system and a fresh factory-default YOLO11n baseline, testing sensitivity to validation selection and reader choice. The final phase used 100 public permanent-dentition bitewing radiographs with annotations from eight dentists and a fixed 60/20/20 image split. V4.1 selected a two-model fusion from 75 validation configurations and was evaluated exploratorily on a previously viewed 20-image holdout. Against reader E0 on this holdout, V4.1 achieved precision 0.415, recall 0.449, F1 0.431, AP50 0.491, and 1.55 false-positive boxes per image; V4 achieved 0.412, 0.286, 0.337, 0.271, and 1.00, and the factory-default baseline achieved 0.750, 0.306, 0.435, 0.458, and 0.25. Only three of the 75 validation configurations were within 0.05 F1 of the winner, and at threshold 0.16 V4.1 F1 ranged from 0.140 to 0.448 across the eight reader references. A precision-oriented threshold of 0.37 reduced V4.1 false positives to 0.50 per image with recall 0.367, and a conditional three-fold out-of-fold stress test produced F1 0.538 and AP50 0.510. V4.1 thus improved recall and AP50 over both comparators, but its F1 was essentially the same as the factory-default baseline and it produced more false-positive boxes. Performance depended strongly on reader choice and threshold, and the small reused holdout, concentrated validation winner, non-nested cross-validation, and absence of patient identifiers prevent clinical or causal claims.

Keywords

Artificial intelligence, dental caries, bitewing radiography, YOLO, object detection, multi-reader evaluation, precision-recall

ACM Reference Format:
Chinmay Karthik Amancha. 2026. Iterative Development of an AI Prototype for Dental Caries Detection and Localization with Informal Clinical Feedback. In *Proceedings of International Journal of Secondary Computing and Applications Research (IJSCAR VOL. 3, ISSUE 4).* ACM, New York, NY, USA, 6 pages. https://doi.org/10.67149/yhjs2024.5/03kanv0l

1 Introduction

Bitewing radiographs support detection of proximal caries, but interpretation varies among dentists and lesion boundaries are not perfectly reproducible. Deep-learning systems have reported promising results, yet reviews continue to identify small datasets, heterogeneous reference standards, limited external validation, and risk of bias [1–4]. These concerns are especially important for a 100-image pilot.

This project began with broad pediatric panoramic experiments and later narrowed to one task—caries localization—on permanent-dentition bitewings after informal consultations with dentists. In particular, Dr. Mark Y. Wang, DMD, DMSc, a prosthodontist practising in Concord, Massachusetts, reviewed early prototype demonstrations and discussed missed or ambiguous caries findings. His feedback informed the research questions and focus on permanent-dentition caries; it did not constitute a clinical reader study or validation, and this observational history cannot establish that feedback caused any measured performance improvement.

The revised analysis asks a narrower question: how did a validation-locked V4.1 detector compare with the preceding V4 detector on the same reused holdout, and how stable were the conclusions across candidate configurations, thresholds, reader references, and an internal resampling analysis?

2 Materials and Methods

2.1 Study design and data

Versions 1–3 were formative experiments on a separate 100-image pediatric panoramic subset of the public Children's Dental Panoramic Radiographs Dataset [8] and are summarized in Supplement S1. The quantitative study concerns V4 and V4.1, which used the public Dental Caries in Bitewing Radiographs dataset [6]. It contains 100 bitewings annotated independently by E0, E1–E4, and N1–N3. E0 supplied the primary training labels. The fixed split contained 60 training images (176 E0 boxes), 20 validation images (44 boxes), and 20 reused-holdout images (49 boxes). The release lacked patient identifiers, so patient-grouped independence could not be verified.

2.2 Comparators

V4 used a COCO-pretrained YOLO11n detector, 768-pixel inputs, batch size 4, AdamW with initial learning rate 0.001, seed 42, deterministic mode, a maximum of 40 epochs, early-stopping patience 12, horizontal flip probability 0.5, rotation up to 5 degrees, translation 0.05, scale 0.20, and brightness augmentation 0.10. Mosaic, mixup, copy-paste, and vertical flipping were disabled. The validation-selected threshold was 0.24 for the full-frame 1024-pixel inference comparator used in the paired report. This comparator was not an untouched factory-default YOLO11 training run.

The fresh factory-default comparator started from the official generic YOLO11n checkpoint and used Ultralytics defaults: 100

This paper is published under the Creative Commons Attribution-NonCommercial-NoDerivs 4.0 International (CC-BY-NC-ND 4.0) license. Authors reserve their rights to disseminate the work on their personal and corporate Web sites with the appropriate attribution.
IJSCAR VOL. 3, ISSUE 4
© 2026 The Author(s). Published by IJSCAR.

epochs, 640-pixel inputs, batch size 16, seed 0, automatic optimizer selection, standard default augmentation, and best validation-fitness checkpoint selection. Only the dataset path, CPU device, zero data-loader workers, and output location were specified. A confidence threshold of 0.35 was locked on the validation set by maximum lesion F1, then precision, then the higher threshold; the holdout was evaluated only after lock.

2.3 V4.1 training and selection

V4.1 trained YOLO11n at 768 and 1024 pixels for up to 80 epochs and YOLO11s at 768 pixels for up to 60 epochs. The YOLO11n runs used batches of 4 and 2 and learning rate 0.001; YOLO11s used batch 2 and learning rate 0.0005. All used COCO-pretrained weights, AdamW, seed 42, and patience 20. The reported software environment and training settings are listed in the Supplement. Checkpoint identities were recorded by SHA-256 in the archived provenance files.

The validation grid included 72 single-checkpoint candidates and three two-model fusion variants. Ranking used lesion F1 at IoU 0.50, then AP50, precision, and lower complexity. The selected fusion combined a YOLO11n-768 checkpoint with full-frame 1024-pixel inference and a YOLO11n-1024 checkpoint with horizontally flipped 1024-pixel inference. The system and confidence threshold 0.16 were locked before holdout scoring.

2.4 Evaluation and sensitivity analyses

Lesion predictions were matched one-to-one to reference boxes at IoU at least 0.50. We report precision, recall, F1, false-positive boxes per image, 101-point AP50, and mAP50–95. Image-level flags used the highest detection score per image and are reported separately. Paired image bootstrap intervals used 5,000 resamples with seed 42. Saved V4.1 predictions were rescored separately against each of the eight readers at thresholds 0.16 and 0.37. Detections unmatched to E0 were also checked for overlap with each alternative reader. These reader-specific comparisons measure agreement, not diagnostic truth.

A completed three-fold out-of-fold stress test trained the selected recipe within the original 60-image training subset and applied the fixed threshold 0.16. Because model selection had already seen the full training subset, this is conditional recipe stability, not nested cross-validation or independent validation.

3 Results

3.1 Comparator results

3.2 Validation leaderboard

The selected validation F1 was 0.695. Three candidates tied at F1 0.695; all three were the same two-model fusion with different final suppression. The next-best single-checkpoint candidate had F1 0.643, 0.052 below the winner. The median candidate F1 was 0.556. This pattern is not a broadly flat leaderboard and reinforces the risk that a narrow winning cluster was selected from a small validation sample.

Table 1: Lesion-level results on the same 20-image reused holdout. Thresholds were selected on validation: V4 0.24, factory-default 0.35, and V4.1 0.16. The paired V4.1-minus-V4 bootstrap interval for F1 was 0.009–0.172; no bootstrap comparison was prespecified for the newly trained baseline.

Metric	V4	Factory default	V4.1
True positive lesions	14	15	22
False-positive boxes	20	5	31
Missed lesions	35	34	27
Precision	0.412	0.750	0.415
Recall	0.286	0.306	0.449
F1	0.337	0.435	0.431
AP50	0.271	0.458	0.491
mAP50–95	0.100	0.187	0.222
False positives per image	1.00	0.25	1.55

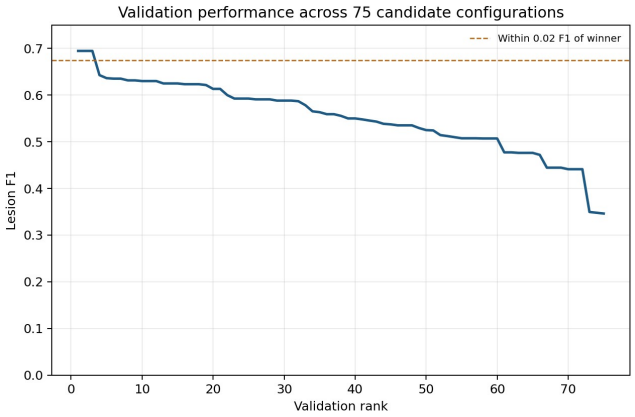


Figure 1: Lesion F1 for all 75 validation configurations, ordered by validation rank.

Table 2: Two validation-selected operating points applied unchanged to the reused holdout. The higher threshold substantially reduced false-positive boxes but missed more E0 lesions.

Threshold	Precision	Recall	F1	FP per image
0.16 balanced	0.415	0.449	0.431	1.55
0.37 precision-oriented	0.643	0.367	0.468	0.50

3.3 Threshold trade-off

At the image level, the balanced threshold produced sensitivity 0.867, specificity 0.800, accuracy 0.850, balanced accuracy 0.833, and ROC AUC 0.933. Specificity was based on only five E0-negative images; one image changes the estimate by 0.20. An image-level flag indicates that at least one detection exceeded threshold and does not establish that every lesion was localized.

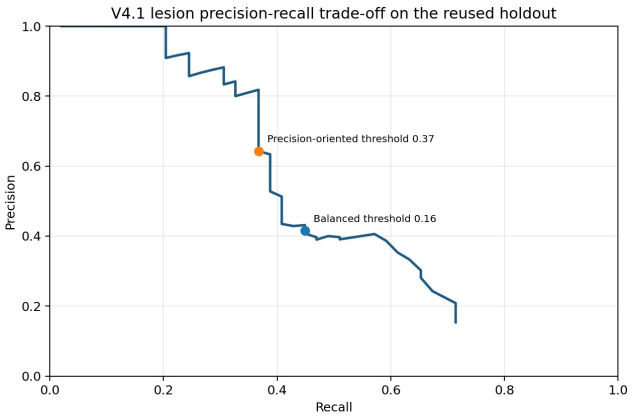


Figure 2: Precision-recall trade-off reconstructed from saved V4.1 holdout predictions and E0 boxes. AP50 was 0.491.

Table 3: V4.1 at the locked threshold 0.16, rescored separately against all eight readers. The 20 holdout images contained 47–85 boxes depending on reader, and F1 ranged from 0.140 to 0.448.

Reader	Boxes	Precision	Recall	F1	FP/image
E0	49	0.415	0.449	0.431	1.55
E1	85	0.302	0.188	0.232	1.85
E2	50	0.283	0.300	0.291	1.90
E3	49	0.358	0.388	0.373	1.70
E4	47	0.132	0.149	0.140	2.30
N1	81	0.566	0.370	0.448	1.15
N2	67	0.321	0.254	0.283	1.80
N3	63	0.434	0.365	0.397	1.50

3.4 Multi-reader sensitivity

Of 31 detections unmatched to E0, 13 overlapped at least one other reader's box, 9 overlapped at least two, 5 overlapped at least three, and 3 overlapped at least four. The remainder cannot be assumed to be true disease or true error without adjudication. These results show that the reported false-positive count is reference-dependent.

3.5 Conditional cross-validation stress test

Across three out-of-fold partitions of the original 60-image training subset, the fixed-threshold results were 82 true positives, 47 false positives, and 94 missed lesions among 176 E0 boxes: precision 0.636, recall 0.466, F1 0.538, and 0.783 false positives per image. AP50 was 0.510 and mAP50–95 was 0.188. Image-level accuracy was 0.950 and ROC AUC 0.995, but these image-level estimates again do not measure complete lesion localization. Because recipe selection was not nested, the stress test is supportive rather than confirmatory.

3.6 Evaluated output

4 Discussion

V4.1 improved recall, F1, AP50, and mAP50–95 relative to the documented V4 comparator. Against the fresh factory-default baseline, V4.1 improved recall and AP50 but had nearly the same F1 (0.431 versus 0.435) and substantially more false-positive boxes (1.55 versus 0.25 per image). The precision-oriented V4.1 threshold illustrates the trade-off: reducing alerts to 0.50 per image increased precision but reduced recall. A clinical operating point cannot be chosen from this pilot; that choice would require a specified workflow, error costs, calibration, and independent validation.

The multi-reader analysis materially changes interpretation. Agreement was strongest with E0 and N1 and weakest with E4, and 42% of E0-unmatched detections overlapped at least one alternative reader. Therefore, "false positive" means unmatched to a selected reader under the specified IoU rule, not proven absence of caries. Consensus or adjudicated labels are needed for stronger claims.

The validation leaderboard was concentrated rather than flat. Although the configuration was locked before holdout scoring, selecting a narrow winning fusion from 75 candidates on 20 images creates a genuine validation-overfitting risk. The three-fold analysis is reassuring about recipe stability but is not nested and cannot remove this bias.

Published context also limits interpretation. Tichý and colleagues evaluated the ensemble developed in the companion Part I study [5], trained on 3,989 separate bitewings, against the original annotator on the same 100-image evaluation dataset and reported precision 0.78, recall 0.73, and F1 0.75 [7]. Those values were verified directly in the primary Part II paper. The system used far more training data and multiple detector families, so it provides context rather than a fair head-to-head benchmark for this 60-image training split.

The reviewer-requested factory-default comparison was completed as a fresh 100-epoch YOLO11n run. Its high precision and low false-positive burden accompanied low recall, while V4.1 recovered more E0 lesions and achieved higher AP50. Because all systems used the same small image split and reused holdout, this comparison does not solve the more important limitations of sample size, holdout reuse, and reference uncertainty.

The informal consultations influenced project framing and visualization questions, but neither clinician annotated the evaluation set or participated in a controlled experiment. The manuscript therefore does not claim that clinical feedback caused the measured gain. The insurance idea is limited to one future-work sentence because no insurer, claim outcome, or workflow endpoint was studied.

5 Limitations

This study used 100 images, a 20-image validation set, and a previously examined 20-image holdout. Patient identifiers were unavailable. E0 labels were used for training and primary selection, candidate search was extensive relative to validation size, and cross-validation was not nested. The factory-default comparator was a single run rather than a repeated-seed estimate. Confidence scores were not calibrated probabilities, and robustness tests were artificial. The consultations were informal, no diagnostic reader study was performed, and no external cohort, histologic reference, or adjudicated consensus was available.

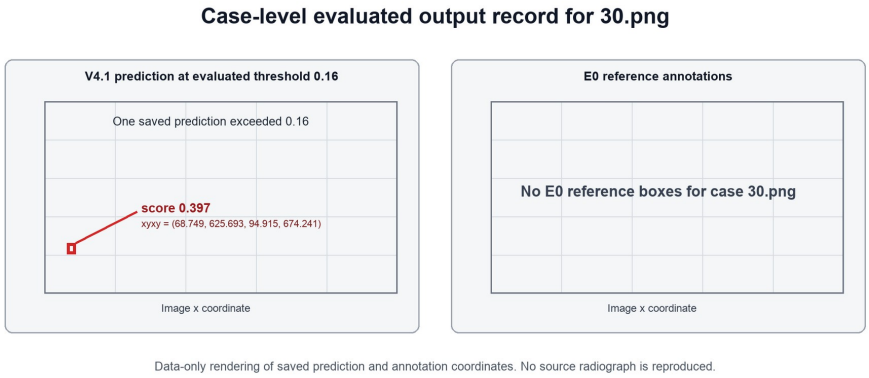


Figure 3: Data-only case record for 30.png at the evaluated threshold 0.16. The single saved V4.1 prediction (score 0.397) was unmatched because E0 supplied no reference box. Coordinates are shown without reproducing the source radiograph.

6 Conclusion

V4.1 improved recall and AP50 over both the documented V4 comparator and the fresh factory-default baseline, but its F1 was essentially tied with the default baseline and it produced more false-positive boxes. Results varied substantially with reader choice and operating threshold. The evidence supports continued research, not clinical use, and does not establish that clinician feedback caused the observed differences.

7 Future Work

Priorities are a larger patient-grouped cohort, a locked independent external test, multi-expert consensus or adjudication, repeated-seed comparator runs, nested model-selection assessment, score calibration, and a formal reader study. Visualization styles and insurance-documentation hypotheses should be tested separately from detector accuracy.

Acknowledgments

The author thanks Dr. Mark Y. Wang, DMD, DMSc, a prosthodontist practising in Concord, Massachusetts, for reviewing early prototype demonstrations and offering informal advice about missed or ambiguous caries findings. Dr. Wang did not annotate the evaluation set, participate in blinded reader testing, or clinically validate the system.

Ethics, Data, Funding, and Disclosures

Ethics: The project used public de-identified datasets; no new participants were recruited or imaged.

Data availability and attribution: the V4 and V4.1 analyses used Dental caries in bitewing radiographs, Version 1, contributed by Jan Kybic, Antonín Tichý, and Lukáš Kunt, Mendeley Data, DOI 10.17632/4fbdxs7s7w.1, licensed CC BY-NC 3.0. The fixed analysis split and derived outputs used in this study are described in the Supplement. Figure 3 is an original data-only visualization created for this study from saved model output; it does not reproduce source radiograph pixels. Its coordinates and E0 reference status derive from the licensed dataset.

Funding: none.

Conflict of interest: the author reports no financial interest in dental-AI companies mentioned.

AI use: OpenAI ChatGPT assisted with technical explanation, scripted analysis checks, organization, drafting, and language refinement. The author reviewed the materials and retains responsibility for accuracy and conclusions; no independent human statistician or external analyst performed a separate validation.

References

[1] Nour Ammar and Jan Kühnisch. 2024. Diagnostic performance of artificial intelligence-aided caries detection on bitewing radiographs: a systematic review and meta-analysis. *Japanese Dental Science Review* 60 (2024), 128–136. https://doi.org/10.1016/j.jdsr.2024.02.001

[2] Ibrahim Sevki Bayrakdar, Kaan Orhan, Serdar Akarsu, Özer Çelik, Samet Atasoy, Adem Pekince, Yasin Yasa, Elif Bilgir, Hande Sağlam, Ahmet Faruk Aslan, and Alper Odabaş. 2022. Deep-learning approach for caries detection and segmentation on dental bitewing radiographs. *Oral Radiology* 38 (2022), 468–479. https://doi.org/10.1007/s11282-021-00577-9

[3] Anselmo Garcia Cantu, Sascha Gehrung, Joachim Krois, Akhilanand Chaurasia, Jesus Gomez Rossi, Robert Gaudin, Karim Elhennawy, and Falk Schwendicke. 2020. Detecting caries lesions of different radiographic extension on bitewings using deep learning. *Journal of Dentistry* 100 (2020), 103425. https://doi.org/10.1016/j.jdent.2020.103425

[4] Kristof Sebastian Hansson Horvath, Nils Roar Gjerdet, and Xie-Qi Shi. 2026. Advancements in caries diagnostics using bitewing radiography: a systematic review of deep learning approaches. *Caries Research* 60 (2026), 127–151. https://doi.org/10.1159/000546448

[5] Lukáš Kunt, Jan Kybic, Valéria Nagyová, and Antonín Tichý. 2023. Automatic caries detection in bitewing radiographs: Part I – deep learning. *Clinical Oral Investigations* 27 (2023), 7463–7471. https://doi.org/10.1007/s00784-023-05335-1

[6] Jan Kybic, Antonín Tichý, and Lukáš Kunt. 2023. Dental caries in bitewing radiographs. Mendeley Data, Version 1. https://doi.org/10.17632/4fbdxs7s7w.1

[7] Antonín Tichy, Lukáš Kunt, Valéria Nagyova, and Jan Kybic. 2024. Automatic caries detection in bitewing radiographs – Part II: experimental comparison. *Clinical Oral Investigations* 28 (2024), 133. https://doi.org/10.1007/s00784-024-05528-2

[8] Yifan Zhang, Fan Ye, Lingxiao Chen, Feng Xu, Xiaodiao Chen, Hongkun Wu, Mingguo Cao, Yunxiang Li, Yaqi Wang, and Xingru Huang. 2023. Children's dental panoramic radiographs dataset for caries segmentation and dental disease detection. *Scientific Data* 10 (2023), 380. https://doi.org/10.1038/s41597-023-02237-5 Dataset: doi:10.6084/m9.figshare.21621705 (CC0 1.0).

Supplementary Methods and Results

Supporting material for: Iterative Development of an AI Prototype for Dental Caries Detection and Localization with Informal Clinical Feedback

S1 Formative Versions 1 to 3

Versions 1–3 used a 100-image subset of the Children's Dental Panoramic Radiographs Dataset (Zhang et al., *Scientific Data*, 2023;10:380 [8]; article DOI: 10.1038/s41597-023-02237-5). The source dataset is hosted by Springer Nature Figshare (dataset DOI: 10.6084/m9.figshare.21621705) under the CC0 1.0 Universal Public Domain Dedication. Version 1 tested binary image classification; Version 2 expanded to six labels but several labels had very few positives; Version 3 attempted six-class localization and performed weakly. These experiments motivated narrowing to caries on permanent-dentition bitewings. Informal consultation with Dr. Mark Y. Wang, DMD, DMSc, a prosthodontist practising in Concord, Massachusetts, included review of early prototype demonstrations and discussion of missed or ambiguous findings; his input was design feedback rather than expert reference annotation, independent reader testing, or clinical validation. The author confirms that Dr. Wang consented to his name, credentials, and this description appearing in the manuscript and supplement. They do not enter the V4 versus V4.1 quantitative comparison. The V4 and V4.1 work used the Dental caries in bitewing radiographs, Version 1, contributed by Jan Kybic, Antonín Tichý, and Lukáš Kunt, Mendeley Data, DOI 10.17632/4fbdxs7s7w.1, under CC BY-NC 3.0. The release contains 100 bitewings with bounding-box annotations from eight dentists. Figure 3 in the main manuscript is an original data-only visualization from saved model output and does not reproduce radiograph pixels; the displayed coordinates and E0 reference status derive from this dataset.

S2 Reproducible Training Settings

Table S1: Training settings for each run.

Run	Model	Pixels	Epochs	Batch	LR	Optimizer	Pretraining
V4	YOLO11n	768	40	4	0.001	AdamW	COCO
V4.1 n768	YOLO11n	768	80	4	0.001	AdamW	COCO
V4.1 n1024	YOLO11n	1024	80	2	0.001	AdamW	COCO
V4.1 s768	YOLO11s	768	60	2	0.0005	AdamW	COCO
Factory default	YOLO11n	640	100	16	Auto	Auto to AdamW	COCO

V4 and V4.1 used seed 42, deterministic mode, no vertical flip, no mosaic, no mixup, no copy-paste, horizontal flip 0.5, rotation 5 degrees, translation 0.05, scale 0.20, and brightness augmentation 0.10. V4 used patience 12; V4.1 used patience 20. The fresh factory-default run used Ultralytics defaults, including seed 0, patience 100, mosaic 1.0, translation 0.10, scale 0.50, HSV augmentation, and automatic optimizer selection, which resolved to AdamW. Only the dataset path, CPU device, zero workers, and output path were specified. Software versions were Python 3.13.9, PyTorch 2.13.0, Ultralytics 8.4.128, NumPy 2.3.5, pandas 2.3.3, and scikit-learn 1.7.2.

S3 Validation Candidate Analysis

Table S2: Top ten validation candidates by lesion F1 (A, B, and C defined in the text below).

Rank	Candidate	F1	P	R	AP50
1	Fusion A + B (NMS .50)	0.695	0.647	0.750	0.651
2	Fusion A + B (NMS .30)	0.695	0.647	0.750	0.650
3	Fusion A + B (NMS .50 and nested .80)	0.695	0.647	0.750	0.650
4	A (NMS .30)	0.643	0.675	0.614	0.567
5	B (NMS .50 and nested .80)	0.636	0.636	0.636	0.606
6	A (NMS .50)	0.635	0.659	0.614	0.574
7	A (NMS .50 and nested .80)	0.635	0.659	0.614	0.563
8	B (NMS .50)	0.632	0.750	0.545	0.610
9	B (NMS .30)	0.632	0.750	0.545	0.609
10	C (NMS .50)	0.630	0.793	0.523	0.578

Display key: A is YOLO11n-768 with full-frame 1024-pixel inference; B is YOLO11n-1024 with horizontally flipped 1024-pixel inference; C is YOLO11n-768 with horizontally flipped 1024-pixel inference. Among 75 candidates, best F1 was 0.695 and median F1 was 0.556. Three candidates were within 0.05 F1 of the winner; all were final-suppression variants of the same A plus B fusion.

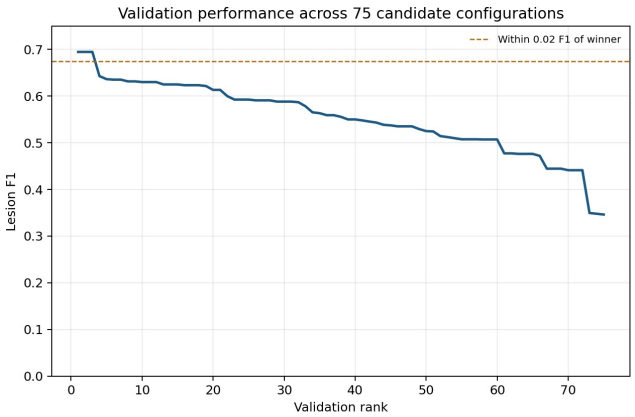


Figure S1: Full validation leaderboard.

S4 Multi-reader Results

The eight independent references are not interchangeable ground truths. At threshold 0.16, 31 predictions were unmatched to E0; 13 overlapped at least one other reader, 9 at least two, 5 at least three, and 3 at least four.

Table S3: V4.1 rescored against each of the eight readers at thresholds 0.16 and 0.37.

Reader	Boxes	P .16	R .16	F1 .16	P .37	R .37	F1 .37	AP50
E0	49	0.415	0.449	0.431	0.643	0.367	0.468	0.491
E1	85	0.302	0.188	0.232	0.500	0.165	0.248	0.138
E2	50	0.283	0.300	0.291	0.464	0.260	0.333	0.251
E3	49	0.358	0.388	0.373	0.536	0.306	0.390	0.279
E4	47	0.132	0.149	0.140	0.179	0.106	0.133	0.042
N1	81	0.566	0.370	0.448	0.786	0.272	0.404	0.388
N2	67	0.321	0.254	0.283	0.464	0.194	0.274	0.172
N3	63	0.434	0.365	0.397	0.607	0.270	0.374	0.386

S5 Threshold Results

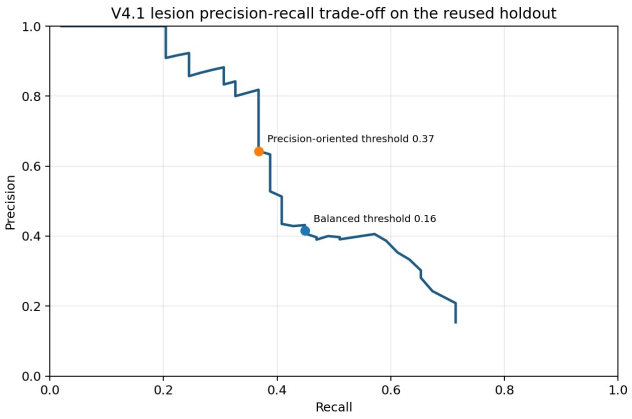


Figure S2: Holdout precision-recall trade-off against E0.

At 0.16: 22 TP, 31 FP, 27 FN, precision 0.415, recall 0.449, F1 0.431, and 1.55 FP/image. At 0.37: 18 TP, 10 FP, 31 FN, precision 0.643, recall 0.367, F1 0.468, and 0.50 FP/image. ROC AUC for the image-level maximum-score flag was 0.933; it is threshold-independent and based on 20 images.

S6 Conditional Three-fold Stress Test

The three folds covered the original 60 training images. At threshold 0.16, pooled out-of-fold detection yielded precision 0.636, recall 0.466, F1 0.538, 0.783 FP/image, AP50 0.510, and mAP50–95 0.188. This is not nested cross-validation because the recipe was informed by development on the same 60-image pool.

S7 Factory-default Comparator and Verification Status

The fresh factory-default YOLO11n run completed 100 epochs from the official generic checkpoint using unmodified training defaults. Validation selected threshold 0.35 before holdout evaluation. On the reused holdout it produced 15 TP, 5 FP, and 34 FN: precision 0.750, recall 0.306, F1 0.435, 0.25 FP/image, AP50 0.458, and mAP50–95 0.187. This single-run comparison is exploratory and does not correct the small reused holdout or missing patient identifiers. The available project package describes raw predictions, annotations, evaluation scripts, training logs, cross-validation assignments, configuration files, and checkpoint hashes. The reported values were recomputed through scripted checks prepared with AI assistance and reviewed by the author. This was not an independent human statistical audit or external validation.

Causal Faithfulness of SAE Feature Attributions in a Probe Pipeline

Samuel Xie
Seven Lakes High School
Katy, Texas, USA
samjh8xie@gmail.com

Noura Al-Moubayed
Durham University
Durham, United Kingdom
noura.al-moubayed@durham.ac.uk

Abstract

Sparse autoencoders (SAEs) decompose language model activations into interpretable features, but interpreting those features remains largely correlational. Attribution methods could estimate which features matter, yet it is unclear whether such estimates track causal effects on the model. We test this in a probe-mediated pipeline on GPT-2 Small and SST-2: we train linear and MLP probes on a layer-7 SAE, attribute probe outputs to features with SHapley Additive exPlanations (SHAP), Integrated Gradients (IG), and Gradient×Input (GA), and score each attribution against the effect of ablating or steering that feature on the language model's logit difference. Because this combines attribution error with any mismatch between what the probe encodes and what the model uses, our results characterize the full pipeline rather than attribution alone. The three methods nearly coincide (and are analytically identical for linear probes), but this agreement does not indicate causal validity. Importance faithfulness survives multiple-comparison correction but is modest and no better than a simple background-free baseline, while directional faithfulness is near zero and nonsignificant. In the linear arm, probe-side controls trace the directional failure to centering attributions on a background mean. Local per-example attribution outranks global averaging on importance only in the MLP arm and only when inactive features are included; among active features we find no advantage. In minimal pairs with known ground truth, attribution does not recover changed features beyond what feature activity alone provides. Importance and directional faithfulness are separable properties that require separate evaluation, and agreement among attribution methods should not substitute for causal validation. Code is available at https://github.com/froge79/shap-sae.

Keywords

Sparse autoencoders, Mechanistic interpretability, Feature attribution, Faithfulness evaluation, Causal interventions, Shapley values

ACM Reference Format:
Samuel Xie and Noura Al-Moubayed. 2026. Causal Faithfulness of SAE Feature Attributions in a Probe Pipeline. In *Proceedings of International Journal of Secondary Computing and Applications Research (IJSCAR VOL. 3, ISSUE 4)*. ACM, New York, NY, USA, 13 pages. https://doi.org/10.67149/yhjs2024.5/adr9jtxz

This paper is published under the Creative Commons Attribution-NonCommercial-NoDerivs 4.0 International (CC-BY-NC-ND 4.0) license. Authors reserve their rights to disseminate the work on their personal and corporate Web sites with the appropriate attribution.
IJSCAR VOL. 3, ISSUE 4
© 2026 The Author(s). Published by IJSCAR.

1 Introduction

Sparse autoencoders (SAEs) have emerged as a leading tool for decomposing the internal representations of language models into interpretable features. By training a sparse dictionary on residual stream activations, SAEs split superposed representations into nearly monosemantic directions that can be inspected individually [3, 7]. Recent work has scaled this approach to frontier models, recovering millions of interpretable features [4, 18].

Despite this progress, SAE feature interpretation remains largely correlational. The dominant pipeline identifies top-activating examples for each feature and assigns a semantic label manually or with automated interpretability models. This reveals the contexts in which a feature activates, but not which features causally drive a model's behavior. Attribution methods offer a more direct estimate of feature influence. Logit-weight analysis, gradient-based attribution, and attribution patching have all been applied to SAE features, and some have been validated against activation patching [9, 11, 17]. Yet post-hoc attribution is well known to be unreliable in general: explanations can diverge from the model's actual computation [14], no attribution method can satisfy all desirable properties at once [2], and apparent explanatory success can arise from methods that would also "explain" noise [12]. Why should attribution fare better on SAE features?

One reason it might is that SAE latents are sparse and, ideally, closer to disentangled than neurons or dense directions. This reduces, though does not eliminate, the feature entanglement that complicates attribution in dense bases. If attributions do not track causal effects even here, we should be cautious about trusting them elsewhere. A natural design for studying this is to attribute a trained probe's output to SAE features. A probe gives a fixed, interpretable target, and for linear probes Shapley values can be computed exactly. However, the probe is only a proxy for what the language model does, so a faithfulness score measures attribution error compounded with any mismatch between what the probe encodes and what the model uses. We treat this as a property of the design to be characterized rather than hidden.

As shown in Figure 1, we evaluate this probe-mediated pipeline on sentiment classification (SST-2) with GPT-2 Small and a 32,768-feature layer-7 SAE. We compare SHapley Additive exPlanations (SHAP) [10], Integrated Gradients (IG), Gradient×Input (GA), and probe-weight baselines across linear and MLP probes, scoring each against the effect of feature ablation and activation steering on the language model's logit difference, and validating on minimal sentiment-word edits with known ground truth.

Our contributions are:

Probe-mediated evaluation of attribution on SAE features.

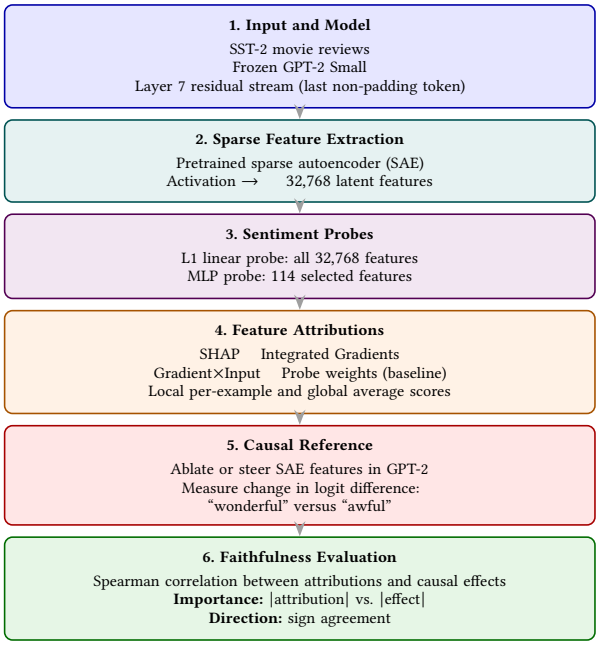


Figure 1: Pipeline for evaluating the causal faithfulness of SAE feature attributions.

We apply SHAP, Integrated Gradients, Gradient×Input, and probe-weight baselines to linear and MLP probes trained on SAE latents. For linear probes the first three are analytically identical, so we treat that arm as a control and use the MLP arm to test whether the methods diverge. They largely do not (Spearman $\rho = 0.98$–0.995 on global importance).

A causal evaluation of pipeline faithfulness. We score each attribution against the effect of feature ablation and activation steering on the language model's logit difference. This measures the composite of attribution error and probe–LM mismatch. Probe-side controls in the linear arm partly separate the two, and we discuss what the design can and cannot distinguish.

A decomposition of global directional failure. Importance faithfulness is significant but modest and no better than a background-free baseline, while directional faithfulness is near zero. In the linear arm, probe-side controls trace this to centering attributions on a background mean. We also compare local and global attribution: local attribution ranks features by effect better than the global score only when inactive features are included, where it benefits from knowing which features are active, and we find no advantage among active features.

A minimal-pair validation with known ground truth. Across four edit types with known sentiment changes, the language model's readout tracks the edit direction, but attribution does not identify the changed features beyond what feature activity alone provides: an attribution-free baseline (whether a feature is active at the edited last token) outperforms every method, and local and global SHAP fall below chance once distractors are activity-matched.

2 Related Work

2.1 Sparse Autoencoders for Interpretability.

Sparse autoencoders decompose residual stream activations into sparse, nearly monosemantic features, addressing the polysemanticity that makes individual neurons difficult to interpret [3, 7]. Recent work has scaled this approach to frontier models [4, 18]. Feature interpretation in this literature remains largely correlational, relying on top-activating examples or auto-interpretability pipelines rather than causal validation.

2.2 Attribution Methods.

Shapley-based attribution [10] and Integrated Gradients [16] are standard tools for identifying which inputs drive model predictions. Attribution patching [9, 17] applies gradient-based attribution to transformer internals as an efficient approximation to activation patching, and has been used to attribute effects to SAE features and validated against patching [11]. These works approximate patching effects inside the language model. We instead evaluate attribution methods applied through a trained probe, including the local-versus-global distinction and insertion versus removal. More broadly, post-hoc attribution is known to be unreliable [2, 12, 14]; we ask whether SAE features, being sparse, are a setting where it fares better.

2.3 Faithfulness Evaluation.

Faithfulness measures the degree to which an explanation reflects a model's true decision-making process [8]. Prior evaluation often relies on input perturbation metrics, which face a basic difficulty: removing features pushes inputs off the distribution the model was trained on, so a drop in performance can reflect distribution shift rather than the importance of the removed features. ROAR [6] addresses this by removing the features an attribution ranks as important and retraining the model, and finds that many widely used attribution methods perform no better than a random ranking. We instead score attributions against the effect of feature ablation and activation steering on the language model's output, a causal reference rather than a perturbation proxy. Like ROAR, we compare each attribution against trivial rankings (random selection, raw probe weights, and a background-free baseline) rather than reading its score in isolation. Unlike ROAR, we do not retrain, so our interventions may move activations off-distribution.

2.4 Probing.

Linear probes over internal representations are a standard tool for identifying whether a concept is encoded at a given layer [1]. High probe accuracy alone does not show that a representation encodes a property, because an expressive probe can learn the task itself. Prior work addresses this with control tasks, which assign labels to word types at random so that accuracy can only come from the probe's capacity to memorize, and define selectivity as the gap between accuracy on the real task and on the control [5]. More expressive probes can achieve high accuracy with low selectivity, and recommend simpler probes. We use probes over SAE activations to define a prediction function over the latent feature space, through which attribution methods are applied. We use an L1-penalized linear probe and a small MLP, but we do not run

a control task. Our checks on probe validity are instead indirect: a bag-of-words baseline matches probe accuracy (Appendix E.6), and the probe's own intervention response predicts the language model's response only moderately. Because the probe is a proxy for the language model, faithfulness here is measured for the probe-attribution pipeline as a whole.

3 Methodology

3.1 Experimental Setup

We evaluate on SST-2 [15], a binary sentiment dataset of movie-review sentences. We pool the full-sentence release, drop duplicates and any sentence contained in another, and re-split 70/15/15 with a fixed seed. A leakage check confirms that no validation or held-out text is a duplicate or sub-text of a training text (Appendix A.1, Table 5). The train set is used for fitting probes, attribution backgrounds and baselines, and calibrating the steering parameter α. The validation set is used for probe hyperparameters, the feature mask, and reported probe accuracy. The held-out split is used only for attribution and causal evaluation. Evaluation and background rows are nested prefixes of one seeded permutation (Appendix A.1).

We use frozen GPT-2 Small [13] with the `gpt2-small-resid-post-v5-32k` SAE from `sae-lens` [4], which decomposes layer 7 residual-stream activations into 32,768 latents. Activations are taken at the last non-padding token.

3.2 Probing Models

We attribute a trained probe's output to SAE features because a probe gives a fixed, interpretable target. The cost is that the probe is only a proxy for the language model. We train two probes, each evaluated with identical attribution and causal methods (Appendix A.2).

Linear probe. An L1-penalized logistic regression over all 32,768 features. Its gradient is constant, so IG, GA, and signed-mean SHAP coincide in closed form; this arm is therefore a control in which their agreement is expected by construction.

MLP probe. A one-hidden-layer MLP on the masked feature subset. It is only weakly nonlinear (a linear fit to its logit has $R^2 = 0.99$), so this arm is a mild departure from the linear control, not a strong test of nonlinear attribution. The mask depends on the validation set, a label-free but non-independent dependency that makes its validation accuracy mildly optimistic.

Feature mask. The mask keeps features with a non-zero linear-probe coefficient and an activation frequency above 5% on the validation split. It yields 114 features and restricts only the MLP input and the steering candidate pool.

3.3 Attribution Methods

Each method attributes the same scalar: the log-odds margin ($wx+b$) in the linear arm and the pre-sigmoid logit in the MLP arm. Scales are not comparable across arms.

Zero-cost baseline. Raw probe weight (linear) and first-layer saliency $\|W_1[i,:]\|_2$ (MLP) rank features with no input-specific computation, so any gain from attribution is measured against them.

SHAP. We use `shap.LinearExplainer` (exact) for the linear arm and `shap.DeepExplainer` with enforced additivity for the MLP arm, which approximates Shapley values. Local SHAP $\phi(x)$ uses a 100-row training background. Global importance is the mean absolute value $\overline{|\phi|}_j = \mathbb{E}_e|\phi_j(x_e)|$ over 1000 held-out rows against the full training set as background. We use mean $|\phi|$ because the signed mean $\Phi_j = \mathbb{E}_e[\phi_j(x_e)]$ cancels across examples and mainly reflects how evaluation rows differ from the background. Φ (100-row background) is kept wherever a direction is needed, since an unsigned score cannot supply one.

Integrated Gradients and Gradient×Input. IG integrates the gradient along a path from a baseline to the input (Appendix B); GA evaluates it at the input, $\mathrm{GA}_j = (x_j - \mathrm{baseline}_j)\,\partial f/\partial x_j(x)$. Both use the mean training row as baseline and are aggregated like SHAP, so methods differ in the attribution and not the aggregation. For a linear probe both reduce to $w_j(x_j - \mathrm{baseline}_j)$ and equal SHAP.

3.4 Feature-Ranking Agreement

We compute pairwise Spearman ρ between methods on features with non-zero probe scores. In the linear arm, agreement ($\rho = 1$) is a sanity check. MLP agreement is computed on |score| because raw saliency has no sign.

3.5 Causal Evaluation

Attribution scores the probe alone. To test whether scores track the language model, we intervene on SAE feature activations at the layer-7 residual stream and measure the change in logit difference. A faithful attribution should rank features by effect size and predict its sign. Because the attribution explains the probe while the intervention acts on the language model, this tests the probe–attribution–LM pipeline as a whole. For feature i with activation a_i:

Ablation:

$$\tilde{a}_i = 0, \quad \tilde{z} = \mathrm{Dec}(\tilde{\mathbf{a}}) \tag{1}$$

Steering (additive, signed):

$$\tilde{a}_i = a_i + \alpha_i, \quad \tilde{z} = \mathrm{Dec}(\tilde{\mathbf{a}}) \tag{2}$$

A no-edit encode–decode pass is the baseline, isolating SAE reconstruction error. Interventions apply at every token. Because attribution is computed at the last token, we also run a last-token-only condition that matches its scope; it is the main condition (Table 2), with all-token results in Appendix E.7.

Readout. The readout is the final-token logit difference,

$$\Delta = \ell(\text{“wonderful”}) - \ell(\text{“awful”}), \tag{3}$$

compared before and after intervention. We use $|\Delta|$ for importance and signed Δ for direction. Ablation effects are negated so positive ρ always means greater faithfulness.

Steering calibration. We set α on the training split as the smallest multiple of the median natural positive activation that gives a measurable but unsaturated shift on a layer-11 probe (Appendix C.1). Steering is scaled per feature to normalize for differing activation magnitudes, with unscaled α as a robustness check.

Feature selection. We evaluate three candidate sets: k random features, the top k by |SHAP|, and all 114 masked features (Appendix C.2). Inference comes from the full set, with one Spearman correlation per method and correlation type. Random draws are repeated over 10 seeds and summarized descriptively, since the draws overlap and are not independent replicates. Multiple comparisons are controlled with Benjamini–Hochberg (FDR 0.05) within a family fixed by rule: the tests of one run. We also report wider families and Benjamini–Yekutieli (Appendix C.3).

Reuse and background sensitivity. Candidate sets that do not depend on attribution are rescored from saved intervention effects rather than re-measured. To check that the background does not drive global scores, we also score Φ and $\overline{|\phi|}$ under backgrounds of different sizes and under bootstrapped 100-row backgrounds (Appendix C.5).

Probe-side controls. A low faithfulness ρ cannot by itself separate attribution error from probe–LM mismatch. For the linear probe, the probe's own response to each intervention is closed form: steering moves the margin by $P_j = w_j\alpha_j$, and ablation by $A_j = -\mathbb{E}_e[w_j x_{ej}]$. We therefore (i) score attributions against P_j and A_j, which isolates attribution error; (ii) correlate P_j with the measured logit-difference effect, which isolates how well the probe predicts the model; and (iii) add a background-free baseline $w_j\bar{x}_j$. We also test a class-matched background (Appendix C.6). These controls are unavailable for the MLP arm.

3.6 Local Faithfulness

Global attribution averages over examples, which could hide per-example quality. We test whether local attributions track each sentence's intervention effects better than the global score, for importance (does |attribution| rank features by effect size?) and direction (does the signed attribution match the sign of Δ?). For each sentence x, candidates are the union of the top-k features by $|\phi(x)|$ and by global $\overline{|\phi|}$, so both are scored on identical sets. We compute per-sentence Spearman ρ for each and report the mean and the fraction of sentences where local wins. The global directional score is signed Φ.

Confound. Globally top features are often inactive in a given sentence, and ablating an inactive feature changes the logit by exactly zero, so global attribution's lower correlation may reflect inactive features rather than worse ranking. Block [A] uses all candidates; Block [B] keeps only active features (activation > 0).

Significance tests. For both blocks and correlation types, we test the mean gap $\rho_{\text{local}} - \rho_{\text{global}}$ with a crossed bootstrap that resamples sentences and features independently ($B = 1000$). Sentences share most of their candidate features (mean pairwise Jaccard ≈ 0.7), so a sentence-only test understates uncertainty by roughly 4–9×. Two follow-ups quantify the confound: a regression of the per-example Block [A] gap on the active fraction of candidates, and a partial Spearman controlling continuously for activation magnitude.

3.7 Known-Ground-Truth Validation

Natural SST-2 text has no known feature importance, so a near-zero ρ could mean attribution failure or an insensitive harness. Minimal pairs resolve this because the source of the sentiment change is known. We use four edits crossing edit type with position relative to the final token: *removal* of a sentiment word, *substitution* with an opposite-polarity word, *insertion* of "it was ⟨word⟩ ." at the end of a neutral sentence, and *interior insertion* of the same clause mid-sentence.

We check three quantities in dependency order:

(1) **Readout validity:** whether the logit-difference sign tracks the known edit direction.
(2) **Feature recovery:** AUROC of each method at identifying features whose last-token activation changed (chance 0.5; Appendix D), against attribution-free baselines based on activation magnitude and whether a feature is active.
(3) **Causal recovery:** correlation between per-feature ablation effects and attribution scores on the edited sentence.

Two confounds are controlled. Using the pair's own activations as the SHAP background makes scores proportional to activation change, which tautologically inflates AUROC, and selecting distractors by top global SHAP biases it downward. We therefore use the training background and random distractors. Because random distractors are mostly inactive, we also report an activity-matched variant. Readout sign accuracy is reported with Wilson 95% intervals against a null-edit noise floor.

3.8 Lexical Simplicity Control

We test whether SST-2 is lexically simple enough that interaction-aware attribution cannot outperform simpler methods, using the full held-out split (Appendix E.6). We run three analyses: (1) **co-activation**, the number of active features per example and the overlap of the top-20 features; (2) **baselines**, comparing the SAE probe's held-out accuracy with logistic regressions on a sentiment lexicon, bag-of-words variants, and the dense layer-7 residual; and (3) **word-detector tests**, how well bag-of-words and lexicon features predict each top feature's activation and the probe logit. Reading rules were fixed in advance: the probe is no better than bag-of-words if the best validation-selected variant is within 0.03 accuracy, and a feature is bag-of-words-explained if held-out $R^2 \geq 0.25$.

4 Results

4.1 Probe Performance

Both probes perform well above chance and comparably to each other (Table 1); the MLP probe's validation score is mildly optimistic because of its pre-filtered inputs.

Table 1: Probe architectures and performance.

Arm	Input dim	Val. Accuracy	Val. AUROC
Linear	32,768	0.815	0.898
MLP	114 (filtered)	0.822	0.899

4.2 Attribution Method Agreement

With all three methods aggregated the same way, IG, GA and SHAP agree almost perfectly on global importance, while probe saliency

correlates only weakly with all three (Figure 2, Table 12). An earlier version aggregated IG and GA as signed means, which lowered IG–SHAP agreement; that gap reflected the aggregation, not the methods (Appendix C.5).

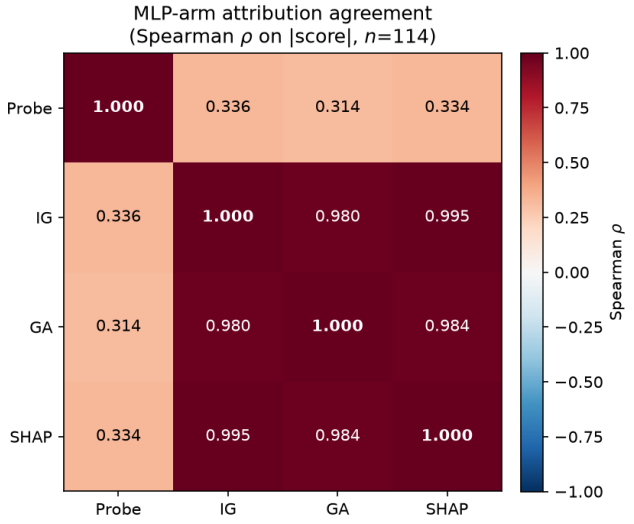


Figure 2: MLP-arm attribution agreement (Spearman ρ on |score|, $n = 114$ filtered features). IG, GA, and SHAP (all mean $|\cdot|$) form a near-identical cluster; saliency correlates only weakly with all three.

4.3 Causal Faithfulness

Table 2 reports global faithfulness over all 114 masked features under last-token steering, BH-corrected within arm. SHAP, IG and GA share the same aggregation, so they are identical in the linear arm and nearly so in the MLP arm.

Importance faithfulness is significant for all three methods in both arms but only small-to-moderate ($\rho \approx 0.42$–0.46, so each explains roughly a fifth of the variation in feature effects). The zero-cost baselines are lower, and in the MLP arm saliency is indistinguishable from zero.

Directional faithfulness is weaker: SHAP, IG and GA are near zero and nonsignificant in both arms, while probe weights alone are significant in the linear arm ($\rho_{\text{dir}} = 0.305$; 0.399 all-token). The weights beat SHAP by 0.41 (95% CI [0.09, 0.72], resampling features), which is notable because attribution adds instance-specific information to the same weights. A likely reason is that steering adds a fixed shift, so its effect follows the sign of the weight, whereas attribution multiplies each weight by a deviation ($x_j - \text{baseline}_j$) that can change sign. The probe-side controls below test this. Neither conclusion depends on how the test family is defined (Appendix C.3). All-token steering gives similar or higher importance faithfulness, and the same tests survive FDR correction (Appendix E.7). Matching the intervention scope to the attributed token position therefore does not improve measured faithfulness.

Global score and background. Scoring importance with the signed mean Φ instead of mean $|\phi|$ lowers importance faithfulness in both

Table 2: Last-token global faithfulness ($k = 114$, steering, all filtered features). Cells show Spearman ρ with BH-corrected q in parentheses.

Arm	Method	ρ_{imp} (q)	ρ_{dir} (q)
Linear	SHAP/IG/GA	+0.455 (<0.0001)*	−0.108 (0.254)
Linear	Probe	+0.270 (0.006)*	+0.305 (0.002)*
MLP	SHAP	+0.425 (<0.0001)*	−0.105 (0.464)
MLP	Probe saliency	+0.032 (0.859)	n/a
MLP	IG	+0.423 (<0.0001)*	−0.053 (0.804)
MLP	GA	+0.444 (<0.0001)*	−0.014 (0.884)

* Survives FDR 0.05. SHAP, IG and GA importance are mean absolute attribution against the full training set (identical in the linear arm); direction is the signed mean Φ (same for all three in the linear arm). Linear arm: 5/8 tests survive (SHAP, IG and GA counted separately; 3/4 distinct); MLP arm: 3/7. The same tests survive when BH is pooled over all 30 tests (both arms, both intervention scopes).

arms and scopes, and makes it depend strongly on the background draw. The same holds for IG and GA, so SHAP's earlier apparent advantage over them came from aggregation, not the method. The background does not change the directional result: Φ's directional ρ stays near zero under the full training set and under bootstrapped backgrounds (Table 9).

Probe-side controls. Because attributions explain the probe and interventions act on the language model, a low ρ could reflect either. For the linear probe we can score attributions against the probe's own closed-form response (Section 3.5, Table 11). Against the probe's steering effect, Φ is anti-correlated ($\rho_{\text{dir}} = -0.37$ with the full training set), while the background-free product $w_j \bar{x}_j$ reaches +0.95. The directional failure is therefore attribution error relative to the probe, produced by centring on a background mean, and does not require any probe–LM mismatch. A class-matched background leaves it unchanged, which excludes class balance as the cause.

The probe–LM link is itself only moderate: the probe's steering effect predicts the measured effect with $\rho = +0.31$ (95% CI [+0.10, +0.49]), so the language-model test has limited resolution. At that level the background-free product and raw weight predict direction better than Φ, and for importance the background-free product matches mean $|\phi|$, so SHAP importance does not beat a baseline with no background. Ablation gives the same picture (Appendix C.6). We therefore read the directional null as a property of Φ rather than evidence about the language model, and the importance results as showing that SHAP is no better than simple probe-based scores. The design cannot show that any probe-based score is unfaithful to the language model beyond the level set by the probe–LM link.

4.4 Local Faithfulness

Figure 3 and Table 3 compare local and global SHAP, with global scored by mean $|\phi|$. Our headline is Block B (active features only), since Block A is dominated by a tied block of inactive features that local $|\phi(x)|$ identifies by construction.

In Block B the importance gap is negative in both arms, nominally significant in the linear arm but not after FDR correction, and not significant in the MLP arm. No directional gap is significant in either block. In Block A, local SHAP ranks importance better than

the global score in the MLP arm, the one local result that survives correction. It is fragile: rescoring the original candidate sets with a 100-row background gives $p = 0.054$ (Appendix C.5). The active fraction of candidates explains only a modest share of the between-example variation in the Block A gap ($R^2 = 0.08$–0.19). Local SHAP therefore ranks features better than the global score only when inactive features are included; among active features we find no evidence that it is more faithful, and the point estimates favour the global score.

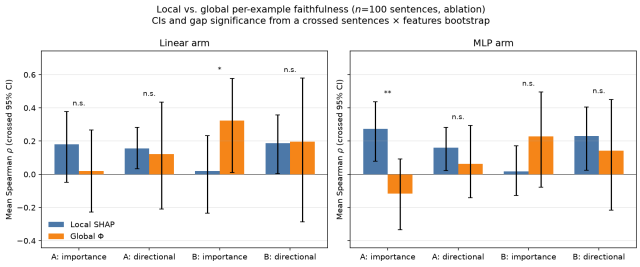


Figure 3: Local vs. global attribution, per-example faithfulness ($n = 100$ sentences, ablation). Bars are mean Spearman ρ across examples with 95% CIs from a crossed (sentences × features) bootstrap; asterisks mark the *uncorrected* crossed-bootstrap p for the local–global gap ($p < 10^{-2}$, * $p < 0.05$, n.s. otherwise); only the MLP-arm Block A importance gap survives FDR correction (Appendix C.3). Global = mean $|\phi|$. Block A = all top-20-local ∪ top-20-global candidates; Block B = active-only control.**

Why global directional attribution may fail. For a linear probe, $\Phi_j = w_j(\bar{x}_j^{\text{eval}} - \mathbb{E}_{\text{bg}}[x_j])$, so global attribution mostly reflects how the evaluation rows differ from the background rather than feature importance. Its magnitude tracks the size of this mean shift more closely than the size of the weights ($\rho = 0.85$ vs. 0.42), and only 40% of the top-20 $|\Phi|$ features have Φ matching the sign of their weight. Neither class balance nor a larger background removes the mismatch: against all training rows Φ matches the weight's sign for only 43% of the 114 masked features, and directional ρ over bootstrapped 100-row backgrounds is centred on zero. Because evaluation and background rows come from the same distribution, the shift has no reason to share the weight's sign. It is not pure noise, though: held-out means sit slightly below training means for most features, and the shift is negatively related to a feature's activation scale. Since the steering size α_j scales with that activation scale, this makes Φ anti-correlated with the probe's steering effect. This follows from defining Φ as a signed mean, not from the size of the background, and dropping the background ($w_j\bar{x}_j$) restores a positive directional ρ.

4.5 Known Ground-Truth Validation and Lexical Simplicity Control

Table 4 reports 100 minimal pairs for each of four edits. The readout tracks the known direction in all four, but is weakest for removal and interior insertion, where effects sit close to a null-edit noise floor (Appendix D).

Table 3: Local vs. global attribution, per-example faithfulness ($n = 100$ sentences, ablation). Mean ρ across examples; 95% CI and two-sided p (uncorrected; only MLP Block A importance survives BH, Appendix C.3) for the gap from a crossed (sentences × features) bootstrap. Global score: mean $|\phi|$ for importance, signed Φ for direction. Block A: all candidates; Block B: active features only.

Block, kind	Local	Global	Gap	95% CI	p	Win frac.
Linear arm						
A, importance	+0.181	+0.020	+0.161	[−0.105, +0.445]	0.206	0.760
A, directional	+0.157	+0.123	+0.034	[−0.356, +0.442]	0.852	0.560
B, importance	+0.019	+0.324	−0.305	[−0.573, −0.020]	0.036	0.137
B, directional	+0.188	+0.197	−0.009	[−0.514, +0.560]	0.938	0.505
MLP arm						
A, importance	+0.273	−0.115	+0.389	[+0.109, +0.665]	0.002	0.970
A, directional	+0.161	+0.062	+0.098	[−0.225, +0.370]	0.532	0.610
B, importance	+0.017	+0.229	−0.213	[−0.534, +0.138]	0.290	0.240
B, directional	+0.230	+0.142	+0.088	[−0.383, +0.565]	0.756	0.635

Against random distractors, local SHAP is modestly above chance in three of four edits and global SHAP is near chance. Local SHAP's apparent advantage on insertion is a position effect: appending the clause beats placing it mid-sentence, a difference that holds across seeds. Feature recovery is also not shown to exceed what activity alone gives: the attribution-free "active at the edited last token" baseline beats every method, and once distractors are activity-matched, both SHAP variants fall below chance in every seed.

Causal recovery is more informative. It is positive for local SHAP in all four edits and for global SHAP in most, but local SHAP's edge over global is absent or reversed except for appended insertion. We read this as weak evidence that |attribution| tracks ablation effect, not that attribution identifies the changed features.

A hand-built sentiment lexicon is a weak baseline, but a full-vocabulary bag-of-words logistic regression is statistically indistinguishable from the SAE probe (Appendix E.6). SST-2 sentiment is therefore largely recoverable from word presence, and we do not claim the probe carries signal beyond it. The probe does trail a logistic regression on the dense layer-7 residual. The lexicon explains almost none of any top feature's variance, so the earlier lexicon-correlation test was uninformative; full-vocabulary bag-of-words explains more, so top features are neither simple lexicon detectors nor free of word identity. Because the task is largely lexical, it gives interaction-aware attribution little room to differ from simpler methods, which limits what these experiments can show.

5 Discussion

Agreement among methods is not evidence of faithfulness. SHAP, Integrated Gradients, and Gradient×Input form a near-identical cluster once they are aggregated the same way (MLP arm: $\rho = 0.98$–0.995; Figure 2). In the linear arm they are analytically identical, so agreement there is guaranteed by construction. This cluster is only modestly related to causal effects. Importance faithfulness is statistically reliable ($\rho_{\text{imp}} = 0.42$–0.46, surviving every family-wise correction we tried) but explains only about 18–21% of the variance

Table 4: Minimal-pair ground-truth validation (seed 42, $n = 100$ pairs per edit, linear arm). Sign acc. is readout validity (Wilson 95% CI); AUROC is feature recovery (chance = 0.5) with random or activity-matched distractors (global SHAP is mean $|\phi|$ against the full training set, which equals mean-$|\cdot|$ IG and GA for the linear probe); ρ is causal recovery (random distractors). The baseline row scores whether a feature is active at the edited last token, with no attribution.

Edit	Method	Sign acc.	AUROC rand.	AUROC act.-matched	ρ
Remove	Local SHAP	0.76 [0.67, 0.83]	0.553	0.420	+0.186
	Global SHAP		0.482	0.442	+0.057
	Probe weights		0.395	0.508	−0.148
	Active baseline		0.851	–	–
Substitute	Local SHAP	0.96 [0.90, 0.98]	0.565	0.427	+0.355
	Global SHAP		0.512	0.421	+0.276
	Probe weights		0.466	0.522	+0.013
	Active baseline		0.803	–	–
Insert (final)	Local SHAP	0.91 [0.84, 0.95]	0.622	0.486	+0.219
	Global SHAP		0.472	0.370	−0.034
	Probe weights		0.406	0.490	−0.200
	Active baseline		0.822	–	–
Insert (interior)	Local SHAP	0.79 [0.70, 0.86]	0.492	0.379	+0.194
	Global SHAP		0.425	0.405	+0.054
	Probe weights		0.410	0.475	−0.151
	Active baseline		0.789	–	–

in feature effects, and directional faithfulness is near zero and non-significant for all three methods. Practitioners should therefore not treat agreement among attribution methods as evidence that they identify causally important SAE features.

Attribution adds little over simple probe-based scores. The probe-side controls show that SHAP importance does not beat a baseline with no background: the product $w_j \bar{x}_j$ reaches $\rho_{\text{imp}} = 0.47$, against 0.46 for mean $|\phi|$. For direction, the raw probe weight ($\rho = +0.31$) and $w_j \bar{x}_j$ (+0.35) both outperform the signed mean Φ (−0.11). In the linear arm, Φ is anti-correlated with the probe's own steering response ($\rho = -0.28$ to -0.37), whereas $w_j \bar{x}_j$ reaches +0.95. The directional failure is therefore attribution error relative to the probe, produced by centering on a background mean, and does not require any probe–LM mismatch. These results concern signed-mean aggregation of background-centered attributions. They do not show that all attribution designs fail.

Local versus global attribution depends on the regime. Local SHAP ranks features by effect better than the global score only in one setting: the MLP arm, with inactive features included (Block A, gap +0.389, $q = 0.008$). That result is fragile, since rescoring the original candidate sets gives +0.222 ($p = 0.054$), and it largely reflects local attribution knowing which features are active, because ablating an inactive feature moves the logit by exactly zero. Among active features (Block B), the point estimates favor the global score (−0.305 linear, −0.213 MLP), though only the linear gap is nominally significant and it does not survive correction. No directional gap between local and global attribution is significant in either block. We therefore find no evidence that local attribution is more faithful than global attribution once feature activity is controlled, and we do not recommend one over the other on the basis of these data.

Attribution does not recover changed features beyond feature activity. In minimal pairs, the language model's readout tracks the known edit direction (sign accuracy 0.76–0.96), so the harness is sensitive. Attribution nonetheless does not identify the changed features. Local SHAP is modestly above chance against random distractors in three of four edits (AUROC 0.55–0.62; 0.49 for interior insertion), but an attribution-free baseline, whether a feature is active at the edited last token, reaches 0.79–0.85. Once distractors are activity-matched, local and global SHAP fall below chance (0.37–0.49) in every seed. The one consistent pattern is positional: appended insertion (AUROC 0.622) outperforms interior insertion (0.492), positive in 4 of 4 seeds, which suggests attribution read at the final token is more informative about edits adjacent to it. Causal recovery among active candidates is positive for both local and global SHAP ($\rho = 0.11$–0.38), but local SHAP's edge is absent or reversed except for appended insertion. We read this as weak evidence that |attribution| tracks ablation effect, not that attribution identifies the changed features.

The task is largely lexical. A full-vocabulary bag-of-words regression reaches 0.794–0.806 held-out accuracy, within 0.03 of the SAE probe (0.810; Table 17), and bag-of-words explains 0.39 of the probe logit's variance. SST-2 sentiment is thus largely recoverable from word presence, which leaves interaction-aware attribution little room to differ from simpler methods. This limits what these experiments can show, and it means the near-identical behavior of SHAP, IG, and GA may be partly a property of the task.

5.1 Limitations

Probe–LM confound. Attributions explain a trained probe over SAE activations, while faithfulness is scored against the language model's logit difference. Our estimates combine attribution error with any mismatch between what the probe encodes and what the model uses. The probe-side controls separate the two only in the linear arm, and the probe's effect predicts the measured effect at only $\rho \approx 0.3$–0.4, so the language-model test has limited resolution.

Method equivalence in the linear arm. SHAP, IG, and GA are analytically identical for linear probes. The MLP arm is only weakly nonlinear (linear fit to its logit $R^2 = 0.99$) and uses approximate DeepSHAP, so it adds little beyond the linear control.

Limited scope. All results come from one model (GPT-2 Small), one SAE and layer (layer 7), and one binary sentiment task (SST-2). We make no claim that the patterns, including the local–global comparison and the position effect in minimal pairs, hold elsewhere.

Statistical power and post hoc choices. Most per-seed $k = 20$ associations do not survive pooled correction, so inference rests on the $k = 114$ run. The mean-$|\phi|$ importance scoring was adopted after the final results were seen, and the wider test families were added post hoc. Significant effects should be read as suggestive.

Untested explanation. Our account of the global directional failure is supported by the linear-arm probe-side controls, but we did not verify it in the MLP arm, which has no closed-form probe response.

Steering as causal reference. Additive steering and ablation in SAE space may move activations off-distribution, and the steering strength α was calibrated against a different readout (a layer-11 probe) than the one used for evaluation. Intervention effects are a causal reference, not ground truth. Only the minimal-pair edits have known ground truth, and they cover a narrow class of sentiment-word changes.

5.2 Future Directions

The most informative next step is to separate attribution error from probe–model mismatch, either by attributing directly to the language model's logit difference or by intervening on the probe's output and comparing against the language model's response. Attribution designs that avoid centering on a background mean, or that are scored against a probe's own response in the nonlinear case, would test whether the directional failure is specific to signed-mean aggregation. Because the decomposition-and-control methodology is not specific to SHAP, it could be applied to other attribution methods, models, layers, SAE architectures, and tasks that are less lexically solvable than SST-2, with larger feature samples to improve power. Finally, a broader set of edits with known ground truth would test whether the position effect in minimal pairs is a general property of attribution or specific to this setup.

6 Conclusion

We evaluated SHAP, Integrated Gradients, and Gradient×Input on SAE latent features in a probe-mediated pipeline, comparing their attributions with the effects of feature ablation and steering on the language model's logit difference. In our GPT-2 Small, layer-7, SST-2 setting, importance faithfulness was statistically significant but modest and no better than a simple background-free baseline, while directional faithfulness was near zero. Probe-side controls in the linear arm traced the directional failure to centering attributions on a background mean. Although the three methods agreed closely with one another, this agreement did not imply correspondence with causal effects. Local attribution outperformed global averaging on importance only in the MLP arm when inactive features were included; among active features, we found no advantage. In minimal pairs with known ground truth, attribution did not identify changed features beyond what feature activity alone provided. These findings show that, in this experimental setting, agreement between attribution methods is insufficient evidence of causal faithfulness, and that importance and directional faithfulness can yield different assessments of an attribution method. Practitioners should therefore evaluate these properties separately and validate probe-mediated attributions against interventions before using them to make claims about model behavior. Because our design combines attribution error with potential probe–model mismatch, future work should separate these sources of error—for example, by comparing attributions with interventions on the probe itself—and test the robustness of these results across models, layers, and tasks.

7 Use of AI Tools

The authors used Claude Sonnet 5 and Opus 5 to assist with generating portions of the experimental code and with editing the paper's prose. The authors reviewed, tested, and verified all AI-assisted code and text, and take full responsibility for the experiments, results, and conclusions reported here.

References

[1] Guillaume Alain and Yoshua Bengio. 2016. Understanding intermediate layers using linear classifier probes. *arXiv preprint arXiv:1610.01644* (2016).

[2] Blair Bilodeau, Natasha Jaques, Pang Wei Koh, and Been Kim. 2024. Impossibility theorems for feature attribution. *Proceedings of the National Academy of Sciences* 121, 2 (2024), e2304406120.

[3] Trenton Bricken, Adly Templeton, Joshua Batson, Brian Chen, Adam Jermyn, Tom Conerly, Nick Turner, Cem Anil, Carson Denison, Amanda Askell, Robert Lasenby, Yifan Wu, Shauna Kravec, Nicholas Schiefer, Tim Maxwell, Nicholas Joseph, Zac Hatfield-Dodds, Alex Tamkin, Karina Nguyen, Brayden McLean, Josiah E Burke, Tristan Hume, Shan Carter, Tom Henighan, and Christopher Olah. 2023. Towards Monosemanticity: Decomposing Language Models With Dictionary Learning. *Transformer Circuits Thread* (2023). https://transformer-circuits.pub/2023/monosemantic-features/index.html.

[4] Leo Gao, Tom Dupre la Tour, Henk Tillman, Gabriel Goh, Rajan Troll, Alec Radford, Ilya Sutskever, Jan Leike, and Jeffrey Wu. 2025. Scaling and evaluating sparse autoencoders. In *International Conference on Learning Representations*, Vol. 2025. 26721–26754.

[5] John Hewitt and Percy Liang. 2019. Designing and interpreting probes with control tasks. In *Proceedings of the 2019 conference on empirical methods in natural language processing and the 9th international joint conference on natural language processing (emnlp-ijcnlp)*. 2733–2743.

[6] Sara Hooker, Dumitru Erhan, Pieter-Jan Kindermans, and Been Kim. 2019. A benchmark for interpretability methods in deep neural networks. *Advances in neural information processing systems* 32 (2019).

[7] Robert Huben, Hoagy Cunningham, Logan Smith, Aidan Ewart, and Lee Sharkey. 2024. Sparse autoencoders find highly interpretable features in language models. In *International Conference on Learning Representations*, Vol. 2024. 7827–7845.

[8] Alon Jacovi and Yoav Goldberg. 2020. Towards faithfully interpretable NLP systems: How should we define and evaluate faithfulness?. In *Proceedings of the 58th annual meeting of the association for computational linguistics*. 4198–4205.

[9] János Kramár, Tom Lieberum, Rohin Shah, and Neel Nanda. 2024. Atp*: An efficient and scalable method for localizing llm behaviour to components. *arXiv preprint arXiv:2403.00745* (2024).

[10] Scott M Lundberg and Su-In Lee. 2017. A unified approach to interpreting model predictions. *Advances in neural information processing systems* 30 (2017).

[11] Samuel Marks, Can Rager, Eric Michaud, Yonatan Belinkov, David Bau, and Aaron Mueller. 2025. Sparse feature circuits: Discovering and editing interpretable causal graphs in language models. In *International Conference on Learning Representations*, Vol. 2025. 23888–23923.

[12] Maxime Méloux, Giada Dirupo, François Portet, and Maxime Peyrard. 2025. The dead salmons of ai interpretability. *arXiv preprint arXiv:2512.18792* (2025).

[13] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al. 2019. Language models are unsupervised multitask learners. *OpenAI blog* 1, 8 (2019), 9.

[14] Cynthia Rudin. 2019. Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead. *Nature machine intelligence* 1, 5 (2019), 206–215.

[15] Richard Socher, Alex Perelygin, Jean Wu, Jason Chuang, Christopher D Manning, Andrew Y Ng, and Christopher Potts. 2013. Recursive deep models for semantic compositionality over a sentiment treebank. In *Proceedings of the 2013 conference on empirical methods in natural language processing*. 1631–1642.

[16] Mukund Sundararajan, Ankur Taly, and Qiqi Yan. 2017. Axiomatic attribution for deep networks. In *International conference on machine learning*. PMLR, 3319–3328.

[17] Aaquib Syed, Can Rager, and Arthur Conmy. 2024. Attribution patching outperforms automated circuit discovery. In *Proceedings of the 7th BlackboxNLP Workshop: Analyzing and Interpreting Neural Networks for NLP*. 407–416.

[18] Adly Templeton, Tom Conerly, Jonathan Marcus, Jack Lindsey, Trenton Bricken, Brian Chen, Adam Pearce, Craig Citro, Emmanuel Ameisen, Andy Jones, Hoagy Cunningham, Nicholas L Turner, Callum McDougall, Monte MacDiarmid, C. Daniel Freeman, Theodore R. Sumers, Edward Rees, Joshua Batson, Adam Jermyn, Shan Carter, Chris Olah, and Tom Henighan. 2024. Scaling Monosemanticity: Extracting Interpretable Features from Claude 3 Sonnet. *Transformer Circuits Thread* (2024). https://transformer-circuits.pub/2024/scaling-monosemanticity/index.html

A Experimental Setup Details

A.1 Task and Dataset

The HuggingFace `stanfordnlp/sst2` train set consists of phrases, and a random split of it leaked sub-phrases across splits (83.8% of a held-out sample were contiguous sub-phrases of a train phrase). We therefore pool the sentence-level SST-2 release (6,920 / 872 / 1,821 sentences), drop exact duplicates, label-conflicted texts and any sentence contained in another, and re-split 70/15/15 with a fixed seed (Table 5). The split script aborts if any validation or held-out text is a duplicate or sub-text of a train text; the final split has none.

Table 5: Data split sizes and purposes.

Split	n
Probe train	6,696
Probe val	1,435
Held-out ("shap")	1,436

Canonical row sampling. All evaluation and background rows are *prefixes* of one seeded permutation (seed= 42), not independent draws. The 100-row local sample is the first 100 rows of the 1000-row global sample, so local and global results join by row position.

A.2 Model and Probes

GPT-2 Small and the layer-7 `resid_post` SAE are frozen throughout. SAE decoder rows are approximately unit norm (median 0.9996), so we apply no decoder-norm correction to steering vectors.

Linear probe. `sklearn` `LogisticRegression` with an L1 penalty ($C = 1$, `liblinear`, fixed `random_state`) over the full 32,768-d activation. It keeps ~600 non-zero coefficients, which seed the 114-feature mask; the mask is a property of the reported run, not a fixed constant.

MLP probe. `sklearn` `MLPClassifier` with one hidden layer of width 32, ReLU activation, Adam, and L2 regularization, trained on the 114 masked features only.

A.3 Glossary

Table 6 defines the notation and shorthand used in the results.

B Attribution Method Details

Global SHAP.. Global importance is $\overline{|\phi|}_j = \mathbb{E}_e|\phi_j(x_e)|$ over the 1000-row held-out sample, with every training row as the background (6,696 rows). For the linear probe $\phi_j(x) = w_j(x_j - \mu_j)$ with μ the background mean, so $\overline{|\phi|}_j = |w_j|\,\mathbb{E}_e|x_{e,j} - \mu_j|$ in closed form (checked against `LinearExplainer`). For the MLP we run DeepSHAP in chunks of 125 evaluation rows with additivity enforced. The signed mean $\Phi_j = \mathbb{E}_e[\phi_j(x_e)]$ (100-row background) is used wherever a direction is needed. Signed values cancel across examples, so Φ is not an importance score (Appendix C.5).

Table 6: Glossary of notation and terms used in the results.

Term	Meaning
$\phi_j(x)$	Local SHAP value of feature j for example x: how much the feature moves the probe output away from its expectation over a background set.
$\lvert\phi\rvert_j$	Global importance: $\mathbb{E}_e\lvert\phi_j(x_e)\rvert$, the mean absolute SHAP value over evaluation rows. Unsigned, so it supplies importance but no direction.
Φ_j	Signed global attribution: $\mathbb{E}_e[\phi_j(x_e)]$, the mean SHAP value over evaluation rows. Used wherever a direction is needed.
Imp. (importance)	Does |attribution| rank features by effect size $\lvert\Delta\rvert$? Scored with Spearman ρ_{imp}.
Dir. (directional)	Does the signed attribution match the sign of the effect Δ? Scored with Spearman ρ_{dir}.
Δ	Change in the language model's logit difference, ℓ("wonderful") $-$ ℓ("awful"), after ablating or steering a feature.
Block A	Local-versus-global comparison over all candidate features (top-20 local $\cup$ top-20 global). Dominated by inactive features, whose ablation effect is exactly zero.
Block B	Same comparison restricted to active features (activation > 0). Our headline local-versus-global test, since it removes the activity confound.

Integrated Gradients.

$$\mathrm{IG}_j = (x_j - b_j) \cdot \int_0^1 \frac{\partial f}{\partial x_j}\big(b + \alpha(x - b)\big)\, d\alpha, \tag{4}$$

approximated in the MLP arm with a 50-step trapezoidal Riemann sum. IG and GA share the baseline b, the *mean* of the train background rather than a zero vector (100 rows for signed means; all 6,696 rows for importance, matching SHAP). SAE features are very sparse, so a zero baseline would make GA degenerate to 0 whenever $x_j = 0$. For the linear probe, IG, GA, and Φ all reduce to $w_j \cdot \mathbb{E}_e[x_{e,j} - \mu_j]$, which we verify numerically (Appendix E.1).

Global importance for IG and GA is aggregated like SHAP, as

$$\mathbb{E}_e\big|\mathrm{IG}_j(x_e)\big| \quad \text{and} \quad \mathbb{E}_e\big|\mathrm{GA}_j(x_e)\big|. \tag{5}$$

Scoring them as $|\mathbb{E}_e[\cdot]|$ while SHAP used $\mathbb{E}_e|\cdot|$ would confound method with aggregation.

C Causal Evaluation Details

C.1 Steering-Strength (α) Calibration

We sweep $\{0.5, 1, 2, 4\}\times$ the median natural positive-activation scale over three top-|SHAP| pilot features on 200 train examples, reading out a layer-11 residual probe's P(positive) rather than the evaluation logit difference. α is a rough "measurable, unsaturated" magnitude, not a hyperparameter tuned against the evaluation metric. We select the smallest α with mean$|\Delta P| \geq 0.02$ and newly saturated fraction ≤ 0.25, giving $\alpha = 0.4936$ (Table 7). This is the smallest grid value, so the criterion does not bind. The mean signed shift is negative at every α under this readout but positive under the evaluation logit-difference readout on the same pilot, so the sign reflects the readout mismatch and not the features' effect on the language model.

Table 7: Raw α-calibration pilot sweep (probe-readout, 3 pilot features, 200 train examples).

α	mean$\lvert\Delta P\rvert$	mean signed ΔP	newly saturated frac.
0.4936	0.0490	−0.0301	0.028
0.9871	0.0961	−0.0571	0.085
1.9743	0.1791	−0.0981	0.182
3.9485	0.2969	−0.1422	0.368

Per-feature scaling. In the primary **scaled** mode, $\alpha_i = \alpha \cdot \text{scale}_i / \text{median}(\text{scale})$, where scale_i is feature i's mean positive train activation. This equalizes the relative nudge across features. The **constant** mode (flat α, robustness check) confounds the |attribution|-vs-|Δ| correlation, since |Δ| then partly reflects how hard a feature was pushed relative to its own scale.

C.2 Candidate Selection

`top` selection takes the top-k by |SHAP| and scores all methods on that set. This range-restricts only SHAP's scored variance, which mechanically deflates SHAP's ρ relative to the other methods. `all` scores every masked feature ($n = 114$). At $n = 20$ the confidence interval on Spearman ρ spans most of $[-1, 1]$, so a single $k = 20$ random draw cannot distinguish a null from a moderate effect.

Provenance of the `all` *analysis.* The `all` selection was introduced after an earlier analysis on a phrase-level split (later found to leak) whose single $k = 20$ draw looked null for several tests, which is outcome-dependent. We have since rebuilt the split at sentence level and regenerated every number here. The `all` selection, the BH family and its rule were fixed before this final analysis was run, so the design was chosen on the earlier data and not on the final results. The analysis is not fully confirmatory, however: the global SHAP importance score was changed from the signed mean Φ to mean $|\phi|$ (with IG and GA aggregated to match) after the final results had been seen, in response to review. Both versions are reported (Appendix C.5); the directional results, which use Φ, are unchanged, but the mean-$|\phi|$ importance results should be read as post hoc. All $k = 20$ results are reported (Table 16, and the seed sweep below). We treat `all` as the inferential run because it uses the whole filtered pool with no sampling; the final $k = 20$ draw agrees with it.

C.3 Multiple-Comparison Correction

Family rule. We apply Benjamini–Hochberg (BH, FDR 0.05) with the following primary family: the tests of one `all` run (one arm, one intervention scope; 8 linear, 7 MLP), and, for local faithfulness, the four crossed-bootstrap tests (Block A/B × importance/directional) of one local run. The per-run rule was specified before the final analysis, but after the initial analysis had been examined. The wider families below are post hoc; we report them to show whether any verdict depends on the choice of family.

Duplicate tests. In the linear arm IG and GA have identical (ρ, p) by construction, as do their directional and importance tests, so the 8 linear tests per run are 4 distinct ones. Counting them separately inflates survivors and, when pooled, BH power, so the sensitivity analysis counts identical tests once (22 distinct global tests; raw 16/30).

Sensitivity to the family. Counts are recomputed from the per-test p-values of every inferential test in the paper under each family definition. The paper-wide family has 30 tests: the 22 distinct global tests and the 8 local crossed-bootstrap tests. The sign tests, minimal-pair summaries, seed sweep and language-model diagnostic carry no independent inferential claim and are excluded.

Table 8: Tests surviving FDR 0.05 under alternative family definitions. BY is Benjamini–Yekutieli (arbitrary dependence).

Family	BH	BY
Global, per run (primary)	12/22	12/22
Global, pooled over arms, scopes, or both	12/22	12/22
Global, raw (SHAP/IG/GA counted separately)	16/30	16/30
Paper-wide, per run / per local source (primary)	13/30	13/30
Paper-wide, global and local pooled separately	13/30	13/30
Paper-wide, one family of all inferential tests	13/30	13/30

No verdict depends on the family: the same 12 global tests and one local test survive under BH and BY in every definition. The local test is the MLP-arm Block A importance gap. Importance faithfulness of SHAP, IG and GA survives all families (except MLP probe saliency), no directional test of SHAP, IG or GA has $q < 0.25$ in any, and no Block B or linear-arm local test survives. A test that fails the FDR criterion is not evidence that its effect is zero.

Dependence. The two scopes share features and eval rows, and the local runs share sentences, so the tests are not independent. BH assumes positive dependence, which we do not verify; BY does not.

Seed sweep. The multi-seed sweep uses a different convention. Its 10 seeds × 8 (linear) or 7 (MLP) per-seed tests each use $n = 20$ features that overlap across seeds, so the pooled BH count over them (39 distinct linear tests, 70 MLP) is descriptive only.

C.4 Local Faithfulness

Local analyses use $k_{\text{local}} = k_{\text{global}} = 20$ and $n_{\text{examples}} = 100$ in both arms. A separate script (`local_shap_faithfulness`) re-implements the MLP-arm analysis against the language model on the same sentences and reproduces the means to within 0.001; it is a replication check, not an independent test.

C.5 Global Baseline Sensitivity

Table 9 compares mean $|\phi|$ with the signed mean Φ and shows how both depend on the background; Table 10 shows the effect of the choice on the local-versus-global gap. Global-faithfulness values rescore the saved intervention effects, so they are exact. IG and GA follow SHAP: their signed means give the Φ columns (lower in the MLP arm, 0.32–0.44 last-token), and mean $|\cdot|$ gives the mean-$|\phi|$ columns.

The Block A MLP gap depends on the candidate sets. The table above rebuilds them from the mean-$|\phi|$ top-20 and reruns the interventions. Rescoring the original candidate sets (built from signed

Table 9: Global faithfulness (Spearman ρ, $k = 114$ steering) for the signed mean Φ and mean $|\phi|$ under different backgrounds. "Boot." gives the 95% range over bootstrapped 100-row backgrounds (500 draws linear, 40 MLP). The directional score for mean $|\phi|$ is undefined (unsigned).

Arm, scope	Φ imp.	Φ dir.	$\overline{\lvert\phi\rvert}$ imp., 100-row	$\overline{\lvert\phi\rvert}$ imp., full	Φ imp., boot.	$\overline{\lvert\phi\rvert}$ imp., boot.
Linear, all-token	+0.377	−0.097	+0.437	+0.430	[+0.20, +0.42]	[+0.41, +0.45]
Linear, last-token	+0.303	−0.108	+0.461	+0.455	[+0.21, +0.44]	[+0.43, +0.47]
MLP, all-token	+0.347	−0.091	+0.487	+0.476	[+0.23, +0.43]	[+0.46, +0.50]
MLP, last-token	+0.233	−0.105	+0.452	+0.425	[+0.13, +0.39]	[+0.41, +0.45]
Directional ρ of Φ under other backgrounds						
Linear, all-token	full training set −0.058; bootstrap −0.03 [−0.21, +0.17]					
Linear, last-token	full training set −0.153; bootstrap −0.05 [−0.24, +0.17]					
MLP, all-token	3,000 rows −0.101; bootstrap −0.04 [−0.18, +0.12]					
MLP, last-token	3,000 rows −0.123; bootstrap −0.05 [−0.21, +0.11]					

Table 10: Local-minus-global importance gap ($n = 100$ sentences, ablation) with the earlier signed-Φ global baseline and with mean $|\phi|$ (used in the paper; candidate sets rebuilt from its top-20 and the interventions rerun). Crossed-bootstrap p, uncorrected.

Arm	Block	Signed Φ gap (p)	Mean $\lvert\phi\rvert$ gap (p)
Linear	A, importance	+0.294 (0.064)	+0.161 (0.206)
Linear	B, importance	−0.305 (0.104)	−0.305 (0.036)
MLP	A, importance	+0.396 (0.024)	+0.389 (0.002)
MLP	B, importance	−0.310 (0.216)	−0.213 (0.290)

Φ) with mean $|\phi|$ and a 100-row background instead gives +0.144 ($p = 0.132$, linear) and +0.222 ($p = 0.054$, MLP) in Block A, so the one local result that survives correction is sensitive to this choice.

C.6 Probe-Side Controls

Linear arm, $k = 114$ steering. $P_j = w_j \alpha_j$ is the probe's own response to steering feature j (exact under last-token steering, direct-only under all-token), and D_j is the measured logit-difference effect. "Class-matched" uses a background whose class mix equals the evaluation rows (51.5% positive). $w\bar{x}$ is $\mathbb{E}_e[w_j x_{ej}]$ with no background. Intervals resample the 114 features and are descriptive.

Table 11: Spearman ρ of each score against the probe's own steering effect P (no language model) and against the measured effect D, last- and all-token. Dir. = signed, imp. = magnitude; the unsigned mean $|\phi|$ has no directional score.

	vs. P		vs. D, last-token		vs. D, all-token	
Score	dir.	imp.	dir.	imp.	dir.	imp.
Φ, 100-row background	−0.28	+0.76	−0.11	+0.30	−0.10	+0.38
Φ, full training set	−0.37	+0.78	−0.15	+0.39	−0.06	+0.37
Φ, class-matched	−0.37	+0.78	−0.15	+0.39	−0.06	+0.37
$w\bar{x}$ (no background)	+0.95	+0.81	+0.35	+0.47	+0.41	+0.44
Probe weight w	+0.99	+0.95	+0.31	+0.27	+0.40	+0.29
Mean $\lvert\phi\rvert$, full set	–	+0.89	–	+0.46	–	+0.43
P itself (probe → LM)	–	–	+0.31 [+0.10, +0.49]	+0.40 [+0.23, +0.55]	+0.39 [+0.20, +0.56]	+0.38 [+0.20, +0.54]

Matching the class mix leaves the sign agreement of Φ with the weight essentially unchanged (43.0% of the pool against 46.5% for the canonical background), as expected since the evaluation and training sets are balanced to within 0.1 points. Over 500 random 100-row backgrounds, with or without stratification to the evaluation class mix, directional ρ against D is centred near zero (about −0.05, range roughly $[-0.26, +0.17]$).

Ablating all 114 features gives the same picture: the probe's ablation effect $-\mathbb{E}_e[w_j x_{ej}]$ predicts the measured effect with directional $\rho \approx +0.34$ in both scopes, and the raw weight (+0.29 to +0.31) beats Φ (negative). The directional gap between the raw weight and Φ (100-row background) is +0.41 [+0.09, +0.72] last-token and +0.50 [+0.17, +0.79] all-token under steering, and similar under ablation (5,000 paired feature resamples). The earlier $k = 20$ ablation estimate (+0.70, $n = 20$) was a small-sample overestimate and is superseded. The probe-to-LM correlation is not a strict ceiling: scores such as $w\bar{x}$ and mean $|\phi|$ also use the activation distribution and can exceed it.

D Minimal-Pair Stimuli

`remove` pairs take held-out sentences containing *exactly one* word from a hand-curated sentiment lexicon and delete it, leaving the rest byte-identical (with two sentiment words, the edit would no longer have a single cause). `insert` pairs append " it was ⟨word⟩ ." to lexicon-free sentences. `substitute` pairs swap the lexicon word for a random word of opposite polarity. `insert-interior` pairs place "it was ⟨word⟩," at the midpoint of a lexicon-free sentence, so the final token and its context are unchanged. The type/position confound thus concerns distance from the final token, not a replaced token. Pools hold 411–886 pairs per edit, from which 100 are drawn.

Changed-feature threshold. A candidate counts as changed if $|\Delta a_j| \geq 0.10 \max_i |\Delta a_i|$ within the pair. Local-SHAP AUROC is stable across relative thresholds 0.05–0.50 for insertion (0.62–0.68) but not for removal (0.55 at 0.10, 0.40 at 0.50). A threshold-free per-pair Spearman of |score| against $|\Delta a|$ gives local SHAP 0.04 (removal) and 0.26 (insertion), against 0.57 and 0.67 for activation magnitude at the edited last token.

Readout noise floor. Null edits (a neutral word deleted or appended) give a median $|\Delta|$ of 0.14 (removal-type) and 0.23 (insertion-type) logits, against mean effects of 0.28 and 0.77. Restricting to pairs above the null 90th percentile raises sign accuracy (0.85 removal, 0.94 insertion) and leaves AUROC and ρ within 0.06 of the full-sample values, with the same method ordering.

Seeds. Seeds 0–3 redraw pairs and distractors from the same pool, so they overlap and are descriptive. Local-SHAP AUROC varies by at most 0.03 (SD) per edit, and no SHAP run exceeds 0.5 under activity-matched distractors.

E Supplementary Tables and Figures

E.1 Linear-Arm Identity Check and MLP-Arm Exact p-Values

Closed-form identity check (linear probe, margin target): $\max|\mathrm{IG} - \Phi| = \max|\mathrm{GA} - \Phi| = 1.72 \times 10^{-7}$. IG, GA and signed-mean SHAP Φ coincide analytically for a linear probe, giving pairwise $\rho = 1.000$ ($n = 615$ non-zero-coefficient features) as an identity, not an empirical result. The identity also holds under mean-$|\cdot|$ aggregation, where the probe weights correlate with all three at $\rho = 0.39$. Aggregating IG and GA

as |signed mean| against mean-$|\phi|$ SHAP would give only $\rho = 0.81$, an aggregation effect and not a difference between methods.

Table 12: MLP-arm attribution agreement (Spearman ρ on |score|, $n = 114$; SHAP, IG and GA are all mean $|\cdot|$ against the full training set).

Pair	ρ	p
Probe vs. IG	0.336	0.0003
Probe vs. SHAP	0.334	0.0003
Probe vs. GA	0.314	0.0007
IG vs. SHAP	0.995	< 0.0001
IG vs. GA	0.980	< 0.0001
SHAP vs. GA	0.984	< 0.0001

E.2 Selection-Bias and Sign-Stratified Checks

Table 13: SHAP faithfulness across candidate-selection strategies (steering), $k = 20$ except `all` ($k = 114$). `top` biases SHAP's ρ downward relative to other methods; `top-pos`/`top-neg` test the sign-inversion hypothesis without mixed-sign cancellation.

Arm : kind	random	top	top-pos	top-neg	all (k=114)
Linear : imp	$+0.636^{**}$	$+0.496^{*}$	+0.293	+0.397	$+0.430^{**}$
Linear : dir	−0.077	−0.083	+0.268	−0.226	−0.097
MLP : imp	$+0.747^{**}$	+0.107	+0.322	$+0.648^{**}$	$+0.476^{**}$
MLP : dir	−0.147	−0.161	+0.168	−0.376	−0.091

$^{*}p < 0.05$, $^{**}p < 0.01$, uncorrected; all other cells $p > 0.08$. Importance for `random` and `all` is mean $|\phi|$ (full training background). The `top`, `top-pos`, and `top-neg` sets are chosen by signed Φ and keep it as the score: scoring a Φ-selected set with a different score would remove the range restriction they test. With signed Φ the `random` and `all` cells are +0.555 and +0.377 (linear) and +0.744 and +0.347 (MLP).

E.3 Alpha-Mode Robustness (Scaled vs. Constant)

Table 14: Alpha-mode sensitivity: scaled vs. constant per-feature α (steering, random selection, $k = 20$).

Arm	Method	ρ_{imp} (scaled / constant)	ρ_{dir} (scaled / constant)
Linear	SHAP/IG/GA	+0.636 / +0.290	−0.077 / +0.072
Linear	Probe	+0.519 / +0.226	+0.624 / +0.567
MLP	SHAP	+0.747 / +0.486	−0.147 / −0.011
MLP	IG	+0.764 / +0.508	−0.146 / +0.000
MLP	GA	+0.707 / +0.406	−0.171 / −0.032
MLP	Probe saliency	+0.400 / +0.365	n/a

Constant-α importance is markedly lower, consistent with its numbers partly reflecting activation scale; we report both modes.

E.4 Multi-Seed Replication

The sweep is descriptive. Seeds draw overlapping subsets of one 114-feature pool and reuse the same per-feature effects, so they are not independent replicates; inference comes from the `all` run (Appendix C.2). Each per-seed test uses only $n = 20$ features, and most do not survive a pooled BH correction (8/39 distinct linear tests, 3/70 MLP). Mean importance ρ is positive in every method–arm cell and SHAP, IG and GA directional ρ is centred near zero, matching the `all` run.

Table 15: Multi-seed faithfulness sweep (10 seeds, $k = 20$, scaled α). Descriptive only. Ref p is the fraction of seeds with $|\rho|$ at least that of the published seed-0 draw.

Method	Type	Mean ρ	SD	Min	Max	Frac. > 0	Ref p
Linear arm							
SHAP/IG/GA	imp	+0.355	0.282	−0.260	+0.636	0.90	0.100
SHAP/IG/GA	dir	−0.047	0.160	−0.356	+0.141	0.50	0.700
Probe	imp	+0.236	0.268	−0.260	+0.582	0.80	0.200
Probe	dir	+0.422	0.162	+0.162	+0.624	1.00	0.100
MLP arm							
SHAP	imp	+0.383	0.262	−0.171	+0.747	0.90	0.100
SHAP	dir	−0.005	0.191	−0.377	+0.202	0.50	0.700
IG	imp	+0.410	0.266	−0.159	+0.764	0.90	0.100
GA	imp	+0.405	0.232	−0.062	+0.707	0.90	0.100

Imp: importance; dir: directional. Pooled BH (descriptive): 8/39 distinct tests survive (linear), 3/70 (MLP), at FDR 0.05.

E.5 Single-draw $k = 20$ causal validation results

Continuous (partial-Spearman) activity control. We compute a partial Spearman of |attribution| vs. $|\Delta|$ controlling continuously for |activation| (rather than the binary active/inactive split of Block B), keeping the whole candidate set. Local SHAP's mean partial ρ is near zero (−0.002 linear, +0.096 MLP), versus −0.144 and −0.117 for the global score.

E.6 Lexical Simplicity Control

This analysis uses the full 1,436-row held-out split, not the 1000-row canonical eval sample used elsewhere.

The best validation-selected bag-of-words (TF-IDF) is within 0.03 accuracy of the probe, so by our pre-fixed rule the probe is no better than word presence on this split. On average 24.3 filtered features are active per example, with mean pairwise Jaccard of the top-20 features 0.223.

Word-detector tests. Held-out R^2 of a ridge regression from bag-of-words to feature activation has median 0.171 for the top-20 $|\Phi|$ features with full vocabulary (0.106 for the other masked features) and 0.003 with the lexicon only; four of the top 20 reach $R^2 \geq 0.25$. Full bag-of-words explains 0.389 of the probe logit's variance (0.148 with the lexicon only), a lower bound on reducibility given the linear fit. Lexicon correlations are small for the top-20 features (mean $|r| \leq 0.07$, including for lexicon words in the last three tokens), and sign-agreement of Φ with the net lexicon score is near zero (−0.006). The lexicon explains almost none of any feature's variance, so the earlier lexicon-presence test could not have revealed word-detection.

Table 16: Single-draw causal faithfulness ($k = 20$, steering and ablation, random selection, $n_{\text{eval}} = 1000$). Cells show Spearman ρ with uncorrected p in parentheses; at $n = 20$ these are descriptive and not used for inference. For the linear arm, SHAP, IG and GA are analytically identical and listed once. SHAP, IG and GA importance are mean absolute attribution; direction is the signed mean (Φ for SHAP).

Method	ρ_{imp} (p)	ρ_{dir} (p)
Linear arm, steering		
SHAP/IG/GA	+0.636 (0.003)	−0.077 (0.748)
Probe	+0.519 (0.019)	+0.624 (0.003)
Linear arm, ablation		
SHAP/IG/GA	+0.535 (0.015)	−0.427 (0.060)
Probe	+0.347 (0.133)	+0.648 (0.002)
MLP arm, steering		
SHAP	+0.747 (0.0002)	−0.147 (0.535)
Probe saliency	+0.400 (0.081)	n/a
IG	+0.764 (0.0001)	−0.146 (0.540)
GA	+0.707 (0.0005)	−0.171 (0.470)
MLP arm, ablation		
SHAP	+0.525 (0.018)	−0.408 (0.075)
Probe saliency	+0.281 (0.230)	n/a
IG	+0.513 (0.021)	−0.421 (0.065)
GA	+0.552 (0.012)	−0.453 (0.045)

Table 17: Lexical simplicity control (held-out, $n = 1{,}436$). Models are fit on train with regularisation chosen on validation. Diff. is probe minus baseline accuracy with a paired-bootstrap 95% interval; p is McNemar.

Model	Features	Accuracy	Diff. [95% CI]	p
SAE probe (held-out)	114 masked / 32768	0.810	–	–
Lexicon-count LR	158	0.611	+0.198 [+0.166, +0.229]	$< 10^{-30}$
Unigram BoW LR (binary)	13,605	0.794	+0.016 [−0.010, +0.041]	0.26
Unigram+bigram BoW LR	79,812	0.800	+0.010 [−0.015, +0.034]	0.49
TF-IDF unigram+bigram LR	79,812	0.806	+0.003 [−0.022, +0.028]	0.83
Dense layer-7 residual LR	768	0.870	−0.060 [−0.080, −0.040]	$< 10^{-8}$

E.7 All-token global faithfulness

Table 18: All-token global faithfulness ($k = 114$, steering, all filtered features). Cells show Spearman ρ with BH-corrected q in parentheses.

Arm	Method	ρ_{imp} (q)	ρ_{dir} (q)
Linear	SHAP/IG/GA	+0.430 (<0.0001)*	−0.097 (0.303)
Linear	Probe	+0.289 (0.003)*	+0.399 (0.0001)*
MLP	SHAP	+0.476 (<0.0001)*	−0.091 (0.473)
MLP	Probe saliency	+0.122 (0.343)	n/a
MLP	IG	+0.483 (<0.0001)*	−0.058 (0.630)
MLP	GA	+0.500 (<0.0001)*	−0.015 (0.875)

* Survives FDR 0.05. Linear arm: 5/8 tests survive (SHAP, IG and GA counted separately; 3/4 distinct); MLP arm: 3/7. The same tests survive when BH is pooled over all 30 tests (both arms, both intervention scopes).

F Additional Limitations

(1) **Attribution error and probe–LM mismatch are only partly separable.** The probe-side controls (Appendix C.6) identify the signed-Φ directional failure as attribution error, but they exist only for the linear arm, where the probe's response is closed form. The probe's steering effect predicts the language-model effect at $\rho \approx 0.3$–0.4, so a faithfulness ρ in that range cannot distinguish a faithful attribution from a partly faithful one, and we make no claim about unfaithfulness to the language model beyond it.

(2) **We calibrate α against a different readout than we evaluate it with** (Appendix C.1). We use a layer-11 residual probe's $P(\text{positive})$ at calibration time, and a GPT-2 logit difference at evaluation time.

(3) **The MLP arm's probe saliency score is magnitude-only** ($\|W_1[i,:]\|_2$, no sign). This is valid for importance/ranking comparisons, invalid for any directional or causal-sign comparison.

(4) **`top` selection biases SHAP's faithfulness ρ downward** relative to the other three methods, by range-restricting only SHAP's scored variance (Appendix C.2). We never present a `top` number without this caveat.

(5) **The MLP arm is only weakly nonlinear and its attributions are approximate.** A linear fit to the MLP logit has $R^2 = 0.99$ on the evaluation rows, and IG, GA and SHAP agree at $\rho = 0.98$–0.995, so this arm adds little beyond the linear control. DeepSHAP approximates Shapley values.

(6) **The MLP probe's reported validation accuracy is mildly optimistic**: we select its inputs using a val-derived (label-free) frequency filter.

Interpreting Character-Level Transformers for Addition and Multiplication Through Integrated Gradients

Abhinav Damera
Northern Burlington County Regional High School
New Jersey, USA
27damera@nburlington.com

Abstract

Transformer models have demonstrated strong performance across a variety of reasoning tasks. However, understanding how these models perform arithmetic reasoning remains an open challenge. This study focuses on investigating the factors that influence arithmetic performance and the interpretability of character-level transformer models. These models were trained on increasingly complex tasks including one-digit arithmetic, two-digit arithmetic, and natural language prompts containing arithmetic expressions embedded within them. Model performance was evaluated by examining the effects of altering training iterations, dataset size, and model configurations. In order to account for training variability, representative two-digit configurations were trained using five independent random seeds and evaluated using a standardized prompt-level procedure. In order to perform the interpretability analysis, Integrated Gradients was utilized to calculate token-level attribution for each input token across zero embedding and mean embedding baselines. The results show that model performance generally improved as training iterations, dataset size, and model configurations increased. Additionally, expanding the character vocabulary allowed the model to process natural language prompts containing arithmetic expressions while generally assigning lower attribution scores to contextual letters and whitespace than for arithmetic tokens. The findings show that the character-level transformer models achieved high accuracy on arithmetic expressions embedded within natural language, while attribution scores were generally higher for arithmetic tokens than for contextual letters and whitespace. This study examines factors associated with the performance of character-level transformer models and provides interpretability evidence of the internal features that contribute to model predictions.

Keywords

Transformer Models, Arithmetic Reasoning, Integrated Gradients, Explainable Artificial Intelligence, Interpretability

ACM Reference Format:
Abhinav Damera. 2026. Interpreting Character-Level Transformers for Addition and Multiplication Through Integrated Gradients. In *Proceedings of International Journal of Secondary Computing and Applications Research (IJSCAR VOL. 3, ISSUE 4)*. ACM, New York, NY, USA, 10 pages. https://doi.org/10.67149/yhjs2024.5/88bi6k5z

This paper is published under the Creative Commons Attribution-NonCommercial-NoDerivs 4.0 International (CC-BY-NC-ND 4.0) license. Authors reserve their rights to disseminate the work on their personal and corporate Web sites with the appropriate attribution.
IJSCAR VOL. 3, ISSUE 4
© 2026 The Author(s). Published by IJSCAR.

1 Introduction

Transformer-based models have become crucial in artificial intelligence in the past decade [2, 10, 15]. These machine learning models have the ability to solve simple tasks such as language generation or even more complex operations involving deep learning and reasoning abilities [2, 10, 16]. Although they are effective, these models often operate as black boxes, making it hard for users to understand what is truly happening behind the scenes. It has become increasingly important to interpret the reasoning of these models because this lack of transparency poses a challenge when attempting to determine which specific parts of an input contribute most to model outputs. Additionally, increased transparency may help determine whether successful predictions are due to meaningful representations or other patterns that may form during model training.

As artificial intelligence becomes increasingly embedded within everyday life, understanding the specific reasoning behind model predictions has become a significant problem. Interpretability methods are useful for addressing this issue, as they allow users to investigate the level of influence each feature and input has on the model's final prediction [8, 11, 13]. These methods can improve transparency and trust in the system by providing insight into the model behavior. Specifically, this is significant when models produce correct or incorrect outputs and do not provide explicit explanations demonstrating how the predictions were made. Token-level attribution shows the influence of each specific input token in an input sequence on model outputs.

Studying interpretability on large-scale language models is difficult due to their architectural complexity, size, and the wide range of information contained within natural language prompts. Arithmetic problems could be used to test interpretability as they provide more controlled environments. Furthermore, utilizing arithmetic can be used to quantitatively and objectively understand model behavior by examining the model's output. This allows us to distinguish the impact of different pieces of each prompt. Beginning with single digit arithmetic and progressively increasing task complexity provides a controlled framework for evaluation and allows us to understand how the model performs against increasingly difficult tasks. Furthermore, embedding arithmetic within natural language prompts can be useful in determining how the model identifies relevant numerical information among the unrelated textual context.

Previous studies have focused on transformer reasoning and interpretability methods for understanding neural network predictions. However, little research has been done on token-level attribution in controlled arithmetic tasks while evaluating how training conditions and model architecture affect numerical predictions. By combining quantitative performance evaluation with

token-level attribution, researchers can understand how models produce successful predictions and what input portions influence those predictions.

This study focuses on small character-level transformer models rather than large-scale language models. Therefore, the results should be interpreted as evidence about transformer behavior in this controlled setting rather than as a direct characterization of arithmetic reasoning in frontier-scale language models. The models are trained on one-digit and two-digit arithmetic operations, allowing individual variables and tokens to be examined systematically, and are evaluated quantitatively. The models are evaluated on the number of training iterations, dataset size, and model capacity. Finally, the model's reasoning behavior is evaluated through Integrated Gradients. The results show how transformer architecture influences performance and provide insight into input features associated with predictions.

2 Related Work

Modern machine learning models have increasingly relied on transformer-based architectures due to their ability to use self-attention mechanisms, allowing each individual token to use information from other tokens in the input sequence, rather than recurrent computations [2, 10, 15]. The attention operation computes weighted combinations of value vectors based on query-key similarity:

$$Attention(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \tag{1}$$

where Q represents the query, K represents the key, and V represents the value. Subsequent autoregressive transformer models further demonstrate that large language models can perform increasingly sophisticated reasoning tasks, including arithmetic through next-token prediction [2, 10, 16].

Recent research has focused on understanding the ability of transformer models to perform arithmetic reasoning. These works suggest that language models are able to learn numerical patterns from training data even if they were not explicitly designed for symbolic computation [9, 12, 14]. However, these studies often lack insight into the mechanisms that lead to success in transformer models when completing operations as they primarily focus on predictive performance.

A major challenge as transformer models become more complex is understanding their reasoning for each output they produce. Explainable AI methods such as LIME and SHAP aim to address this issue by identifying features and inputs that influence model predictions [8, 11]. These interpretability techniques focus on utilizing local approximations and feature contribution analysis. Integrated Gradients, a gradient based attribution method, has become widely adopted because it satisfies properties such as Sensitivity and Implementation Invariance [13]. Integrated Gradients creates attribution scores for each input token, making it a useful attribution-based method as it does not rely fully on a single token. Although Integrated Gradients has become widely used, few studies have focused on utilizing it to understand arithmetic reasoning behavior in transformer-based language models.

Recent studies have examined attention distributions, neuron activations, and feature attribution techniques to better understand transformer reasoning [1, 3, 4]. This study focuses on addressing these gaps by using quantitative evaluation methods on increasingly difficult arithmetic tasks and then using Integrated Gradients to perform an interpretability analysis on the final natural language processing model to demonstrate how token-level attribution relates to arithmetic reasoning.

3 Methodology

During model training and evaluation, datasets were distributed with an equal number of multiplication and addition expressions. Additionally, positive numbers were used throughout all experiments. In order to investigate arithmetic reasoning in transformer models, three datasets were created that progressively became more complex. The first dataset consisted of one-digit multiplication and addition expressions, ranging from 0 to 9, to evaluate whether a character-level transformer could learn basic arithmetic relationships. The second dataset consisted of two-digit multiplication and addition expressions, ranging from 10 to 99. In order to successfully complete two-digit operations, the model had to learn more nuanced relationships between numerical values. These nuances include carrying operations in addition and multi-digit computation in multiplication. The increased numerical complexity allowed for the evaluation of the model's ability to scale beyond simple arithmetic tasks. The last dataset further increased in complexity. The numerical values still ranged from 10 to 99 similar to the second dataset. However, this dataset included natural language embedded with arithmetic operations. The natural language prompts were originally constructed manually and tested one by one. Eventually, three different 1,000-prompt test sets (one for the one-digit model, one for the two-digit model, and one for the natural language model) were generated separately and used to evaluate each model's performance. These evaluation prompts were kept fixed across repeated runs of the same configuration so that models trained with different random seeds were evaluated under identical conditions. This helped solidify our findings by expanding the range of linguistic contexts and reducing dependency on manually selected examples. The natural language characters served as irrelevant linguistic context to test whether the model would still perform arithmetic operations while ignoring unrelated information.

Each dataset was generated using custom Python scripts. Each of them was converted into the binary training format required by nanoGPT. Separate training and validation splits were prepared for each dataset using a 90/10 split. The model was trained on the training portion while it used the validation portion to monitor model performance during training. After training, the model was evaluated using the fixed 1,000-prompt test set to measure its performance.

The evaluation prompts consisted of the same numerical ranges as the corresponding training datasets. Specific operand combinations were not deliberately held out from the training data. Rather, the evaluation prompts were generated separately from the same numerical ranges.

The model used character-level tokenization where each individual character (letters, digits, operators, punctuation) was represented by a unique token. The model produces the probability distribution of the next token given the preceding sequence:

$$P(x_t \mid x_1, x_2, \ldots, x_{t-1}) = \text{softmax}(Wh_t + b) \tag{2}$$

where x_t represents the predicted token, h_t is the hidden representation, while W and b are output parameters. This allowed the model to perform arithmetic operations even with natural language prompts embedded under one unified vocabulary. Transformer configurations were modified in order to test model scaling. Each experiment varied in the number of transformer layers, attention heads, and embedding dimensions. However, the model maintained its overall architecture throughout all experiments. A checkpoint was saved for each experiment to allow consistent comparison across models.

Models were trained independently on each dataset using supervised character prediction. Cross-entropy loss was minimized between the predicted and target token distributions:

$$L = -\frac{1}{N}\sum_{i=1}^{N} y_i \log(\hat{y}_i) \tag{3}$$

where y_i represents the target token distribution and $\hat{y}_i$ represents the predicted probability distribution. The AdamW optimizer was used during training with learning rate warmup and cosine learning rate decay following the nanoGPT framework [7]. Unless specified by the experiment, models were trained for 20,000 iterations using a batch size of 64, a block size of 32, a learning rate of 1×10^{-3}, and a dropout rate of 0.2. During the model scaling experiment, the model depth, number of attention heads, and embedding dimension were varied while the remaining model parameters were held constant. Training and validation loss were observed during each experiment. Larger model configurations were tested in order to understand whether increasing model capacity improves arithmetic performance. The final and most advanced model, consisting of natural language, used the largest architecture and was used to perform the interpretability analysis.

The performance of each model was evaluated quantitatively using prompt-based inference. The evaluation involved introducing a fixed set of arithmetic expressions to each model based on the model's configurations. Arithmetic problems in the test set did not contain the answers to each problem. The model's predictions were compared against the correct arithmetic answers. The number of correctly solved problems was divided by the total number of prompts in the test set to produce the accuracy. Every prompt contributed to the model's accuracy. The same procedure was utilized for all experiments and random seeds.

The natural language model also focused on solving arithmetic expressions correctly while the prompts involved distractor letters and numbers. Multiple prompt styles were tested with the model, including both two-digit multiplication and addition, so that robustness across different arithmetic tasks and sentence contexts could be tested. The model was evaluated using the same prompt-level procedure.

To account for variability of random initialization and training, representative configurations were trained using five independent random seeds. The accuracy for these configurations is presented as the mean and standard deviation across each of the five runs. The repeated seed evaluation was used for the two-digit model when testing the 5,000 example and 20,000 example dataset as well as the expanded two-digit model that trained on the same 20,000 example dataset. The natural language model utilized the expanded model architecture and the same 20,000 example dataset. This model and the subsequent Integrated Gradients analysis was conducted using a single trained model rather than multiple random seeds.

In order to understand which input tokens were associated with the model's predictions, Integrated Gradients (IG) from the Captum library [6] was utilized to complete interpretability experiments:

$$IG_i(x) = (x_i - x_i') \int_0^1 \frac{\partial F(x' + \alpha(x - x'))}{\partial x_i}\, d\alpha \tag{4}$$

where x represents the original input embeddings, x' represents the baseline embeddings, $F(x)$ represents the logit corresponding to the model's predicted next token, and i is the input token being analyzed. Integrated Gradients computes attribution scores by accumulating gradients along a continuous path between a baseline input and the observed input. During the interpretability analysis, 100 integration steps were used to compute attribution scores for all 1,000 prompts in the natural language test set. Two different baseline configurations were utilized: a zero embedding baseline and a mean embedding baseline. For each prompt, the target was the model's highest-probability next-token prediction at the final input position. A single attribution value was obtained for each input token by summing attribution values across the embedding dimensions. The resulting scores were aggregated by token category: digits, operators, letters, punctuation, and whitespace. This was done to examine whether the observed attribution patterns were consistent across the evaluation set. Larger positive attribution scores suggest a greater influence on the overall prediction. The resulting attribution scores were used to determine whether the transformer mostly relied on the arithmetic tokens (digits and operators) or whether the unrelated natural language influenced predictions more heavily. This analysis provided an additional insight on the input features associated with the model's predictions beyond traditional prediction accuracy metrics.

4 Experimental Evaluation

The experimental evaluation was structured such that each experiment progressively increased in complexity. We first establish a baseline for the arithmetic performance before testing the effect of training iterations, numerical complexity, dataset size, model capacity, and additional contextual information. In the end, the interpretability analysis is performed where we analyze the token-level attribution of the model by utilizing Integrated Gradients. In all, these experiments not only test the effect of increasing complexity on the model's output, but also identify whether relevant information influences predictions and to what extent the linguistic context impacts these predictions.

4.1 Experiment 1: Baseline Arithmetic Evaluation

The first experiments were created to test whether transformer-based models could learn to perform arithmetic operations by learning arithmetic relationships through supervised training. A synthetic dataset, containing around 20,000 examples including one-digit arithmetic multiplication and addition expressions, was

created using a custom Python script. Additionally, these examples contained the actual answers to the problems for model training while the prompts did not contain correct outputs during evaluation. There was an equal amount of addition and multiplication examples. The model was set up with 1 layer, 1 attention head, an embedding dimension of 120, dropout rate of 0.2, batch size of 64, and a block size of 32. The training hardware which was used was a CPU.

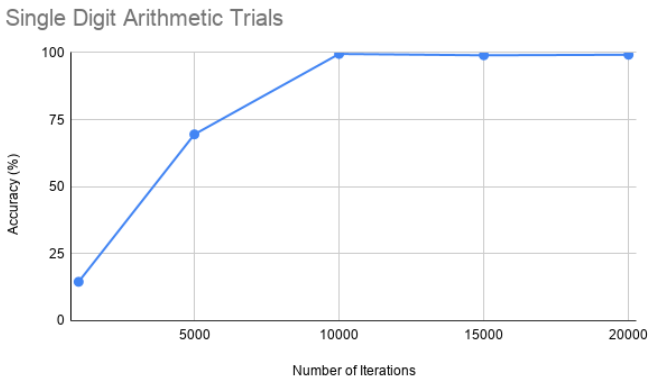


Figure 1: Accuracy improvement of the single-digit baseline arithmetic model with increasing training iterations.

Table 1: Single-digit baseline arithmetic model performance across training iterations.

Trial	Training Iterations	Accuracy (%)
1	1,000	14.51
2	5,000	69.50
3	10,000	99.50
4	15,000	99.00
5	20,000	99.20

The effect of increasing training iterations on model accuracy is shown in Figure 1 and Table 1. The results in Table 1 are from a single training run and therefore do not quantify variability across random seeds. The model was trained using supervised next-character prediction. The objective of the model was to predict the next character in an arithmetic expression. The model's performance was evaluated using arithmetic prompts to verify that the network generated the right numerical outputs. Addition and multiplication problems were evaluated together. These initial experiments established the proof of concept that the model was capable of learning simple arithmetic operations. It allowed us to see whether changes to the model had to be made to allow for it to be able to process arithmetic prompts before moving onto the more rigorous evaluations.

The initial model was adjusted to test the effect of altering the number of training iterations to see if that led to improved accuracy. Model checkpoints were evaluated throughout the training process to determine how the additional optimization impacted performance. Due to the usage of a CPU over a GPU, training took longer than expected. We noticed as we increased the number of training iterations, the training time substantially increased. Specifically, when the number of iterations was set to 1,000, the training time was less than 15 minutes. However, when the number of iterations was set to 20,000, the training time increased to over 2.5 hours. Accuracy generally improved as the number of training iterations increased, rising from 14.51% at 1,000 iterations to 99.20% at 20,000 iterations. The 1,000 prompt test set contained 199 unique arithmetic expressions. All of the unique prompts appeared in the training portion which means that the evaluation set did not assess generalization to unseen one-digit operand–operator combinations. Therefore, these results should be interpreted as performance on problems represented in the training data. Furthermore, the results are from a single run.

The model performance seemed to improve nonlinearly as the number of training iterations increased. Although increasing the number of training iterations did lead to improved performance, the model's performance seemed to rapidly increase from 1,000 to 10,000 iterations and plateaued once it reached around 10,000 training iterations. This suggests that the model had already learned the necessary arithmetic relationships to perform single digit arithmetic operations (see Figure 1 and Table 1). The final model's configurations were used as the baseline for the second experiment. Additionally, 20,000 training iterations were used for subsequent experiments in order to allow each model to have enough training before evaluation.

4.2 Experiment 2: Two-Digit Arithmetic Evaluation

After the baseline model was evaluated, the model was expanded to include two-digit arithmetic multiplication and addition operations in order to determine whether the transformer could perform more complex arithmetic relationships. The two-digit arithmetic model contained most of the same parameters as the single digit model. However, the two-digit model contained 4 layers, 4 attention heads, and an embedding dimension of 128 as preliminary results showed that the model was initially unable to perform arithmetic calculations. Additionally, the two-digit model was trained on 20,000 iterations.

Initially, the two-digit arithmetic dataset only contained 5,000 examples. Similar to the one-digit dataset, only numerical prompts were included. However, the dataset included values from 10 to 99. Single digit numbers were not included in the new dataset. Even though combining one-digit and two-digit examples into one dataset would still have increased complexity, this would introduce different levels of complexity within one dataset making it hard to determine how the model responds to two-digit prompts specifically. By restricting the dataset to only two-digit, we can strictly analyze the model's ability to learn more complex numerical relationships associated with two-digit operations. Specifically, when performing addition operations, the model had to consider carrying between the ones and tens positions. For multiplication, the model had to consider the relationship between multiple digits. The increased numerical complexity was associated with the decrease in performance by the original model configuration.

Each model was trained against five independent random seeds in order to account for the variation caused by random initialization and was evaluated using the standardized prompt-level evaluation procedure. By training on the 5,000 example dataset, the model produced a mean accuracy of 68.74% with a standard deviation of 1.16 percentage points (see Table 2). The relatively small standard deviation indicated that model results were similar across the five random seeds. After the model trained on the original 5,000 example dataset, the performance of the two-digit arithmetic model was noticeably lower compared to the performance of the one-digit arithmetic model. We were initially reluctant to assume that the dip in accuracy was purely due to the increased complexity of the numerical tasks since the initial two-digit dataset was smaller compared to the one-digit dataset. This presented a potential confounding factor that could have affected the model's accuracy and its ability to create more complex relationships between numerical values. In particular, the dataset of the two-digit model only contained a quarter of the number of examples compared to the one-digit dataset. Therefore, it was not clear whether the performance decreased due to an increase in the complexity of the arithmetic operations or whether it was due to the reduced dataset size.

To isolate the effect of dataset size, the two-digit arithmetic dataset was expanded to include 20,000 examples to match the size of the one-digit arithmetic dataset while containing an equal distribution of two-digit multiplication and addition problems. The model architecture, training procedure, and training iterations were all held constant while the dataset size was the only experimental variable between both two-digit trials. Across the five random seeds, the model produced a mean accuracy of 86.00% and a standard deviation of 2.29 percentage points (see Table 2). The mean accuracy increased by 17.26 percentage points from the initial dataset to the expanded dataset.

The evaluation prompts overlapped with the training data for both dataset sizes, but at different extents. For the 5,000 example dataset, 243 of the 1,000 evaluation prompt occurrences (24.30%) matched arithmetic problems represented in the training portion. On the other hand, 670 of the 1,000 evaluation prompt occurrences (67.00%) matched arithmetic problems represented in the training portion for the 20,000 example dataset. When considering only unique prompts, 233 of 970 problems (24.02%) overlapped with the 5,000 example dataset and 652 of 970 (67.22%) overlapped with the 20,000 example dataset. Therefore, the 17.26 percentage point increase may partly reflect greater exposure to evaluated problems during training. Operand–operator combinations were not deliberately held out. So, these results do not necessarily establish improved generalization to unseen arithmetic problems.

The results suggest that increasing the amount of training data from 5,000 to 20,000 examples substantially improved performance (see Figure 2). The mean accuracy of the two-digit model remained lower than the accuracy of the one-digit model. This demonstrates that the increased numerical complexity remained a challenge under the baseline architecture. These results motivated the subsequent experiment investigating the effect of increasing model capacity to determine whether that would provide an additional improvement beyond increasing the amount of training data.

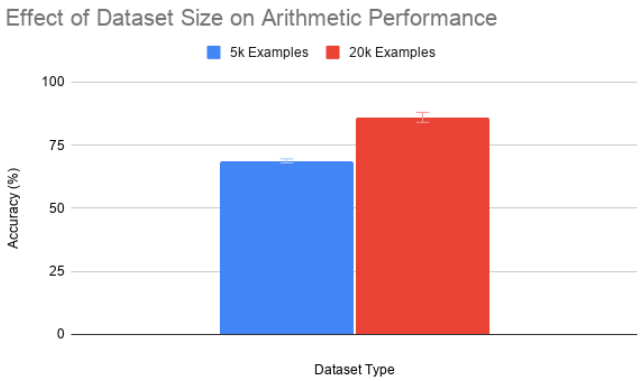


Figure 2: Accuracy comparison between 5,000 and 20,000 example datasets.

Table 2: Two-digit arithmetic accuracy across five random seeds for different dataset sizes.

Metric	5,000 Examples	20,000 Examples
Seed 1	69.70	84.00
Seed 2	67.70	83.10
Seed 3	70.00	87.70
Seed 4	68.90	87.10
Seed 5	67.40	88.10
Mean ± SD	68.74 ± 1.16	86.00 ± 2.29

4.3 Experiment 3: Model Scaling

After testing the effect of increasing the arithmetic dataset for the two-digit arithmetic model from 5,000 examples to 20,000 examples, this motivated the investigation of expanding the model architecture. Additionally, the one-digit model still performed substantially better than the two-digit model. Therefore, we decided to focus on altering the model architecture in order to make it larger to examine whether this factor would help the model learn more arithmetic relationships, allowing it to have a higher accuracy boost.

In order to isolate the effect of model capacity, the number of examples in the dataset was held constant at 20,000 examples. Additionally, the number of training iterations was held constant at 20,000 as well. The initial architecture of the model consisted of 4 transformer layers, 4 attention heads, and an embedding dimension of 128, resulting in 0.79 million trainable parameters. The new model was altered from 4 to 6 transformer layers, 4 to 6 attention heads, and an embedding dimension altered from 128 to 192. The largest configuration consisted of 2.66 million trainable parameters. The other model parameters such as dropout rate, batch size, block size, optimizer, and learning rate were held constant (see Table 3).

The largest two-digit arithmetic dataset, containing 20,000 examples, was used to train the model and was evaluated using the same prompt-based inference procedures as the original two-digit model. By keeping the dataset size, training duration, and evaluation methodology constant, this allowed for the evaluation of the model to be purely based on the increases in the model's capacity

Table 3: Configuration comparison between baseline and expanded two-digit model architectures.

Parameter	Baseline	Expanded
Layers	4	6
Attention Heads	4	6
Embedding Dimension	128	192
Parameters	0.79M	2.66M
Dropout	0.2	0.2
Batch Size	64	64
Block Size	32	32
Optimizer	AdamW	AdamW
Learning Rate	1e-3	1e-3
Training Iterations	20,000	20,000

rather than the influence of external variables. This allowed us to compare the baseline and expanded model architectures in a controlled environment. The experiment went on to show whether larger transformer architectures were beneficial and would lead to improvements in accuracy when tackling tougher arithmetic expressions.

Across the five independent random seeds, the model produced a mean accuracy of 92.66% with a standard deviation of 1.22 percentage points (see Table 4). The model had a 6.66 percentage-point improvement compared to the baseline architecture which achieved an 86.00% mean accuracy (see Figure 3). Both the baseline and expanded models were trained on the same 20,000 example dataset and trained over 20,000 iterations. The evaluation overlap for the expanded model was the same because it used the same 20,000 example dataset and evaluation prompts as the baseline model. Therefore, accuracy improvement indicates better performance under the same data conditions, but does not necessarily establish improved generalization to unseen arithmetic problems.

Table 4: Two-digit arithmetic accuracy across five random seeds for the baseline and expanded model architectures.

Metric	Baseline	Expanded
Seed 1	84.00	93.50
Seed 2	83.10	90.70
Seed 3	87.70	92.90
Seed 4	87.10	93.80
Seed 5	88.10	92.40
Mean ± SD	86.00 ± 2.29	92.66 ± 1.22

The expanded model had an improvement over the baseline architecture. This improvement was smaller compared to the improvement achieved by increasing dataset size from 5,000 to 20,000 examples. However, both changes led to improved performance. Furthermore, the expanded model architecture allowed the model to further approach the accuracy of the one-digit arithmetic model which suggests that increased model capacity was associated with improved performance on two-digit arithmetic operations.

This experiment focuses on investigating overall model capacity rather than isolating the contribution of a single component

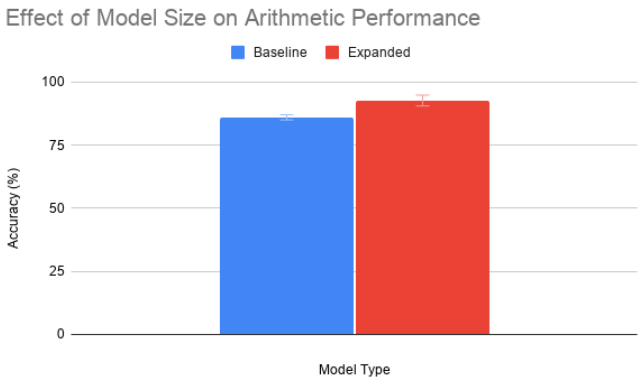


Figure 3: Accuracy comparison between baseline and expanded two-digit models.

of the model's architecture as the expanded architecture simultaneously increased the number of layers, attention heads, and embedding dimensions. Therefore, the results suggest an association between increasing model architecture and improved arithmetic performance.

4.4 Experiment 4: Natural Language Arithmetic Evaluation

The arithmetic-only experiments provided a foundation for understanding the factors that influenced the model's ability to learn and perform arithmetic operations. In order to add another level of complexity, arithmetic expressions were embedded into natural language prompts which meant that the arithmetic expressions were no longer isolated. These prompts contained surrounding words, punctuation, and non-mathematical characters. This was done to evaluate whether the model would still be able to identify relevant parts of the prompt and perform arithmetic operations despite the surrounding linguistic context.

Initially, natural language prompts were inputted into the arithmetic model to test the model. However, the model was unable to process these prompts purely due to a vocabulary limitation, producing key errors. This was the case because the initial character vocabulary only contained numerical values and operators. Therefore, the model was not able to process the natural language prompts because alphabetic characters and punctuation were not part of the tokenizer vocabulary. Examples of these vocabulary errors are shown in Table 5.

Table 5: Examples of vocabulary errors produced by the arithmetic-only model when processing natural language prompts.

Prompt	Output	Cause
I like pizza. 14*15=	KeyError	Unseen Character "I"
What is 88+23=	KeyError	Unseen Character "W"
Today is Sunday. 45+27=	KeyError	Unseen Character "T"

The vocabulary failure demonstrated that the arithmetic-only model was unable to directly evaluate natural language prompts without first modifying the tokenization scheme. Importantly, this did not suggest that the model was completely incapable of providing correct outputs with natural language surrounding arithmetic prompts as the model stopped directly after the first character when processing each prompt. In order to address this limitation, a new character-level vocabulary was created to account for both the original vocabulary, containing numerical values and operators, as well as alphabetic characters and punctuation marks required for the natural language prompts. The upgraded unified vocabulary allowed both mathematical and non-mathematical characters to be represented.

Subsequently, this motivated the construction of a new dataset which not only included numbers and operators, but also contained the letters and punctuation marks that were missing from the original arithmetic-only dataset. The same training framework and the same number of iterations from the previous experiments was used. Additionally, the range of numbers used was 10 to 99 which is the same range of numbers used in Experiments 2 and 3. Furthermore, the model contained the same expanded transformer architecture as Experiment 3. The initial natural language prompts were constructed and tested manually in order to examine whether the model could process the prompts under varied textual formats. Eventually, prompts were generated and tested in larger quantities to minimize the margin of error and reduce human bias.

The expanded natural language model was evaluated using a set of 1,000 prompts containing linguistic context as well as both two-digit multiplication and addition expressions. Similar to previous trials, model outputs were only considered correct if the output contained the correct numerical result. In order to successfully answer prompts, the model not only had to make sure that it developed the right numerical relationships for both addition and multiplication expressions, but it also had to focus on the relevant portion containing the arithmetic expression and reduce its focus on non-mathematical context.

The natural language model was evaluated through the standardized prompt-based evaluation procedure. The model produced the correct output for 943 prompts from the 1,000 prompt set, resulting in a 94.30% accuracy. The results suggest that the model retained substantial arithmetic performance even when numerical expressions were embedded within natural language prompts. The model seemed to perform slightly better than the expanded two-digit arithmetic model. However, this result is not directly comparable to the five-seed mean produced by the expanded two-digit architecture as it was produced by a single run.

4.5 Experiment 5: Interpretability Analysis

After noticing that the natural language arithmetic model was producing the right outputs for the majority of the evaluated prompts, this motivated further investigation in examining which input tokens influenced the behavior of the model, as the raw accuracy alone was insufficient to indicate which portions of the prompts were being used to generate outputs. In order to do so, interpretability experiments were conducted using the Captum library for PyTorch [6] for our fifth and most crucial experiment. Specifically, the model's behavior was analyzed using Integrated Gradients [13] as the primary attribution method because it satisfies the Sensitivity and Implementation Invariance axioms and has the completeness property. Integrated Gradients was used because it estimates how much each input token contributes to the overall output by integrating gradients along a continuous interpolated path between a baseline input and the observed input.

Attribution scores were computed systematically across all 1,000 evaluation prompts which had diverse context prefixes. For every input token in each prompt sequence, attribution was evaluated with respect to the highest probability next-token prediction. Two baseline configurations were investigated: a zero embedding baseline and a mean embedding baseline. 100 integration steps were used for each calculation. The target token was selected as the model's highest probability prediction for the final input position. The embedding-dimension attribution values were summed to produce a single attribution score for each input token. Attribution scores were aggregated by token category to observe whether the attribution scores and patterns of individual prompts were consistent across the test set. The attribution scores were crucial as they provided insight into the contribution of each individual character to the model's output. This helped determine whether arithmetic operators and numerical values had a greater influence on the model's predictions than the surrounding natural language tokens. After evaluating all batches, the distribution of attribution scores revealed a divide between natural language prefixes and arithmetic operations.

After observing the zero embedding baseline, arithmetic operators received the highest mean attribution score (+5.891 ± 2.722) and digits resulted in the second highest mean attribution score (+1.436 ± 1.239). Punctuation seemed to produce an intermediate mean attribution of +0.975 ± 1.528. Whitespace and letters received the lowest attribution scores. Similarly, this pattern was also consistent through the mean embedding baseline as operators (+4.543 ± 2.317) and digits (+1.645 ± 1.262) contributed most to model predictions while the whitespace and letters received substantially lower scores. The results indicate that the attribution patterns were broadly consistent across both baselines (see Table 6).

Table 6: Mean Integrated Gradients attribution ± standard deviation across 1,000 test prompts using zero embedding and mean embedding baselines.

Token	Zero	Mean
Operator	+5.891 ± 2.722	+4.543 ± 2.317
Digit	+1.436 ± 1.239	+1.645 ± 1.262
Punctuation	+0.975 ± 1.528	+0.918 ± 1.824
Whitespace	+0.222 ± 0.653	+0.152 ± 0.493
Letter	+0.055 ± 0.231	+0.109 ± 0.276

For example, in the prompt "Today is Sunday. 42+94=", the highest attribution scores were given to the arithmetic operators (the plus and equals sign) along with the numerical values (4, 2, 9, and 4) that preceded the prediction. On the other hand, lower attribution scores were given to most characters belonging to the natural language prefix (see Figure 4).

(1) **Operational Tokens Attribution:** The arithmetic operators received the highest mean attribution scores across the test prompts. Specifically, the equals sign frequently produced the highest attribution score. This suggests the operators were strongly associated with the model's predicted outputs.
(2) **Operand Digit Attribution:** The second operand's leading digits and the first operand's major positional components attracted the majority of feature weight, suggesting that the model's predictions were more strongly associated with numerical tokens than with most contextual tokens. Digits received the second highest attribution, suggesting they also contributed meaningfully to the model's predictions.
(3) **Contextual Attribution:** The natural language portions of each prompt produced lower attribution scores than the other tokens or even produced negative scores. However, certain punctuation marks sometimes received higher scores than expected, suggesting that some non-mathematical tokens were not entirely irrelevant to the model's predictions.

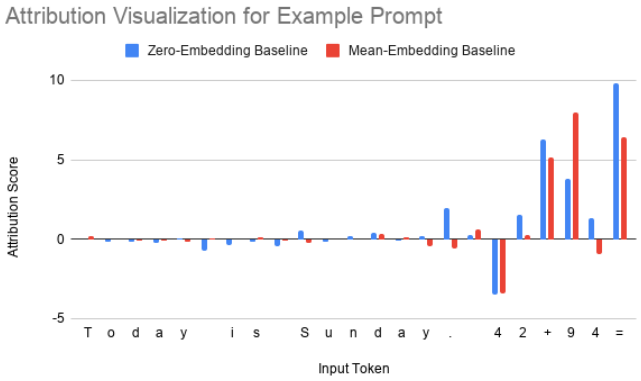


Figure 4: Character-level Integrated Gradients attribution scores for an example arithmetic prompt under zero embedding and mean embedding baselines.

Punctuation was further investigated as it occasionally received relatively high attribution scores. Two experiments were conducted to investigate the role of punctuation in arithmetic prompts. The first experiment focused on removing all punctuation marks. By doing so, we noticed the accuracy decrease from 94.30% on the original prompts to 93.00% on the prompts without punctuation. The 1.30 percentage point decrease in performance suggests that punctuation removal had a modest effect on performance (see Table 7).

Furthermore, the second punctuation experiment focused on shifting the position of the punctuation marks. Specifically, punctuation marks were shifted 5 characters to the left of their initial position. Accuracy decreased to 82.10% for the modified prompts, resulting in a reduction of 12.20 percentage points compared to the 94.30% on the original prompts (see Table 7). The larger decrease in performance suggests that punctuation placement and prompt structure can substantially affect model performance. However, this altered the local context and naturalness of the prompts. Therefore, the decrease in accuracy may not be solely attributed to the positions of punctuation marks.

Table 7: Effect of punctuation modifications on accuracy.

Condition	Accuracy	Δ (Percentage Points)
Original prompts	94.30%	—
Punctuation removed	93.00%	−1.30
Punctuation shifted left	82.10%	−12.20

tegrated Gradients scores for punctuation were examined under both the zero embedding and mean embedding baselines for the original and modified prompts (see Table 8). For the original prompts, the zero embedding baseline produced mean attribution scores of 0.953 for periods and 1.010 for colons while the mean embedding baseline produced mean attribution scores of 1.089 for periods and 0.651 for colons. For the relocated punctuation results, periods had higher mean attributions than colons under both baselines. Under the zero embedding baseline, periods produced a mean attribution score of 1.280 compared to colons which produced a score of 0.330. Under the mean embedding baseline, periods produced a mean attribution score of 1.930 while colons produced a mean attribution of 0.465. At the aggregate punctuation-category level, the mean score decreased from 0.975 to 0.695 under the zero embedding baseline but increased from 0.918 to 1.028 under the mean embedding baseline. These differences indicate that the observed attribution patterns may depend on the baseline and prompt condition. One possible explanation is that punctuation near an arithmetic expression may provide a structural cue associated with the transition from natural-language context to the expression. However, the results do not fully establish that altering punctuation position solely causes the observed changes in accuracy or attribution since relocating the punctuation marks also changes the surrounding text.

Table 8: Attribution statistics across baselines and prompt conditions.

Baseline	Prompt	Period	Colon	All Punctuation
Zero	Original	0.953 ± 1.701	1.010 ± 1.210	0.975 ± 1.528
Zero	Shifted	1.280 ± 4.369	0.330 ± 0.837	0.695 ± 2.824
Mean	Original	1.089 ± 1.846	0.651 ± 1.760	0.918 ± 1.824
Mean	Shifted	1.930 ± 3.966	0.465 ± 0.905	1.028 ± 2.653

These observations suggest that the model placed greater emphasis on the arithmetic portion of the prompt when predicting the next token. After evaluating the 1,000 prompt test set, the average convergence delta was computed for each baseline. The average signed convergence delta was $+8.88\times10^{-4}$ with an average absolute convergence delta of 8.21×10^{-3} for the zero embedding baseline. The mean embedding baseline resulted in an average signed convergence delta of $+2.46\times10^{-4}$ and an average absolute convergence delta of 4.11×10^{-3}. The small average absolute convergence deltas indicate that the attribution calculations had relatively small approximation errors for both baselines.

Although these experiments represent an initial analysis, they provide evidence that operators and numerical values received greater average attribution than letters and whitespace. Additionally, the majority of the prompt outputs were correct regardless of the linguistic information. Furthermore, the consistency between the two baselines indicates that the observed attribution patterns were not solely dependent on a single baseline choice.

5 Discussion & Future Work

The experiments demonstrate that the character-level transformer models used in this study were able to learn and perform one-digit and two-digit arithmetic operations under a controlled evaluation setting. Increasing training iterations improved performance for one-digit operations while the decrease in performance of the initial two-digit model suggested that more complex models required additional training data or a larger model capacity. The two-digit model substantially improved when dataset size increased from 5,000 examples to 20,000 examples, rising from a mean accuracy of 68.74% to 86.00%. Expanding the model architecture, with the same 20,000 example dataset, further increased performance to 92.66%. The results suggest that both additional training data as well as increased model capacity are associated with improved performance [5].

The repeated seed evaluation demonstrated the importance of accounting for training variability. Both datasets and the expanded model were tested against five independent random seeds. The 5,000 example dataset model produced a standard deviation of 1.16 percentage points. The 20,000 example dataset model produced a standard deviation of 2.29 percentage points. The expanded model produced a standard deviation of 1.22 percentage points. These trials helped provide a more complete representation of model performance rather than solely relying on a single run.

Testing the natural language prompts with the arithmetic-only model showed that the model was originally unable to process these prompts, suggesting that adjusting the vocabulary design was necessary to test arithmetic operations beyond purely numerical inputs. After the vocabulary was expanded, the model successfully produced correct outputs for the majority of the prompts even with surrounding natural language, indicating that the initial failure was due to the limited character vocabulary rather than dataset size or model architecture.

Integrated Gradients provided further insight into the features associated with model outputs. Specifically, operators and digits had the highest mean attribution scores across both the zero embedding and mean embedding baselines. On the other hand, whitespace and letters produced substantially lower scores. The intermediate attribution scores produced by punctuation marks suggest that some contextual tokens did have more influence than expected on model predictions. The punctuation experiments also showed that changes to punctuation placement were associated with a substantial decrease in accuracy, although these changes also altered the surrounding text. This suggests that prompt structure may affect model performance, but further experiments are needed to isolate the effect of punctuation position itself. The attribution values identified the associations between inputs and predictions. However, these results should be interpreted as evidence of feature contribution rather than direct evidence that the model possesses human-like reasoning abilities.

This study was focused on one-digit and two-digit arithmetic operations and the model's reasoning behavior was evaluated using Integrated Gradients on the final and most advanced model containing natural language. The controlled setting allowed for the analysis of token-level attribution and individual factors. However, it does not establish how the behaviors observed throughout the experiments would relate to larger language models or other mathematical reasoning tasks. The experiments do not compare the models against a non-transformer baseline. Additionally, model architecture parameters were increased simultaneously. Therefore, the effect of increasing overall model capacity was examined. Future work could isolate individual contributions of transformer layers, attention heads, or embedding dimensions. Additionally, future studies should focus on more complex arithmetic tasks, simpler computational baselines, larger benchmark datasets, other interpretability techniques, and examine whether larger or independently trained models exhibit similar attribution patterns while running trials on a GPU to increase computational speed.

6 Conclusion

This study has unpacked how character-level transformer models can learn arithmetic relationships in a controlled setting. Each examined factor demonstrated changes in arithmetic performance. However, the effect differed based on the factor being analyzed. Increasing dataset size substantially improved performance from a mean accuracy of 68.74% to 86.00% across five random seeds. Expanding the two-digit model's architecture increased performance to 92.66%, further reducing the gap in performance between the two-digit model and the simpler one-digit model. Integrated Gradients showed that the model assigned higher average attribution to arithmetic operators and numerical values than whitespace and letters, while the model achieved high accuracy on arithmetic expressions embedded in natural language. This overall pattern was consistent across both the zero embedding and mean embedding baselines. These findings demonstrate that interpretability methods are useful as they add an additional perspective, complementing accuracy-based evaluations, by identifying the influence of different input tokens on the model's predictions. More broadly, these experiments demonstrate the value of investigating transformer behavior through both accuracy and interpretability techniques rather than solely relying on model accuracy to provide evidence of reasoning.

7 Acknowledgements

The author sincerely thanks Dr. Sumit Kumar Jha of the University of Florida for his guidance and mentorship throughout this research. His support and feedback were especially valuable regarding the interpretability analysis during the study using Integrated Gradients.

References

[1] Abnar, S., Zuidema, W.: Quantifying attention flow in transformers. In: Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics. pp. 4190–4197 (2020)

[2] Brown, T.B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., et al.: Language models are few-shot learners. In: Advances in Neural Information Processing Systems. vol. 33 (2020)
[3] Elhage, N., Nanda, N., Olsson, C., et al.: A mathematical framework for transformer circuits (2021), https://transformer-circuits.pub/2021/framework/index.html
[4] Jain, S., Wallace, B.C.: Attention is not explanation. arXiv preprint arXiv:1902.10186 (2019)
[5] Kaplan, J., McCandlish, S., Henighan, T., Brown, T.B., Chess, B., Child, R., et al.: Scaling laws for neural language models. arXiv preprint arXiv:2001.08361 (2020)
[6] Kokhlikyan, N., Miglani, V., Martin, M., Wang, E., Alsallakh, B., Reynolds, A., et al.: Captum: A unified and generic model interpretability library for pytorch. arXiv preprint arXiv:2009.07896 (2020)
[7] Loshchilov, I., Hutter, F.: Decoupled weight decay regularization. In: International Conference on Learning Representations (2019)
[8] Lundberg, S.M., Lee, S.I.: A unified approach to interpreting model predictions. In: Advances in Neural Information Processing Systems. vol. 30 (2017)
[9] Power, A., Burda, Y., Edwards, H., Babuschkin, I., Misra, V.: Grokking: Generalization beyond overfitting on small algorithmic datasets. arXiv preprint arXiv:2201.02177 (2022)
[10] Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., Sutskever, I.: Language models are unsupervised multitask learners. OpenAI Technical Report (2019)
[11] Ribeiro, M.T., Singh, S., Guestrin, C.: "why should i trust you?": Explaining the predictions of any classifier. In: Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining. pp. 1135–1144 (2016)
[12] Saxton, D., Grefenstette, E., Hill, F., Kohli, P.: Analysing mathematical reasoning abilities of neural models. arXiv preprint arXiv:1904.01557 (2019)
[13] Sundararajan, M., Taly, A., Yan, Q.: Axiomatic attribution for deep networks. In: International Conference on Machine Learning. pp. 3319–3328 (2017)
[14] Trask, A., Hill, F., Reed, S., Rae, J., Dyer, C., Blunsom, P.: Neural arithmetic logic units. arXiv preprint arXiv:1808.00508 (2018)
[15] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, L., Polosukhin, I.: Attention is all you need. In: Advances in Neural Information Processing Systems. vol. 30 (2017)
[16] Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., et al.: Chain-of-thought prompting elicits reasoning in large language models. arXiv preprint arXiv:2201.11903 (2022)